Network Science

Concepts, Examples, and Exercises

Zahra Rezaei

2026-04-03

Table of contents

Chapter 1

Course Overview

1.1 Welcome

This book is designed as the main course text for a university class in **Network Science**. Its purpose is not only to define central ideas, but to help students move back and forth between:

- mathematical language,
- structural intuition,
- computational experimentation in `R`,
- and interpretation of real network phenomena.

The book treats networks as mathematical objects, probabilistic systems, and models of real complex systems all at once. Across the chapters, we move from the language of graphs to randomness, emergence, clustering, hubs, and robustness.

! What This Book Is Trying To Teach

The central goal of the course is to understand how **large-scale network behavior emerges from local structural rules**.
By the end of the book, students should be able to:

- describe networks using precise graph-theoretic definitions,
- analyze random and generative network models,
- interpret degree distributions, clustering, distances, and components,
- compare homogeneous and heterogeneous network structures,
- and use simulation in `R` to test theoretical predictions.

1.2 The Big Questions

The course is organized around a small number of recurring questions:

1. How should we represent a network mathematically?
2. What changes when edges are introduced at random?
3. Why do large connected structures suddenly emerge?
4. How can networks have both short distances and local clustering?
5. Why do some networks produce hubs and heavy-tailed degree distributions?
6. What makes a network resilient, and what makes it fragile?

These questions give the book its internal structure. Each chapter develops one part of this larger story.

1.3 Course Map

After this welcome page, the book begins with a short preface and reader's guide, then moves into the main conceptual chapters.

Chapter	Main theme	Guiding idea
Preface and Reader's Guide	Orientation	Explains how to read the text, what background is assumed, and how theory, code, and exercises fit together.
Graph Theory	Foundations	Networks begin with the language of vertices, edges, paths, matrices, and connectivity.
Random Graphs	Probability on graphs	Structure can be studied statistically when edges appear by chance.
The Evolution of Random Graphs	Emergence and phase transitions	Global connectivity appears through threshold behavior rather than smooth change.
Small-World Networks	Distance and clustering	Real networks can combine short paths with surprisingly strong local cohesion.
Scale-Free Networks	Hubs and heavy tails	Broad degree distributions create heterogeneity, hierarchy, and hub-dominated structure.
Network Robustness	Failure and resilience	The same structure that makes networks efficient can also make them vulnerable.

1.4 Conceptual Progression

The intellectual progression of the book is deliberate:

1. We start with **graph theory**, because every later model depends on a precise structural vocabulary.
2. We then study **random graphs**, where probability becomes part of the model rather than a nuisance.
3. From there, we examine **network evolution**, where component growth and threshold phenomena reveal how global order emerges.
4. We next turn to **small-world structure**, which explains why many large networks remain navigable.
5. We then study **scale-free networks**, where heterogeneity, hubs, and preferential attachment become central.
6. Finally, we investigate **robustness**, asking how network architecture shapes resilience under failure or attack.

This means the book is not just a list of topics. It is a structured argument about how network structure, randomness, growth, and vulnerability fit together.

1.5 How To Use This Book

Each chapter is designed to be used in the same rhythm:

1. **Theory and intuition**: key definitions, explanations, and mathematical results.
2. **Worked examples in `R`**: simulations and visualizations that make the abstract ideas concrete.
3. **Exercises and coding tasks**: problems that move from derivation to interpretation to implementation.

For students, the most effective way to use the book is:

- read the definitions carefully before looking at code,
- use the figures to build intuition about what the formulas mean,
- run and modify the `R` examples,
- and treat the exercises as part of the learning process, not as material left for the end.

> **Classroom Use**
>
> This welcome page is the front door to a course text, not just project documentation. The book is intended to support lectures, homework, discussion, and computational exploration in parallel.

1.6 What Students Should Notice Across Chapters

Although the topics differ, several ideas return throughout the book:

- **Local versus global structure**: a small local rule can produce a large global effect.
- **Typical behavior versus exceptional nodes**: averages matter, but tails and hubs matter too.
- **Randomness versus mechanism**: some patterns arise from chance, others from growth rules or constraints.
- **Interpretation through computation**: simulation helps students see why the mathematics is meaningful.

Seeing these recurring themes is one of the most important learning outcomes of the course.

1.7 Software Setup

Install the packages below once:

```
if (!requireNamespace("renv", quietly = TRUE)) install.packages("renv")
renv::restore()
tinytex::install_tinytex()
```

The same source files produce both:

- an HTML course website,
- and a PDF course handout.

Chapter 2

Preface and Reader's Guide

How to use this book in a university course

2.1 Why this book exists

This book was written as a course text for a university class in **Network Science**. Its role is broader than that of a reference manual. It is meant to support:

- lectures,
- guided reading,
- homework and problem solving,
- computational experimentation in `R`,
- and classroom discussion about how local structure leads to global network behavior.

The subject sits at the intersection of graph theory, probability, statistical physics, computation, and data analysis. Because of that, students often meet ideas here that are individually simple but conceptually powerful when combined. The aim of the book is to help those ideas fit together.

2.2 What this course is about

At the center of the course is one recurring question:

> How do simple structural rules produce complex large-scale network behavior?

That question appears in different forms throughout the book. In one chapter it appears as connectedness and component growth. In another it appears as clustering and short paths. Later it appears as hubs, heterogeneity, and resilience under failure. The chapters are different, but the conceptual thread is continuous.

! Reader's Perspective

This book should be read as a connected argument, not as a disconnected list of topics. Each chapter prepares ideas that become important later.

2.3 Expected background

The book assumes comfort with:

- basic algebra and functions,
- elementary probability,
- reading mathematical notation carefully,
- and writing or modifying short `R` scripts.

Students do **not** need an advanced background in graph theory before starting. The early chapters build the core vocabulary from first principles. What matters most is willingness to move between formulas, pictures, and computational examples.

2.4 How the chapters are built

Most chapters follow the same learning pattern:

1. **Theory and intuition**
   Definitions, explanations, and the main mathematical claims.
2. **Worked examples in `R`**
   Simulations and visualizations that turn abstract structure into something visible.
3. **Exercises and coding tasks**
   Problems designed to move from reproduction to analysis to interpretation.

This structure is deliberate. Network science is difficult to learn well if it is treated only as theory or only as coding. The strongest understanding comes from moving back and forth between the two.

2.5 How to study from this text

The most effective reading strategy is:

1. Read definitions slowly and precisely.
2. Before looking at the code, try to predict what a figure or simulation should show.
3. Run the `R` examples and change parameters to see how the model behaves.
4. Use the exercises to test whether you understand both the formula and its interpretation.

When a chapter introduces a model, ask yourself three questions:

- What is the structural object being studied?
- What is random, fixed, or generated by a rule?
- Which observed behavior is local, and which behavior is global?

Those questions will help you see the deeper connections between chapters.

2.6 Mathematical and computational reading

Some students prefer the proofs and formulas; others learn first through plots and code. This book is designed for both reading styles, but each style is incomplete on its own.

- Mathematics gives precision.
- Simulation gives intuition.
- Visualisation gives pattern recognition.
- Exercises give durable understanding.

For that reason, you should resist the temptation to skip whichever layer feels less familiar. A network model is best understood only when you can:

- define it,
- simulate it,
- interpret its outputs,

- and explain why its behavior matters.

2.7 Conventions used in the book

Throughout the text:

- a graph is usually denoted by $G = (V, E)$,
- V is the vertex set and E is the edge set,
- N often denotes the number of vertices,
- degree is written as k,
- average degree is written as $\langle k \rangle$,
- and probabilities, distributions, clustering, and path lengths are interpreted both analytically and numerically.

When the same quantity appears in several chapters, pay attention to how its role changes. For example, degree begins as a basic graph-theoretic notion, becomes a random variable in random graphs, becomes a source of heterogeneity in scale-free networks, and becomes a determinant of vulnerability in robustness analysis.

2.8 Using the book in class

This book is intended to support a live course environment. In practice, that means:

- lectures should provide conceptual framing,
- the text should provide mathematical structure,
- code examples should provide experimentation,
- and exercises should provide consolidation.

The HTML version is especially useful for browsing, navigation, and code visibility. The PDF version is useful for linear reading, annotation, and offline study.

> **A good habit**
>
> Before each class, skim the chapter headings and learning goals. After class, return to the examples and exercises while the ideas are still fresh.

2.9 Final advice to students

Do not judge your understanding only by whether a formula looks familiar. In this subject, real understanding means being able to explain why a model behaves as it does and what its assumptions imply.

If you can move comfortably between:

- the abstract definition,
- the visual pattern,
- the numerical experiment,
- and the interpretation of the result,

then you are learning network science in the right way.

Chapter 3

Graph Theory

The mathematical language of network structure

3.1 Introduction

Network science rests on a mathematical foundation: **graph theory**. Before studying random graphs, small-world structure, or scale-free degree heterogeneity, it is essential to establish the formal language used to describe networks precisely.

Graph theory is a branch of discrete mathematics concerned with collections of objects and the pairwise relations between them. Its origins trace to Leonhard Euler's 1736 solution to the *Königsberg bridge problem*, in which the question of whether a city could be traversed by crossing each of its seven bridges exactly once was reformulated as a purely combinatorial question about a drawing of vertices and edges. Euler's insight — that only the pattern of connections, not the physical geography, determines the answer — is the founding idea of graph theory (Euler, 1736).

In the context of this course, graph theory provides:

- a precise vocabulary for describing network structure,
- algebraic representations that connect combinatorics to linear algebra,
- combinatorial results that govern global properties such as connectivity and reachability,
- and the rigorous foundation on which the probabilistic models of later chapters are built (Barabási & Pósfai, 2016; Diestel, 2017; Newman, 2010; West, 2001).

Standard graduate references for the material in this chapter are Diestel (2017) for pure graph theory and West (2001) for a combinatorial treatment. For the network science perspective that motivates the choice of topics, see Newman (2010) and Barabási & Pósfai (2016). Spectral properties of graphs, covered in Section 3.7, are treated in depth by Chung (1997).

3.2 Learning goals

By the end of this chapter, a student should be able to:

- represent a network as $G = (V, E)$ and report its order, size, and degree structure,
- translate between edge lists, adjacency matrices, and `igraph` objects,
- state and apply the handshaking lemma and its immediate consequences,
- explain what matrix powers count, and how the Laplacian spectrum encodes connectivity,

- distinguish paths, components, cut vertices, and bridges — the vocabulary used later for robustness and percolation,
- recognise when a graph is bipartite and apply König's criterion,
- compute the summary quantities in Table 3.1 and interpret them structurally.

3.3 Chapter roadmap

The chapter proceeds from basic objects to structural concepts. It begins with the formal definition of a graph and the main graph types, then studies degree and matrix representations, and turns to walks, connectivity, trees, and bipartite structure. It ends with a compact computational toolkit in `R`. Each theoretical definition is paired with a concrete representation that can be explored and verified computationally.

3.4 What is a graph?

3.4.1 Formal definition

Definition 3.1. A **graph** G is an ordered pair

$$G = (V, E)$$

where V is a finite non-empty set called the **vertex set** (or **node set**), and E is a set of unordered pairs $\{u, v\}$ with $u, v \in V$ and $u \neq v$, called the **edge set**.

The number of vertices is denoted $N = |V|$ and called the **order** of the graph. The number of edges is denoted $M = |E|$ and called the **size** of the graph.

Two vertices u and v are **adjacent** (or **neighbors**) if $\{u, v\} \in E$. An edge $\{u, v\}$ is **incident** to each of u and v. The **neighborhood** of v is the set $N(v) = \{u \in V : \{u, v\} \in E\}$.

> **Convention**
>
> Throughout this chapter, all graphs are assumed to be **simple** unless stated otherwise. A simple graph contains no self-loops (edges from a vertex to itself) and no multi-edges (multiple edges between the same pair of vertices).

3.4.2 A motivating example

Consider the graph with vertex set $V = \{1, 2, 3, 4, 5\}$ and edge set

$$E = \{\{1, 2\},\ \{1, 3\},\ \{2, 4\},\ \{3, 4\},\ \{4, 5\}\}.$$

This graph has order $N = 5$ and size $M = 5$. Vertex 4 is the only neighbor of vertex 5 and the only common neighbor of vertices 2 and 3 — structural features that will become precise once degree and connectivity are defined.

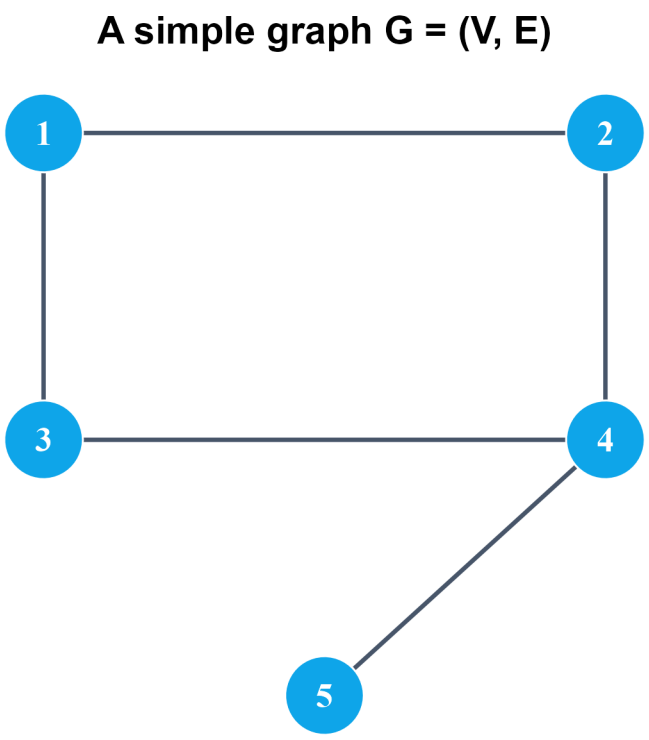


Figure 3.1: A simple graph on five vertices. Vertex labels indicate node identity.

3.5 Directed and undirected graphs

The definition above treats each edge as unordered: the relation $\{u, v\}$ is symmetric. Such a graph is **undirected**. In many real networks, relations are asymmetric — a hyperlink from page u to page v does not imply a link in the reverse direction; a neuron may send signals to a target without receiving signals back.

Definition 3.2. A **directed graph** (or **digraph**) is a pair $G = (V, A)$ where V is the vertex set and A is a set of **ordered** pairs (u, v) with $u \neq v$, called **arcs**. The arc (u, v) has **tail** u and **head** v; it is distinct from (v, u).

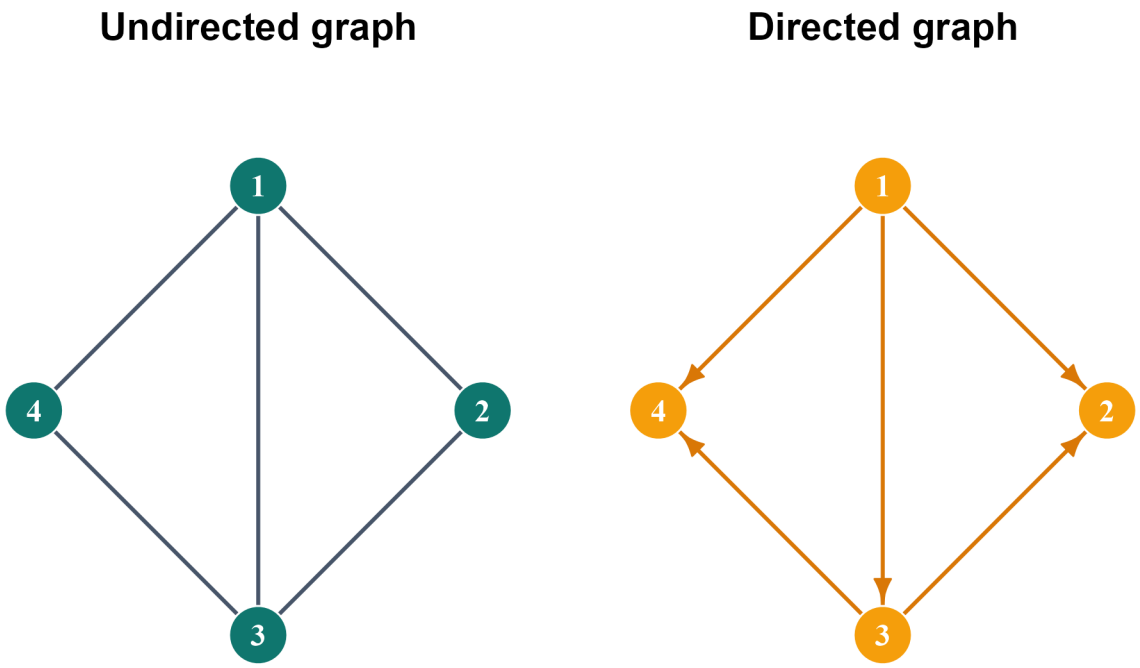


Figure 3.2: Left: an undirected graph. Right: a directed graph on the same vertex set. Arcs carry orientation.

3.5.1 Weighted graphs

Definition 3.3. A **weighted graph** is a graph $G = (V, E)$ together with a weight function $w : E \to \mathbb{R}$, assigning a real-valued weight to each edge. Weights may represent distances, capacities, interaction strengths, or costs, depending on the application.

3.6 Degree and the handshaking lemma

3.6.1 Degree of a vertex

Definition 3.4. Let $G = (V, E)$ be an undirected graph. The **degree** of vertex $v \in V$, denoted k_v or $\deg(v)$, is the number of edges incident to v:

$$k_v = |\{u \in V : \{u, v\} \in E\}| = |N(v)|.$$

A vertex with $k_v = 0$ is **isolated**; one with $k_v = 1$ is a **leaf** or **pendant vertex**. The maximum and minimum degrees of G are denoted $\Delta(G)$ and $\delta(G)$ respectively.

3.6.2 Degree sequence

Definition 3.5. The **degree sequence** of G is the list of vertex degrees arranged in non-increasing order:

$$k_1 \geq k_2 \geq \cdots \geq k_N.$$

The degree sequence is a fundamental invariant of the graph. Not every integer sequence is graphical — the Erdős–Gallai theorem gives a necessary and sufficient condition for a sequence of non-negative integers to be realizable as a degree sequence of a simple graph.

3.6.3 The handshaking lemma

Theorem 3.1. ***Handshaking Lemma.*** *For any undirected graph $G = (V, E)$,*

$$\sum_{v \in V} k_v = 2M.$$

Proof. Consider the incidence sum $\sum_{v \in V} \sum_{e \in E} \mathbf{1}[v \in e]$. Evaluated by vertex, it equals $\sum_v k_v$. Evaluated by edge, each edge $\{u, v\}$ contributes exactly 1 to $v = u$ and exactly 1 to v, so the sum equals $2M$. □

> **i** Immediate corollary
>
> Every graph has an even number of odd-degree vertices. Since $2M$ is even, the sum $\sum_v k_v$ is even; an even sum requires an even count of odd summands.

3.6.4 Average degree

Definition 3.6. The **average degree** of G is

$$\langle k \rangle = \frac{1}{N} \sum_{v \in V} k_v = \frac{2M}{N}.$$

The identity $\langle k \rangle = 2M/N$ follows immediately from the handshaking lemma. It connects average degree, edge count, and graph order in a single formula that recurs throughout this book.

3.6.5 In-degree and out-degree in directed graphs

In a directed graph, each arc contributes asymmetrically to two vertices.

Definition 3.7. Let $G = (V, A)$ be a directed graph and $v \in V$. The **in-degree** of v is

$$k_v^{\text{in}} = |\{u \in V : (u, v) \in A\}|$$

and the **out-degree** of v is

$$k_v^{\text{out}} = |\{u \in V : (v, u) \in A\}|.$$

The directed analogue of the handshaking lemma states:

$$\sum_{v \in V} k_v^{\text{in}} = \sum_{v \in V} k_v^{\text{out}} = |A|.$$

Every arc (u, v) contributes 1 to the out-degree of u and 1 to the in-degree of v.

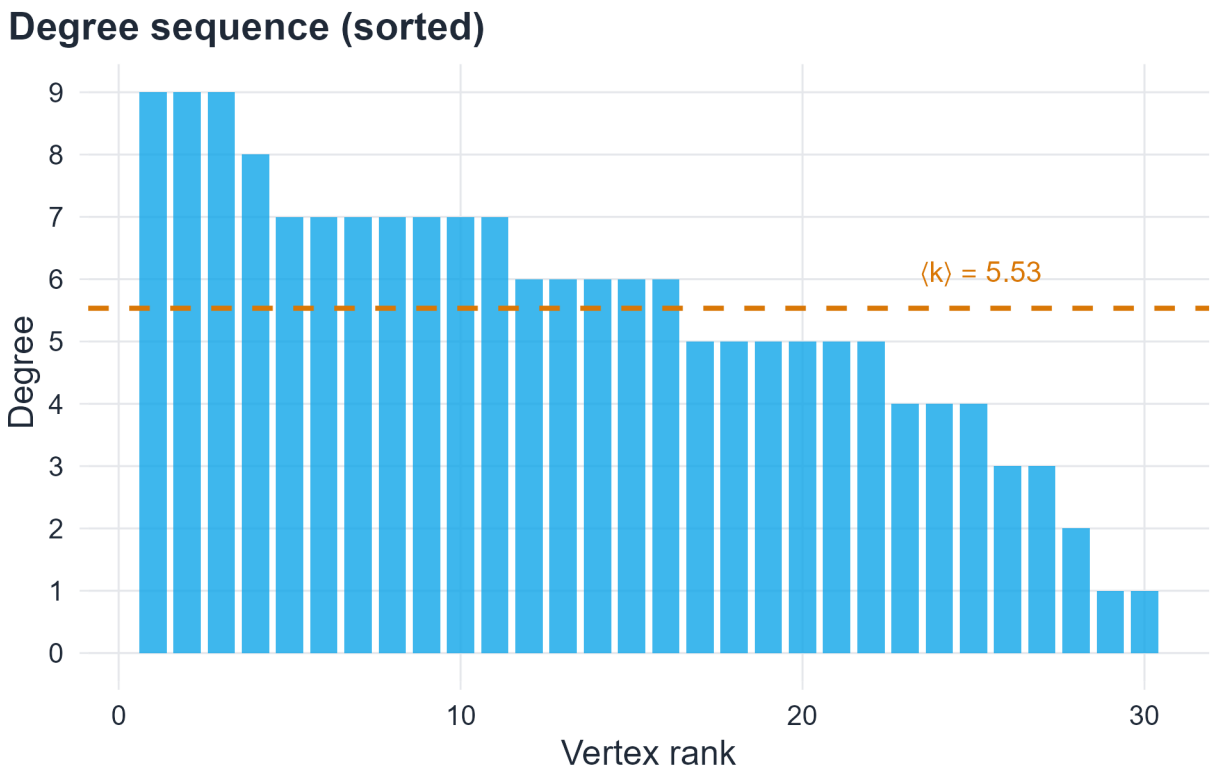


Figure 3.3: Degree sequence (sorted in decreasing order) of a graph on 30 vertices. The dashed line marks the average degree.

3.7 Matrix representations

Matrix representations are central to network science because they translate combinatorial graph structure into linear-algebraic objects, making it possible to study connectivity, diffusion, centrality, and spectral properties computationally (Chung, 1997; Newman, 2010). The adjacency matrix and the Laplacian are the two most important representations; a comprehensive treatment of their spectral theory is given in Chung (1997).

3.7.1 The adjacency matrix

Definition 3.8. Let $G = (V, E)$ be a graph with $V = \{1, 2, \dots, N\}$. The **adjacency matrix** of G is the $N \times N$ matrix A defined entry-wise by

$$A_{ij} = \begin{cases} 1 & \text{if } \{i, j\} \in E, \\ 0 & \text{otherwise.} \end{cases}$$

For an undirected graph, A is **symmetric**: $A_{ij} = A_{ji}$ for all i, j. For a simple graph, $A_{ii} = 0$ for all i.

3.7.1.1 Key properties

Several properties follow immediately from the definition:

- The row sum of row i equals the degree of vertex i: $\sum_j A_{ij} = k_i$.
- The total sum of all entries equals $2M$: $\sum_{i,j} A_{ij} = 2M$.

- The entry $(A^r)_{ij}$ of the r-th matrix power counts the number of **walks** of length r from vertex i to vertex j.

The third property follows from the definition of matrix multiplication: $(A^2)_{ij} = \sum_k A_{ik}A_{kj}$ counts the number of common neighbors of i and j, which is exactly the number of walks of length 2 connecting them. The argument extends by induction to any $r \geq 1$.

3.7.2 The degree matrix

Definition 3.9. The **degree matrix** D is the $N \times N$ diagonal matrix with $D_{ii} = k_i$ and $D_{ij} = 0$ for $i \neq j$.

3.7.3 The Laplacian matrix

Definition 3.10. The **Laplacian matrix** of G is

$$L = D - A.$$

The Laplacian is symmetric and positive semi-definite. Its entries are:

$$L_{ij} = \begin{cases} k_i & \text{if } i = j, \\ -1 & \text{if } \{i, j\} \in E, \\ 0 & \text{otherwise.} \end{cases}$$

Two structural facts about L are fundamental:

1. The smallest eigenvalue of L is always 0, with eigenvector $\mathbf{1}$ (the all-ones vector), because $L\mathbf{1} = D\mathbf{1} - A\mathbf{1} = \mathbf{k} - \mathbf{k} = \mathbf{0}$. The multiplicity of the zero eigenvalue equals the number of connected components.
2. The **Fiedler value** $\lambda_2(L)$ — the second-smallest eigenvalue, introduced by Fiedler (1973) — measures how well-connected the graph is. A larger λ_2 implies greater robustness: more edge-disjoint paths exist between typical pairs of vertices (Chung, 1997).

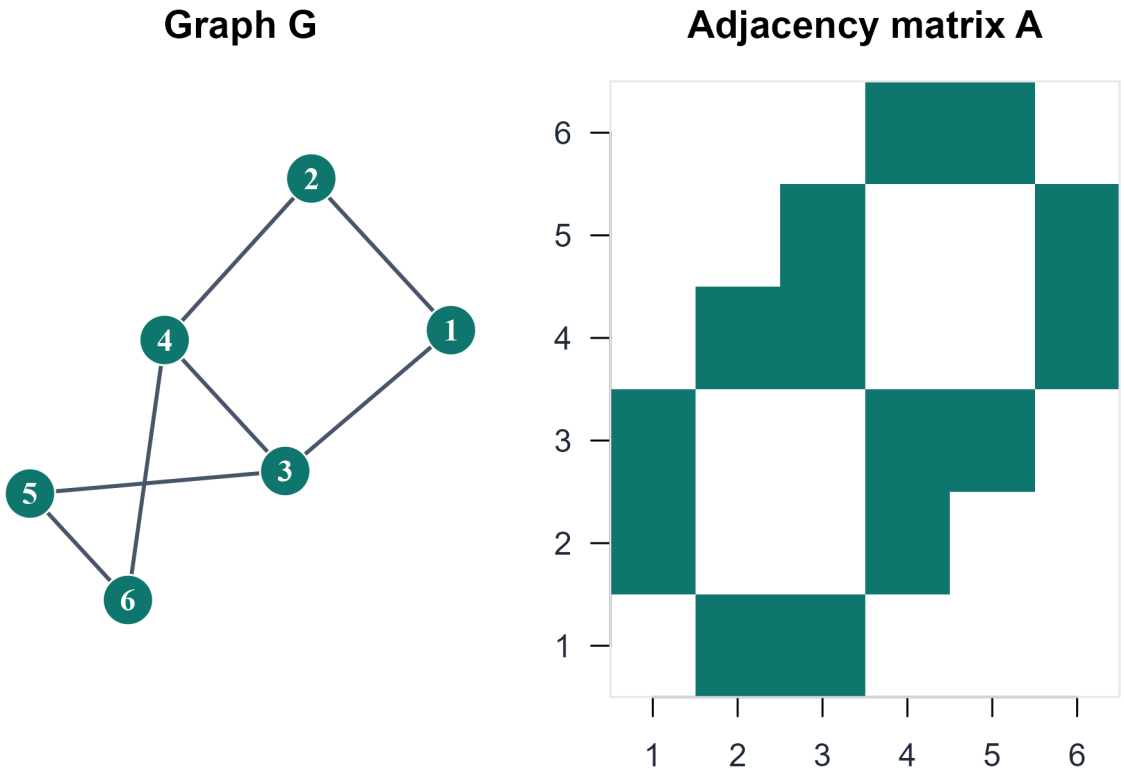


Figure 3.4: Left: a small graph on six vertices. Right: its adjacency matrix as a heatmap. Coloured cells indicate edges.

3.8 Walks, paths, and cycles

3.8.1 Walk, trail, and path

Definition 3.11. A **walk** of length ℓ in G is a finite sequence of vertices

$$v_0, v_1, \dots, v_\ell$$

such that $\{v_{i-1}, v_i\} \in E$ for every $i = 1, \dots, \ell$. A walk may repeat vertices and edges.

Definition 3.12. A **trail** is a walk in which no edge is repeated. A **path** is a walk in which no vertex is repeated (which implies no edge is repeated).

The **distance** $d(u, v)$ between two vertices is the length of a shortest path connecting them (or ∞ if no path exists). The **diameter** $D(G) = \max_{u,v} d(u, v)$ is the maximum distance over all pairs of vertices.

3.8.2 Cycle

Definition 3.13. A **cycle** is a closed walk $v_0, v_1, \dots, v_\ell = v_0$ of length $\ell \geq 3$ in which all vertices $v_0, \dots, v_{\ell-1}$ are distinct. The **girth** of G is the length of its shortest cycle. A graph with no cycles is **acyclic**.

Matrix powers and triangle counting

Because $(A^r)_{ij}$ counts walks of length r from i to j, the diagonal entry $(A^3)_{ii}$ counts closed walks of length 3 through i — that is, the number of triangles containing i. Summing over all vertices and dividing by 6 (two traversal directions $\times$ three starting vertices per triangle) gives:

$$\text{triangles}(G) = \frac{1}{6}\,\text{tr}(A^3).$$

This identity connects combinatorial structure to a straightforward matrix computation.

3.9 Connectivity

3.9.1 Connected components

Definition 3.14. An undirected graph G is **connected** if for every pair of distinct vertices $u, v \in V$, there exists a path from u to v. A **connected component** is a maximal connected subgraph of G.

The number of connected components is denoted $c(G)$; $c(G) = 1$ if and only if G is connected. The multiplicity of the zero eigenvalue of L equals $c(G)$ — providing an algebraic route to detecting components.

3.9.2 Strong and weak connectivity in directed graphs

Definition 3.15. A directed graph is **strongly connected** if for every ordered pair (u, v) there is a directed path from u to v. It is **weakly connected** if the underlying undirected graph (obtained by ignoring arc directions) is connected.

Strong connectivity requires reachability in both directions between every pair of vertices. A directed graph can be weakly connected without being strongly connected.

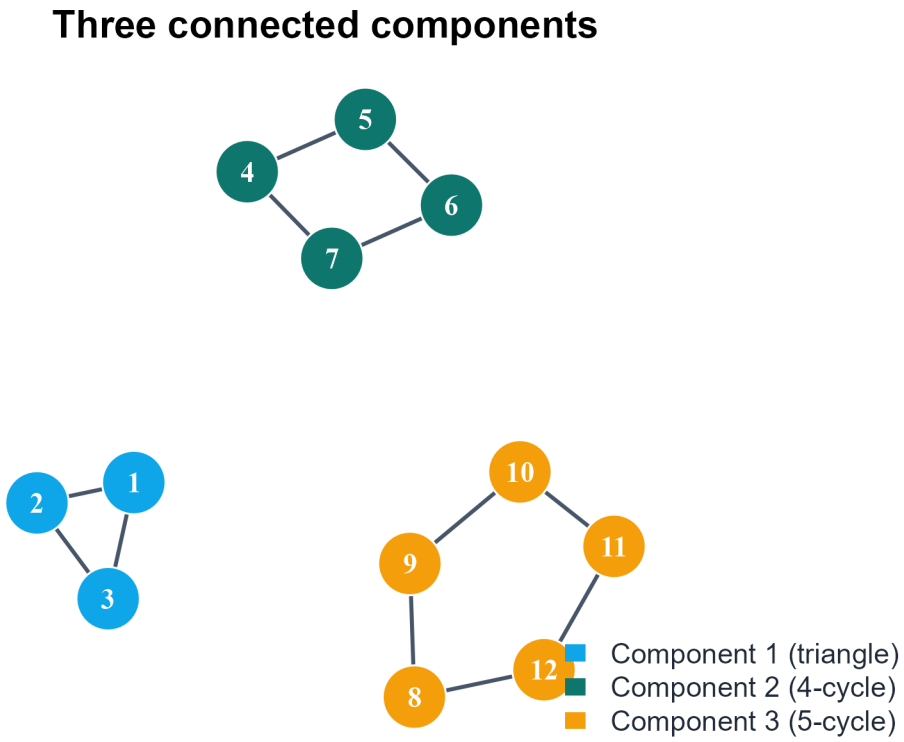


Figure 3.5: A disconnected graph with three connected components shown in distinct colours. Component membership is determined by mutual reachability.

3.9.3 Cut vertices and bridges

Two structural elements are critical for understanding how connectivity can fail.

Definition 3.16. A **cut vertex** (or **articulation point**) is a vertex v whose removal strictly increases the number of connected components: $c(G - v) > c(G)$.

Definition 3.17. A **bridge** is an edge e whose removal strictly increases the number of connected components: $c(G - e) > c(G)$.

Cut vertices and bridges are the most fragile elements of a connected graph. Their removal disconnects the network — a property directly relevant to network robustness, examined in detail in Chapter 7. Note that every bridge $\{u, v\}$ is contained in no cycle: if it were, its removal would leave u and v connected via the cycle, contradicting the definition.

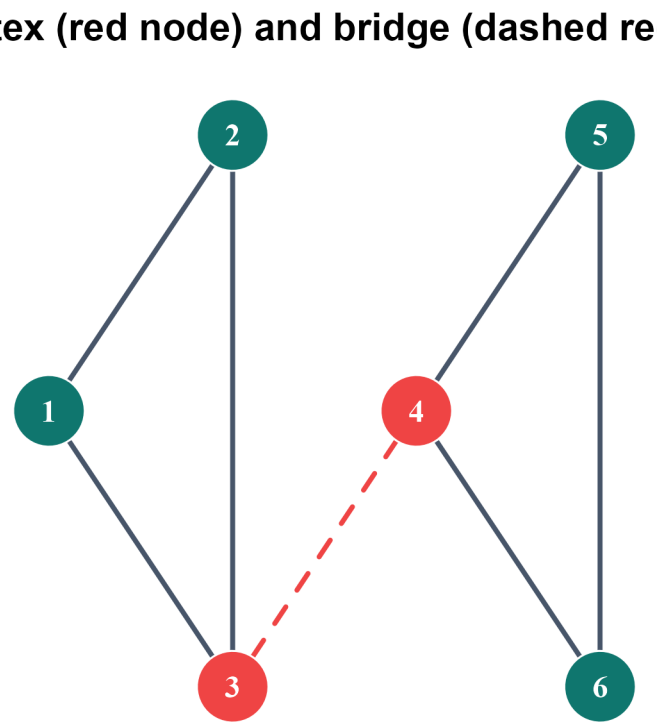


Figure 3.6: A graph with a cut vertex (red) and a bridge (dashed red edge). Removing either one disconnects the graph.

3.10 Trees and spanning trees

3.10.1 Definition and characterisation

Definition 3.18. A **tree** is a connected acyclic graph. A **forest** is an acyclic graph (whose connected components are trees).

Theorem 3.2. ***Theorem.*** *Let G be a graph with N vertices. The following are equivalent:*

1. *G is a tree.*
2. *G is connected and has exactly $N-1$ edges.*
3. *G is acyclic and has exactly $N-1$ edges.*
4. *Any two distinct vertices are connected by exactly one path.*
5. *G is connected, and removing any single edge disconnects it (equivalently, every edge is a bridge).*

The equivalence of these statements is classical; proofs appear in Diestel (2017, Ch. 1) and West (2001, Ch. 2). In particular, $M = N - 1$ is both necessary and sufficient for a connected graph to be a tree. This identity will resurface in Chapter 4 when characterising the giant component of a random graph near the phase transition.

3.10.2 Spanning trees

Definition 3.19. A **spanning tree** of a connected graph $G = (V, E)$ is a subgraph $T = (V, E')$, with $E' \subseteq E$, that is itself a tree — it contains all N vertices with exactly $N - 1$ edges, is connected, and is acyclic.

Every connected graph possesses at least one spanning tree. When edge weights are present, the **minimum spanning tree** minimises total edge weight; the Kruskal and Prim algorithms find it in $O(M \log N)$ time.

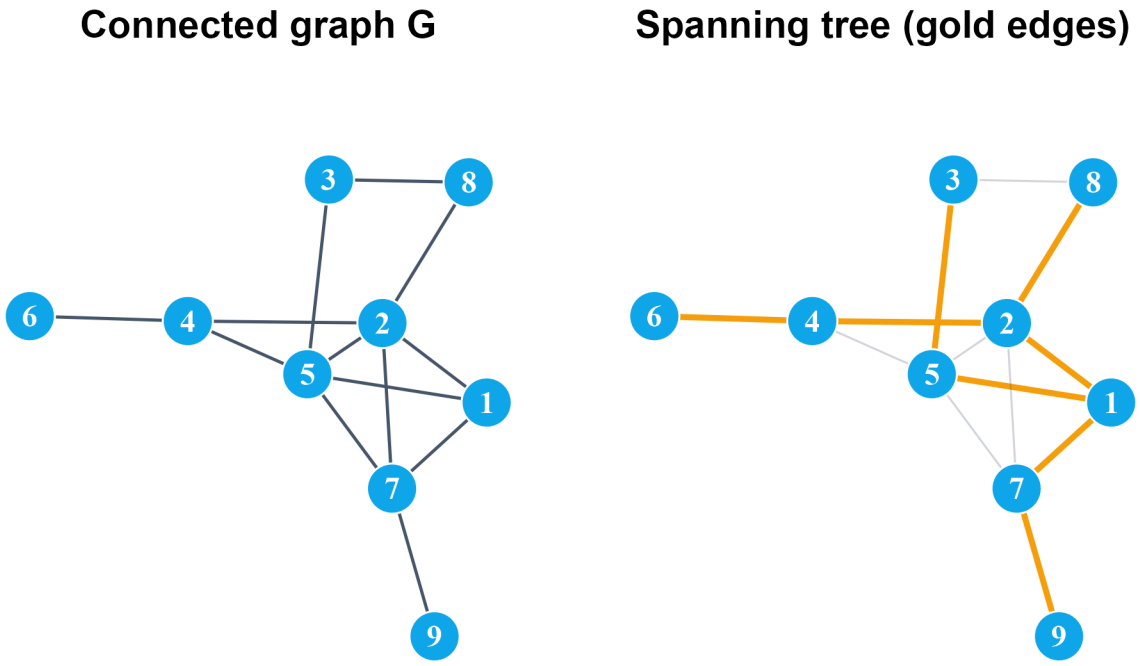


Figure 3.7: Left: a connected graph on nine vertices. Right: one spanning tree — gold edges are in the tree, grey edges are excluded.

3.11 Bipartite graphs

3.11.1 Definition and König's criterion

Definition 3.20. A graph $G = (V, E)$ is **bipartite** if V can be partitioned into two non-empty disjoint sets X and Y such that every edge has one endpoint in X and one in Y. The pair (X, Y) is the **bipartition**.

Bipartite graphs admit no edges within X or within Y. They arise naturally in two-mode network models: authors and papers, genes and diseases, employees and projects.

Theorem 3.3. ***Theorem (Kőnig, 1916).*** *A graph G is bipartite if and only if it contains no cycle of odd length.*

This criterion is both elegant and computationally efficient: bipartiteness is detected by a two-coloring of G, executable in $O(N+M)$ time by breadth-first search (West, 2001). When BFS assigns the same color to both endpoints of any edge, an odd cycle has been found and the graph is not bipartite. Bipartite graphs arise naturally in two-mode network models — authors and papers, genes and diseases, employees and projects — and require adapted statistical tools; see Newman (2010, Ch. 6) for a discussion of projection methods.

3.11.2 Complete bipartite graph

Definition 3.21. The **complete bipartite graph** $K_{m,n}$ is the bipartite graph with $|X| = m$ and $|Y| = n$ in which every vertex of X is adjacent to every vertex of Y. It has mn edges, n vertices of degree m, and m

vertices of degree n.

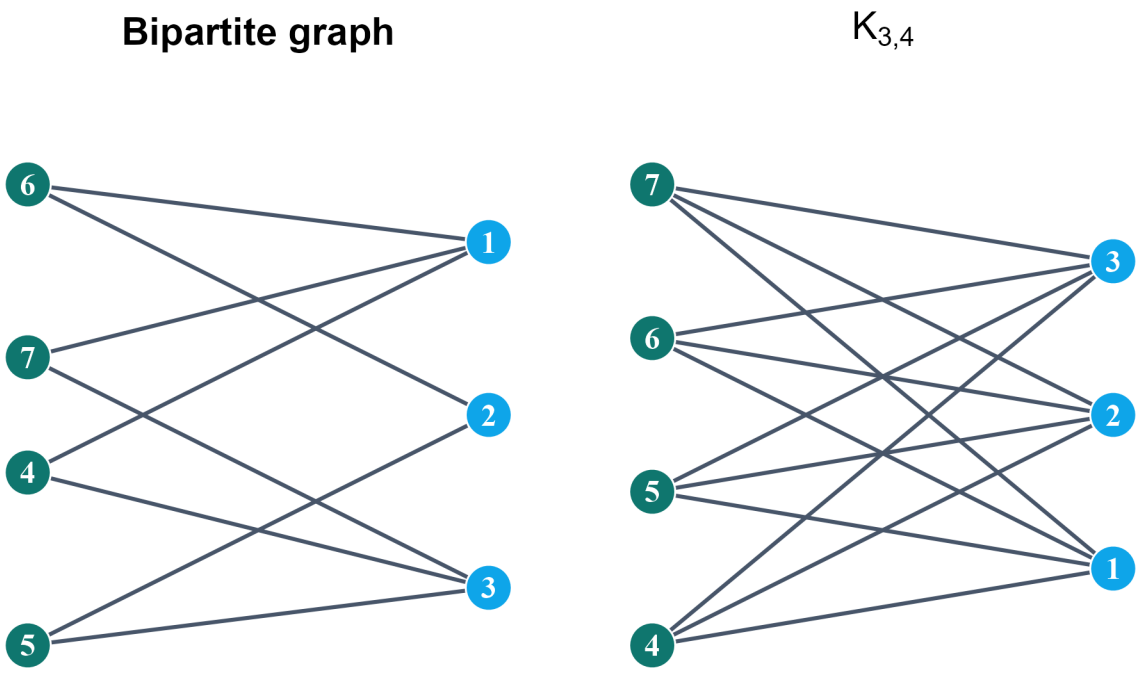


Figure 3.8: Left: a bipartite graph with bipartition shown in blue and teal. Right: the complete bipartite graph K(3,4).

3.12 Special graph families

Several families appear repeatedly in network science as benchmarks and building blocks.

Definition 3.22. The **complete graph** K_N is the simple graph on N vertices in which every pair of distinct vertices is adjacent. It has $M = \binom{N}{2}$ edges and every vertex has degree $N - 1$.

Definition 3.23. The **cycle graph** C_N is a single cycle through all N vertices: every vertex has degree 2 and the graph has N edges.

Definition 3.24. The **path graph** P_N has N vertices forming a single path: the two endpoints have degree 1, all interior vertices have degree 2, and the graph has $N - 1$ edges.

Definition 3.25. A graph is **k-regular** if every vertex has degree exactly k. The complete graph K_N is $(N-1)$-regular; C_N is 2-regular.

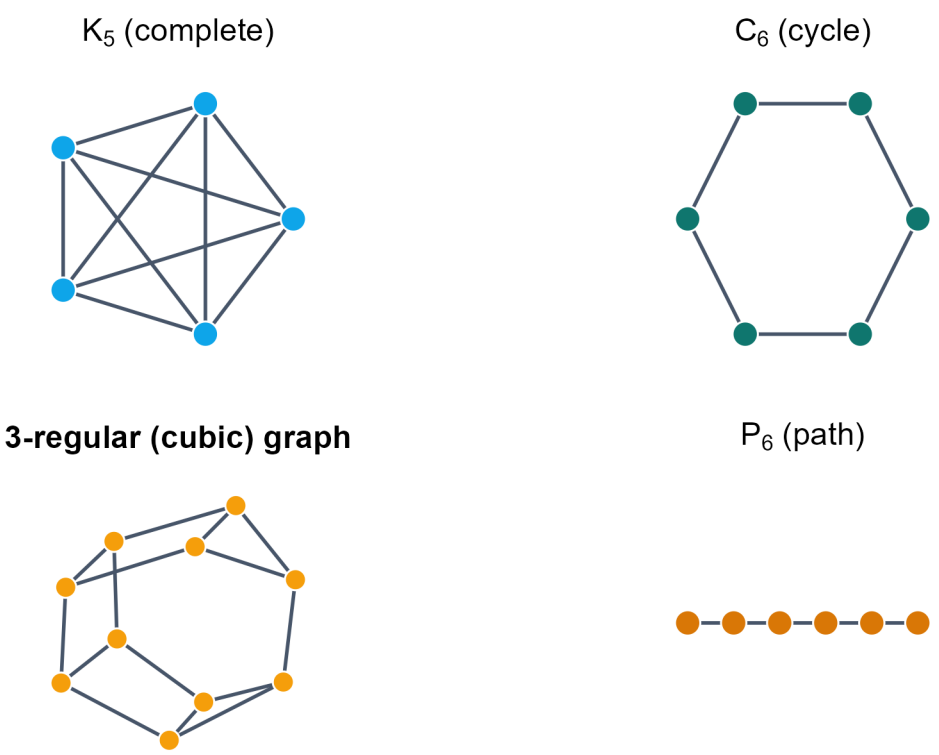


Figure 3.9: Four standard graph families: K5 (complete), C6 (cycle), a 3-regular graph on 10 vertices, and P6 (path).

3.13 Subgraphs and graph isomorphism

Definition 3.26. A graph $H = (V', E')$ is a **subgraph** of $G = (V, E)$ if $V' \subseteq V$ and $E' \subseteq E$. It is an **induced subgraph** on $V' \subseteq V$, written $G[V']$, if E' contains every edge of G with both endpoints in V'.

Definition 3.27. Two graphs $G = (V_1, E_1)$ and $H = (V_2, E_2)$ are **isomorphic**, written $G \cong H$, if there exists a bijection $\phi : V_1 \to V_2$ preserving adjacency:

$$\{u, v\} \in E_1 \iff \{\phi(u), \phi(v)\} \in E_2.$$

The function ϕ is a **graph isomorphism**.

Isomorphic graphs are structurally identical: one is a relabeling of the other. A necessary condition for isomorphism is that the two graphs share the same degree sequence — but this is not sufficient. Constructing non-isomorphic graphs with the same degree sequence is a useful exercise in understanding what degree alone can and cannot determine.

3.14 Working with graphs in R

The `igraph` package provides a comprehensive toolkit for constructing, manipulating, and analysing graphs in R.

3.14.1 Constructing graphs

```
library(igraph)

# From an edge list
g1 <- graph_from_edgelist(
  matrix(c(1,2, 1,3, 2,3, 3,4, 4,5), ncol = 2, byrow = TRUE),
  directed = FALSE
)

# From an adjacency matrix
A_mat <- matrix(c(0,1,1,0,
                  1,0,1,1,
                  1,1,0,0,
                  0,1,0,0), nrow = 4)
g2 <- graph_from_adjacency_matrix(A_mat, mode = "undirected")

# Special constructors
g_cycle  <- make_ring(8)
g_full   <- make_full_graph(6)
g_random <- sample_gnp(n = 20, p = 0.20, directed = FALSE)
```

3.14.2 Basic graph attributes

```
cat("Order (vertices):     ", vcount(g1), "\n")
```

```
Order (vertices):      5
```

```
cat("Size (edges):         ", ecount(g1), "\n")
```

```
Size (edges):          5
```

```
cat("Degree sequence:      ",
    paste(sort(degree(g1), decreasing = TRUE), collapse = " "), "\n")
```

```
Degree sequence:       3 2 2 2 1
```

```
cat("Average degree:       ", round(mean(degree(g1)), 3), "\n")
```

```
Average degree:        2
```

```
cat("Is connected?:        ", is_connected(g1), "\n")
```

```
Is connected?:         TRUE
```

```
cat("Number of components: ", components(g1)$no, "\n")
```

```
Number of components:  1
```

```
cat("Diameter:             ", diameter(g1), "\n")
```

```
Diameter:              3
```

```
cat("Girth:                ", girth(g1)$girth, "\n")
```

```
Girth:                 3
```

3.14.3 Adjacency and Laplacian matrices

```
A_out  <- as.matrix(as_adjacency_matrix(g1))
D_out  <- diag(rowSums(A_out))
L_out  <- D_out - A_out
eigs   <- round(sort(eigen(L_out)$values), 6)

cat("Adjacency matrix:\n")
```

```
Adjacency matrix:
```

```
print(A_out)
```

```
     [,1] [,2] [,3] [,4] [,5]
[1,]    0    1    1    0    0
[2,]    1    0    1    0    0
[3,]    1    1    0    1    0
[4,]    0    0    1    0    1
[5,]    0    0    0    1    0
```

```
cat("\nLaplacian eigenvalues:\n")
```

```

Laplacian eigenvalues:
```

```
print(eigs)
```

```
[1] 0.000000 0.518806 2.311108 3.000000 4.170086
```

```
cat("\nFiedler value (algebraic connectivity):", eigs[2], "\n")
```

```

Fiedler value (algebraic connectivity): 0.518806
```

Since `g1` is connected, L has exactly one zero eigenvalue. The Fiedler value $\lambda_2 > 0$ confirms connectivity; its magnitude quantifies how robustly the graph is connected.

3.14.4 Triangle counting via matrix trace

```
A3    <- A_out %*% A_out %*% A_out
n_tri <- sum(diag(A3)) / 6
cat("Number of triangles (via tr(A^3)/6):", n_tri, "\n")
```

```
Number of triangles (via tr(A^3)/6): 1
```

```
cat("Number of triangles (igraph)        :", sum(count_triangles(g1)) / 3, "\n")
```

```
Number of triangles (igraph)        : 1
```

3.15 Summary of key definitions

The table below collects the most important quantities introduced in this chapter.

3.16 Exercises

The exercises below are designed for MSc-level students. They do not ask you to reproduce derivations from the text. Instead, they ask you to **apply, extend, and connect** the ideas introduced in this chapter.

Table 3.1: Core graph-theoretic quantities, their notation, and formulas.

Symbol	Quantity	Formula
$N = \|V\|$	Order	$\|V\|$
$M = \|E\|$	Size	$\|E\|$
k_v	Degree of v	$\|\{u : uv \in E\}\|$
$\langle k \rangle$	Average degree	$2M/N$
A	Adjacency matrix	$A_{ij} = \mathbf{1}[ij \in E]$
L	Laplacian matrix	$D - A$
$d(u, v)$	Distance	Length of shortest u-v path
$D(G)$	Diameter	$\max_{u,v} d(u, v)$
$c(G)$	Component count	Maximal connected subgraph count
$\lambda_2(L)$	Fiedler value	2nd smallest eigenvalue of L

Several problems have no unique answer — the goal is precise reasoning, not a single correct number.

! Orientation

These problems train you to work with the quantities in Table 3.1 as diagnostic tools. The same tools reappear throughout the course — in random graph theory, small-world models, and network robustness. The habit of computing, interpreting, and questioning each quantity is what this chapter is building.

3.16.0.1 Exercise 1. Degree sequences: existence and uniqueness

1. Give an example of two non-isomorphic graphs on six vertices that share the same degree sequence. Prove they are not isomorphic.
2. State (without proof) the Erdős–Gallai condition (Erdős & Gallai, 1960) for a sequence of non-negative integers to be graphical. Apply it to determine which of the following sequences are degree sequences of a simple graph:
   (a) $(4, 3, 3, 2, 2)$,
   (b) $(5, 3, 3, 2, 1)$,
   (c) $(4, 4, 3, 3, 2, 2)$.
3. Prove that there is no simple graph whose degree sequence is $(5, 4, 3, 2, 1)$ on five vertices. What goes wrong?

3.16.0.2 Exercise 2. The Laplacian as a diagnostic tool

Let G be the graph on $\{1, 2, 3, 4\}$ with edge set $\{\{1, 2\}, \{1, 3\}, \{2, 3\}, \{3, 4\}\}$.

1. Write A and $L = D - A$ explicitly.
2. Compute the eigenvalues of L (numerically is fine). Verify that the smallest is 0 and that its multiplicity matches $c(G)$.
3. Now delete the edge $\{3, 4\}$ and recompute the Laplacian spectrum. What changes, and why? Interpret the Fiedler value before and after deletion.
4. A student claims: *"The Fiedler value tells us how many edges we need to remove to disconnect the graph."* Is this correct? Give a precise statement of what λ_2 does and does not tell us.

3.16.0.3 Exercise 3. Matrix powers and walk counting

Let A be the adjacency matrix of the motivating graph from Section 3.4.2.

1. Compute A^2 and A^3 (by hand or in R).
2. Use $(A^2)_{ij}$ to identify all pairs of vertices at distance exactly 2. Verify against the graph directly.
3. Use $\text{tr}(A^3)/6$ to count triangles. Explain why the factor 6 appears.
4. The entry $(A^r)_{ij}$ counts walks, not paths. Construct a specific example (using this or any other graph) where $(A^2)_{ij} > 0$ but the number of paths of length 2 from i to j is strictly less than $(A^2)_{ij}$. Explain the discrepancy.

3.16.0.4 Exercise 4. Connectivity and fragility

Consider the graph formed by two triangles, $\{1, 2, 3\}$ and $\{4, 5, 6\}$, joined by the single edge $\{3, 4\}$.

1. Identify all cut vertices and bridges. Justify each answer using the definitions in Definition 3.16 and Definition 3.17.
2. Prove that every bridge is contained in no cycle. Use this to explain why P_N has $N - 2$ cut vertices and $N - 1$ bridges, while C_N has neither.
3. Compute the Fiedler value for the two-triangle graph and for C_6 (which has the same number of vertices and edges). Which is larger? Does the Fiedler value correctly predict which graph is more fragile? Discuss any limitations of this comparison.
4. In `igraph`, use `articulation_points()` and an edge-deletion loop to verify your classification. Report your code and output.

3.16.0.5 Exercise 5. Trees: characterisation and counting

1. Prove that a connected graph G with N vertices is a tree if and only if it has exactly $N - 1$ edges. Your proof should use the equivalences in Theorem 3.2.
2. How many distinct labeled trees exist on $N = 4$ vertices? List them explicitly and verify using Cayley's formula N^{N-2} (Cayley, 1889).
3. A graph G has $N = 10$ vertices and $M = 9$ edges. A student concludes it must be a tree. Is this necessarily true? If not, give a counterexample and state the missing condition.
4. In `igraph`, generate a random tree on 15 vertices using `sample_tree(15)`. Verify that it satisfies all five conditions in Theorem 3.2 computationally.

3.16.0.6 Exercise 6. Bipartite structure: theory and detection

1. Prove König's criterion: a graph is bipartite if and only if it contains no odd cycle. *(Hint: for the forward direction, show that a 2-colouring assigns opposite colours to endpoints of every edge; for the reverse, show BFS produces a valid 2-colouring when no odd cycle exists.)*
2. The complete bipartite graph $K_{m,n}$ has mn edges. Derive a formula for its average degree $\langle k \rangle$ in terms of m and n. For what value of m (with n fixed) is $\langle k \rangle$ maximised, and does this make sense intuitively?
3. Consider the following adjacency matrix:

$$A = \begin{pmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{pmatrix}.$$

Without drawing the graph, determine whether it is bipartite using only the eigenvalues of A. *(Hint: a graph is bipartite if and only if its adjacency spectrum is symmetric about zero (Chung, 1997).)*

4. Give a concrete real-world example of a network that is naturally bipartite. Explain what would be structurally wrong with computing the standard clustering coefficient on this network without accounting for its bipartite nature.

3.16.0.7 Exercise 7. Comparative network profile

Using the `igraph` workflow from Section 3.14, construct the following four graphs:

- K_5,
- C_{10},
- P_{10},
- one connected realisation of $G(20, 0.20)$.

For each graph, compute: order, size, $\langle k \rangle$, diameter, $c(G)$, and $\lambda_2(L)$ where applicable. Organise the results in a single table.

1. The path P_{10} and cycle C_{10} have the same number of vertices but very different diameters. Express both diameters in closed form and prove your answers.
2. Rank the four graphs by Fiedler value. Explain, in terms of graph structure, why the ranking comes out as it does.
3. The random graph $G(20, 0.20)$ is not deterministic. Repeat the construction five times. How stable are the quantities in your table across realisations? Which quantity varies most?
4. Write a short paragraph (5–8 sentences) identifying which of these graphs would make the most informative **null model** for comparison when, in the next chapter, we ask what a random graph typically looks like — and why.

3.17 Bridge to the next chapter

This chapter has treated graphs as deterministic mathematical objects: the vertex set and edge set are given, and the task is to reason about the resulting structure. The next chapter changes this viewpoint. Instead of fixing one graph in advance, we study probability models that generate many possible graphs. The language established here — degree, adjacency matrix, connectivity, trees, Laplacian spectrum — becomes the vocabulary used to describe what a random graph typically looks like and how its properties emerge from a simple probabilistic rule.

3.18 Concluding remarks

Graph theory provides the rigorous language on which network science depends. The definitions introduced here — graphs, adjacency matrices, degrees, paths, connectivity, trees, bipartiteness — are not merely formal conventions. They are the precise tools that make it possible to state questions unambiguously, derive results exactly, and implement algorithms correctly.

Several structural themes run through this chapter: a graph encodes only the pattern of connections; the adjacency matrix links combinatorics to linear algebra; the handshaking lemma is a universal constraint; connectivity, trees, and bipartiteness are global properties that determine much of a graph's character; and the Laplacian spectrum connects graph structure to spectral properties, with λ_2 measuring robustness.

These foundations appear in every subsequent chapter. The Erdős–Rényi (Erdős & Rényi, 1959, 1960), Watts–Strogatz (Watts & Strogatz, 1998), and Barabási–Albert (Barabási & Albert, 1999) models are all built on top of the graph-theoretic language established here. Before asking how likely a graph with certain properties is, one must be able to describe those properties precisely — and that is what this chapter has provided.

Chapter 4

Random Graphs

Probability models for network structure — the Erdős–Rényi ensemble

4.1 Introduction

Chapter 2 treated graphs as fixed mathematical objects: the vertex set and edge set were given, and the task was to reason about the resulting structure. This chapter changes that viewpoint. Instead of specifying one graph in advance, we define a **probability distribution over graphs** — a rule that assigns a probability to every possible graph on N labeled vertices. The object of study is no longer a single network but an entire **ensemble** of possible networks, each realised with a specified probability.

This change of perspective is essential for network science. Real networks are not uniquely determined by a few parameters; they reflect historical, stochastic, and context-dependent processes. The ensemble viewpoint lets us ask not *what is the structure of this specific graph?* but *what structure does a graph generated by this rule typically have?* Answering this question requires expected values, distributions, and limit arguments — the tools of probability theory applied to graph-theoretic quantities.

The Erdős–Rényi models, introduced independently by Erdős & Rényi (1959) and Gilbert (1959), are the simplest and most analytically tractable random graph models. The subsequent paper Erdős & Rényi (1960) established the foundational theory of phase transitions in random graphs, including the emergence of a giant component. These models serve as the primary null model in network science: if a real network's properties differ substantially from those of an Erdős–Rényi graph with the same size and density, something structurally non-trivial is present (Barabási & Pósfai, 2016; Newman, 2010).

The rigorous mathematical theory of random graphs is developed in Bollobás (2001). For the network science perspective, the standard references are Newman (2010, Ch. 12–13) and Barabási & Pósfai (2016, Ch. 3). The role of the Erdős–Rényi model as a null model for community detection and other structural inference problems is discussed in Fortunato (2010).

4.2 Learning goals

By the end of this chapter, a student should be able to:

- define a graph ensemble and distinguish $G(N, L)$ from $G(N, p)$,
- derive the expected average degree in both models from first principles,
- derive the exact degree distribution of $G(N, p)$ and identify its parameters,
- carry out the Poisson limit argument in the sparse regime,

- prove that $\mathbb{E}[C_i] = p$ in $G(N, p)$ and interpret the result,
- connect all theoretical results to simulation in `R` and critically evaluate the fit.

4.3 Graph ensembles

Definition 4.1. A **graph ensemble** on N labeled vertices is a probability space $(\mathcal{G}_N, \mathcal{F}, \mathbb{P})$ where $\mathcal{G}_N$ is the set of all simple graphs on vertex set $\{1, \dots, N\}$, $\mathcal{F}$ is a σ-algebra on $\mathcal{G}_N$, and $\mathbb{P}$ is a probability measure assigning $P(G) \geq 0$ to each graph $G \in \mathcal{G}_N$ with $\sum_G P(G) = 1$.

In an ensemble, graph-theoretic quantities — degree, diameter, number of triangles — become **random variables**. The central questions shift from computing exact values to computing expectations, variances, and distributional limits.

4.4 The Erdős–Rényi models

4.4.1 The model $G(N, L)$

Definition 4.2. The ensemble $G(N, L)$ consists of all simple graphs on N labeled vertices with exactly L edges, each assigned equal probability (Erdős & Rényi, 1959).

The total number of possible undirected edges on N vertices is

$$K = \binom{N}{2}.$$

The number of labeled simple graphs with exactly L edges is $\binom{K}{L}$, so the probability of any single graph G in $G(N, L)$ is

$$P(G) = \frac{1}{\binom{K}{L}}.$$

The randomness in $G(N, L)$ lies entirely in **which** L edges are chosen, not in how many. Every realisation of the model has the same number of edges.

4.4.1.1 Example: the ensemble $G(3, 2)$

For $N = 3$: $K = \binom{3}{2} = 3$ possible edges. Choosing exactly $L = 2$ gives $\binom{3}{2} = 3$ distinct labeled graphs, each with probability $\frac{1}{3}$.

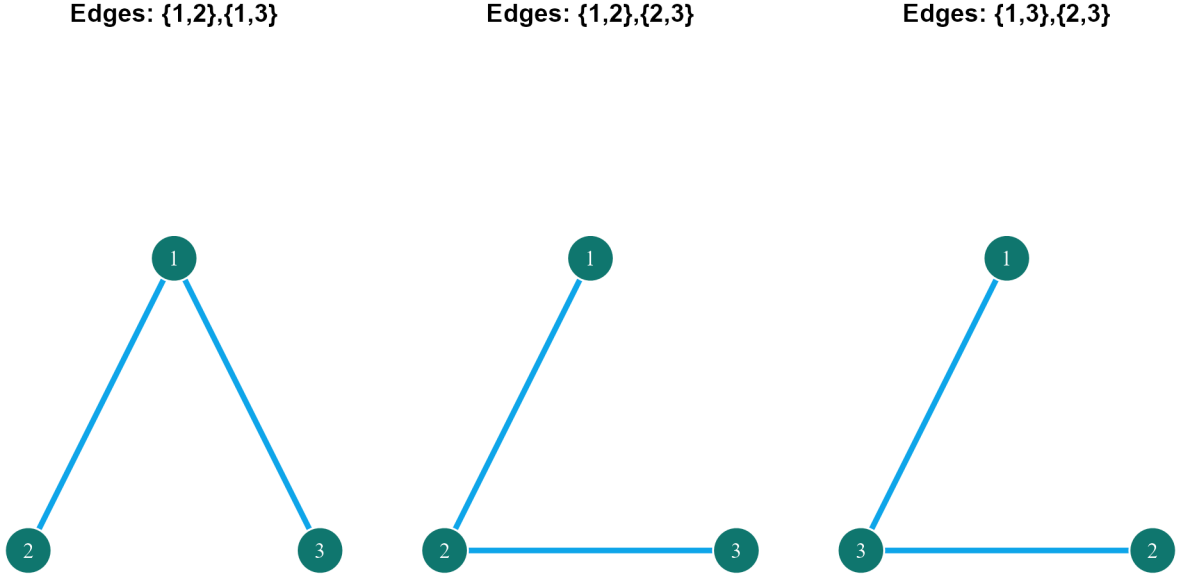


Figure 4.1: The three labeled graphs in the ensemble G(3,2). Each occurs with probability 1/3.

4.4.2 The model $G(N, p)$

Definition 4.3. The ensemble $G(N, p)$ on N labeled vertices includes each of the $K = \binom{N}{2}$ possible edges independently with probability $p \in [0, 1]$ (Gilbert, 1959).

A graph G with exactly L edges occurs with probability

$$P(G) = p^L (1-p)^{K-L}.$$

Unlike $G(N, L)$, the number of edges is not fixed — it is a random variable. Two graphs with the same number of edges are equally probable; two graphs with different numbers of edges generally are not.

4.4.2.1 Example: the ensemble $G(3, p)$

For $N = 3$, there are $2^3 = 8$ possible graphs (each of the three edges is independently present or absent). Their probabilities depend only on L:

L	Number of graphs	Probability per graph
0	1	$(1-p)^3$
1	3	$p(1-p)^2$
2	3	$p^2(1-p)$
3	1	p^3

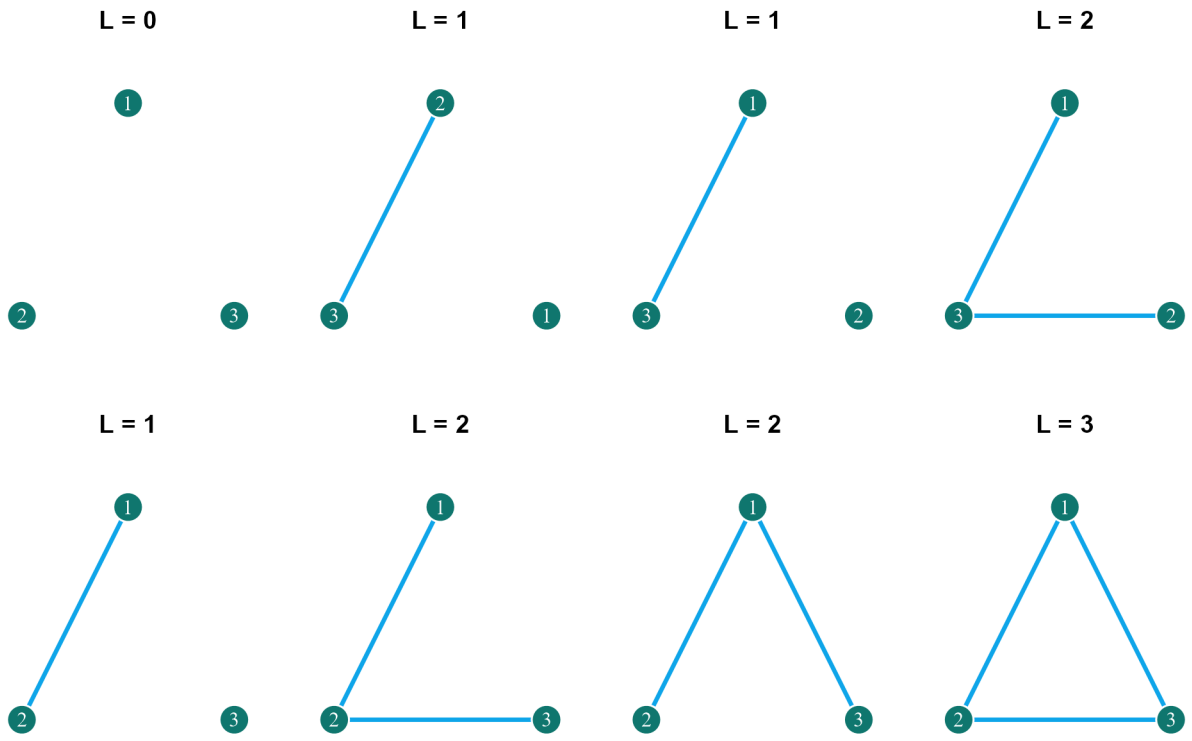


Figure 4.2: The eight labeled graphs in G(3,p), grouped by edge count L.

4.4.3 Comparing the two models

> **!** Key distinction
>
> In $G(N, L)$, every graph has exactly L edges and $P(G) = 1/\binom{K}{L}$ — a uniform distribution on a restricted set.
> In $G(N, p)$, every graph on K possible edges is assigned a probability, but graphs with different numbers of edges have different probabilities. Only within the same edge count are graphs equiprobable.
> The two models are related: $G(N, p)$ with $p = L/K$ has expected edge count $\mathbb{E}[L] = pK$, which matches the fixed L of $G(N, L)$. But they are not identical — in $G(N, p)$ the edge count fluctuates around that expectation.

4.5 Expected average degree

The **average degree** of any graph with N vertices and L edges is

$$\langle k \rangle = \frac{2L}{N},$$

a consequence of the handshaking lemma from Chapter 2. In a random graph ensemble, L may itself be random, so the quantity of interest is $\mathbb{E}[\langle k \rangle]$.

Since N is fixed,

$$\mathbb{E}[\langle k \rangle] = \frac{2\,\mathbb{E}[L]}{N}.$$

4.5.1 In $G(N, L)$

Every graph has exactly L edges, so $\mathbb{E}[L] = L$ and

$$\mathbb{E}[\langle k \rangle] = \frac{2L}{N}.$$

The average degree is not merely expected to be $2L/N$ — it equals $2L/N$ in every realisation of the model.

4.5.2 In $G(N, p)$

Each of the $K = \binom{N}{2}$ possible edges is included independently with probability p, so

$$L \sim \text{Binomial}\left(\binom{N}{2}, p\right), \qquad \mathbb{E}[L] = p\binom{N}{2} = \frac{pN(N-1)}{2}.$$

Substituting:

$$\mathbb{E}[\langle k \rangle] = \frac{2}{N} \cdot \frac{pN(N-1)}{2} = p(N-1).$$

A direct route to the same result: fix any vertex v. It has $N-1$ potential neighbors; each is connected to v independently with probability p, so

$$\mathbb{E}[k_v] = p(N-1).$$

Since all vertices are statistically equivalent, this equals $\mathbb{E}[\langle k \rangle]$.

> **Key formula**
>
> $$\mathbb{E}[\langle k \rangle] = p(N-1) \qquad \text{in } G(N, p).$$
>
> Equivalently, $p = \mathbb{E}[\langle k \rangle]/(N-1)$: to target a specific mean degree in a large network, set p accordingly.

```
n <- 80
p <- 0.08

g <- sample_gnp(n=n, p=p, directed=FALSE, loops=FALSE)
obs_deg <- degree(g)

obs_tbl <- tibble(k=obs_deg) |>
  count(k, name="observed") |>
  complete(k=0:(n-1), fill=list(observed=0)) |>
  mutate(observed_prob = observed / sum(observed))

theory_tbl <- tibble(k=0:(n-1)) |>
  mutate(theory_prob = dbinom(k, size=n-1, prob=p))

plot_tbl <- left_join(obs_tbl, theory_tbl, by="k")
```

```r
ggplot(plot_tbl, aes(x=k)) +
  geom_col(aes(y=observed_prob), fill=course_cols$blue, alpha=0.5, width=0.82) +
  geom_line(aes(y=theory_prob), color=course_cols$gold, linewidth=1.1) +
  geom_point(aes(y=theory_prob), color=course_cols$teal, size=2.2) +
  labs(title="Observed and theoretical degree distribution",
       subtitle=sprintf("Single realisation of G(%d, %.2f)", n, p),
       x="Degree k", y="Probability")
```

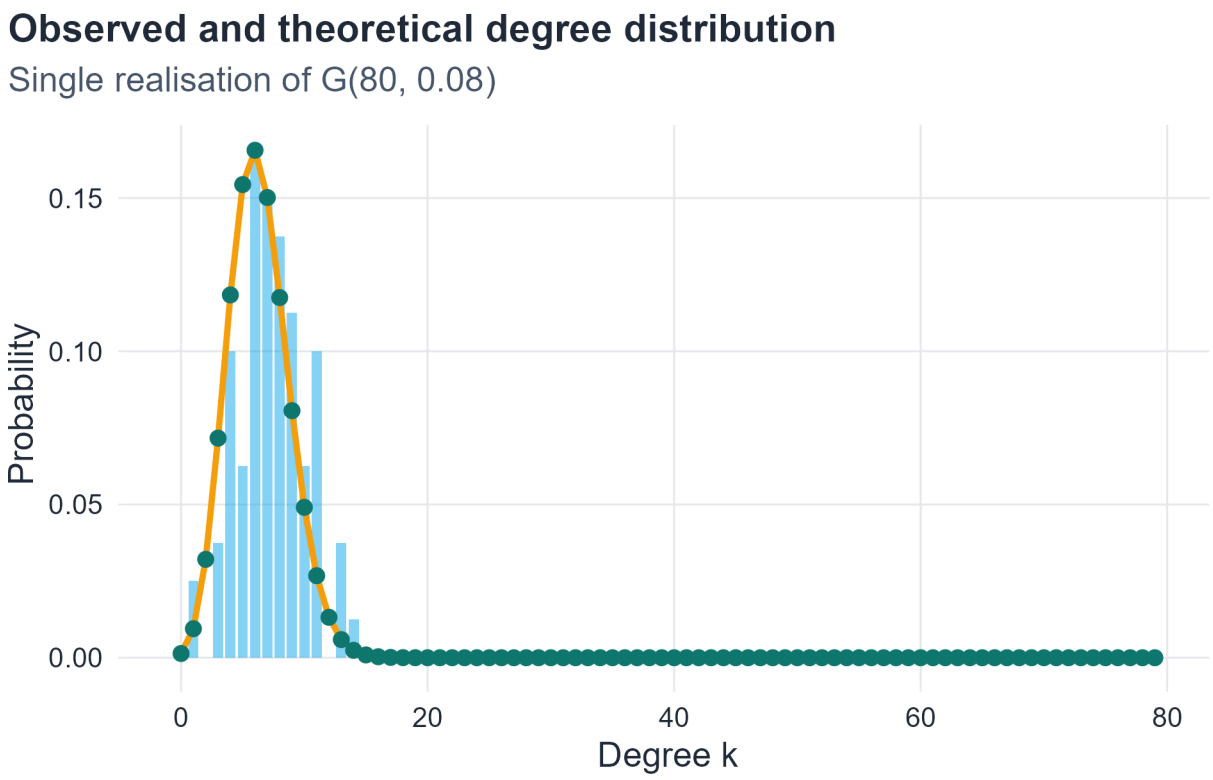


Figure 4.3: Observed degree frequencies in one realisation of G(80, 0.08) compared with the theoretical binomial distribution.

4.6 Degree distribution in $G(N, p)$

Fix a vertex v. It has $N-1$ potential neighbors. Each potential edge is present independently with probability p and absent with probability $1-p$. The degree k_v is therefore the count of successes in $N-1$ independent Bernoulli(p) trials:

$$k_v \sim \text{Binomial}(N-1,\, p).$$

The probability that v has degree exactly k is

$$P(k) = \binom{N-1}{k} p^k \,(1-p)^{N-1-k}, \qquad k = 0, 1, \ldots, N-1.$$

The three factors have clear interpretations:

- $\binom{N-1}{k}$: the number of ways to choose which k of the $N-1$ potential neighbors are connected,
- p^k: the probability that those k edges are present,
- $(1-p)^{N-1-k}$: the probability that the remaining $N-1-k$ edges are absent.

The mean and variance of this distribution are

$$\mathbb{E}[k] = p(N-1), \qquad \mathrm{Var}(k) = p(1-p)(N-1).$$

The mean recovers the expected average degree derived above. The variance quantifies fluctuations around that mean within a single graph.

! Summary

In $G(N,p)$, the degree of any vertex follows a Binomial$(N-1,p)$ distribution. The degree distribution of the model is therefore

$$P(k) = \binom{N-1}{k} p^k (1-p)^{N-1-k}.$$

4.7 Poisson approximation in the sparse regime

The binomial degree distribution is exact. In many network science settings, however, attention focuses on large graphs in the **sparse regime**, where

$$N \to \infty, \quad p \to 0, \quad \lambda := p(N-1) \text{ remains finite.}$$

Sparsity means the average degree stays bounded even as the network grows — a feature shared by many real-world networks.

4.7.1 Derivation of the Poisson limit

Starting from the exact binomial probability:

$$P(k) = \binom{N-1}{k} p^k \, (1-p)^{N-1-k}.$$

Expand the binomial coefficient and substitute $p = \lambda/(N-1)$:

$$\binom{N-1}{k} = \frac{(N-1)(N-2)\cdots(N-k)}{k!}.$$

This gives

$$P(k) = \frac{(N-1)(N-2)\cdots(N-k)}{k!} \cdot \left(\frac{\lambda}{N-1}\right)^k \cdot \left(1 - \frac{\lambda}{N-1}\right)^{N-1-k}.$$

Rearranging:

$$P(k) = \frac{\lambda^k}{k!} \cdot \underbrace{\frac{(N-1)(N-2)\cdots(N-k)}{(N-1)^k}}_{\to 1} \cdot \underbrace{\left(1 - \frac{\lambda}{N-1}\right)^{N-1}}_{\to e^{-\lambda}} \cdot \underbrace{\left(1 - \frac{\lambda}{N-1}\right)^{-k}}_{\to 1}.$$

As $N \to \infty$ with λ fixed:

1. The first product $\frac{(N-1)(N-2)\cdots(N-k)}{(N-1)^k} = \prod_{j=0}^{k-1}\left(1 - \frac{j}{N-1}\right) \to 1$ since each factor tends to 1 and k is fixed.
2. $\left(1 - \frac{\lambda}{N-1}\right)^{N-1} \to e^{-\lambda}$ by the standard limit $\lim_{m\to\infty}(1 - x/m)^m = e^{-x}$.
3. $\left(1 - \frac{\lambda}{N-1}\right)^{-k} \to 1$ since k is fixed and $\lambda/(N-1) \to 0$.

Therefore,

$$\boxed{P(k) \;\longrightarrow\; e^{-\lambda}\frac{\lambda^k}{k!},}$$

the Poisson distribution with mean $\lambda = p(N-1) = \mathbb{E}[k]$.

What the approximation requires

The Poisson approximation is accurate when N is large and p is small such that $\lambda = p(N-1)$ is of moderate size. It fails when p is not small (e.g., in dense graphs), because then $\mathrm{Var}(k) = p(1-p)(N-1) \approx (1-p)\lambda \not\approx \lambda$, and the Poisson assumption of equal mean and variance breaks down.

```
n      <- 500
p      <- 0.02
lambda <- p * (n - 1)
k_vals <- 0:24

compare_tbl <- tibble(
  k = k_vals,
  binomial = dbinom(k_vals, size=n-1, prob=p),
  poisson  = dpois(k_vals, lambda=lambda)
) |>
  pivot_longer(cols=c(binomial, poisson),
               names_to="distribution", values_to="probability") |>
  mutate(distribution = recode(distribution,
                               binomial="Binomial", poisson="Poisson"))

ggplot(compare_tbl, aes(x=k, y=probability, color=distribution)) +
  geom_line(linewidth=1.1) +
  geom_point(size=2.2) +
  scale_color_manual(values=c("Binomial"=course_cols$gold,
                              "Poisson"=course_cols$teal)) +
  labs(title="Binomial and Poisson degree distributions",
       subtitle=sprintf("N = %d, p = %.3f, mean degree = %.2f", n, p, lambda),
       x="Degree k", y="Probability", color=NULL)
```

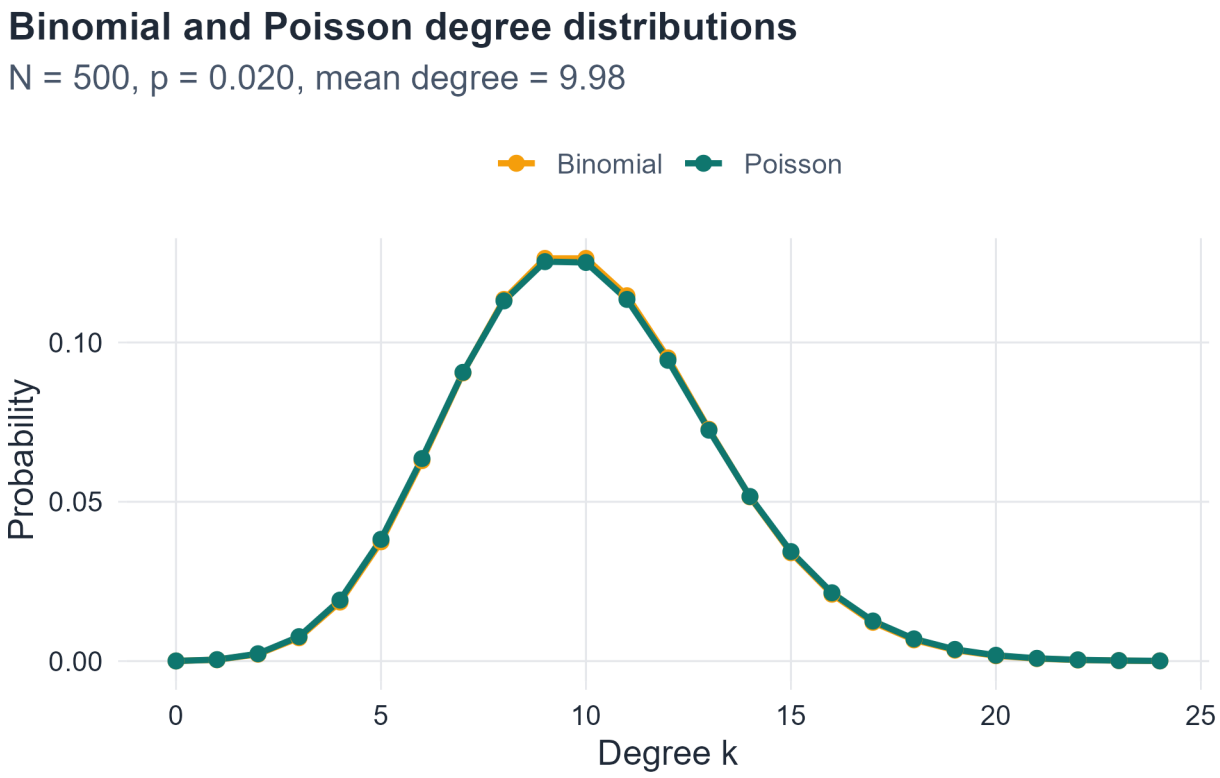


Figure 4.4: Binomial degree distribution and its Poisson approximation for G(500, 0.02). The two curves are nearly indistinguishable in the sparse regime.

4.8 Clustering coefficient

4.8.1 Definition

The **local clustering coefficient** of vertex i with degree k_i measures the fraction of its $\binom{k_i}{2}$ possible neighbor-neighbor edges that are actually present. If L_i denotes the number of edges among the neighbors of i, then

$$C_i = \frac{L_i}{\binom{k_i}{2}} = \frac{2L_i}{k_i(k_i-1)}, \qquad k_i \geq 2.$$

For $k_i \leq 1$, C_i is conventionally set to 0. The value $C_i \in [0,1]$: it equals 0 when no two neighbors of i are connected, and 1 when all $\binom{k_i}{2}$ possible neighbor pairs are connected.

The **average clustering coefficient** of the graph is

$$\bar{C} = \frac{1}{N}\sum_{i=1}^{N} C_i.$$

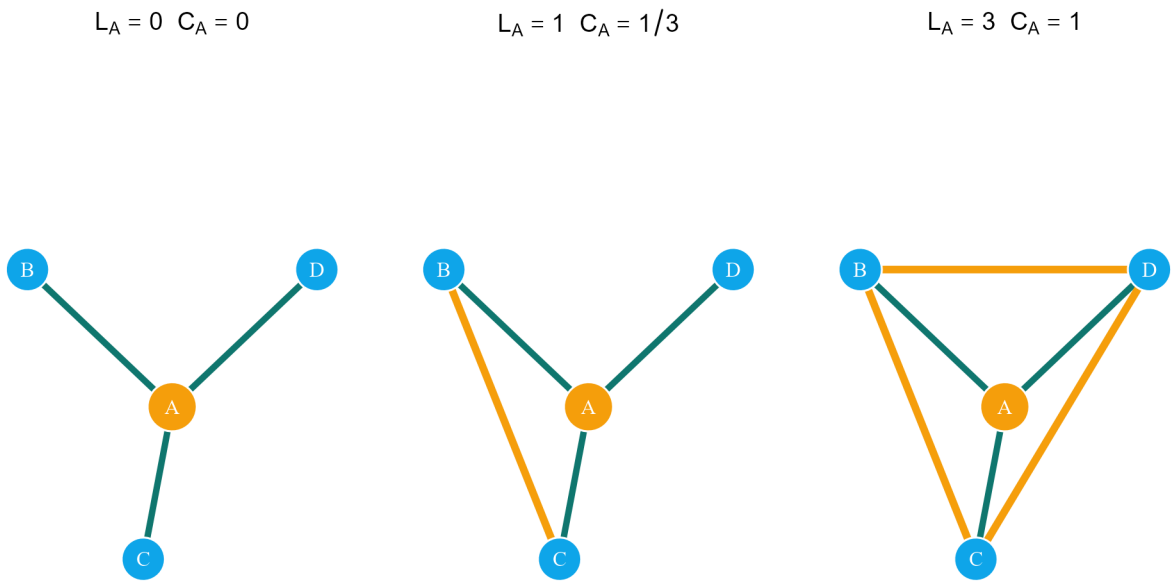


Figure 4.5: Three neighborhoods of the same focal node A, each with degree 3 but different numbers of neighbor-neighbor edges. Teal edges connect A to its neighbors; gold edges connect neighbors to each other.

4.8.2 Expected clustering in $G(N, p)$

Theorem 4.1. ***Proposition.*** *In $G(N, p)$,*

$$\mathbb{E}[C_i] = p \quad \textit{for every vertex } i \textit{ with } k_i \geq 2,$$

and consequently $\mathbb{E}[\bar{C}] = p$.

Proof. Condition on a fixed set of k_i neighbors of vertex i. For each pair (u, v) among those neighbors, the edge $\{u, v\}$ is present independently with probability p — independently of the edges connecting u and v to i, because edges in $G(N, p)$ are mutually independent. Therefore

$$\mathbb{E}[L_i \,|\, k_i] = \binom{k_i}{2} p,$$

which gives

$$\mathbb{E}[C_i \,|\, k_i] = \frac{\mathbb{E}[L_i \mid k_i]}{\binom{k_i}{2}} = p.$$

Taking expectations over k_i: $\mathbb{E}[C_i] = p$. Averaging over all vertices: $\mathbb{E}[\bar{C}] = p$. $\square$

Interpretation

In $G(N, p)$, clustering is entirely determined by the edge probability p. In sparse graphs where p is small, clustering is correspondingly small. This is a direct consequence of edge independence: there is no mechanism favouring triangle closure beyond the baseline probability that any edge exists.
Many empirical social networks have clustering coefficients far exceeding p, even when matched to a random graph on the same N and average degree (Newman, 2010; Watts & Strogatz, 1998). This gap — between Erdős–Rényi clustering and observed clustering — motivates the small-world models studied in the next chapter (Strogatz, 2001; Watts & Strogatz, 1998).

```
n    <- 100
p    <- 0.07
reps <- 250

sim_tbl <- map_dfr(seq_len(reps), function(i) {
  g <- sample_gnp(n=n, p=p, directed=FALSE, loops=FALSE)
  tibble(
    sim            = i,
    avg_degree     = mean(degree(g)),
    avg_clustering = transitivity(g, type="average")
  )
})

summary_tbl <- tibble(
  quantity          = c("Average degree", "Average clustering"),
  empirical_mean    = c(mean(sim_tbl$avg_degree),
                        mean(sim_tbl$avg_clustering, na.rm=TRUE)),
  theoretical_value = c(p*(n-1), p)
)

summary_tbl
```

```
# A tibble: 2 x 3
  quantity           empirical_mean theoretical_value
  <chr>                       <dbl>             <dbl>
1 Average degree             6.93               6.93
2 Average clustering         0.0699             0.07
```

```
ggplot(sim_tbl, aes(x=avg_clustering)) +
  geom_histogram(bins=24, fill=course_cols$gold, alpha=0.65, color="white") +
  geom_vline(xintercept=p, color=course_cols$teal,
             linewidth=1.1, linetype=2) +
  labs(title="Average clustering across simulations",
       subtitle=sprintf("Dashed line = theoretical value p = %.2f", p),
       x="Average clustering", y="Count")
```

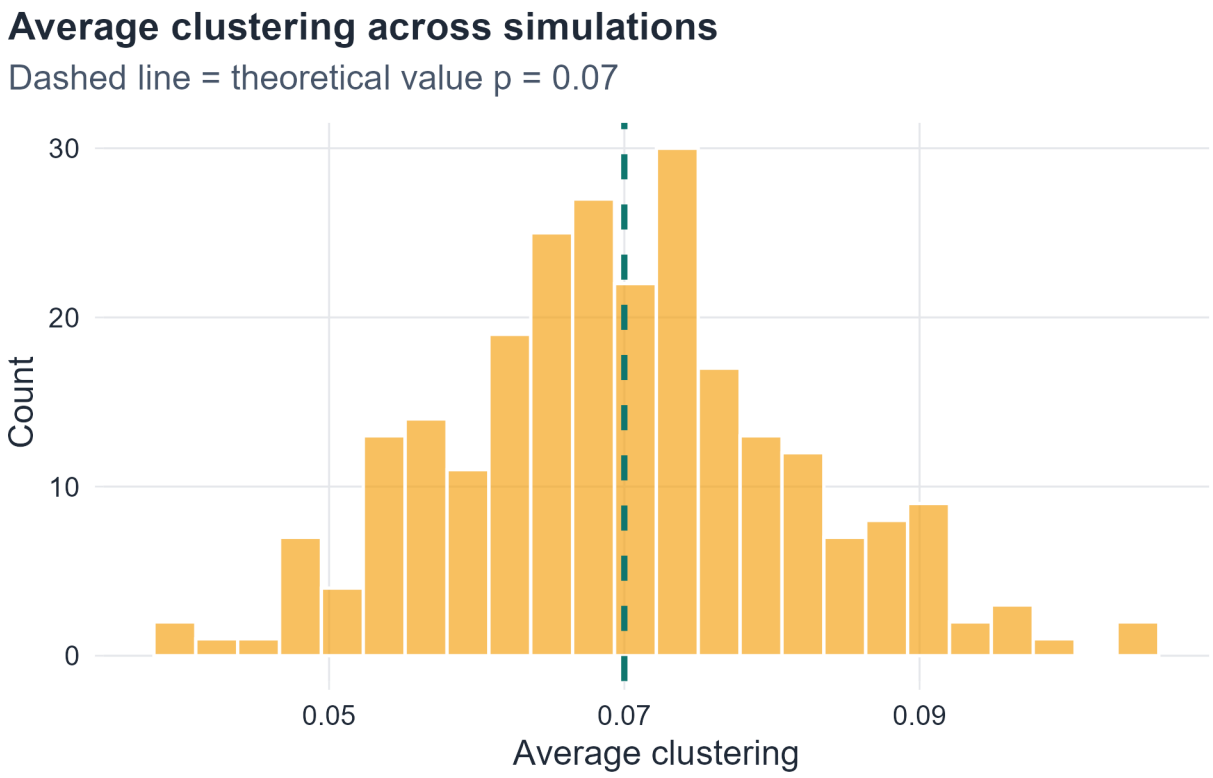


Figure 4.6: Distribution of average clustering across 250 realisations of G(100, 0.07). Dashed line marks the theoretical value p.

4.9 Core formulas

! Summary of key results

For Erdős–Rényi random graphs:

Quantity	$G(N, L)$	$G(N, p)$
$P(G)$	$1/\binom{K}{L}$	$p^L(1-p)^{K-L}$
$\mathbb{E}[\langle k \rangle]$	$2L/N$	$p(N-1)$
Degree distribution	—	$\text{Bin}(N-1, p)$
Poisson limit (sparse)	—	$\text{Poisson}(\lambda)$, $\lambda = p(N-1)$
$\mathbb{E}[\bar{C}]$	—	p

where $K = \binom{N}{2}$.

4.10 Working with random graphs in R

```
n <- 20
p <- 0.12

g_er <- sample_gnp(n=n, p=p, directed=FALSE, loops=FALSE)
```

```r
plot(g_er,
     layout=layout_with_fr(g_er),
     vertex.size=22, vertex.color=course_cols$teal,
     vertex.frame.color="white", vertex.label.color="white",
     edge.color=scales::alpha(course_cols$ink, 0.32),
     vertex.label.cex=0.8,
     main=sprintf("A realisation of G(%d, %.2f)", n, p))
```

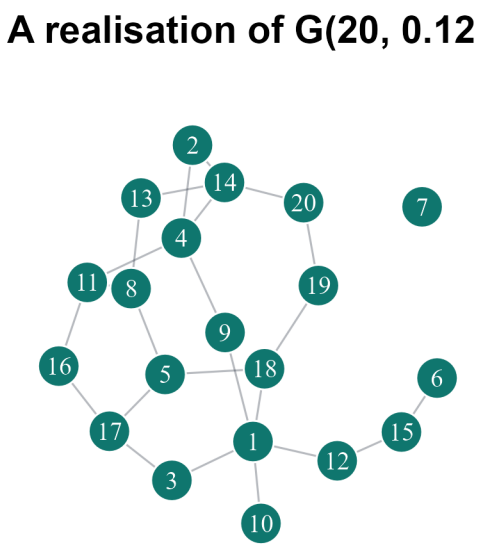
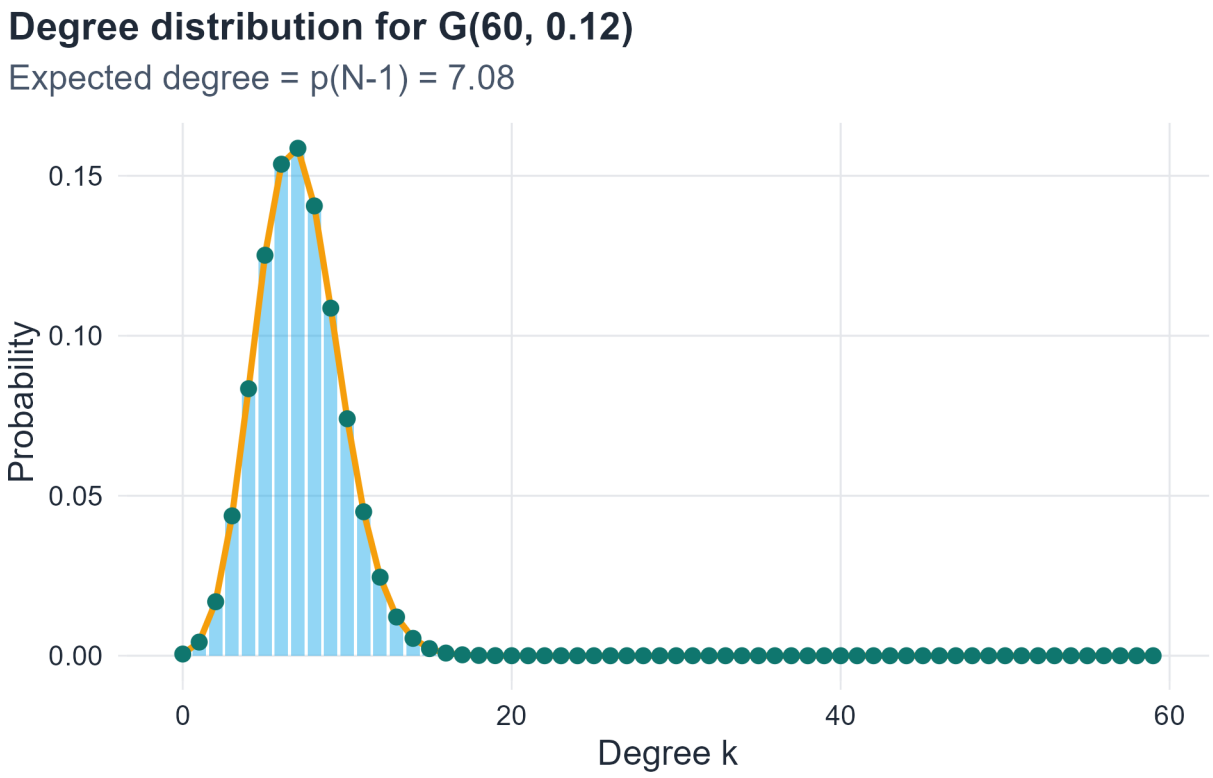


Figure 4.7: One realisation of G(20, 0.12).



4.11 In-class questions

1. What distinguishes a graph ensemble from a single graph?
2. Why does L follow a binomial distribution in $G(N, p)$ but not in $G(N, L)$?
3. What does the Poisson approximation require, and why does it fail in the dense regime?
4. Two graphs have the same N and $\mathbb{E}[\langle k \rangle]$. Must they have the same degree distribution?
5. Why is $\mathbb{E}[\bar{C}] = p$ a consequence of edge independence, not of any structural property of G?
6. A real social network has clustering $\bar{C} = 0.4$ and $p = 0.01$. What does this tell you?

4.12 Exercises

The seven exercises below cover the full content of this chapter. They are designed to require reasoning and derivation, not reproduction of results already in the text. The first two are mathematical, the next three analytical, and the final two computational.

4.12.0.1 Exercise 1. Model structure and equiprobability

Let $G(3, 2)$ and $G(3, p)$ with $p = 2/3$ both be defined on the same three labeled vertices.

1. List all graphs in $G(3, 2)$ and compute $P(G)$ for each. Are all graphs with two edges in $G(3, p)$ assigned the same probability? Are they assigned the same probability as in $G(3, 2)$?
2. In $G(N, p)$, two graphs G_1 and G_2 with $L_1 \neq L_2$ edges satisfy $P(G_1) \neq P(G_2)$ whenever $p \in (0, 1)$. Prove this directly from the probability formula $P(G) = p^L(1-p)^{K-L}$.
3. For what value of p is $G(N, p)$ the uniform distribution over **all** 2^K labeled simple graphs? Justify your answer.

4. In $G(N, L)$, two vertices u and v are connected with probability L/K where $K = \binom{N}{2}$. Derive this result combinatorially. *(Hint: count the graphs in the ensemble that contain the edge $\{u, v\}$.)*

4.12.0.2 Exercise 2. Expected degree: derivation and inversion

1. Derive $\mathbb{E}[\langle k \rangle] = p(N-1)$ in $G(N, p)$ using two different arguments: first via $\mathbb{E}[L]$, then directly from the degree of a single vertex. Confirm that both give the same result.
2. Compute $\mathrm{Var}(\langle k \rangle)$ in $G(N, p)$. Show that the relative fluctuation $\mathrm{SD}(\langle k \rangle)/\mathbb{E}[\langle k \rangle]$ vanishes as $N \to \infty$ with $\lambda = p(N-1)$ fixed. What does this say about how close $\langle k \rangle$ is to its expectation in large graphs?
3. You observe a network with $N = 1000$ vertices and average degree $\langle k \rangle = 6$. If you model it as $G(N, p)$, what value of p should you use? Would you expect the degree distribution to be closer to binomial or Poisson? Justify your answer.

4.12.0.3 Exercise 3. Degree distribution: properties and moments

The degree distribution in $G(N, p)$ is $P(k) = \binom{N-1}{k} p^k (1-p)^{N-1-k}$.

1. Verify that $\sum_{k=0}^{N-1} P(k) = 1$ using the binomial theorem. Show your working.
2. Compute the variance $\mathrm{Var}(k) = p(1-p)(N-1)$ from the definition $\mathrm{Var}(k) = \mathbb{E}[k^2] - (\mathbb{E}[k])^2$.
3. In the sparse regime $\lambda = p(N-1)$ fixed, the Poisson distribution has $\mathrm{Var}(k) = \lambda = \mathbb{E}[k]$. Show that $\mathrm{Var}(k) = p(1-p)(N-1) \to \lambda$ as $N \to \infty$ with λ fixed. What property of the Poisson distribution does this reflect?
4. A student argues: *"Since both the binomial and the Poisson have the same mean λ in the sparse regime, there is no reason to prefer one over the other."* Identify precisely what is wrong with this argument, and state one quantity for which the two distributions give different predictions.

4.12.0.4 Exercise 4. The Poisson limit

1. Carry out the Poisson limit argument from the lecture in full detail for the case $k = 2$. That is, starting from $P(2) = \binom{N-1}{2} p^2 (1-p)^{N-3}$ with $p = \lambda/(N-1)$, show explicitly that $P(2) \to e^{-\lambda}\lambda^2/2$ as $N \to \infty$.
2. The argument uses three separate limit steps. State each one precisely and identify the standard limit result being applied.
3. Suppose instead of fixing $\lambda = p(N-1)$, you fix $\mu = pN$. Does the Poisson approximation still hold? If so, with which mean parameter? If not, explain what goes wrong.
4. For which values of p (as a function of N) is the Poisson approximation *not* appropriate, and why?

4.12.0.5 Exercise 5. Clustering: proof and interpretation

1. The lecture proves $\mathbb{E}[C_i] = p$ by conditioning on the neighborhood of vertex i. Write out this proof in full, being explicit about which independence assumption is used at each step.
2. A student constructs a graph model in which edges are not independent: whenever edge $\{u, v\}$ is present, each of the edges $\{u, w\}$ and $\{v, w\}$ for any third vertex w is present with probability $q > p$ (rather than p). Qualitatively, how would $\mathbb{E}[\bar{C}]$ compare to p in this model? Relate your answer to real social networks.
3. In $G(N, p)$, the expected number of triangles is $\binom{N}{3} p^3$. Derive this result. Then express the expected clustering coefficient in terms of the expected number of triangles and the expected number of connected triples (paths of length 2). Are the two expressions for $\mathbb{E}[\bar{C}]$ consistent?
4. Fix $N = 100$ and vary p from 0.01 to 0.5. Predict how $\mathbb{E}[\bar{C}]$ and $\mathbb{E}[\langle k \rangle]$ scale with p. Does the ratio $\mathbb{E}[\bar{C}]/\mathbb{E}[\langle k \rangle]$ depend on N?

4.12.0.6 Exercise 6. Comparing $G(N, L)$ and $G(N, p)$ numerically

Set $N = 50$, $L = 100$, and $p = L/\binom{N}{2}$.

1. Generate 500 independent graphs from $G(N, p)$ using `sample_gnp`. For each graph record: edge count M, average degree $\langle k \rangle$, and average clustering $\bar{C}$.
2. Plot the distribution of edge counts across the 500 realisations. Mark $L = 100$ and $\mathbb{E}[L] = pK$ on the plot. Comment on the spread and its relationship to $\mathrm{Var}(L) = Kp(1-p)$.
3. Generate one graph from $G(N, L)$ using `sample_gnm(N, L)`. Compare its average degree and clustering to the empirical means from part 1. Is this comparison fair? What does it reveal about the difference between the two models?
4. Compute the empirical degree distribution across all 500 graphs from $G(N, p)$. Overlay the theoretical binomial $\mathrm{Bin}(N-1, p)$ and the Poisson approximation $\mathrm{Poisson}(\lambda)$ on the same plot. At this value of N and p, which approximation fits better? Is this what you expected?

4.12.0.7 Exercise 7. A network diagnostic: ER as a null model

A network analyst observes a real network with the following properties: $N = 200$, $M = 600$, $\bar{C} = 0.18$, and degree sequence ranging from 1 to 25.

1. Fit an Erdős–Rényi model $G(N, p)$ to this network by matching the observed average degree. What value of p do you use?
2. Under your fitted model, compute: (a) $\mathbb{E}[\bar{C}]$, (b) the expected maximum degree, (c) the probability that a randomly chosen vertex has degree 0.
3. The observed clustering $\bar{C} = 0.18$ is much larger than $\mathbb{E}[\bar{C}]$ under your fitted model. Simulate 200 realisations of $G(N, p)$ and plot the distribution of $\bar{C}$. Where does 0.18 fall in this distribution? What does this tell you about the real network?
4. The observed degree sequence has maximum degree 25. Under $G(N, p)$ with your fitted p, what is the probability of observing a vertex of degree ≥ 25? Compute this analytically and verify it by simulation. What structural feature of the real network might explain a higher maximum degree than the ER model predicts?

4.13 Bridge to the next chapter

The Erdős–Rényi model captures one key feature of real networks — sparse random connectivity — but fails on two others: it has low clustering ($\mathbb{E}[\bar{C}] = p \approx 0$ for sparse graphs), and its degree distribution is binomial or Poisson, decaying exponentially for large k. Empirical networks routinely exhibit clustering far above the ER baseline (Watts & Strogatz, 1998) and degree distributions with much heavier tails — some vertices have degrees orders of magnitude larger than the mean (Albert & Barabási, 2002; Barabási & Albert, 1999). These two departures from the ER model motivate the next two chapters: the Watts–Strogatz model (Watts & Strogatz, 1998) introduces a mechanism for generating high clustering at low average degree, and the Barabási–Albert model (Barabási & Albert, 1999) introduces preferential attachment to generate heavy-tailed degree distributions. The degree sequence approach of Molloy & Reed (1995) provides a third generalisation, allowing arbitrary degree distributions to be studied within a random graph framework.

4.14 Concluding remarks

The Erdős–Rényi model is not a realistic description of most real networks. Its value lies elsewhere: it is the simplest probability model over graphs, it is analytically exact (Bollobás, 2001; Erdős & Rényi, 1959,

1960), and it provides a rigorous null hypothesis against which observed network structure can be tested (Newman, 2010). The binomial degree distribution, the Poisson limit, and the clustering formula $\mathbb{E}[\bar{C}] = p$ are all consequences of a single assumption — that edges form independently. When real networks violate this assumption, as they almost always do (Barabási & Albert, 1999; Strogatz, 2001; Watts & Strogatz, 1998), the deviation itself is informative. That is why the Erdős–Rényi model remains the essential starting point for network science.

Chapter 5

The Evolution of Random Graphs

Thresholds, phase transitions, and the emergence of large-scale connectivity

5.1 Introduction

Chapter 3 described static properties of the Erdős–Rényi ensemble at a fixed edge probability p: the degree distribution, expected average degree, and clustering coefficient. This chapter adopts a dynamic viewpoint. Instead of fixing p, we ask how the graph changes as p increases from 0 to 1, and in particular how global connectivity emerges from local random wiring.

The central result is that this process is not gradual and undifferentiated — it is organized by two sharp **threshold phenomena** (Bollobás, 2001; Erdős & Rényi, 1960):

1. Near $\langle k \rangle = 1$, a **giant component** — a connected subgraph containing a positive fraction of all vertices — emerges discontinuously.
2. Near $\langle k \rangle = \ln N$, the graph becomes **fully connected** with high probability.

These two events are separated by a wide intermediate range in which the graph has a macroscopic backbone but also retains isolated vertices and small disconnected pieces. Understanding both thresholds, and the structural regimes between them, is the goal of this chapter.

The theory of random graph evolution was initiated by Erdős & Rényi (1960) and developed rigorously by Bollobás (2001). The network science perspective is covered in Newman (2010, Ch. 12) and Barabási & Pósfai (2016, Ch. 3).

5.2 Learning goals

By the end of this chapter, a student should be able to:

- define connected components precisely and characterise their partition structure,
- state the giant component and connectedness thresholds and express each in terms of p and $\langle k \rangle$,
- derive the self-consistency equation $S = 1 - e^{-\langle k \rangle S}$ from a probabilistic argument,
- carry out the near-threshold expansion and obtain $S \approx 2(\langle k \rangle - 1)$,
- distinguish the four structural regimes and describe the component structure in each,
- connect all theoretical results to simulation in `R` and interpret finite-size effects.

5.3 Connectedness and components

5.3.1 Definitions

Definition 5.1. Let $G = (V, E)$ be an undirected graph. A **connected component** of G is a maximal connected subgraph: a subgraph $H \subseteq G$ that is connected and to which no further vertex of G can be added while preserving connectedness.

The connected components of G partition the vertex set V: every vertex belongs to exactly one component, because if a vertex belonged to two distinct components, those components would share a vertex, contradicting their maximality. An **isolated vertex** — one with degree zero — forms a component of size 1.

The **size** of a component is its number of vertices, $|V(H)|$.

> **Partition property**
>
> Because components are maximal, the component decomposition is unique and the components tile V without overlap. This is the key structural fact that makes component size a well-defined global quantity.

5.3.2 Component structure example

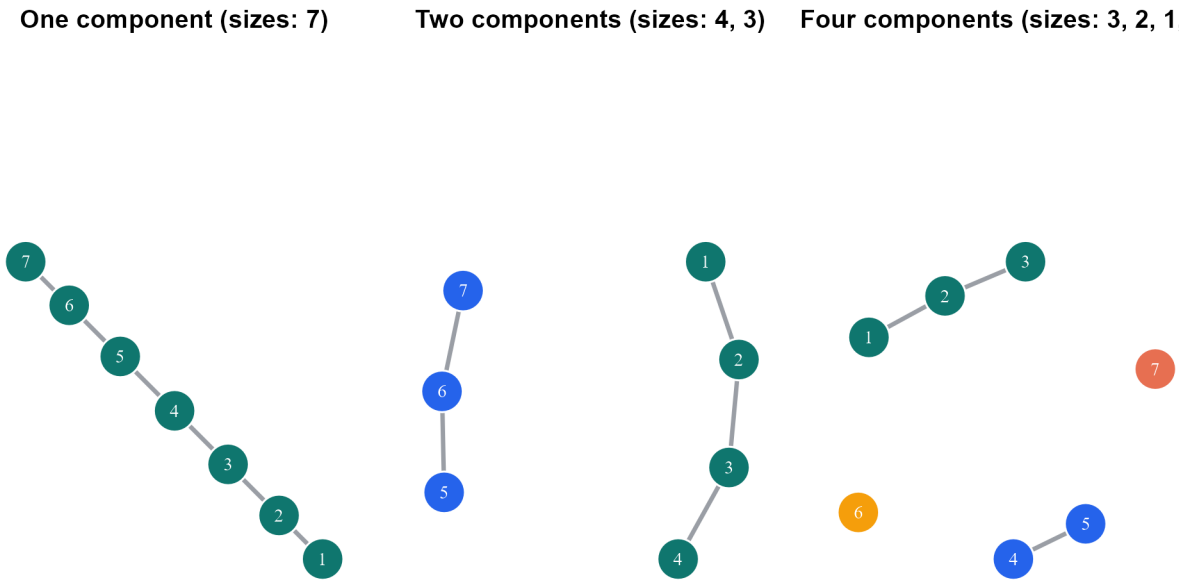


Figure 5.1: Three graphs on seven vertices with different component structures. Vertices in the same component share the same colour.

5.4 The giant component

Definition 5.2. Let $C_{\max}(G_N)$ denote the largest connected component of a graph on N vertices. A sequence of random graphs contains a **giant component** if

$$\frac{|C_{\max}(G_N)|}{N} \xrightarrow{N\to\infty} c > 0.$$

The term *giant* is reserved for components that constitute a **positive fraction** of all vertices in the large-N limit. Every finite graph trivially has a largest component, but that component is giant only when it grows proportionally with the system size. Before the giant component emerges, all components have sizes of order $O(\log N)$ or smaller.

5.5 Evolution of a random graph

Consider $G(N,p)$ with N fixed and p increasing from 0 to 1. The evolution passes through four qualitatively distinct structural regimes, controlled by the expected degree

$$\langle k\rangle = p(N-1).$$

5.5.1 The four regimes

Definition 5.3. Subcritical regime: $\langle k\rangle < 1$. No giant component exists. Almost all connected components have size $O(\log N)$. The largest component has size of order $\log N$ (Bollobás, 2001, Ch. 6). Isolated vertices are common.

Definition 5.4. Critical regime: $\langle k\rangle \approx 1$. The phase transition. The largest component has size of order $N^{2/3}$ — macroscopic, but still a vanishing fraction of N (Erdős & Rényi, 1960). This is the boundary at which the giant component is born.

Definition 5.5. Supercritical regime: $\langle k\rangle > 1$. A unique giant component exists, containing a fraction $S > 0$ of all vertices. The remaining vertices form many small finite components, predominantly tree-like near the threshold. The giant component itself contains cycles.

Definition 5.6. Connected regime: $\langle k\rangle \approx \ln N$. The graph is connected with high probability. More precisely, for $p = (\ln N + c)/N$ with c a constant, the probability that $G(N,p)$ is connected converges to $e^{-e^{-c}}$ as $N\to\infty$ (Erdős & Rényi, 1960; Bollobás, 2001, Ch. 7).

> **!** Two distinct thresholds
>
> The giant component emerges at $\langle k\rangle = 1$ ($p_c \approx 1/N$). The graph becomes fully connected at $\langle k\rangle \approx \ln N$ ($p \approx \ln N/N$). Between these thresholds, the graph has a macroscopic giant component but also retains isolated vertices and small disconnected pieces. A graph with a giant component is **not** necessarily connected.

5.5.2 Monotone evolution

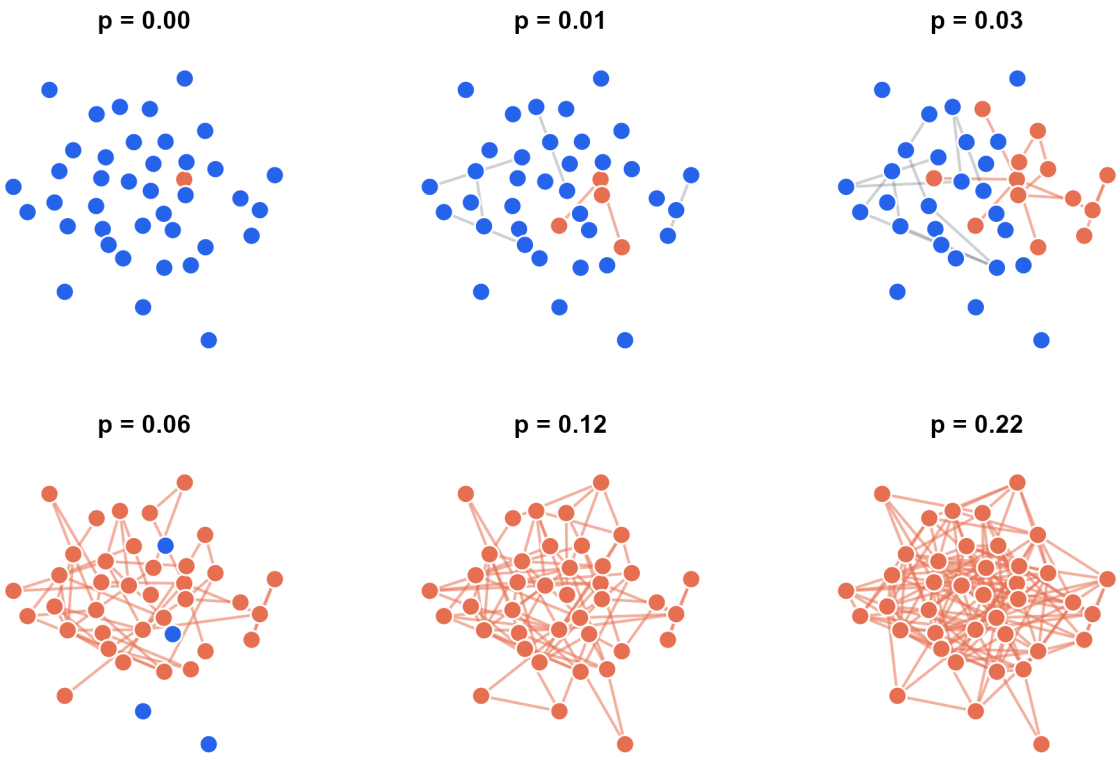


Figure 5.2: Evolution of G(N,p) on a fixed vertex set of 40 vertices. Vertex positions are fixed; edges accumulate monotonically. The largest component is highlighted in red.

5.6 The self-consistency equation

5.6.1 Derivation

The fraction S of vertices in the giant component satisfies a self-consistency equation that can be derived from first principles (Newman, 2010, Ch. 12).

A vertex v belongs to the giant component if and only if at least one of its neighbors also belongs to the giant component. Equivalently, v does **not** belong to the giant component if and only if every one of its neighbors also lies outside the giant component.

Let u be a neighbor of v, reached by following a random edge. The probability that u does not lead to the giant component is $e^{-\langle k\rangle S}$, derived as follows. Following an edge from v to u means u has at least one neighbor (v), so the remaining degree of u (the number of further neighbors) is approximately Poisson($\langle k\rangle$) in the sparse large-N limit. Each of those further neighbors independently fails to reach the giant with probability $1 - S$. Therefore the probability that u itself does not reach the giant is

$$\sum_{j=0}^{\infty} e^{-\langle k\rangle} \frac{\langle k\rangle^j}{j!} (1-S)^j = e^{-\langle k\rangle S}.$$

Since v has Poisson($\langle k\rangle$) neighbors (in the sparse limit), each independently failing to reach the giant with probability $e^{-\langle k\rangle S}$, the probability that v itself is not in the giant component is $e^{-\langle k\rangle \cdot S}$... but S here is the

probability that a *neighbor reached by a random edge* belongs to the giant, which by the self-consistency argument equals S itself. Therefore the probability that v lies outside the giant is $e^{-\langle k\rangle S}$, giving

$$\boxed{S = 1 - e^{-\langle k\rangle S}.}$$

> **Existence of a positive solution**
>
> Define $f(S) = 1 - e^{-\langle k\rangle S}$. Note $f(0) = 0$, so $S = 0$ is always a solution. The derivative at $S = 0$ is $f'(0) = \langle k\rangle$. A positive fixed point exists if and only if $f'(0) > 1$, i.e., $\langle k\rangle > 1$. This is the **giant component threshold**. When $\langle k\rangle \leq 1$, the only solution is $S = 0$; when $\langle k\rangle > 1$, a unique positive solution $S^* \in (0, 1)$ exists (Bollobás, 2001, Ch. 5).

5.6.2 Near-threshold expansion

When $\langle k\rangle = 1 + \varepsilon$ with $\varepsilon > 0$ small, the positive solution S^* is also small. Expand the exponential to second order in S:

$$S = 1 - e^{-\langle k\rangle S} \approx 1 - \left(1 - \langle k\rangle S + \frac{\langle k\rangle^2 S^2}{2} - \cdots\right) = \langle k\rangle S - \frac{\langle k\rangle^2 S^2}{2} + O(S^3).$$

Rearranging (and discarding $O(S^3)$ since S is small):

$$S - \langle k\rangle S + \frac{\langle k\rangle^2 S^2}{2} \approx 0 \implies S\left(1 - \langle k\rangle + \frac{\langle k\rangle^2 S}{2}\right) = 0.$$

The non-zero solution satisfies $\frac{\langle k\rangle^2 S}{2} \approx \langle k\rangle - 1 = \varepsilon$, giving

$$\boxed{S \approx \frac{2(\langle k\rangle - 1)}{\langle k\rangle^2} \approx 2(\langle k\rangle - 1) \qquad \text{as } \langle k\rangle \downarrow 1^+.}$$

The transition is **continuous**: the giant component size grows linearly with the distance $\varepsilon = \langle k\rangle - 1$ from the threshold. There is no jump discontinuity in S — the giant component emerges gradually from zero.

> **Physical interpretation**
>
> The linear growth $S \approx 2(\langle k\rangle - 1)$ means that doubling the excess degree doubles the fraction of vertices absorbed into the giant component, at least near the threshold. Far above the threshold, S saturates toward 1 as the graph approaches full connectivity.

5.6.3 Connectedness threshold

Connectivity requires more than a giant component: every vertex must be reachable from every other vertex. The binding constraint is isolated vertices. A vertex v is isolated if none of its $N - 1$ potential edges is present, which happens with probability $(1 - p)^{N-1}$. The expected number of isolated vertices is

$$\mathbb{E}[\text{isolated}] = N(1 - p)^{N-1} \approx Ne^{-p(N-1)} = Ne^{-\langle k\rangle}.$$

Setting this equal to 1 gives $\langle k\rangle = \ln N$, or equivalently $p = \ln N/N$. This is the **connectedness threshold**: below it, isolated vertices are expected and the graph is disconnected; above it, isolated vertices disappear and the graph becomes connected with high probability.

More precisely (Erdős & Rényi, 1960; Bollobás, 2001, Ch. 7), for $p = (\ln N + c)/N$:

$$P(G(N,p) \text{ is connected}) \xrightarrow{N\to\infty} e^{-e^{-c}}.$$

Setting $c = 0$ gives $P \to e^{-1} \approx 0.368$; the graph is connected with probability $1/2$ when $c = \ln\ln 2 \approx -0.37$.

5.7 Numerical exploration

```
set.seed(2026)
n    <- 500
k_grid <- seq(0.1, 8.0, length.out=45)
p_grid <- k_grid / (n - 1)
reps   <- 80

evolution_tbl <- map_dfr(p_grid, function(p) {
  map_dfr(seq_len(reps), function(rep_id) {
    g    <- sample_gnp(n=n, p=p, directed=FALSE, loops=FALSE)
    comp <- components(g)$csize
    tibble(
      p                          = p,
      mean_degree                = p*(n-1),
      rep                        = rep_id,
      largest_component_fraction = max(comp)/n,
      number_of_components       = length(comp),
      isolated_fraction          = sum(comp==1)/n,
      connected_indicator        = as.integer(length(comp)==1)
    )
  })
})

evolution_summary <- evolution_tbl |>
  group_by(p, mean_degree) |>
  summarise(
    largest_component_fraction = mean(largest_component_fraction),
    number_of_components       = mean(number_of_components),
    isolated_fraction          = mean(isolated_fraction),
    connected_probability      = mean(connected_indicator),
    connected_probability_se   = sqrt(connected_probability*(1-connected_probability)/n()),
    .groups = "drop"
  )

iso_fit <- isoreg(evolution_summary$mean_degree,
                  evolution_summary$connected_probability)
evolution_summary$connected_probability_iso <- iso_fit$yf
```

5.7.1 Giant component fraction

```
# Theoretical S from self-consistency equation
solve_S <- function(k_mean, tol=1e-8, max_iter=1000) {
  if (k_mean <= 0) return(0)
  S <- 0.5
  for (i in seq_len(max_iter)) {
```

```
    S_new <- 1 - exp(-k_mean * S)
    if (abs(S_new - S) < tol) return(S_new)
    S <- S_new
  }
  S
}
theory_tbl <- tibble(
  mean_degree = seq(0.1, 8.0, length.out=200),
  S_theory    = sapply(mean_degree, solve_S)
)

ggplot(evolution_summary, aes(x=mean_degree, y=largest_component_fraction)) +
  geom_point(color=course_cols$gold, size=2.1, alpha=0.8) +
  geom_line(data=theory_tbl, aes(x=mean_degree, y=S_theory),
            color=course_cols$teal, linewidth=1.2) +
  geom_vline(xintercept=1, color=course_cols$coral,
             linewidth=1.0, linetype=2) +
  geom_vline(xintercept=log(n), color=course_cols$blue,
             linewidth=1.0, linetype=3) +
  labs(title="Emergence of the giant component",
       subtitle="Teal curve: theoretical S from self-consistency equation",
       x=expression(paste("Expected degree ", langle*k*rangle)),
       y="Largest component fraction S")
```

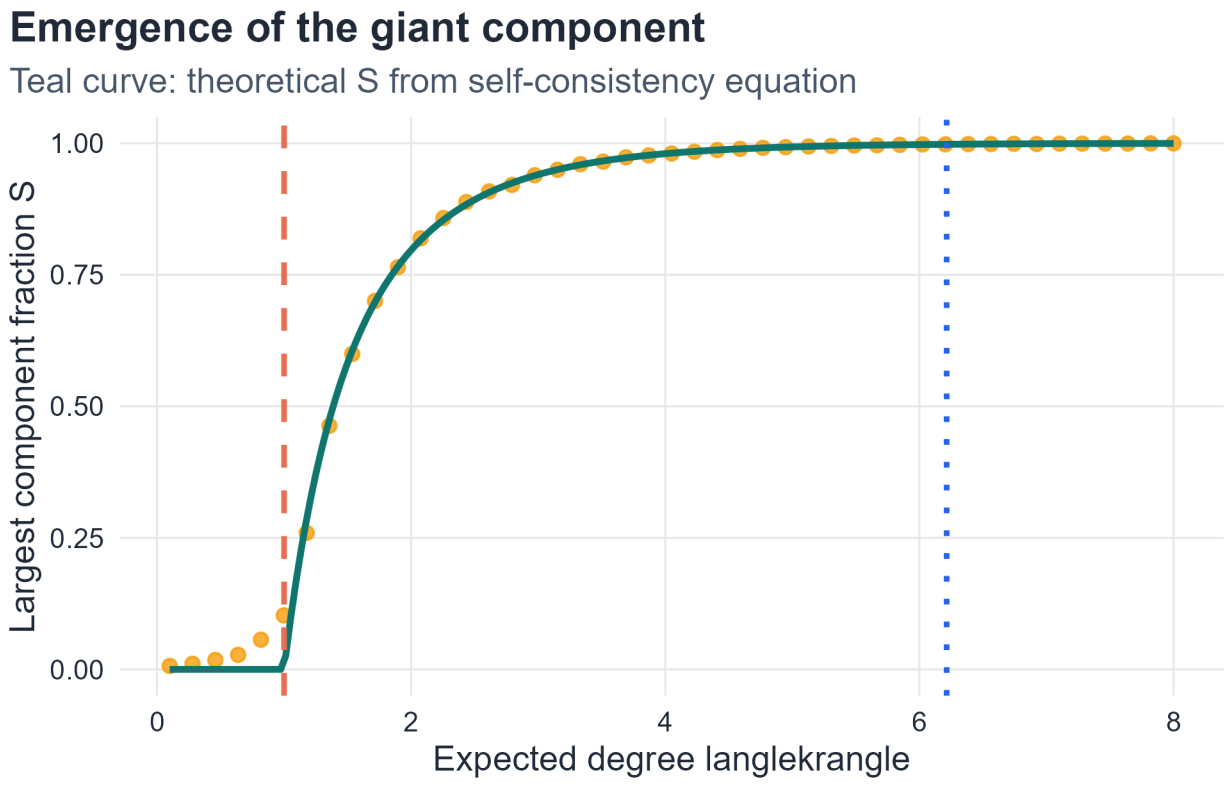


Figure 5.3: Average fraction of vertices in the largest component as a function of expected degree. The theoretical curve from the self-consistency equation is overlaid. Red dashed: giant-component threshold. Blue dotted: connectedness threshold.

5.7.2 Connectedness probability

```
ggplot(evolution_summary, aes(x=mean_degree)) +
  geom_ribbon(
    aes(ymin=pmax(connected_probability - 1.96*connected_probability_se, 0),
        ymax=pmin(connected_probability + 1.96*connected_probability_se, 1)),
    fill=course_cols$blue, alpha=0.12) +
  geom_point(aes(y=connected_probability), color=course_cols$gold, size=1.9) +
  geom_line(aes(y=connected_probability_iso), color=course_cols$coral, linewidth=1.2) +
  geom_vline(xintercept=1, color=course_cols$coral,
             linewidth=1.0, linetype=2) +
  geom_vline(xintercept=log(n), color=course_cols$blue,
             linewidth=1.0, linetype=3) +
  labs(title="Connectedness probability",
       subtitle=sprintf("N = %d; dashed = giant threshold; dotted = connectedness threshold (ln N = %.2f)", n, log(n)),
       x=expression(paste("Expected degree ", langle*k*rangle)),
       y="P(connected)")
```

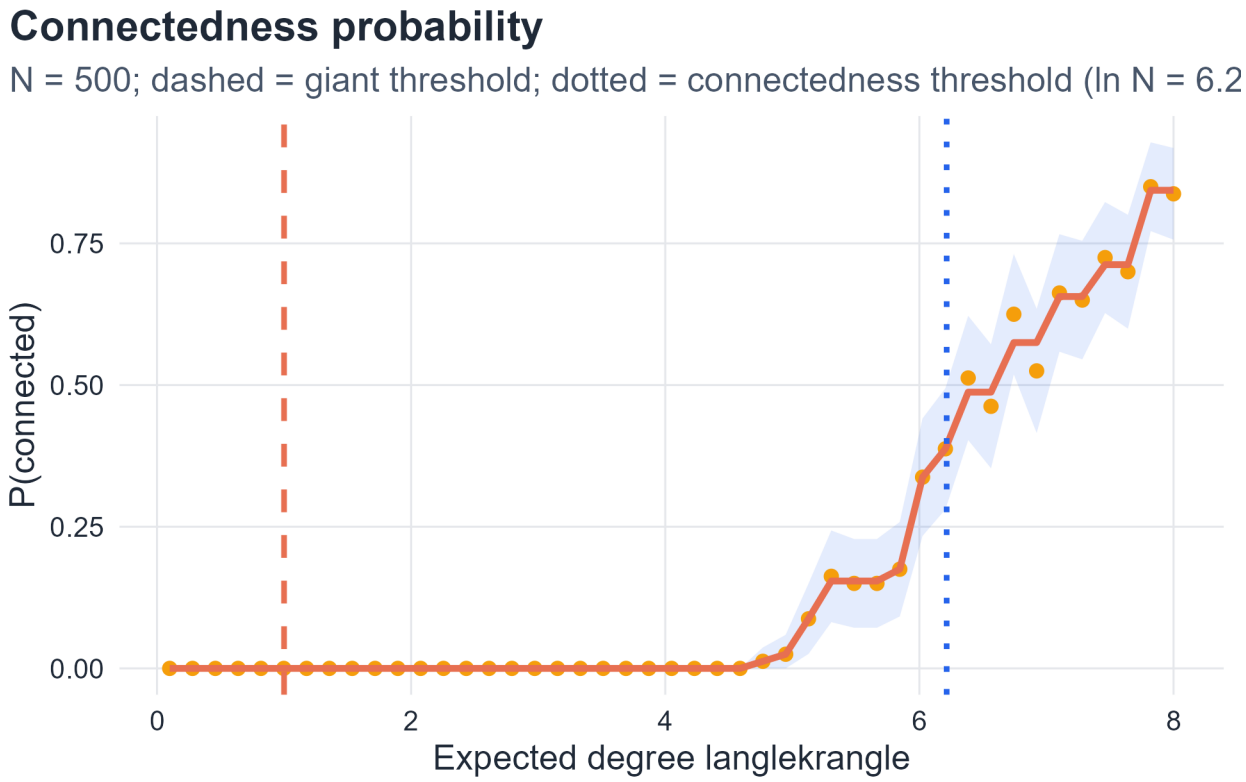


Figure 5.4: Empirical connectedness probability with 95% confidence bands. The monotone isotonic fit (solid line) enforces the theoretical constraint that P(connected) is non-decreasing in p.

The two plots together make the structural distinction explicit: the giant component fraction begins rising sharply near $\langle k \rangle = 1$, while the connectedness probability remains near zero until $\langle k \rangle \approx \ln N \approx 6.2$ for $N = 500$.

5.8 Summary of regimes

Regime	Condition	Largest component	Isolated vertices	Connected?
Subcritical	$\langle k \rangle < 1$	$O(\log N)$	Common	No
Critical	$\langle k \rangle \approx 1$	$O(N^{2/3})$	Present	No
Supercritical	$1 < \langle k \rangle < \ln N$	$\Theta(N)$	Decreasing	No
Connected	$\langle k \rangle \gtrsim \ln N$	N	Absent (w.h.p.)	Yes

5.9 Core formulas

! Summary

Result	Formula
Giant-component threshold	$\langle k \rangle = 1$, i.e., $p_c \approx 1/N$
Self-consistency equation	$S = 1 - e^{-\langle k \rangle S}$
Near-threshold size	$S \approx 2(\langle k \rangle - 1)$ for $\langle k \rangle \downarrow 1^+$
Connectedness threshold	$\langle k \rangle \approx \ln N$, i.e., $p \approx \ln N / N$
Isolated vertex count	$\mathbb{E}[\text{isolated}] \approx N e^{-\langle k \rangle}$

5.10 In-class questions

1. Why do the connected components of a graph partition the vertex set?
2. A graph has 1000 vertices and its largest component contains 600. Is this a giant component? Justify using the definition.
3. Why does $f'(0) > 1$ imply the existence of a positive fixed point of $f(S) = 1 - e^{-\langle k \rangle S}$?
4. Why does the connectedness threshold depend on $\ln N$ rather than a constant?
5. A student observes that a graph with $N = 100$ and $\langle k \rangle = 1.5$ has no isolated vertices. Does this mean the graph is connected? Explain.
6. In the near-threshold expansion, why is it valid to truncate the Taylor series at second order?

5.11 Exercises

The seven exercises below cover the full theoretical and computational content of this chapter. They require derivation, analysis, and synthesis — not reproduction of lecture material.

5.11.0.1 Exercise 1. Components, partitions, and the giant component criterion

1. Let $G = (V, E)$ be any graph. Prove that the connected components partition V: every vertex belongs to exactly one component. Your proof must use the definition of maximality precisely.
2. Let $G_N = P_N$ (the path graph on N vertices). Compute $|C_{\max}(G_N)|/N$ and determine whether P_N contains a giant component in the sense of Definition 5.2.
3. Let $G_N = \lfloor N/2 \rfloor \cdot K_2$ (a perfect matching: $\lfloor N/2 \rfloor$ disjoint edges). Compute $|C_{\max}(G_N)|/N$. Does this sequence contain a giant component?
4. Construct a sequence of graphs $\{G_N\}$ such that $|C_{\max}(G_N)|/N \to 1/3$. Is this a giant component? Compare to the Erdős–Rényi case: at what value of $\langle k \rangle$ does the self-consistency equation give $S = 1/3$?

5.11.0.2 Exercise 2. The self-consistency equation: analysis and phase transition

The self-consistency equation is $S = 1 - e^{-\langle k\rangle S}$, with $f(S) = 1 - e^{-\langle k\rangle S}$.

1. Show that $S = 0$ is always a solution. Show that any other fixed point $S^* > 0$ satisfies $S^* < 1$ for any finite $\langle k\rangle$.
2. Prove rigorously that a positive fixed point exists if and only if $\langle k\rangle > 1$. *(Hint: consider the slope $f'(0) = \langle k\rangle$ and the fact that $f(S) < S$ for all $S \in (0, 1)$ when $\langle k\rangle \leq 1$.)*
3. For $\langle k\rangle = 2$, find S^* numerically and verify that $S^* = 1 - e^{-2S^*}$. Repeat for $\langle k\rangle \in \{1.5, 3, 5\}$. Plot S^* as a function of $\langle k\rangle$ for $\langle k\rangle \in [0, 5]$, overlaying the simulation estimates from Figure 5.3.
4. The function $\langle k\rangle \mapsto S^*(\langle k\rangle)$ is continuous and equals zero for $\langle k\rangle \leq 1$. This continuity is why the transition is called **second-order** (continuous) rather than **first-order** (discontinuous). Explain what it would mean physically if S jumped discontinuously at $\langle k\rangle = 1$.

5.11.0.3 Exercise 3. Near-threshold expansion

The lecture derives $S \approx 2(\langle k\rangle - 1)$ near $\langle k\rangle = 1$.

1. Carry out the Taylor expansion of $e^{-\langle k\rangle S}$ to **third** order in S. Show that the correction to the leading term $S \approx 2(\langle k\rangle - 1)$ is of order $(\langle k\rangle - 1)^2$, so the linear approximation is valid to leading order.
2. Write $\langle k\rangle = 1 + \varepsilon$ and expand $S^*(\varepsilon)$ as a power series in ε: $S^* = a_1\varepsilon + a_2\varepsilon^2 + \cdots$. Determine a_1 and a_2.
3. For $N = 1000$ and $\langle k\rangle = 1.2$, compare: (a) the theoretical prediction $S \approx 2(\langle k\rangle - 1) = 0.4$; (b) the exact numerical solution of the self-consistency equation; and (c) the simulation estimate. Which is more accurate at this distance from the threshold?
4. The near-threshold formula is $S \approx 2\varepsilon$ with $\varepsilon = \langle k\rangle - 1$. The analogous quantity in percolation theory is $\varepsilon = p - p_c$, where p_c is the critical bond probability. Does the linear scaling $S \sim \varepsilon$ hold in bond percolation on a 2D lattice? *(You need not derive this — look it up and cite your source.)*

5.11.0.4 Exercise 4. The connectedness threshold

1. Show that the expected number of isolated vertices in $G(N, p)$ is $N(1-p)^{N-1}$. Using the approximation $(1-p)^{N-1} \approx e^{-p(N-1)}$, derive the condition under which the expected number of isolated vertices equals 1. Show this gives $p \approx \ln N/N$.
2. The exact result (Erdős & Rényi, 1960) is: for $p = (\ln N + c)/N$, $P(G(N, p) \text{ connected}) \to e^{-e^{-c}}$. Verify numerically for $N = 500$: (a) estimate the probability of connectivity by simulation for several values of $c \in [-2, 3]$; (b) overlay the theoretical curve $e^{-e^{-c}}$; (c) comment on the agreement.
3. Express the ratio of the connectedness threshold to the giant-component threshold as a function of N:

$$\frac{\langle k\rangle_{\text{connected}}}{\langle k\rangle_{\text{giant}}} = \frac{\ln N}{1}.$$

For $N = 10^3$, 10^6, 10^9 (sizes of typical real-world networks), compute this ratio. What does this say about the range of $\langle k\rangle$ in which a network has a giant component but is not fully connected?

4. The connectedness threshold is driven by isolated vertices — the **last** structural obstacle to full connectivity. Demonstrate this numerically: for $N = 500$, plot simultaneously (a) the fraction of isolated vertices and (b) the connectedness probability as functions of $\langle k\rangle$. Show that the two curves cross near the same threshold.

5.11.0.5 Exercise 5. Structural regimes: classification and transitions

1. A social network has $N = 10^5$ nodes and average degree $\langle k \rangle = 3.2$. Classify it into one of the four regimes of Section 5.5.1. Under the Erdős–Rényi model with the same parameters, compute: (a) the theoretical giant component fraction S^*; (b) the expected number of isolated vertices; (c) whether the graph is likely connected.
2. A network analyst reports observing two networks with the same $N = 500$ and $\langle k \rangle = 4$, but different largest component fractions: Network A has $S = 0.85$, Network B has $S = 0.43$. Under the ER model, S^* is uniquely determined by $\langle k \rangle$. What does the discrepancy between Networks A and B suggest about their structure relative to the ER null model?
3. The **second-largest** component in the supercritical ER model has size $O(\log N)$ — much smaller than the giant. Verify this numerically for $N = 1000$ and $\langle k \rangle \in \{1.5, 2, 3\}$: generate 100 graphs at each value and plot the distribution of second-largest component sizes. Does the $O(\log N)$ prediction match?
4. In the critical regime $\langle k \rangle = 1$, the largest component has size $O(N^{2/3})$. For $N \in \{100, 500, 1000, 5000\}$, simulate 100 graphs at $\langle k \rangle = 1$ and plot the mean largest component size against N on a log-log scale. Estimate the exponent empirically and compare with the theoretical value $2/3$.

5.11.0.6 Exercise 6. Finite-size effects and convergence

The thresholds derived in this chapter are sharp in the limit $N \to \infty$. For finite networks, transitions are rounded.

1. For $N \in \{100, 500, 1000, 5000\}$, simulate the largest-component fraction over a grid of $\langle k \rangle \in [0, 4]$. Plot all four curves on the same axes. How does the sharpness of the transition near $\langle k \rangle = 1$ change with N? Define a quantitative measure of sharpness (e.g., the width of the interval where S goes from 0.1 to 0.9) and compute it for each N.
2. Define the **empirical threshold** $\hat{k}_c(N)$ as the value of $\langle k \rangle$ at which the largest-component fraction first exceeds 0.5 in simulation. Plot $\hat{k}_c(N)$ against N for $N \in \{50, 100, 250, 500, 1000\}$. Does $\hat{k}_c(N) \to 1$ as N grows? At what rate?
3. In finite graphs, the transition is governed by fluctuations of order $N^{-1/3}$ around the critical point — a result from the theory of random graph evolution (Bollobás, 2001, Ch. 5). Explain qualitatively why larger graphs have sharper transitions, using the law of large numbers as an analogy.

5.11.0.7 Exercise 7. The ER model as a null model for connectivity

A network scientist observes a real network with $N = 500$ vertices, $M = 800$ edges, and a largest component containing 480 vertices.

1. Fit an ER null model by computing $\langle k \rangle = 2M/N$ and the corresponding p. Solve the self-consistency equation to get the theoretical S^*. How does it compare to the observed value $480/500 = 0.96$?
2. Simulate 500 realisations of $G(N, p)$ with your fitted parameters. Plot the distribution of the largest component fraction. Where does the observed value 0.96 fall in this distribution?
3. For the fitted model, compute: (a) the expected number of isolated vertices; (b) the probability that the graph is connected; (c) the expected size of the second-largest component. Compare each to what you would observe in the real network if it were ER.
4. Now suppose the real network has clustering $\bar{C} = 0.35$ whereas the ER model predicts $\mathbb{E}[\bar{C}] = p \approx 0.006$. The large clustering coefficient is known to reduce the effective size of the giant component relative to the ER prediction (Newman, 2010, Ch. 13). Explain intuitively why high clustering — many triangles — makes it harder for the giant component to span the network efficiently.

5.12 Bridge to the next chapter

The Erdős–Rényi process shows how a giant component emerges from uniform random wiring, but it produces networks with two properties that contradict empirical observations: negligibly low clustering ($\mathbb{E}[\bar{C}] = p \to 0$ in the sparse regime) and short average path length of order $\ln N / \ln\langle k \rangle$. Real networks — social, biological, technological — simultaneously exhibit much higher clustering and comparably short path lengths (Milgram, 1967; Watts & Strogatz, 1998). This combination, which cannot be produced by either a regular lattice or an Erdős–Rényi graph alone, defines the **small-world** property studied in the next chapter (Strogatz, 2001; Watts & Strogatz, 1998).

5.13 Concluding remarks

The evolution of a random graph is one of the clearest examples of emergent global structure arising from simple local randomness. Starting from a collection of isolated vertices, adding edges at random produces first small tree-like components, then a macroscopic giant component at $\langle k \rangle = 1$, and finally a fully connected graph at $\langle k \rangle = \ln N$. Each transition is governed by a precise mathematical threshold, derived from the self-consistency equation and the isolated vertex argument (Bollobás, 2001; Erdős & Rényi, 1960).

The two thresholds are conceptually distinct: the giant component threshold reflects the onset of global reachability for a positive fraction of vertices, while the connectedness threshold reflects the elimination of the last isolated pieces. Between them lies a wide supercritical regime that is the most relevant for real sparse networks: $\langle k \rangle$ is typically of order 3–10 for biological and social networks, well above 1 but far below $\ln N$ for large N. Understanding the structure of this intermediate regime — giant component size, second-largest component, cycle structure — provides the analytical foundations for the models studied in subsequent chapters.

Chapter 6

Small-World Networks

Short Paths, High Clustering, and the Watts–Strogatz Model

6.1 Introduction

The small-world phenomenon is one of the central ideas of modern network science. In everyday language it appears in phrases such as *six degrees of separation*: even in a very large social system, two people may be connected by only a short chain of acquaintances (Milgram, 1967; Watts & Strogatz, 1998). The mathematical content of the idea, however, is broader and more precise than the slogan itself.

The real question is not whether the number is exactly six. The important structural observation is that in many large networks the typical distance between vertices is surprisingly small. This raises an immediate benchmark question: small relative to what? The answer only becomes clear when we compare different classes of graphs.

Random graphs typically have short path lengths, often growing roughly logarithmically with network size (Newman, 2010; Strogatz, 2001). Regular lattices, by contrast, have strong local order but much larger distances, with polynomial growth in system size. Real networks often combine the most interesting aspects of both worlds: they retain strong local clustering while also exhibiting short global distances (Strogatz, 2001; Watts & Strogatz, 1998).

This chapter follows that logical progression. We first define the basic metric quantities, then explain why random graphs naturally produce short paths, then show why that result is surprising from the perspective of lattice geometry, and finally introduce the Watts–Strogatz model as a mechanism that combines short paths with high clustering. The goal is not only to define a small-world network, but to understand why the phenomenon is mathematically nontrivial and structurally important.

6.2 Chapter roadmap

This chapter develops in the following order:

1. metric quantities: distance, diameter, average path length, and clustering
2. logarithmic distance scaling in random graphs
3. the contrast with regular lattices
4. the Watts–Strogatz construction and the role of shortcuts
5. numerical identification of the small-world regime
6. the strengths and limitations of the model

6.3 Learning goals

By the end of this chapter, it should be possible to:

- define graph distance, diameter, average path length, and clustering coefficient precisely
- explain the small-world property in mathematical terms
- derive the logarithmic distance estimate for random-like networks
- explain why the small-world effect is surprising relative to lattice geometry
- define the Watts–Strogatz model and interpret its parameters
- explain why shortcuts reduce distances much faster than they reduce clustering
- identify the small-world regime numerically in `R`
- discuss what the Watts–Strogatz model explains well and what it does not explain

6.4 Distance, diameter, average path length, and clustering

6.4.1 Distance

Definition 6.1. Let $G = (V, E)$ be a connected undirected graph. For vertices $u, v \in V$, the **distance** between u and v, denoted by

$$d(u, v),$$

is the length of a shortest path from u to v.

Distance is the basic metric quantity in a graph. It counts the minimum number of edges that must be traversed to move from one vertex to another. If two vertices are adjacent, then their distance is 1; if they share a common neighbor but are not adjacent, then their distance is 2.

6.4.2 Diameter

Definition 6.2. Let $G = (V, E)$ be a connected graph. The **diameter** of G is

$$D(G) = \max_{u,v \in V} d(u, v).$$

The diameter measures the largest distance between any pair of vertices. It is therefore a notion of the graph's maximum metric extent. Because it depends on extreme pairs, it can be sensitive to rare structural features.

6.4.3 Average path length

Definition 6.3. Let $G = (V, E)$ be a connected graph with $N = |V|$ vertices. The **average path length** of G is

$$L(G) = \frac{1}{N(N-1)} \sum_{\substack{u,v \in V \\ u \neq v}} d(u, v).$$

This quantity measures the typical separation between two vertices. In practice it is often more informative than the diameter because it describes typical rather than extreme reachability.

6.4.4 Why average path length is usually more stable

The distinction between diameter and average path length is important.

- The diameter is controlled by a small number of extreme paths.
- The average path length is computed over all ordered vertex pairs.

For this reason, average path length is usually much more stable in both theory and simulation. In this chapter, it will be our main distance observable.

6.4.5 Clustering coefficient

Definition 6.4. Let $G = (V, E)$ be an undirected graph. The **average clustering coefficient** of G is

$$\bar{C}(G) = \frac{1}{N} \sum_{i=1}^{N} C_i,$$

where C_i denotes the local clustering coefficient of vertex i.

Clustering is a local measure. It asks whether neighbors of a vertex also tend to be connected to one another, thereby closing triangles. The central question of the chapter is whether a network can have, at the same time,

- small global distances, and
- strong local clustering.

That coexistence is the essence of the small-world phenomenon (Strogatz, 2001; Watts & Strogatz, 1998).

6.5 Why random graphs can have short distances

A first explanation of small distances comes from a branching approximation. The logic is rough but powerful.

6.5.1 A branching approximation

Suppose a network has average degree $\langle k \rangle$ and that local neighborhoods are approximately tree-like. Starting from one vertex, one expects roughly:

- about $\langle k \rangle$ vertices at distance 1
- about $\langle k \rangle^2$ vertices at distance 2
- about $\langle k \rangle^3$ vertices at distance 3
- and in general about $\langle k \rangle^d$ vertices at distance d

This argument ignores neighborhood overlap, loops, and degree fluctuations, so it is not exact. But it captures the main mechanism: when the number of reachable vertices grows exponentially with distance, only a small number of steps is needed to cover a large network (Newman, 2010).

6.5.2 Number of vertices within distance d

Under the same approximation,

$$N(d) \approx 1 + \langle k \rangle + \langle k \rangle^2 + \cdots + \langle k \rangle^d.$$

This is a geometric series, so

$$N(d) \approx \frac{\langle k \rangle^{d+1} - 1}{\langle k \rangle - 1}.$$

When $\langle k \rangle > 1$, the number of reachable vertices expands very quickly with distance.

6.5.3 Logarithmic estimate for distance

Since $N(d)$ cannot exceed the total number of vertices N, the largest relevant distance scale satisfies approximately

$$N(d_{\max}) \approx N.$$

For large N, the final term dominates the geometric sum, giving the simplified relation

$$\langle k \rangle^{d_{\max}} \approx N.$$

Taking logarithms yields

$$d_{\max} \approx \frac{\ln N}{\ln \langle k \rangle}.$$

This is the first mathematical form of the small-world phenomenon. In many random-like networks, the average path length obeys a comparable logarithmic law (Newman, 2010; Strogatz, 2001).

> **Key idea**
>
> When neighborhoods grow approximately exponentially with distance, metric distances grow only logarithmically with system size.

6.5.4 A rough social-network illustration

Take very rough values

$$N \approx 7 \times 10^9, \qquad \langle k \rangle \approx 10^3.$$

Then

$$\frac{\ln(7 \times 10^9)}{\ln(10^3)} \approx 3.28.$$

The exact number is not the point. The point is the growth law: logarithmic scaling makes it plausible that a huge system may still have surprisingly short paths.

6.6 Why this is surprising: the lattice benchmark

The phrase *small world* only becomes mathematically interesting when we compare random-like logarithmic scaling with the slower growth of regular lattices.

6.6.1 Distance scaling in regular lattices

In a one-dimensional lattice,

$$L(G) \sim D(G) \sim N.$$

In a two-dimensional square lattice,

$$L(G) \sim D(G) \sim N^{1/2}.$$

In a three-dimensional cubic lattice,

$$L(G) \sim D(G) \sim N^{1/3}.$$

More generally, in a d-dimensional regular lattice,

$$L(G) \sim D(G) \sim N^{1/d}.$$

These are polynomial growth laws, not logarithmic ones.

6.6.2 Why the contrast matters

If a large system were organized like a low-dimensional lattice, distances would grow much faster than in a random graph. The small-world effect is therefore not merely the fact that distances are finite. It is the much stronger statement that they are dramatically smaller than what geometric lattice intuition would suggest (Strogatz, 2001; Watts & Strogatz, 1998).

6.7 Simulation: scaling of average path length

The following simulation compares the growth of average path length across four model families:

- a one-dimensional lattice
- a two-dimensional lattice
- a three-dimensional lattice
- a connected Erdős–Rényi random graph with expected degree about 8

The goal is not to force all families to use exactly the same values of N. The goal is to reveal the scaling trend clearly.

```
mean_apl <- function(g) {
  igraph::mean_distance(g, directed = FALSE, unconnected = FALSE)
}

simulate_connected_er_apl <- function(n, mean_degree = 8, reps = 6) {
  p <- mean_degree / (n - 1)

  vals <- replicate(reps, {
    repeat {
      g <- igraph::sample_gnp(n, p, directed = FALSE, loops = FALSE)
      if (igraph::components(g)$no == 1) break
    }
    mean_apl(g)
  })
```

```
  mean(vals)
}

apl_lattice <- function(...) {
  g <- igraph::make_lattice(dimvector = c(...), periodic = FALSE)
  mean_apl(g)
}

line_sizes   <- c(20, 30, 40, 50, 60, 80, 100)
cube_sides   <- c(3, 4, 5, 6, 7, 8)
random_sizes <- c(20, 30, 40, 50, 60, 80, 100, 140, 180, 240, 320)

rect_dims <- tibble::tribble(
  ~a, ~b,
   4,  4,
   5,  5,
   6,  6,
   7,  7,
   8,  8,
   9,  9,
  10, 10,
  10, 12,
  12, 12,
  12, 15,
  15, 15,
  18, 18
)

distance_scaling_tbl <- dplyr::bind_rows(
  tibble::tibble(
    N = line_sizes,
    avg_distance = purrr::map_dbl(line_sizes, ~ apl_lattice(.x)),
    model = "1D lattice"
  ),
  tibble::tibble(
    N = rect_dims$a * rect_dims$b,
    avg_distance = purrr::map2_dbl(rect_dims$a, rect_dims$b, apl_lattice),
    model = "2D lattice"
  ),
  tibble::tibble(
    N = cube_sides^3,
    avg_distance = purrr::map_dbl(cube_sides, ~ apl_lattice(.x, .x, .x)),
    model = "3D lattice"
  ),
  tibble::tibble(
    N = random_sizes,
    avg_distance = purrr::map_dbl(random_sizes, ~ simulate_connected_er_apl(.x, mean_degree = 8, reps =
    model = "Random graph"
  )
) |>
  dplyr::mutate(
    model = factor(model, levels = c("1D lattice", "2D lattice", "3D lattice", "Random graph")),
    logN = log(N),
```

```
    logd = log(avg_distance)
  )
```

```
ggplot(distance_scaling_tbl, aes(x = N, y = avg_distance, color = model)) +
  geom_line(linewidth = 1.15) +
  geom_point(size = 2.2) +
  scale_color_manual(
    values = c(
      "1D lattice"   = course_cols$blue,
      "2D lattice"   = course_cols$teal,
      "3D lattice"   = course_cols$gold,
      "Random graph" = course_cols$slate
    )
  ) +
  labs(
    title = "Average path length versus system size",
    subtitle = "Lattice models show polynomial growth, while the random graph grows much more slowly",
    x = "Number of vertices, N",
    y = "Average path length",
    color = NULL
  )
```

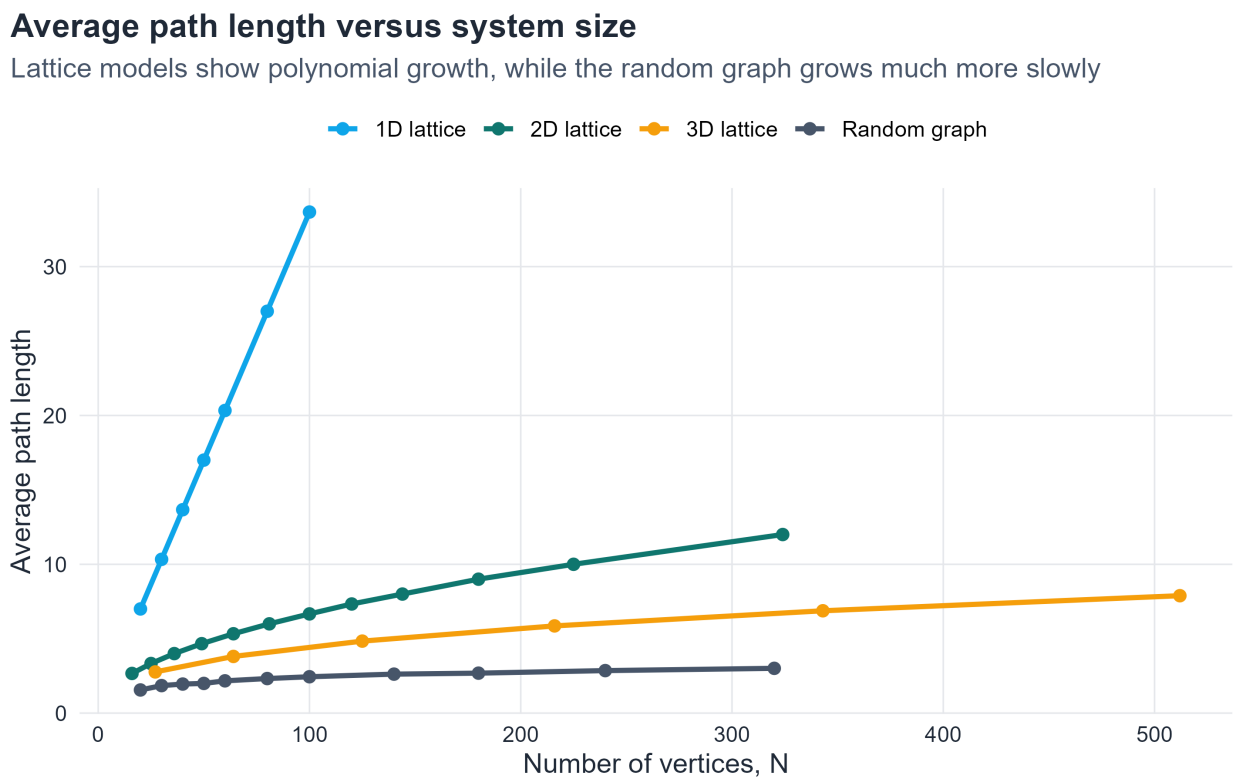


Figure 6.1: Average path length as a function of system size for one-, two-, and three-dimensional lattices and for connected random graphs.

As shown in Figure 6.1, the one-dimensional lattice grows fastest, the two-dimensional and three-dimensional lattices grow more slowly, and the random graph grows far more slowly still. Even on a linear scale, the contrast is already visible.

```r
ggplot(distance_scaling_tbl, aes(x = logN, y = logd, color = model)) +
  geom_line(linewidth = 1.1) +
  geom_point(size = 2.0) +
  scale_color_manual(
    values = c(
      "1D lattice"   = course_cols$blue,
      "2D lattice"   = course_cols$teal,
      "3D lattice"   = course_cols$gold,
      "Random graph" = course_cols$slate
    )
  ) +
  labs(
    title = "A logarithmic view of distance scaling",
    subtitle = "The lattice families follow polynomial trends; the random graph remains much more compa
    x = "ln N",
    y = "ln average path length",
    color = NULL
  )
```

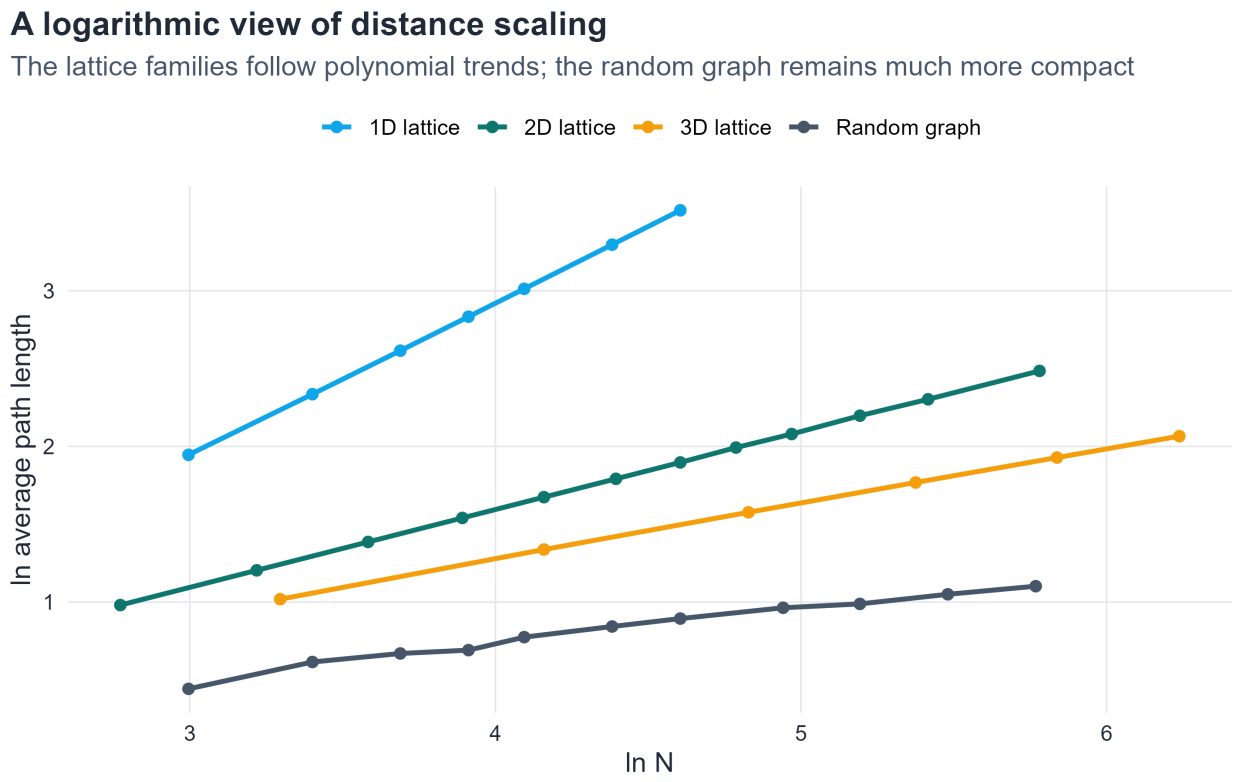


Figure 6.2: The same simulation displayed on logarithmic axes. The lattice curves are consistent with polynomial growth, while the random graph is consistent with much slower, near-logarithmic growth.

Figure Figure 6.2 makes the growth laws easier to read. The lattice families align with the expected polynomial scaling, while the random graph remains far below them. This is the structural meaning of the small-world effect: large networks can remain metrically compact when even a modest amount of randomness is present.

6.8 The Watts–Strogatz model

The previous section explains why short distances can arise in random-like networks, but it does not explain how a network can keep high clustering at the same time. That is the problem addressed by the Watts–Strogatz model (Watts & Strogatz, 1998).

6.8.1 Motivation

Two empirical observations often appear together in real systems:

1. short average path lengths, and
2. clustering much higher than in a comparable random graph.

These point in different structural directions.

- A regular lattice has strong local order and therefore high clustering, but large path lengths.
- A random graph has short path lengths, but typically much lower clustering.

The Watts–Strogatz model interpolates between these two extremes. It shows that a graph can maintain local order while acquiring short global distances after only a small amount of random rewiring (Strogatz, 2001; Watts & Strogatz, 1998).

6.8.2 Parameters of the model

The standard parameters are:

- N: number of vertices
- K: degree of each vertex in the initial ring lattice, assumed even
- $\beta \in [0, 1]$: rewiring probability

The useful regime is

$$2 \leq K < N, \qquad K \text{ even},$$

and in asymptotic discussions one often assumes

$$N \gg K \gg \ln N \gg 1.$$

6.8.3 Regular ring lattice

Definition 6.5. Let N and K be as above, with K even. The **regular ring lattice** is the graph on vertex set

$$V = \{0, 1, \dots, N-1\}$$

in which each vertex is connected to its $K/2$ nearest neighbors on each side around the ring.

Every vertex has degree exactly K, so the average degree is

$$\langle k \rangle = K.$$

This graph is highly ordered and has substantial clustering, but travel across the graph still requires many local steps.

6.8.4 Definition of the Watts–Strogatz model

Definition 6.6. The **Watts–Strogatz model** with parameters (N, K, β) is constructed as follows.

1. Start from the regular ring lattice on N vertices with degree K.
2. For each vertex, consider only the $K/2$ clockwise edges.
3. Independently, with probability β, rewire each such edge to a uniformly chosen new endpoint.
4. Self-loops and multiple edges are forbidden.

Acting only on the clockwise half of the edges prevents the same undirected edge from being treated twice.

6.8.5 Interpretation of the rewiring probability

The parameter β controls the amount of randomness.

- If $\beta = 0$, the graph remains a regular lattice.
- If $0 < \beta \ll 1$, only a small fraction of edges is rewired, creating a few long-range shortcuts.
- If $\beta = 1$, the graph is highly randomized.

Because the number of edges is preserved, the mean degree remains equal to K throughout the interpolation.

6.8.6 Ring-lattice benchmarks

For the regular ring lattice one has the approximate path-length formula

$$L(0) \approx \frac{N}{2K}$$

and the exact clustering coefficient

$$C(0) = \frac{3(K-2)}{4(K-1)}.$$

Thus the ring lattice starts from

- large clustering, and
- large average path length.

That is exactly why it is a useful starting point for the model (Watts & Strogatz, 1998).

6.9 Visualizing the transition from order to randomness

The next code block builds a progressive rewiring construction. The purpose is pedagogical: the sequence will look like a genuine evolution from order to randomness while keeping the vertex positions fixed.

```
make_ring_edges_K <- function(n, K) {
  stopifnot(K %% 2 == 0)
  tibble::tibble(
    from = rep(seq_len(n), each = K / 2),
    shift = rep(seq_len(K / 2), times = n)
  ) |>
    dplyr::mutate(to = ((from + shift - 1) %% n) + 1) |>
    dplyr::select(from, to)
}

build_progressive_ws_graph_K <- function(n, K, beta, meta) {
  base_edges <- meta$base_edges
```

```
  adj <- matrix(FALSE, n, n)

  for (i in seq_len(nrow(base_edges))) {
    from <- base_edges$from[i]
    original_to <- base_edges$to[i]

    if (meta$u[i] <= beta) {
      candidates <- meta$candidates[[i]]
      idx <- which(!adj[from, candidates] & candidates != from)
      to <- if (length(idx) == 0) original_to else candidates[idx[1]]
    } else {
      to <- original_to
    }

    adj[from, to] <- TRUE
    adj[to, from] <- TRUE
  }

  igraph::graph_from_adjacency_matrix(adj, mode = "undirected", diag = FALSE)
}

make_edge_df <- function(g, K, nodes_df) {
  edge_pairs <- igraph::ends(g, igraph::E(g), names = FALSE) |>
    as.data.frame() |>
    stats::setNames(c("from", "to")) |>
    tibble::as_tibble()

  edge_pairs |>
    dplyr::mutate(
      cyclic_dist = purrr::map2_dbl(from, to, \(a, b) {
        d <- abs(a - b)
        min(d, igraph::vcount(g) - d)
      }),
      edge_type = dplyr::if_else(cyclic_dist <= K / 2, "local", "rewired")
    ) |>
    dplyr::left_join(nodes_df |> dplyr::rename(from = id, x = x, y = y), by = "from") |>
    dplyr::left_join(nodes_df |> dplyr::rename(to = id, xend = x, yend = y), by = "to")
}

plot_progressive_ws <- function(beta_value, title_text, n_vis = 32, K_vis = 4, ws_meta_vis, nodes_df) {
  g <- build_progressive_ws_graph_K(n_vis, K_vis, beta_value, ws_meta_vis)
  edges_df <- make_edge_df(g, K_vis, nodes_df)

  local_edges <- dplyr::filter(edges_df, edge_type == "local")
  rewired_edges <- dplyr::filter(edges_df, edge_type == "rewired")

  ggplot() +
    geom_curve(
      data = local_edges,
      aes(x = x, y = y, xend = xend, yend = yend),
      curvature = 0.38,
      linewidth = 0.55,
      colour = course_cols$teal,
```

```
      lineend = "round"
    ) +
    geom_curve(
      data = rewired_edges,
      aes(x = x, y = y, xend = xend, yend = yend),
      curvature = -0.18,
      linewidth = 0.8,
      colour = course_cols$gold,
      lineend = "round"
    ) +
    geom_point(
      data = nodes_df,
      aes(x = x, y = y),
      size = 2.8,
      shape = 21,
      fill = course_cols$blue,
      colour = "white",
      stroke = 0.5
    ) +
    coord_equal(
      xlim = c(-1.08, 1.08),
      ylim = c(-1.08, 1.08),
      expand = FALSE,
      clip = "off"
    ) +
    labs(title = title_text) +
    theme_void() +
    theme(
      plot.title = element_text(hjust = 0.5, size = 10.5, face = "bold", margin = margin(b = 2)),
      plot.margin = margin(0, 0, 0, 0)
    )
}
```

```
set.seed(2032)

n_vis <- 32
K_vis <- 4
beta_vis <- c(0, 0.01, 0.05, 0.20, 1.00)

base_edges_vis <- make_ring_edges_K(n_vis, K_vis)

ws_meta_vis <- list(
  base_edges = base_edges_vis,
  u = runif(nrow(base_edges_vis)),
  candidates = lapply(
    seq_len(nrow(base_edges_vis)),
    function(i) sample(setdiff(seq_len(n_vis), base_edges_vis$from[i]))
  )
)

g0 <- build_progressive_ws_graph_K(n_vis, K_vis, 0, ws_meta_vis)
layout_ws <- igraph::layout_in_circle(g0)

nodes_df <- tibble::tibble(
```

```
  id = seq_len(n_vis),
  x = layout_ws[, 1],
  y = layout_ws[, 2]
)

ws_plots <- purrr::map(
  beta_vis,
  ~ plot_progressive_ws(.x, paste0("\u03B2 = ", sprintf("%.2f", .x)), n_vis, K_vis, ws_meta_vis, nodes_
)

patchwork::wrap_plots(ws_plots, nrow = 1)
```

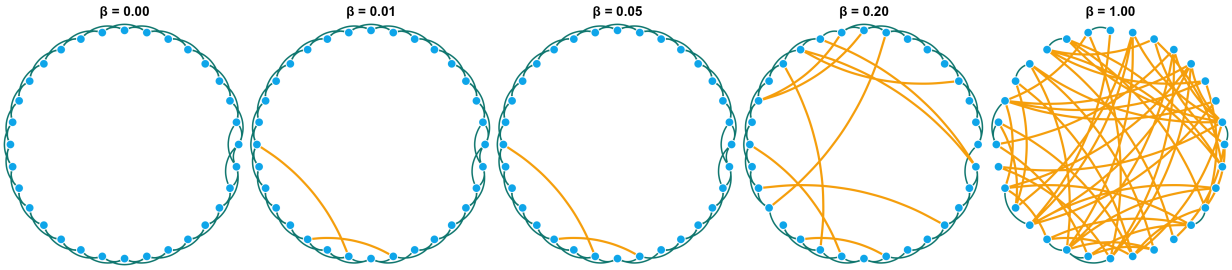


Figure 6.3: From regular lattice to random network in the Watts–Strogatz model. The same vertex positions are used throughout. Local edges are shown in teal and rewired shortcut edges in gold.

Figure Figure 6.3 makes the mechanism visually transparent. At $\beta = 0$, every edge is local and the graph is fully ordered. For small positive β, only a few edges are rewired, yet those few shortcuts already connect distant parts of the ring. As β increases, the graph loses its geometric appearance and becomes progressively more random.

6.9.1 Why shortcuts are so effective

A shortcut has a disproportionate effect on path length. In a ring lattice, moving from one side of the graph to another requires many local steps. A single long-range edge can bypass a large part of that travel.

Clustering behaves differently. If only a small fraction of edges is rewired, most local neighborhoods still resemble the original lattice, so clustering changes much more slowly.

> **!** Core asymmetry of the Watts–Strogatz model
>
> A small amount of random rewiring has a much stronger effect on path length than on clustering.
> - Path length drops sharply because shortcuts bypass long lattice segments.
> - Clustering decreases slowly because most local neighborhoods remain lattice-like.

This asymmetry is the structural source of the small-world regime (Strogatz, 2001; Watts & Strogatz, 1998).

6.10 Numerical identification of the small-world regime

To identify the small-world regime quantitatively, it is useful to normalize both observables relative to their lattice values:

$$\frac{L(\beta)}{L(0)}, \qquad \frac{C(\beta)}{C(0)}.$$

A small-world regime is characterized by a substantial drop in normalized path length while normalized clustering remains comparatively high.

```
sample_connected_ws_K <- function(size, K, beta, max_iter = 100) {
  stopifnot(K %% 2 == 0)
  for (i in seq_len(max_iter)) {
    g <- igraph::sample_smallworld(dim = 1, size = size, nei = K / 2, p = beta)
    if (igraph::components(g)$no == 1) return(g)
  }
  stop("Could not generate a connected Watts-Strogatz graph.")
}
```

```
set.seed(2033)

n_ws <- 700
K_ws <- 10
beta_grid <- unique(c(0, 10^seq(-4, 0, length.out = 22)))
reps <- 8

ws_tbl <- purrr::map_dfr(beta_grid, function(beta) {
  purrr::map_dfr(seq_len(reps), function(rep_id) {
    g <- sample_connected_ws_K(size = n_ws, K = K_ws, beta = beta)

    tibble::tibble(
      beta = beta,
      rep = rep_id,
      clustering = igraph::transitivity(g, type = "average"),
      avg_distance = igraph::mean_distance(g, directed = FALSE, unconnected = FALSE)
    )
  })
})

ws_summary <- ws_tbl |>
  dplyr::group_by(beta) |>
  dplyr::summarise(
    clustering = mean(clustering),
    avg_distance = mean(avg_distance),
    .groups = "drop"
  )

C0 <- ws_summary$clustering[ws_summary$beta == 0]
L0 <- ws_summary$avg_distance[ws_summary$beta == 0]

ws_summary_long <- ws_summary |>
  dplyr::mutate(
    clustering_norm = clustering / C0,
    distance_norm = avg_distance / L0
  ) |>
  tidyr::pivot_longer(
    cols = c(clustering_norm, distance_norm),
    names_to = "quantity",
    values_to = "value"
  ) |>
  dplyr::mutate(
    quantity = dplyr::recode(
```

```
      quantity,
      clustering_norm = "C(beta) / C(0)",
      distance_norm = "L(beta) / L(0)"
    )
  )
```

```
ggplot(ws_summary_long, aes(x = beta, y = value, color = quantity)) +
  geom_line(linewidth = 1.2) +
  geom_point(size = 1.8) +
  scale_x_log10() +
  scale_color_manual(
    values = c(
      "C(beta) / C(0)" = course_cols$teal,
      "L(beta) / L(0)" = course_cols$gold
    )
  ) +
  labs(
    title = "The Watts--Strogatz transition",
    subtitle = "Distances become small before clustering has time to disappear",
    x = "Rewiring probability, beta",
    y = "Normalized quantity",
    color = NULL
  )
```

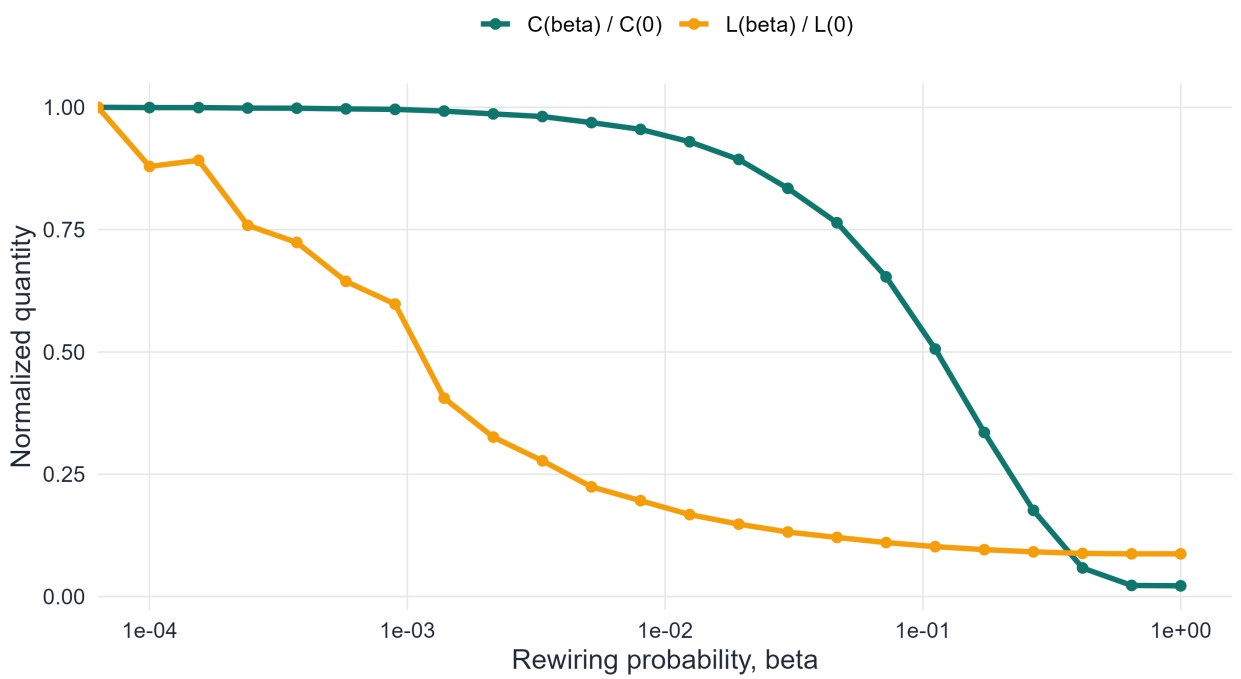


Figure 6.4: Normalized average path length and normalized clustering coefficient in the Watts–Strogatz model. Path length drops quickly while clustering remains high over a broader range of rewiring probabilities.

Figure Figure 6.4 contains the main quantitative message of the model. The normalized average path length

falls sharply even when β is still very small, whereas the normalized clustering coefficient decreases much more slowly. There is therefore a range of rewiring probabilities for which the network already has short path lengths but still retains strong local clustering. That interval is the small-world regime.

Definition 6.7. In the Watts–Strogatz model, the **small-world regime** is the range of rewiring probabilities β for which

1. the average path length is substantially smaller than in the regular lattice, and
2. the clustering coefficient remains substantially larger than in a comparable random graph.

6.11 Quantifying small-worldness

The phrase *small-world* is often used qualitatively, but one can also build a simple numerical index. A common choice compares the observed graph with a random graph having similar size and average degree (Humphries & Gurney, 2008; Telesford et al., 2011).

Let C and L denote the clustering coefficient and average path length of the observed graph, and let C_{rand} and L_{rand} denote the corresponding quantities for a comparable random graph. One widely used index is

$$\sigma = \frac{C/C_{\text{rand}}}{L/L_{\text{rand}}}.$$

A graph is often regarded as small-world when

$$\sigma > 1,$$

because this indicates that clustering is much larger than random-graph clustering, while path length remains close to the random benchmark.

The next simulation compares three networks of the same size and approximately the same mean degree:

- a regular ring lattice,
- a Watts–Strogatz network in the small-world regime,
- an Erdős–Rényi random graph.

```
set.seed(2034)

n_cmp <- 400
K_cmp <- 10
p_cmp <- K_cmp / (n_cmp - 1)

repeat {
  g_er_cmp <- igraph::sample_gnp(n_cmp, p_cmp, directed = FALSE, loops = FALSE)
  if (igraph::components(g_er_cmp)$no == 1) break
}

g_ring_cmp <- igraph::make_lattice(dimvector = n_cmp, nei = K_cmp / 2, circular = TRUE)
g_ws_cmp <- sample_connected_ws_K(size = n_cmp, K = K_cmp, beta = 0.02)

network_stats <- function(g, label, g_rand) {
  C <- igraph::transitivity(g, type = "average")
  L <- igraph::mean_distance(g, directed = FALSE, unconnected = FALSE)
  C_rand <- igraph::transitivity(g_rand, type = "average")
  L_rand <- igraph::mean_distance(g_rand, directed = FALSE, unconnected = FALSE)

  tibble::tibble(
```

```
    model = label,
    clustering = C,
    avg_distance = L,
    sigma = (C / C_rand) / (L / L_rand)
  )
}

comparison_tbl <- dplyr::bind_rows(
  network_stats(g_ring_cmp, "Ring lattice", g_er_cmp),
  network_stats(g_ws_cmp, "Watts--Strogatz", g_er_cmp),
  network_stats(g_er_cmp, "Random graph", g_er_cmp)
) |>
  dplyr::mutate(model = factor(model, levels = c("Ring lattice", "Watts--Strogatz", "Random graph")))
```

```
comparison_long <- comparison_tbl |>
  tidyr::pivot_longer(
    cols = c(clustering, avg_distance, sigma),
    names_to = "measure",
    values_to = "value"
  ) |>
  dplyr::mutate(
    measure = factor(measure, levels = c("clustering", "avg_distance", "sigma"),
                     labels = c("Clustering", "Average path length", "Small-worldness sigma"))
  )

ggplot(comparison_long, aes(x = model, y = value, fill = model)) +
  geom_col(width = 0.72, show.legend = FALSE) +
  facet_wrap(~ measure, scales = "free_y") +
  scale_fill_manual(values = c(
    "Ring lattice" = course_cols$teal,
    "Watts--Strogatz" = course_cols$gold,
    "Random graph" = course_cols$blue
  )) +
  labs(
    title = "Three structural regimes at the same scale",
    subtitle = "The small-world network combines high clustering with relatively short paths",
    x = NULL,
    y = NULL
  ) +
  theme(axis.text.x = element_text(angle = 15, hjust = 1))
```

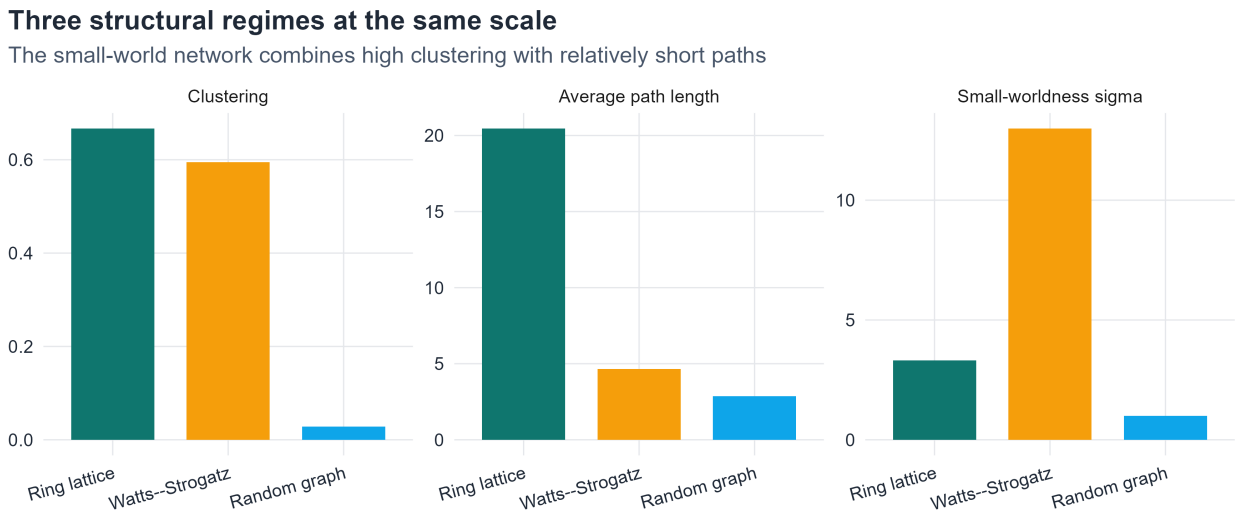


Figure 6.5: Comparison of clustering coefficient, average path length, and small-worldness index for a ring lattice, a Watts–Strogatz network, and a random graph of comparable size and average degree.

Figure Figure 6.5 summarizes the chapter's structural comparison. The ring lattice has high clustering but also large distances. The random graph has short distances but weak clustering. The Watts–Strogatz network occupies the intermediate regime that matters most: its clustering remains much higher than random, while its path length has already moved much closer to the random benchmark. That is exactly the structural sense in which it is a small-world network.

Structural comparison

Model	Average path length	Clustering coefficient
Regular lattice	large ($\sim N/2K$)	high ($\sim 3/4$)
Random graph ($G(N,p)$)	small ($\sim \ln N / \ln\langle k\rangle$)	low ($\sim p$)
Watts–Strogatz (small-world)	small	high

Only the small-world model combines both properties.

The table above highlights the central trade-off of the chapter. A regular lattice preserves local neighborhood structure, so triangles remain common, but long-range communication is inefficient. A random graph greatly improves reachability, yet it does so by sacrificing most of the local order. The Watts–Strogatz model occupies the narrow but important intermediate regime in which a small number of shortcuts lowers path length dramatically while much of the original clustering survives. For teaching purposes, this is the key structural message to keep in mind: small-world behavior is not pure randomness, but a controlled balance between local order and global efficiency (Strogatz, 2001; Watts & Strogatz, 1998).

6.12 What the Watts–Strogatz model explains and what it does not

The Watts–Strogatz model successfully explains one important empirical coexistence:

- high clustering, and
- short average path lengths.

This is a major achievement. But the model does not explain all structural properties observed in real networks.

Its degree distribution remains relatively narrow. Because rewiring preserves the number of edges and introduces only limited degree heterogeneity, the model does not naturally generate hubs or heavy-tailed degree distributions. Nor does it naturally reproduce the degree-dependent clustering patterns often seen in hierarchical or scale-free networks (Albert & Barabási, 2002; Barabási & Pósfai, 2016).

Therefore, the model captures one key aspect of real systems, but not the full range of structural signatures found in empirical data.

6.13 Applications of small-world networks

The small-world idea matters because many real systems balance local organization with global reachability.

6.13.1 Social systems

In social networks, high clustering reflects the tendency of acquaintances of the same person to know one another. At the same time, short path lengths support rapid information flow, rumor spreading, and social search (Milgram, 1967; Watts & Strogatz, 1998).

6.13.2 Neural systems

In brain and neural networks, strong local clustering is associated with local specialization, while short path lengths support efficient long-range communication. The result is often interpreted as a compromise between segregation and integration (Strogatz, 2001).

6.13.3 Technological and infrastructure systems

Transportation and communication networks require efficient long-range reachability without eliminating regional organization. Shortcut-like connections can reduce travel or transmission time significantly while preserving local substructure.

6.13.4 Why these applications matter

The practical importance of small-world structure comes from a trade-off:

- purely local systems are organized but globally slow,
- purely random systems are globally efficient but locally weak,
- small-world systems combine much of the benefit of both.

6.14 Bridge to the next chapter

The Watts–Strogatz model explains how short paths and high clustering can coexist, but it does not explain another major empirical feature of many real systems: the presence of hubs and strongly heterogeneous degree distributions. In the model, most vertices still have similar degree, even after rewiring. For that reason, the small-world chapter naturally leads to a new question: how can networks produce a few highly connected vertices together with many low-degree ones? That question takes us to the theory of scale-free networks, where the main focus shifts from shortcuts and clustering to heterogeneity, power laws, and growth mechanisms (Albert & Barabási, 2002; Barabási & Pósfai, 2016).

6.15 Core formulas to remember

! Summary

For small-world networks and the Watts–Strogatz model:

1. **Random-network distance estimate:**

$$L(G) \approx \frac{\ln N}{\ln\langle k\rangle}$$

2. **Lattice distance scaling in dimension d:**

$$L(G) \sim D(G) \sim N^{1/d}$$

3. **Ring-lattice average path length:**

$$L(0) \approx \frac{N}{2K}$$

4. **Ring-lattice clustering coefficient:**

$$C(0) = \frac{3(K-2)}{4(K-1)}$$

5. **Small-worldness index:**

$$\sigma = \frac{C/C_{\text{rand}}}{L/L_{\text{rand}}}$$

6. **Small-world regime:**

$$L(\beta)/L(0) \ll 1, \qquad C(\beta)/C(0) \approx 1$$

6.16 In-class questions

1. What does "small" mean mathematically in the phrase *small world*?
2. Why is the estimate $\ln N/\ln\langle k\rangle$ associated with random-like networks?
3. Why is the small-world effect surprising from the viewpoint of lattice geometry?
4. Why do shortcuts affect path length more strongly than clustering?
5. Why is the Watts–Strogatz model structurally different from an Erdős–Rényi random graph?
6. Why is average path length usually more stable than diameter?
7. What does the Watts–Strogatz model explain successfully, and what does it fail to explain?
8. What is the purpose of the small-worldness index σ?

6.17 End-of-chapter exercises

The exercises are divided into three groups:

1. mathematical exercises,
2. conceptual and analytical exercises,
3. simulation and coding exercises.

6.17.1 Part I. Mathematical exercises

6.17.1.1 Exercise 1. Geometric-series derivation

Starting from the branching approximation

$$N(d) \approx 1 + \langle k \rangle + \langle k \rangle^2 + \cdots + \langle k \rangle^d,$$

derive the closed-form expression

$$N(d) \approx \frac{\langle k \rangle^{d+1} - 1}{\langle k \rangle - 1}.$$

State clearly the assumption under which this approximation is meaningful.

6.17.1.2 Exercise 2. Logarithmic distance estimate

Starting from

$$N(d_{\max}) \approx N,$$

derive the estimate

$$d_{\max} \approx \frac{\ln N}{\ln \langle k \rangle}.$$

Explain each algebraic step carefully.

6.17.1.3 Exercise 3. Why the estimate is not exact

The relation

$$L(G) \approx \frac{\ln N}{\ln \langle k \rangle}$$

is only an approximation. Explain why it is not exact. Your answer should discuss:

- overlap between neighborhoods,
- loops and finite-size effects,
- the difference between average distance and diameter.

6.17.1.4 Exercise 4. Lattice scaling laws

Show that in a d-dimensional regular lattice with N vertices, the characteristic linear size is of order

$$N^{1/d}.$$

Explain why this leads to the scaling

$$L(G) \sim D(G) \sim N^{1/d}.$$

6.17.1.5 Exercise 5. Average degree in the ring lattice

Let G be the regular ring lattice with parameters (N, K).

1. Prove that every vertex has degree K.
2. Deduce that
$$|E| = \frac{NK}{2}.$$
3. Compute the average degree.

6.17.1.6 Exercise 6. Normalized observables

Let $C(\beta)$ and $L(\beta)$ be the clustering coefficient and average path length in the Watts–Strogatz model.

1. Define the normalized quantities
$$\frac{C(\beta)}{C(0)}, \qquad \frac{L(\beta)}{L(0)}.$$
2. Explain why these ratios are useful when studying the transition from order to randomness.
3. State the behavior expected in the small-world regime.

6.17.1.7 Exercise 7. Ring-lattice path length

Give a heuristic derivation of the estimate

$$L_{\text{lattice}} \approx \frac{N}{2K}$$

for the regular ring lattice.

6.17.1.8 Exercise 8. Ring-lattice clustering

Using the ring lattice with parameter K, derive the formula

$$C(0) = \frac{3(K-2)}{4(K-1)}.$$

6.17.2 Part II. Conceptual and analytical exercises

6.17.2.1 Exercise 9. Six degrees and logarithmic scaling

Explain why the statement "all vertices are separated by only a few steps" should not be interpreted as a fixed constant independent of system size. Your answer should distinguish carefully between a constant bound and logarithmic growth.

6.17.2.2 Exercise 10. Why small worlds are surprising

Write a short mathematical discussion comparing the growth laws

$$\ln N, \qquad N^{1/3}, \qquad N^{1/2}, \qquad N.$$

Explain why the logarithmic law is fundamentally different from polynomial lattice scaling.

6.17.2.3 Exercise 11. Random graph versus small-world network

A random graph and a Watts–Strogatz network can both have short average path lengths. Explain why they should still be considered structurally different models. Your answer should address clustering, local order, and global reachability.

6.17.2.4 Exercise 12. Why shortcuts are powerful

Explain why a small number of long-range edges can drastically reduce average path length. Use the language of graph distance and distinguish between local and long-range edges.

6.17.2.5 Exercise 13. Limitations of the Watts–Strogatz model

Discuss why the Watts–Strogatz model does not naturally explain hub-dominated degree distributions or strong degree-dependent clustering.

6.17.2.6 Exercise 14. Interpreting the small-worldness index

Explain what it means if a graph has $\sigma > 1$. Why is it not enough to examine clustering alone?

6.17.3 Part III. Simulation and coding exercises

6.17.3.1 Exercise 15. Distance scaling in random networks

For several values of N and fixed average degree, simulate random graphs and estimate the average path length. Compare the empirical results with the prediction

$$L(G) \approx \frac{\ln N}{\ln\langle k\rangle}.$$

6.17.3.2 Exercise 16. Visualizing the Watts–Strogatz interpolation

Produce your own sequence of Watts–Strogatz graphs for several values of β. Use a fixed circular layout so that the effect of rewiring can be seen clearly.

6.17.3.3 Exercise 17. Detecting the small-world regime numerically

Simulate the Watts–Strogatz model for a grid of rewiring probabilities. Plot both

$$\frac{L(\beta)}{L(0)} \qquad \text{and} \qquad \frac{C(\beta)}{C(0)}$$

on the same graph, and identify the regime in which the network is small-world.

6.17.3.4 Exercise 18. Comparing ring lattice, Watts–Strogatz, and random graph

Construct three graphs of comparable size and average degree: a ring lattice, a Watts–Strogatz graph, and an Erdős–Rényi graph. Compute clustering, average path length, and σ for each one. Interpret the results.

6.18 References

Chapter 7

Scale-Free Networks

Power Laws, Hubs, and Generative Mechanisms

7.1 Introduction

Many real networks are dominated by degree heterogeneity. In citation systems, a small number of papers receive enormous attention while most receive few citations. In transportation systems, a small number of airports serve as major transfer hubs while most airports remain modest in connectivity. In the World Wide Web, some pages attract extraordinarily many links while the overwhelming majority remain comparatively peripheral. (Albert & Barabási, 2002; Barabási & Albert, 1999; Newman, 2010)

This chapter studies the mathematics of that phenomenon. Our goal is not merely to present the Barabási–Albert model, but to distinguish clearly between three different levels of description:

1. the **structural signature** of a scale-free network,
2. the **statistical regularities** associated with heavy-tailed degree distributions,
3. the **generative mechanisms** that can produce such patterns.

The chapter therefore proceeds in two parts. We first study scale-free structure independently of any particular growth model. After that, we introduce growth and preferential attachment, and then present the Barabási–Albert model as one important mechanism that generates heavy-tailed degree distributions.

7.2 Learning goals

By the end of this chapter, it should be possible to:

- explain what degree heterogeneity and hub structure mean
- define a power-law degree distribution and interpret its exponent
- explain why the term *scale-free* is used
- interpret degree distributions on log-log axes
- explain the ideas of universality and ultra-small distances in heavy-tailed networks
- generate a network with an arbitrary degree distribution using the configuration model
- describe growth and preferential attachment conceptually and mathematically
- define the Barabási–Albert model
- interpret degree dynamics in a growing preferential-attachment network
- evaluate the strengths and limitations of the BA model

7.3 Roadmap

The first half of the chapter develops the structural language of scale-free networks: hubs, heavy tails, power laws, universality, and the configuration model. The second half turns to growth-based explanations. There we introduce preferential attachment, the BA model, degree dynamics, and several empirical signatures of that model. This structure parallels the general organization used elsewhere in the book: structural property first, mechanistic model second, and numerical interpretation throughout.

7.4 Hubs, degree heterogeneity, and broad degree distributions

7.4.1 Degree heterogeneity

Definition 7.1. A network exhibits **degree heterogeneity** when vertices with very different degrees coexist in substantial numbers, so that the degree distribution is broad rather than concentrated around a single typical value.

In a homogeneous graph, most degrees lie in a relatively narrow band around the mean. In a heterogeneous graph, such a summary is inadequate: many vertices have small degree, a moderate number have intermediate degree, and a few may have extremely large degree. This broadness already suggests that the graph cannot be described adequately by average degree alone.

7.4.2 Hubs

Definition 7.2. A **hub** is a vertex whose degree is substantially larger than the degrees of most other vertices in the same network.

Hubs matter because they can change the geometry and function of the network. They can shorten paths, centralize traffic, accelerate spreading processes, and create vulnerability under targeted removal. For this reason, the existence of hubs is not merely a visual feature; it is a structural fact with dynamical consequences.

7.4.3 Visual contrast: homogeneous and heterogeneous networks

```
layout_tbl <- function(g, model_name) {
  lay <- igraph::layout_with_fr(g)
  nodes <- tibble(
    id = seq_len(vcount(g)),
    x = lay[, 1],
    y = lay[, 2],
    degree = igraph::degree(g),
    model = model_name
  )

  edges <- igraph::as_data_frame(g, what = "edges") |>
    transmute(from = from, to = to) |>
    left_join(nodes |> select(from = id, x, y), by = "from") |>
    rename(x_from = x, y_from = y) |>
    left_join(nodes |> select(to = id, x, y), by = "to") |>
    rename(x_to = x, y_to = y)

  list(nodes = nodes, edges = edges)
}

g_er_small <- igraph::sample_gnp(n = 55, p = 0.08, directed = FALSE, loops = FALSE)
g_ba_small <- igraph::sample_pa(n = 55, power = 1, m = 2, directed = FALSE)
```

```r
er_plot <- layout_tbl(g_er_small, "Homogeneous random graph")
ba_plot <- layout_tbl(g_ba_small, "Heterogeneous network with hubs")

nodes_tbl <- bind_rows(er_plot$nodes, ba_plot$nodes) |>
  group_by(model) |>
  mutate(degree_band = ntile(degree, 3)) |>
  ungroup() |>
  mutate(degree_band = factor(degree_band, levels = 1:3, labels = c("low", "middle", "high")))

edges_tbl <- bind_rows(er_plot$edges, ba_plot$edges, .id = "panel") |>
  mutate(model = c("Homogeneous random graph", "Heterogeneous network with hubs")[as.integer(panel)])

ggplot() +
  geom_segment(
    data = edges_tbl,
    aes(x = x_from, y = y_from, xend = x_to, yend = y_to),
    color = alpha(course_cols$ink, 0.20), linewidth = 0.45
  ) +
  geom_point(
    data = nodes_tbl,
    aes(x = x, y = y, size = degree, fill = degree_band),
    shape = 21, color = "white", stroke = 0.35, alpha = 0.95
  ) +
  scale_size_continuous(range = c(3.5, 12), guide = "none") +
  scale_fill_manual(values = c(low = course_cols$blue, middle = course_cols$teal, high = course_cols$go
  facet_wrap(~model, nrow = 1) +
  labs(
    title = "Homogeneous versus heterogeneous degree structure",
    subtitle = "Preferential attachment creates visibly dominant hubs",
    x = NULL, y = NULL
  ) +
  theme(
    axis.text = element_blank(),
    axis.title = element_blank(),
    axis.ticks = element_blank(),
    panel.grid = element_blank(),
    strip.text = element_text(face = "bold", color = course_cols$ink)
  )
```

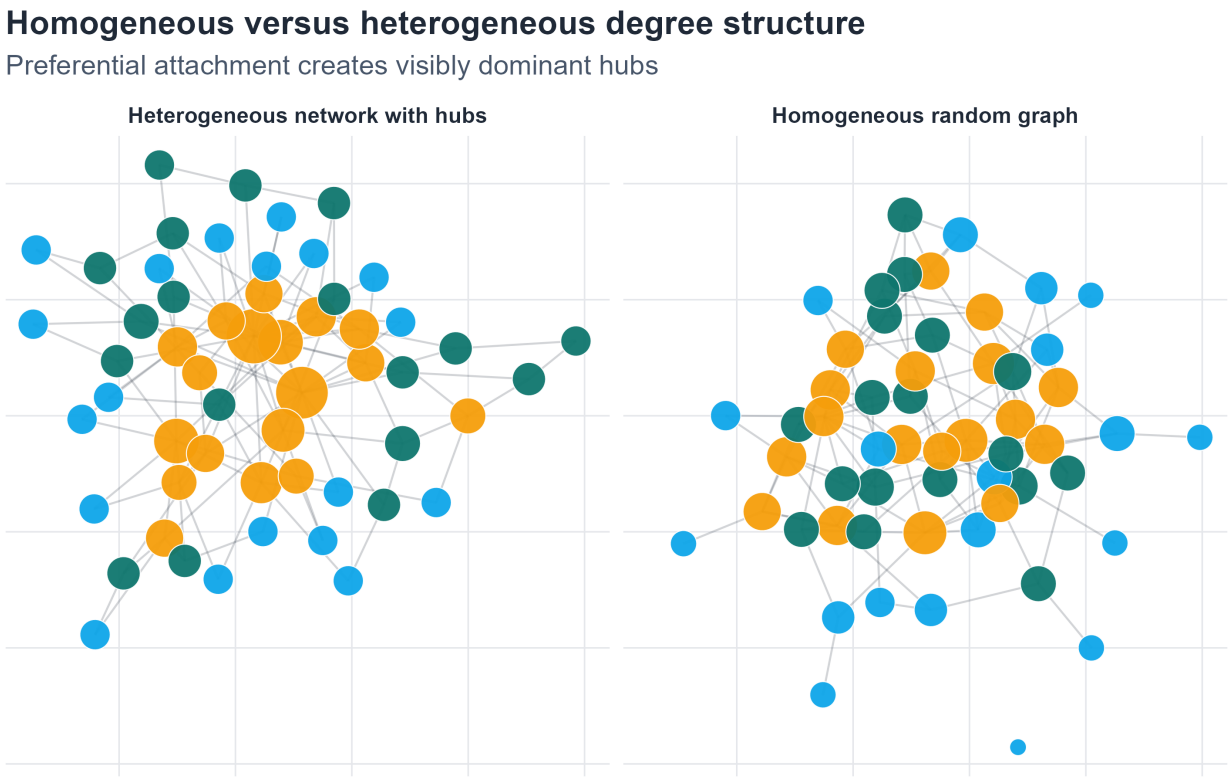


Figure 7.1: A visual comparison between a homogeneous Erdős–Rényi graph and a heterogeneous graph generated by preferential attachment. The two networks have roughly similar average degree, but the second contains a few much larger vertices, reflecting hub formation.

As shown in Figure 7.1, the contrast is qualitative before it is quantitative. In the Erdős–Rényi graph, vertex sizes remain relatively comparable. In the heterogeneous network, a small number of vertices dominate visually. This motivates the need for a more precise statistical description of broad degree structure.

7.5 Power-law tails and the meaning of scale-free structure

7.5.1 Power-law degree distributions

Definition 7.3. A degree distribution follows a **power law** if, for sufficiently large degree k,

$$P(k) \sim Ck^{-\gamma},$$

where $C > 0$ and $\gamma > 0$.

The notation $\sim$ means asymptotic proportionality. In other words,

$$\lim_{k\to\infty} \frac{P(k)}{Ck^{-\gamma}} = 1.$$

So a power law is a statement about tail behavior. It does not claim that the full distribution must follow the same expression at all degrees.

7.5.2 Why such networks are called scale-free

Suppose

$$P(k) \sim Ck^{-\gamma}.$$

Then for any fixed scaling factor $a > 0$,

$$P(ak) \sim C(ak)^{-\gamma} = a^{-\gamma}Ck^{-\gamma}.$$

So rescaling the degree changes the tail only by a multiplicative factor. There is no characteristic degree scale governing the tail. This is the basic reason for the term **scale-free**.

Definition 7.4. A network is called **scale-free** if its degree distribution has a power-law tail over a substantial range of degrees (Albert & Barabási, 2002; Barabási & Albert, 1999; Newman, 2010).

In practice, this definition should be interpreted carefully. Real finite networks rarely exhibit a perfect power law over all degrees. What matters is the approximate tail behavior over a meaningful scaling region (Clauset et al., 2009).

7.5.3 Why the tail matters

Heavy tails differ sharply from binomial, Poisson, or exponential tails. In a heavy-tailed degree distribution:

1. low-degree vertices remain common,
2. high-degree vertices are rare but not negligible,
3. extreme degrees occur much more often than in narrow distributions.

This makes the second moment of the degree distribution especially important. When the tail is sufficiently heavy, quantities depending on $\langle k^2 \rangle$ can behave very differently from what one expects in homogeneous random graphs.

7.6 How to read degree distributions on ordinary and log-log scales

For empirical work, it is useful to distinguish three related objects:

1. the raw degree distribution $P(k)$,
2. its representation on log-log axes,
3. the complementary cumulative distribution function, or CCDF, $P(K \geq k)$.

A raw histogram can obscure the tail because high-degree observations are rare. Log-log coordinates often make heavy-tailed behavior easier to inspect, but approximate linearity on log-log axes is only suggestive, not a mathematical proof of a power law. For this reason, cumulative representations are often preferred in practice.

```
k_vals <- 1:80
plot_tbl <- tibble(
  k = rep(k_vals, 2),
  prob = c(dpois(k_vals, lambda = 6), {
    p <- k_vals^(-2.7)
    p / sum(p)
  }),
  distribution = rep(c("Poisson", "Power law"), each = length(k_vals))
)
```

```r
p1 <- ggplot(plot_tbl, aes(x = k, y = prob, color = distribution)) +
  geom_line(linewidth = 1.1) +
  scale_color_manual(values = c("Poisson" = course_cols$teal, "Power law" = course_cols$gold)) +
  labs(title = "Ordinary axes", x = "Degree k", y = "Probability", color = NULL) +
  theme(legend.position = "top")

p2 <- ggplot(plot_tbl, aes(x = k, y = prob, color = distribution)) +
  geom_line(linewidth = 1.1) +
  scale_color_manual(values = c("Poisson" = course_cols$teal, "Power law" = course_cols$gold)) +
  scale_x_log10() +
  scale_y_log10() +
  labs(title = "Log-log axes", x = "Degree k", y = "Probability", color = NULL) +
  theme(legend.position = "none")

gridExtra::grid.arrange(p1, p2, ncol = 2)
```

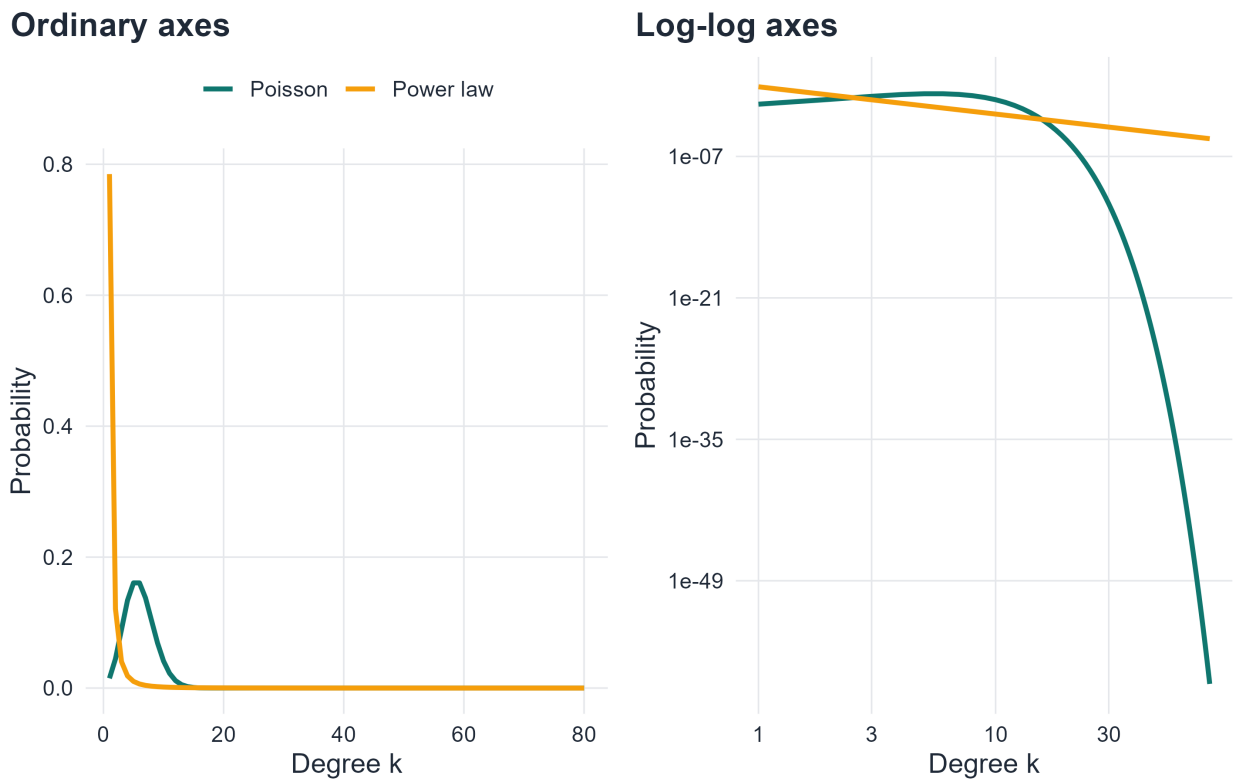


Figure 7.2: A comparison between a Poisson distribution and a power-law distribution on ordinary and log-log scales. On ordinary axes, the heavy tail is harder to interpret. On log-log axes, the slower tail decay becomes much clearer.

Figure Figure 7.2 illustrates why log-log plots are so common in this literature. On ordinary axes, the mass near small degrees dominates the picture. On log-log axes, the difference in tail decay becomes much easier to see. At the same time, one should not over-interpret visual linearity. Finite-size noise, truncation, and sampling variability can all distort the tail (Clauset et al., 2009).

7.7 Universality and the scale-free viewpoint

One reason scale-free networks attracted so much attention is that broad degree distributions appear in many systems that are otherwise very different: technological, biological, informational, and social networks can all exhibit strong degree heterogeneity. This led to the idea of **universality**: systems with very different microscopic details may display similar large-scale statistical signatures. (Albert & Barabási, 2002; Barabási & Pósfai, 2016)

This does not mean that all real networks are power-law networks, nor that all heavy-tailed networks arise from the same mechanism. The point is more subtle. Similar tail behavior can emerge from very different domains, which suggests that some statistical regularities are more fundamental than the details of the underlying application.

For a mathematical treatment, the important lesson is that scale-free structure should be studied first as a structural phenomenon. A generative model such as the BA model is then one explanation of that phenomenon, not the definition of the phenomenon itself.

7.8 Ultra-small distances

```
sample_powerlaw_degrees <- function(n, gamma, k_min = 2, k_max = 80) {
  ks <- k_min:k_max
  probs <- ks^(-gamma)
  probs <- probs / sum(probs)
  deg <- sample(ks, size = n, replace = TRUE, prob = probs)

  if (sum(deg) %% 2 == 1) {
    i <- sample(seq_len(n), 1)
    deg[i] <- deg[i] + 1
  }

  deg
}

make_scale_free_cfg <- function(n, gamma, k_min = 2, k_max = 80) {
  deg <- sample_powerlaw_degrees(n, gamma, k_min, k_max)
  g <- igraph::sample_degseq(deg, method = "vl")
  g <- igraph::simplify(g, remove.multiple = TRUE, remove.loops = TRUE)
  comps <- igraph::components(g)
  giant_id <- which.max(comps$csize)
  igraph::induced_subgraph(g, which(comps$membership == giant_id))
}

simulate_mean_distance_connected <- function(generator, sizes, reps = 8) {
  map_dfr(sizes, function(n) {
    vals <- numeric(reps)
    for (r in seq_len(reps)) {
      repeat {
        g <- generator(n)
        if (igraph::components(g)$no == 1) break
      }
      vals[r] <- igraph::mean_distance(g, directed = FALSE, unconnected = FALSE)
    }
    tibble(n = n, mean_distance = mean(vals))
  })
```

```
}
```

Scale-free degree heterogeneity can also affect the metric structure of a network. In many classical random graph models, average path length grows on the order of $\log N$. In sufficiently heavy-tailed networks, especially when $2 < \gamma < 3$, the growth can be even slower. This regime is often described as **ultra-small**, because distances may scale like $\log \log N$ rather than $\log N$ in the large-size limit (Barabási & Pósfai, 2016; Dorogovtsev et al., 2003; Newman et al., 2001).

The exact asymptotics require care, but the central idea is intuitive: a small number of extremely connected hubs can create shortcuts across the graph. As a result, the network may remain globally close even as it becomes very large.

```
sizes <- c(120, 180, 260, 380, 550, 800)

er_path_tbl <- simulate_mean_distance_connected(
  generator = function(n) igraph::sample_gnp(n = n, p = 6 / (n - 1), directed = FALSE, loops = FALSE),
  sizes = sizes,
  reps = 6
) |>
  mutate(model = "Erdos-Renyi")

sf_path_tbl <- simulate_mean_distance_connected(
  generator = function(n) make_scale_free_cfg(n = n, gamma = 2.5, k_min = 2, k_max = 70),
  sizes = sizes,
  reps = 6
) |>
  mutate(model = "Heavy-tailed configuration model")

path_tbl <- bind_rows(er_path_tbl, sf_path_tbl)

ggplot(path_tbl, aes(x = n, y = mean_distance, color = model)) +
  geom_line(linewidth = 1.1) +
  geom_point(size = 2) +
  scale_color_manual(values = c("Erdos-Renyi" = course_cols$teal,
                                "Heavy-tailed configuration model" = course_cols$gold)) +
  labs(
    title = "Distance growth in homogeneous and heavy-tailed networks",
    subtitle = "Heavy-tailed graphs tend to remain metrically closer",
    x = "Number of vertices, N",
    y = "Average path length",
    color = NULL
  )
```

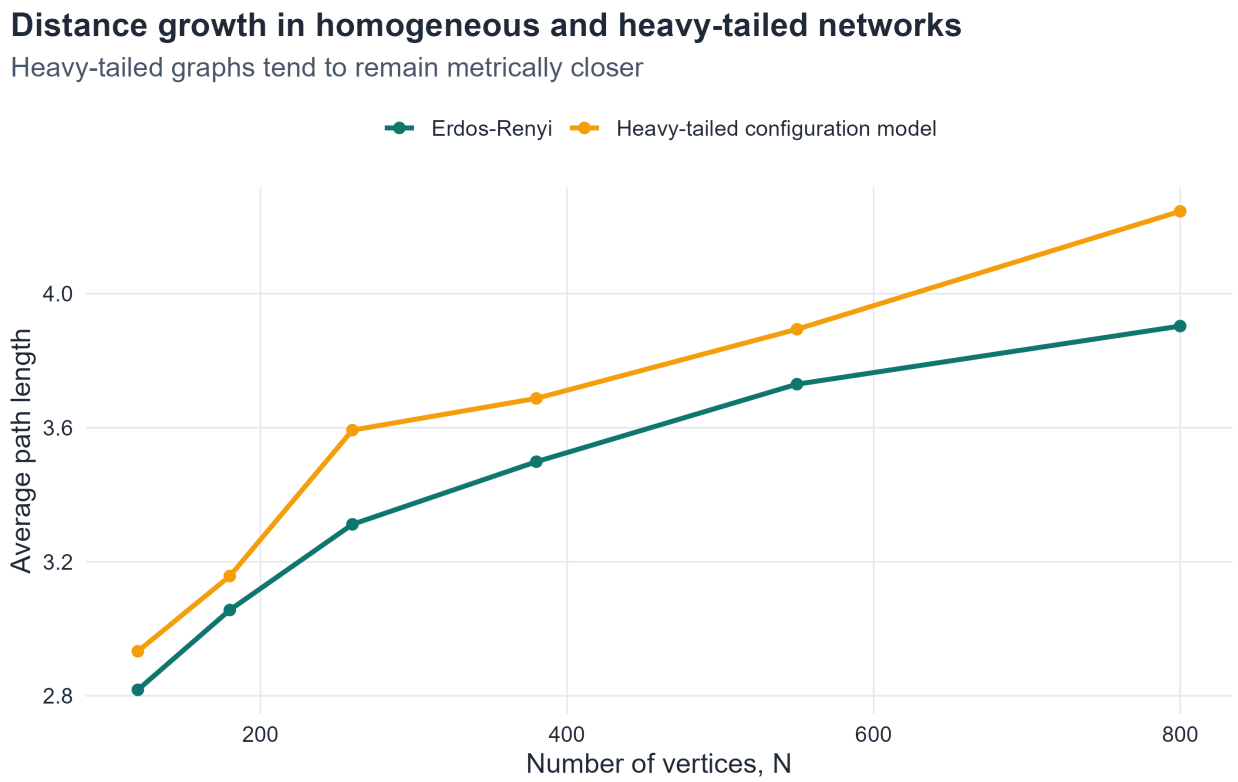


Figure 7.3: Average path length versus system size for Erdős–Rényi graphs and heavy-tailed configuration-model graphs. The heavy-tailed networks tend to maintain shorter distances, illustrating the qualitative idea behind ultra-small behavior.

As shown in Figure 7.3, the heavy-tailed configuration-model networks tend to maintain shorter paths over the same range of system sizes. This finite simulation does not prove the asymptotic law $\log \log N$, but it supports the broader structural message: degree heterogeneity can compress distances very strongly.

7.9 Generating arbitrary degree distributions: the configuration model

```
emp_degree_table <- function(g) {
  tibble(k = igraph::degree(g)) |>
    count(k, name = "count") |>
    filter(k > 0) |>
    arrange(k) |>
    mutate(prob = count / sum(count))
}
```

The scale-free viewpoint is broader than any single growth model. If our objective is simply to construct a graph with a given degree sequence or degree distribution, we can do so without invoking growth or preferential attachment at all (Newman et al., 2001; Newman, 2010).

Definition 7.5. The **configuration model** constructs a random graph from a prescribed degree sequence by assigning a number of half-edges to each vertex equal to its degree and then pairing those half-edges at random.

This idea is important conceptually. It shows that a heavy-tailed degree distribution is not the same thing

as a growth rule. One can impose the distribution first and randomize the rest of the structure (Molloy & Reed, 1995; Newman et al., 2001).

7.9.1 Why the configuration model matters

The configuration model is useful because it allows us to separate two questions:

1. What happens because of the degree distribution itself?
2. What happens because of the particular mechanism that generated the network?

If two networks share the same degree distribution but differ in clustering, age structure, or community organization, then the differences cannot be attributed to degree sequence alone.

```
g_cfg <- make_scale_free_cfg(n = 1800, gamma = 2.6, k_min = 2, k_max = 90)
cfg_tbl <- emp_degree_table(g_cfg)

ggplot(cfg_tbl, aes(x = k, y = prob)) +
  geom_point(color = course_cols$violet, size = 1.8, alpha = 0.85) +
  scale_x_log10() +
  scale_y_log10() +
  labs(
    title = "Heavy-tailed degree distribution from the configuration model",
    subtitle = "The tail is imposed through the degree sequence rather than produced by growth",
    x = "Degree k",
    y = "P(k)"
  )
```

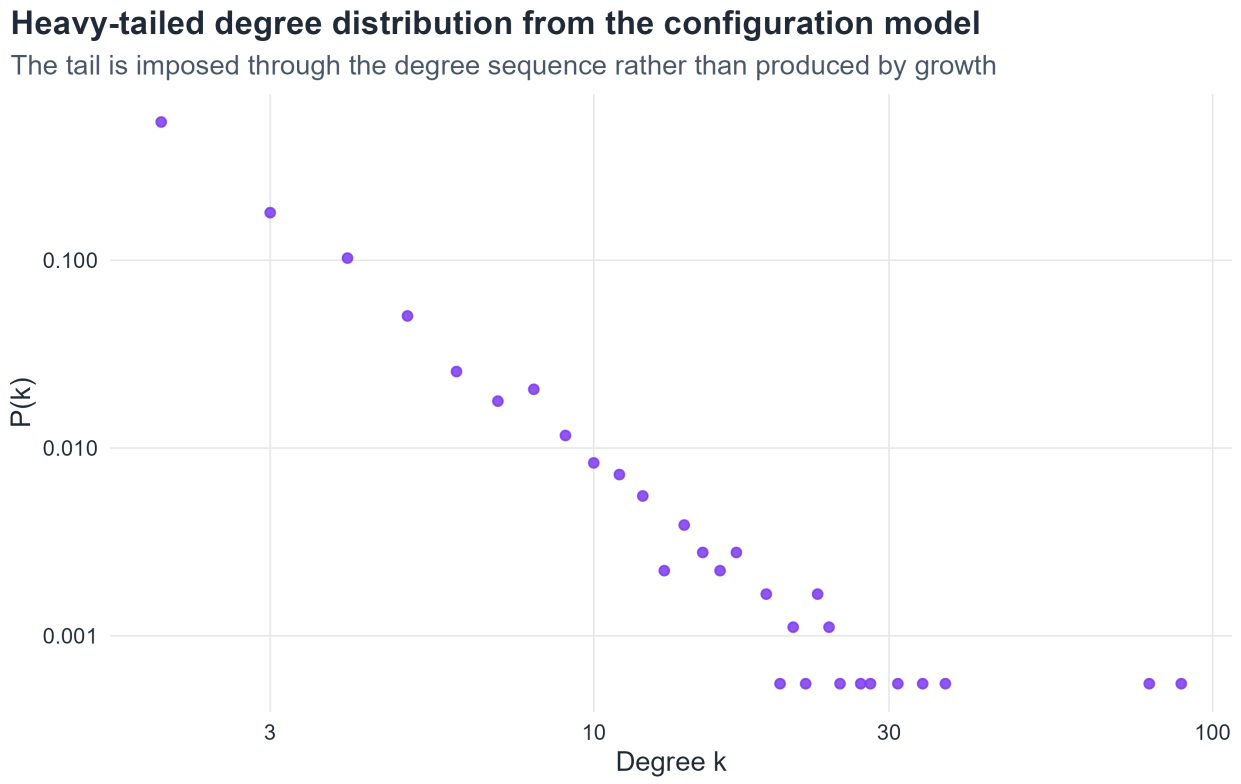


Figure 7.4: Empirical degree distribution of a heavy-tailed network generated by the configuration model. The model reproduces a prescribed broad degree sequence without using growth or preferential attachment.

Figure Figure 7.4 makes the main point visible. A broad, approximately power-law degree distribution can be produced without any growth mechanism. This is why the configuration model is such a useful baseline in network science: it isolates the role of degree heterogeneity from other structural ingredients.

7.10 From structural signature to generative mechanism

We have now described scale-free networks as structural objects: they exhibit broad degree distributions, heavy tails, hubs, and often unusually short distances. The next question is different in character:

> What dynamic mechanism can generate such structure?

One influential answer is based on two ideas:

1. networks grow over time,
2. new vertices do not attach uniformly, but prefer already well-connected vertices.

This is the logic of growth plus preferential attachment.

7.11 Growth and preferential attachment

7.11.1 Conceptual idea

Preferential attachment is a cumulative-advantage mechanism. If a vertex has already accumulated many links, it becomes more likely to receive additional links in the future. This creates positive feedback: large degree tends to become even larger.

In social and informational systems, this idea is often plausible. Popular pages attract more links. Well-cited papers attract more citations. Highly connected airports attract more routes. The phrase "the rich get richer" is informal, but it captures the intuition well.

7.11.2 Mathematical form

Definition 7.6. A growing network evolves by **preferential attachment** if the probability that a new vertex connects to an existing vertex i is proportional to the current degree of i.

If vertex i has degree $k_i(t)$ at time t, then the attachment probability is

$$\Pi_i(t) = \frac{k_i(t)}{\sum_j k_j(t)}.$$

Since $\sum_j k_j(t) = 2L(t)$, this can also be written as

$$\Pi_i(t) = \frac{k_i(t)}{2L(t)}.$$

> **Interpretation**
>
> Preferential attachment does not say that only large-degree vertices receive new links. It says that they receive them with higher probability. The mechanism is probabilistic, not deterministic.

7.12 The Barabási–Albert model

Definition 7.7. The **Barabási–Albert (BA) model** is defined by the following rule:

1. begin with a connected seed graph,
2. at each time step add one new vertex,
3. connect the new vertex to m existing vertices,
4. choose each target with probability proportional to degree.

The BA model therefore combines growth with preferential attachment. Neither ingredient alone is enough to produce the characteristic degree dynamics of the model.

7.12.1 Average degree

At each time step, one new vertex and m new edges are added. Therefore, for large system size,

$$\langle k \rangle \approx 2m.$$

So the parameter m controls density, while the preferential-attachment rule controls the heavy-tailed structure.

7.13 Simulating BA growth

```
layout_tbl <- function(g, stage_name) {
  lay <- igraph::layout_with_fr(g)
  nodes <- tibble(
    id = seq_len(vcount(g)),
    x = lay[, 1],
    y = lay[, 2],
    degree = igraph::degree(g),
    stage = stage_name
  )
  edges <- igraph::as_data_frame(g, what = "edges") |>
    left_join(nodes |> select(from = id, x, y), by = "from") |>
    rename(x_from = x, y_from = y) |>
    left_join(nodes |> select(to = id, x, y), by = "to") |>
    rename(x_to = x, y_to = y) |>
    mutate(stage = stage_name)
  list(nodes = nodes, edges = edges)
}

g1 <- igraph::sample_pa(n = 12, power = 1, m = 2, directed = FALSE)
g2 <- igraph::sample_pa(n = 28, power = 1, m = 2, directed = FALSE)
g3 <- igraph::sample_pa(n = 60, power = 1, m = 2, directed = FALSE)

p1 <- layout_tbl(g1, "Early stage")
p2 <- layout_tbl(g2, "Intermediate stage")
p3 <- layout_tbl(g3, "Later stage")

nodes_tbl <- bind_rows(p1$nodes, p2$nodes, p3$nodes)
edges_tbl <- bind_rows(p1$edges, p2$edges, p3$edges)

ggplot() +
  geom_segment(
    data = edges_tbl,
    aes(x = x_from, y = y_from, xend = x_to, yend = y_to),
    color = alpha(course_cols$ink, 0.20), linewidth = 0.45
```

```
  ) +
  geom_point(
    data = nodes_tbl,
    aes(x = x, y = y, size = degree),
    fill = course_cols$gold, color = "white", shape = 21, stroke = 0.35, alpha = 0.95
  ) +
  scale_size_continuous(range = c(3.5, 12), guide = "none") +
  facet_wrap(~stage, nrow = 1) +
  labs(
    title = "Growth snapshots in the BA model",
    subtitle = "Early vertices can accumulate visible degree advantage over time",
    x = NULL, y = NULL
  ) +
  theme(
    axis.text = element_blank(),
    axis.title = element_blank(),
    axis.ticks = element_blank(),
    panel.grid = element_blank(),
    strip.text = element_text(face = "bold", color = course_cols$ink)
  )
```

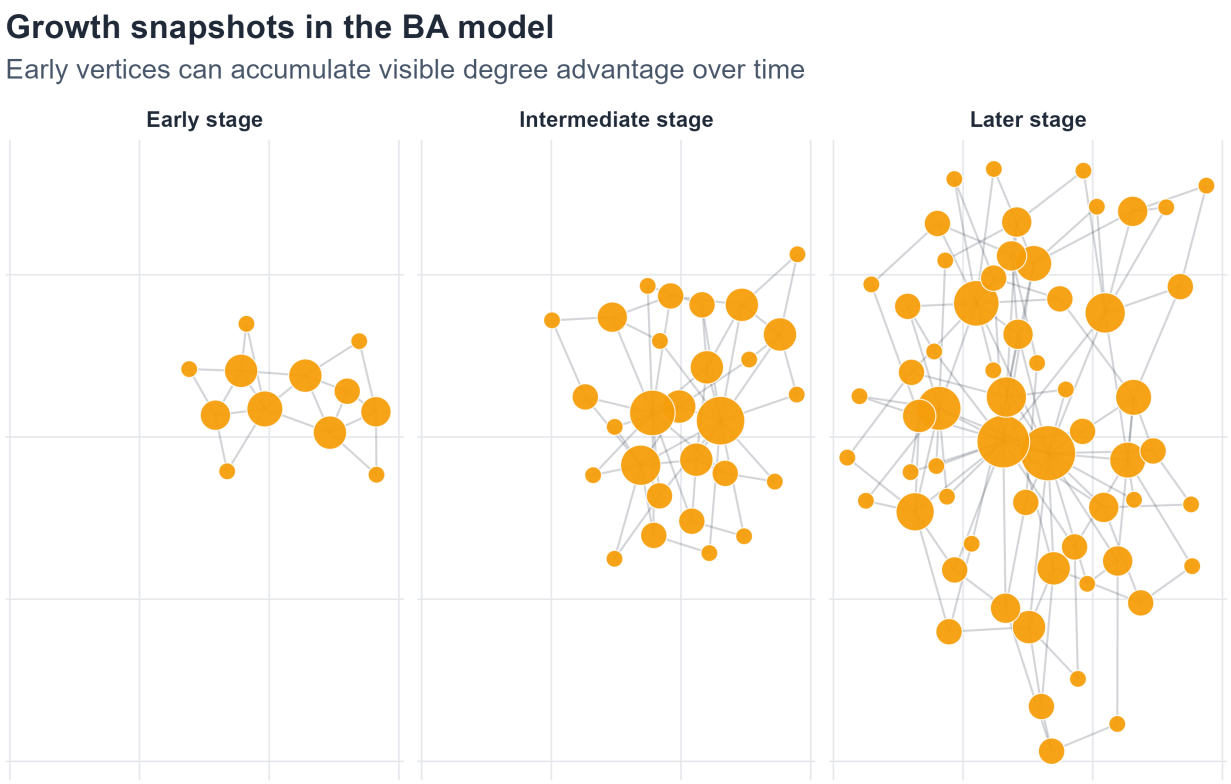


Figure 7.5: Snapshots of a Barabási–Albert network at three stages of growth. Older vertices with early degree advantage can accumulate disproportionately many links as the network expands.

As shown in Figure 7.5, the BA model is intrinsically time-asymmetric. Vertices that enter early have more opportunities to accumulate links, and preferential attachment can amplify that early advantage. The emergence of hubs is therefore tied both to present degree and to time of arrival.

7.14 Degree dynamics in a growing network

```
track_ba_degree_dynamics <- function(n = 300, m = 2, tracked = c(1, 2, 5, 10, 30, 80)) {
  tracked <- sort(unique(tracked))
  tracked <- tracked[tracked <= n]

  deg <- rep(0, n)
  adj <- matrix(0, nrow = n, ncol = n)

  active <- seq_len(m + 1)
  for (i in active) {
    for (j in active) {
      if (i < j) {
        adj[i, j] <- 1
        adj[j, i] <- 1
      }
    }
  }
  deg[active] <- rowSums(adj[active, active, drop = FALSE])

  record_step <- function(t) {
    tibble(time = t, vertex = tracked, degree = deg[tracked])
  }

  out <- list(record_step(m + 1))

  for (t in (m + 2):n) {
    probs <- deg[1:(t - 1)] / sum(deg[1:(t - 1)])
    targets <- sample(1:(t - 1), size = m, replace = FALSE, prob = probs)
    adj[t, targets] <- 1
    adj[targets, t] <- 1
    deg[targets] <- deg[targets] + 1
    deg[t] <- m
    out[[length(out) + 1]] <- record_step(t)
  }

  bind_rows(out)
}
```

One of the most instructive ways to understand preferential attachment is to follow a few vertices through time. Early vertices often accumulate degree much faster than later ones, because they have both temporal advantage and higher exposure to subsequent attachment events.

```
dyn_tbl <- track_ba_degree_dynamics(
  n = 320,
  m = 2,
  tracked = c(1, 2, 5, 10, 30, 80)
) |>
  mutate(vertex = paste0("v", vertex))

ggplot(dyn_tbl, aes(x = time, y = degree, color = vertex)) +
  geom_line(linewidth = 1.0) +
  labs(
    title = "Degree dynamics in the BA model",
```

```
    subtitle = "Early arrival and preferential attachment reinforce one another",
    x = "Time",
    y = "Degree",
    color = "Tracked vertex"
  )
```

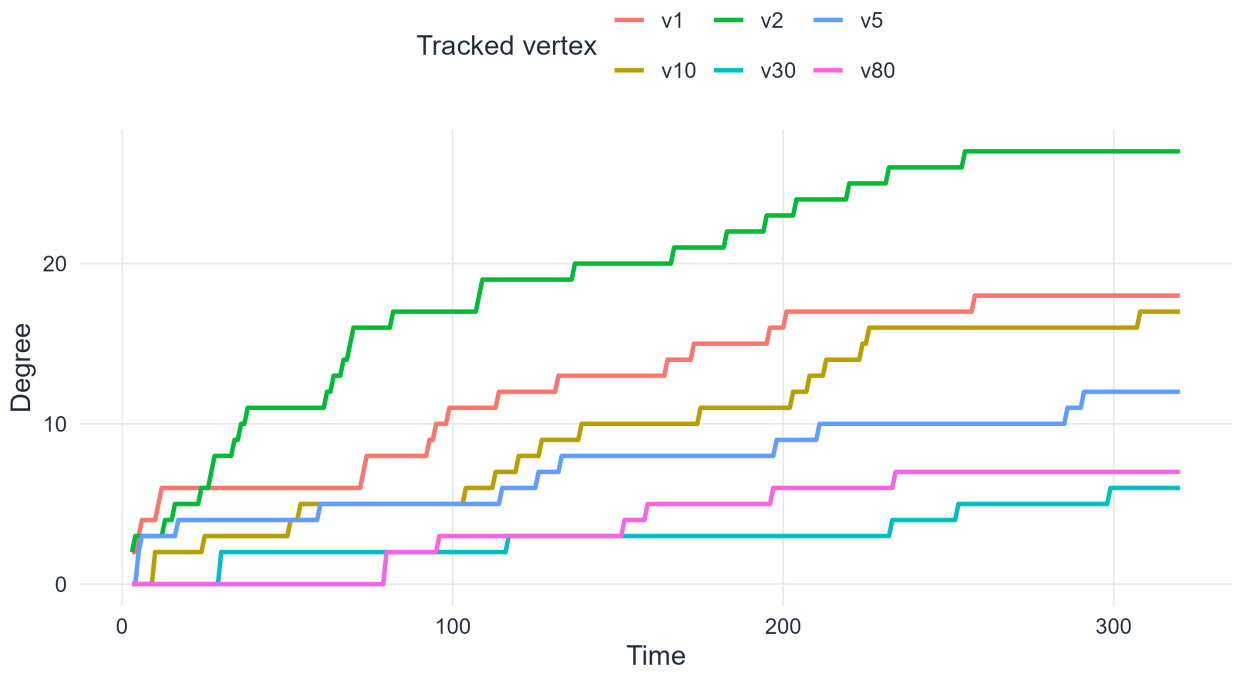


Figure 7.6: Degree trajectories for several vertices in a growing BA network. Earlier vertices tend to accumulate degree for longer and often remain ahead of later vertices.

Figure Figure 7.6 highlights a central message: preferential attachment is not just a statement about the final degree distribution. It is a dynamic rule that shapes the entire history of the network. The trajectories also show that stochasticity remains important. Early arrival creates opportunity, but not every early vertex becomes a dominant hub.

7.15 Degree distribution in the BA model

```
emp_ccdf_table <- function(g) {
  emp_degree_table(g) |>
    arrange(desc(k)) |>
    mutate(ccdf = cumsum(prob)) |>
    arrange(k)
}
```

A classical result for the BA model is that the degree distribution has asymptotic tail (Barabási & Albert, 1999; Bollobás et al., 2001)

$$P(k) \sim Ck^{-3}.$$

So the BA model produces a scale-free network with exponent

$$\gamma = 3.$$

This is one of the reasons the model became so influential: a very simple growth rule produces hub-dominated degree structure automatically.

7.15.1 A note on finite-size effects

In finite simulations, one should not expect a perfect straight line on a log-log plot. The extreme tail is sparse, noisy, and often truncated. This is why cumulative plots are also useful.

```
set.seed(2040)
n_ba <- 3000
m_ba <- 2

g_ba_large <- igraph::sample_pa(n = n_ba, power = 1, m = m_ba, directed = FALSE)
deg_tbl <- emp_degree_table(g_ba_large)
cum_tbl <- emp_ccdf_table(g_ba_large)
```

```
ggplot(deg_tbl, aes(x = k, y = prob)) +
  geom_point(color = course_cols$teal, size = 1.9, alpha = 0.82) +
  scale_x_log10() +
  scale_y_log10() +
  labs(
    title = "Degree distribution of a BA network",
    subtitle = "Heavy-tailed structure is visible on log-log axes",
    x = "Degree k",
    y = "P(k)"
  )
```

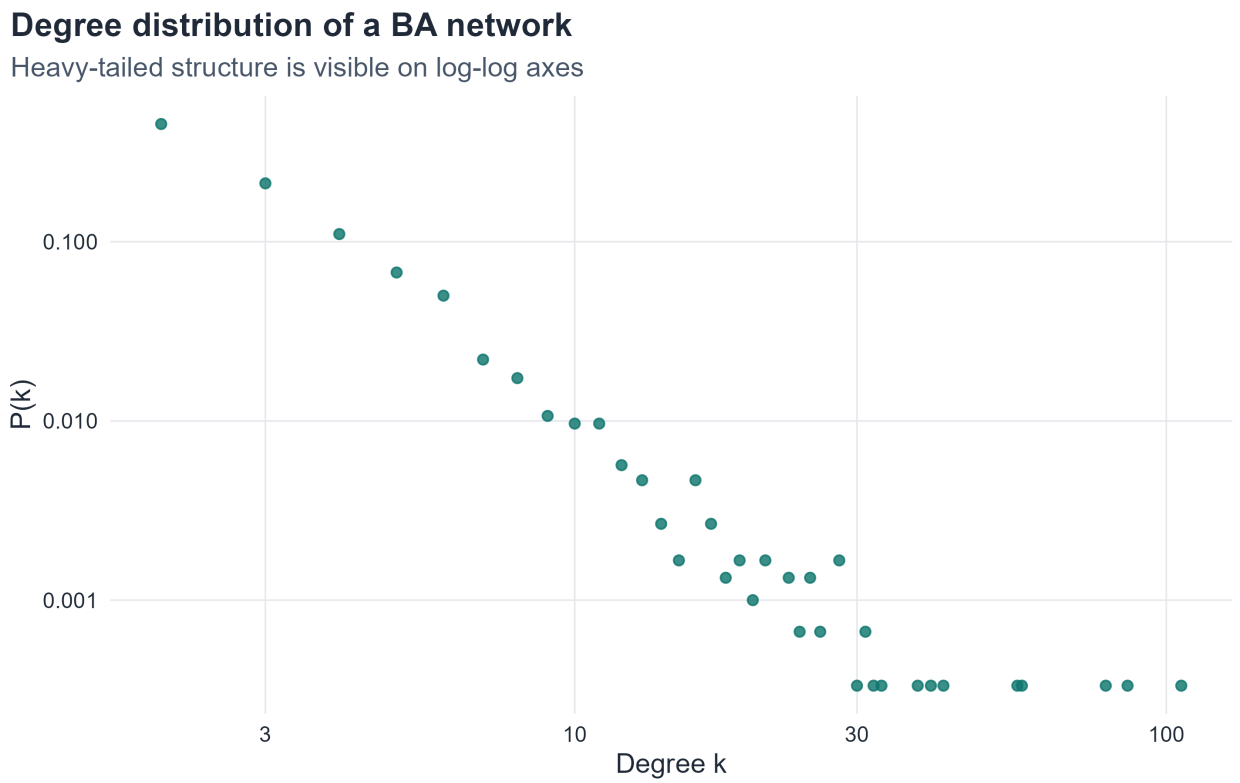


Figure 7.7: Empirical degree distribution of a Barabási–Albert network on log-log axes. The broad tail is clear, although the far tail is noisy because of finite-size effects.

```
ggplot(cum_tbl, aes(x = k, y = ccdf)) +
  geom_point(color = course_cols$gold, size = 1.9, alpha = 0.82) +
  scale_x_log10() +
  scale_y_log10() +
  labs(
    title = "Cumulative degree distribution of a BA network",
    subtitle = "The CCDF gives a smoother view of the tail",
    x = "Degree k",
    y = "P(K >= k)"
  )
```

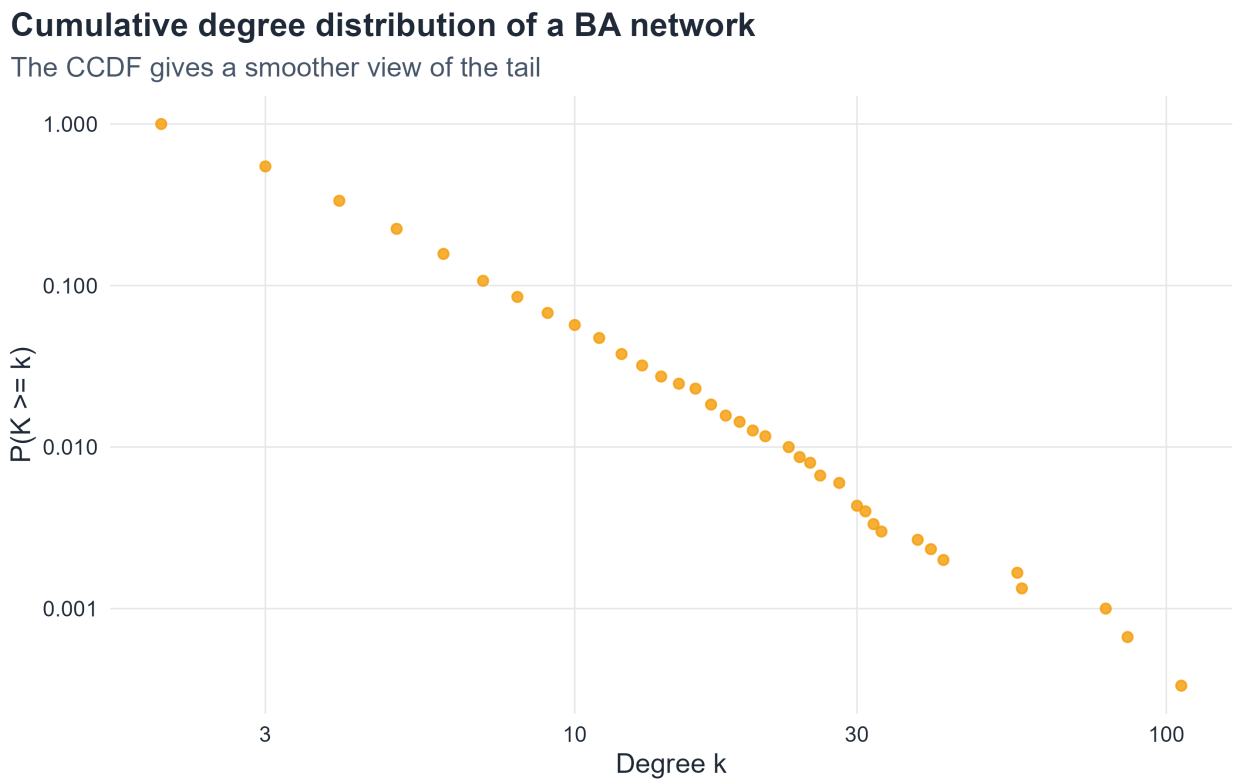


Figure 7.8: Empirical complementary cumulative degree distribution of a Barabási–Albert network on log-log axes. The cumulative representation smooths the tail and often gives a more stable visual summary.

Taken together, Figure 7.7 and Figure 7.8 show the same structural message in two complementary ways. The raw distribution emphasizes the broadness of the tail, while the CCDF gives a cleaner view of the decay at large degree. In a textbook setting, it is useful for students to see both because empirical work often moves back and forth between them.

7.16 Measuring preferential attachment in simulation

```
estimate_attachment_kernel_ba <- function(n = 1200, m = 2) {
  deg <- rep(0, n)
  adj <- matrix(0, nrow = n, ncol = n)

  active <- seq_len(m + 1)
  for (i in active) {
    for (j in active) {
      if (i < j) {
        adj[i, j] <- 1
        adj[j, i] <- 1
      }
    }
  }
  deg[active] <- rowSums(adj[active, active, drop = FALSE])

  records <- list()
```

```
  for (t in (m + 2):n) {
    existing_deg <- deg[1:(t - 1)]
    probs <- existing_deg / sum(existing_deg)
    targets <- sample(1:(t - 1), size = m, replace = FALSE, prob = probs)

    hit <- integer(t - 1)
    hit[targets] <- 1L

    records[[length(records) + 1]] <- tibble(k = existing_deg, received = hit)

    adj[t, targets] <- 1
    adj[targets, t] <- 1
    deg[targets] <- deg[targets] + 1
    deg[t] <- m
  }

  bind_rows(records) |>
    group_by(k) |>
    summarise(mean_new_links = mean(received), count = n(), .groups = "drop") |>
    filter(count >= 30)
}
```

If the BA mechanism is truly degree-proportional, then vertices of degree k should receive new links at a rate that grows approximately linearly with k. This can be checked empirically from a simulation.

```
attach_tbl <- estimate_attachment_kernel_ba(n = 1400, m = 2)

ggplot(attach_tbl, aes(x = k, y = mean_new_links)) +
  geom_point(color = course_cols$coral, size = 1.8, alpha = 0.85) +
  geom_smooth(method = "lm", se = FALSE, color = course_cols$ink, linewidth = 0.9) +
  labs(
    title = "Empirical attachment rate versus current degree",
    subtitle = "Degree-proportional growth appears as an approximately linear trend",
    x = "Current degree k",
    y = "Mean probability of receiving a new link"
  )
```

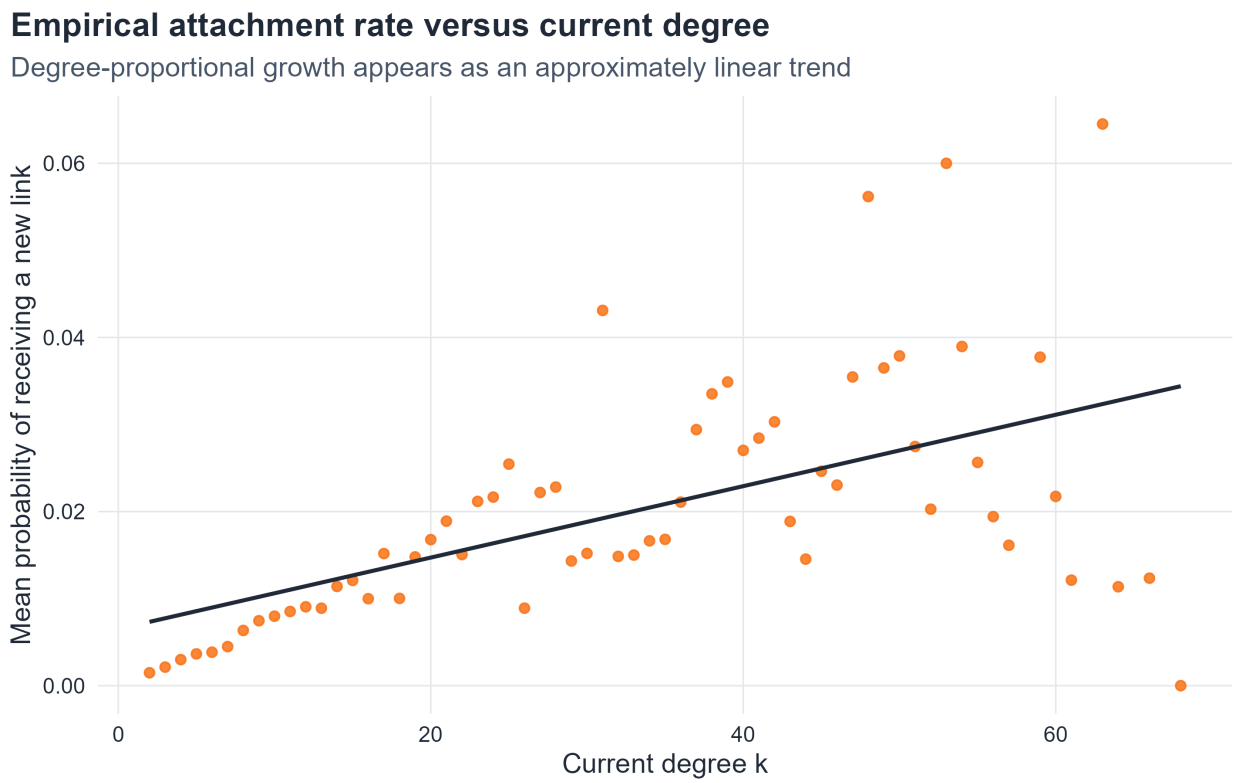


Figure 7.9: Estimated attachment kernel in a BA simulation. The mean rate of receiving new links grows approximately linearly with current degree, consistent with preferential attachment.

As shown in Figure 7.9, the simulated attachment rate increases roughly linearly with current degree. This does not replace the formal definition of the model, but it gives an empirical confirmation of what the mechanism means in practice (Barabási & Albert, 1999; Barabási & Pósfai, 2016).

7.17 Beyond degree distribution: clustering and distances in the BA model

```
compare_stats <- function(n, m, reps = 10) {
  map_dfr(seq_len(reps), function(r) {
    g_ba <- igraph::sample_pa(n = n, m = m, power = 1, directed = FALSE)
    p_er <- (2 * m) / (n - 1)

    repeat {
      g_er <- igraph::sample_gnp(n = n, p = p_er, directed = FALSE, loops = FALSE)
      if (igraph::components(g_er)$no == 1) break
    }

    tibble(
      model = c("Barabasi-Albert", "Erdos-Renyi"),
      mean_distance = c(
        igraph::mean_distance(g_ba, directed = FALSE),
        igraph::mean_distance(g_er, directed = FALSE)
      ),
```

```
      transitivity = c(
        igraph::transitivity(g_ba, type = "global"),
        igraph::transitivity(g_er, type = "global")
      )
    )
  })
}
```

A degree distribution is not the whole story of a network. Two models may differ not only in their tails, but also in their path length, clustering, and other global properties. It is therefore useful to compare the BA model with an Erdős–Rényi graph of similar average degree.

```
compare_stats <- function(n, m, reps = 10) {
  map_dfr(seq_len(reps), function(r) {
    g_ba <- igraph::sample_pa(n = n, m = m, power = 1, directed = FALSE)
    p_er <- (2 * m) / (n - 1)
    repeat {
      g_er <- igraph::sample_gnp(n = n, p = p_er, directed = FALSE, loops = FALSE)
      if (igraph::components(g_er)$no == 1) break
    }

    tibble(
      model = c("Barabasi-Albert", "Erdos-Renyi"),
      mean_distance = c(
        igraph::mean_distance(g_ba, directed = FALSE),
        igraph::mean_distance(g_er, directed = FALSE)
      ),
      transitivity = c(
        igraph::transitivity(g_ba, type = "global"),
        igraph::transitivity(g_er, type = "global")
      )
    )
  })
}

stats_tbl <- compare_stats(n = 250, m = 2, reps = 5) |>
  pivot_longer(cols = c(mean_distance, transitivity),
               names_to = "metric", values_to = "value") |>
  mutate(metric = recode(metric,
                         mean_distance = "Average path length",
                         transitivity = "Global clustering"))
ggplot(stats_tbl, aes(x = model, y = value, fill = model)) +
  geom_boxplot(alpha = 0.85, width = 0.65, outlier.alpha = 0.4) +
  facet_wrap(~ metric, scales = "free_y") +
  scale_fill_manual(values = c("Barabasi-Albert" = course_cols$gold,
                               "Erdos-Renyi" = course_cols$teal)) +
  labs(
    title = "Structural comparison of BA and Erdős--Rényi models",
    subtitle = "Hub structure shortens distances, but clustering is not automatically large",
    x = NULL,
    y = NULL,
    fill = NULL
  )
```

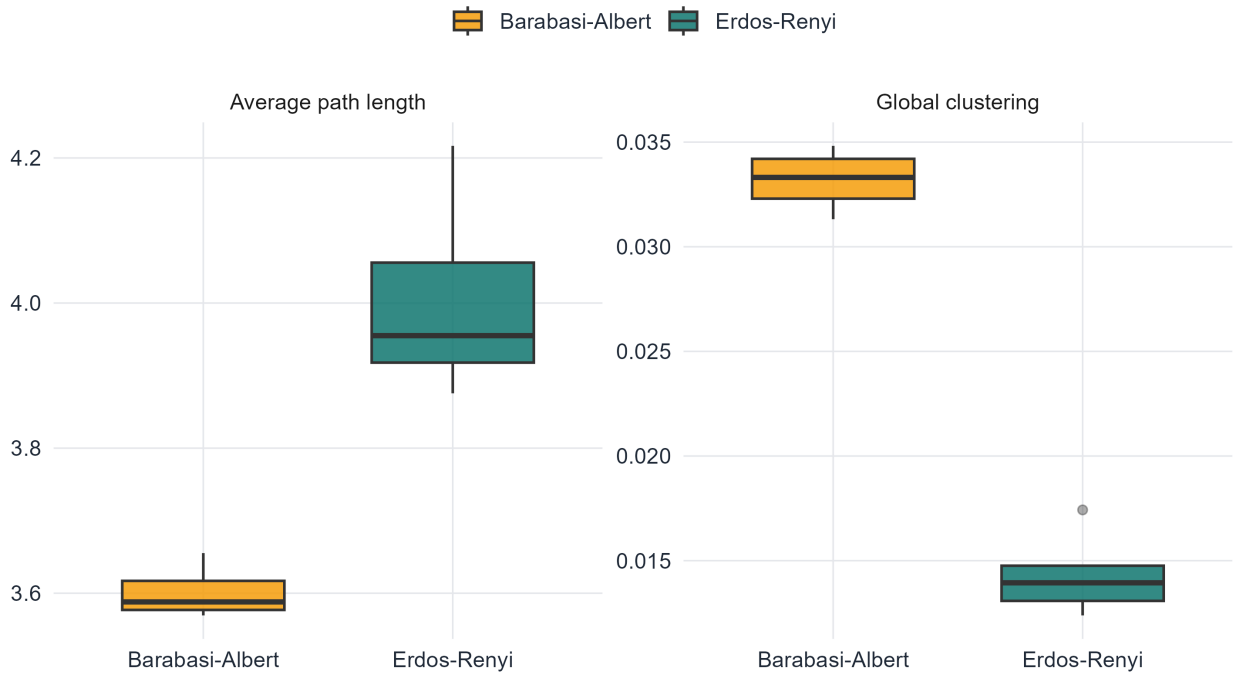


Figure 7.10: A comparison of average path length and transitivity for BA and Erdős–Rényi graphs with approximately matched average degree. The BA model tends to produce shorter paths because of hubs, while clustering is not automatically high.

Figure Figure 7.10 shows why one should not reduce the BA model to its power-law tail alone. The model typically creates shorter paths than a matched Erdős–Rényi graph because hubs provide shortcuts. At the same time, its clustering is not necessarily large enough to match many empirical networks. This is one reason later models introduced additional mechanisms beyond simple preferential attachment (Albert & Barabási, 2002; Barabási & Pósfai, 2016).

7.18 What the BA model explains and what it does not

The BA model is historically important because it demonstrates that a very simple mechanism can generate heavy-tailed degree distributions and hub formation. It explains, in a mathematically transparent way, how growth plus cumulative advantage can lead to scale-free structure (Albert & Barabási, 2002; Barabási & Albert, 1999; Barabási & Pósfai, 2016).

However, the model also has clear limitations:

1. the exponent is quite specific, with asymptotic value $\gamma = 3$,
2. clustering is often lower than in many real systems,
3. communities, geometry, and hidden attributes are absent,
4. not every heavy-tailed network is generated by pure preferential attachment.

So the BA model should be viewed as a foundational mechanism, not as a complete description of all scale-free networks.

7.19 Core formulas to remember

! Summary

For scale-free structure and the BA model, the most important formulas in this chapter are:

1. **Power-law tail:**
$$P(k) \sim Ck^{-\gamma}$$
2. **Scale-free scaling relation:**
$$P(ak) \sim a^{-\gamma}P(k)$$
3. **Complementary cumulative tail:**
$$P(K \geq k) \sim C'k^{-(\gamma-1)} \qquad (\gamma > 1)$$
4. **Preferential attachment probability:**
$$\Pi_i(t) = \frac{k_i(t)}{\sum_j k_j(t)} = \frac{k_i(t)}{2L(t)}$$
5. **Average degree in the BA model:**
$$\langle k \rangle \approx 2m$$
6. **BA tail exponent:**
$$P(k) \sim Ck^{-3}$$

7.20 In-class questions

1. Why is a heavy-tailed degree distribution different from a narrow degree distribution even when the mean degree is the same?
2. Why does approximate linearity on a log-log plot not by itself prove a power law?
3. What is gained conceptually by studying scale-free structure before introducing the BA model?
4. Why does the configuration model help separate degree-sequence effects from growth effects?
5. Why do early vertices often have an advantage in the BA model?
6. In what sense is the BA model explanatory, and in what sense is it incomplete?

7.21 End-of-chapter exercises

The exercises below are divided into four groups:

1. conceptual and structural exercises,
2. mathematical exercises,
3. model-based analytical exercises,
4. simulation and coding exercises.

7.21.1 Part I. Conceptual and structural exercises

7.21.1.1 Exercise 1. Hubs and heterogeneity

Explain why average degree is not enough to describe a network with strong degree heterogeneity. In your answer, discuss the role of hubs and the limitations of using only a "typical" degree.

7.21.1.2 Exercise 2. What "scale-free" means

Assume that $P(k) \sim Ck^{-\gamma}$.

1. Show that
$$P(ak) \sim a^{-\gamma} P(k)$$
for every fixed $a > 0$.
2. Explain why this motivates the term *scale-free*.
3. Discuss why finite real networks should not be expected to follow a perfect power law at all degrees.

7.21.1.3 Exercise 3. Universality

Write a short analytical note explaining what is meant by universality in the context of scale-free networks. Why does the appearance of heavy tails in many different systems attract theoretical interest?

7.21.1.4 Exercise 4. Ultra-small distances

Explain why a network with strong degree heterogeneity may have shorter typical path length than a homogeneous random graph with comparable average degree. What is the qualitative meaning of the phrase *ultra-small*?

7.21.2 Part II. Mathematical exercises

7.21.2.1 Exercise 5. Cumulative power-law tail

Assume that

$$P(k) \sim Ck^{-\gamma}$$

with $\gamma > 1$.

Show heuristically that

$$P(K \geq k) \sim C' k^{-(\gamma-1)}.$$

Explain why the CCDF is often easier to interpret numerically than the raw degree distribution.

7.21.2.2 Exercise 6. Preferential-attachment probability

Suppose vertex i has degree $k_i(t)$ at time t.

1. Explain why the preferential-attachment rule has the form
$$\Pi_i(t) = \frac{k_i(t)}{\sum_j k_j(t)}.$$
2. Show that
$$\sum_i \Pi_i(t) = 1.$$
3. Give a probabilistic interpretation of this normalization.

7.21.2.3 Exercise 7. Average degree in the BA model

In the BA model, one new vertex and m new edges are added at each time step.

1. Show that for large network size, the number of edges grows approximately linearly in N.
2. Deduce that
$$\langle k \rangle \approx 2m.$$
3. Explain why m controls density but not the tail exponent.

7.21.3 Part III. Model-based analytical exercises

7.21.3.1 Exercise 8. Configuration model versus BA model

Write a careful comparison between the configuration model and the BA model. Your answer should address:

- what each model keeps fixed,
- whether the model is static or growing,
- whether the heavy tail is imposed or generated,
- what structural information is not captured by degree sequence alone.

7.21.3.2 Exercise 9. Early vertices and cumulative advantage

Use the idea of degree trajectories to explain why early vertices often maintain an advantage in the BA model. Does early arrival guarantee that a vertex becomes a hub? Explain carefully.

7.21.3.3 Exercise 10. Limits of the BA model

Discuss at least three limitations of the BA model as a model of real networks. Why is it useful despite those limitations?

7.21.4 Part IV. Simulation and coding exercises

7.21.4.1 Exercise 11. Ordinary versus log-log plots

Generate a heavy-tailed degree sequence and compare its empirical distribution on ordinary axes and on log-log axes. Explain which aspects of the tail become visible only after rescaling the axes.

7.21.4.2 Exercise 12. Configuration-model experiment

Generate a power-law-like degree sequence and use the configuration model to build a graph.

1. Plot its degree distribution on log-log axes.
2. Compute its average degree and maximum degree.
3. Compare the result with an Erdős–Rényi graph of similar mean degree.

7.21.4.3 Exercise 13. BA growth snapshots

Write a short `R` script that generates snapshots of a BA network at several sizes. Use the figure to explain how hubs emerge over time.

7.21.4.4 Exercise 14. Degree dynamics

Simulate the BA model while tracking the degree of several vertices introduced at different times.

1. Plot degree versus time.
2. Compare early and late vertices.
3. Discuss how the figure illustrates cumulative advantage.

7.21.4.5 Exercise 15. Measuring preferential attachment

In a BA simulation, record the current degree of a vertex and whether it receives a new link at the next step.

1. Estimate the empirical attachment rate as a function of current degree.
2. Plot the result.
3. Explain why approximate linearity supports the preferential-attachment hypothesis.

7.21.4.6 Exercise 16. BA versus Erdős–Rényi properties

Generate BA and Erdős–Rényi networks with approximately the same average degree and compare:

- average path length,
- clustering,
- maximum degree,
- the visual appearance of the network.

Write a short discussion explaining which differences arise mainly because of hub structure.

7.21.5 Suggested mini-project

A strong mini-project for this chapter is the following:

> Compare three ways of thinking about heavy-tailed degree structure: descriptive power-law analysis, the configuration model, and the Barabási–Albert growth model.

A complete submission should contain:

- precise mathematical definitions,
- at least three figures,
- one configuration-model simulation,
- one BA growth simulation,
- one comparison with a homogeneous random graph,
- a concluding discussion separating structural signature from generative mechanism.

7.22 References

Chapter 8

Network Robustness

Percolation, Failure, and Resilience in Complex Networks

8.1 Introduction

All previous chapters have studied how networks are built and what structural properties they possess. This chapter asks a different question: *what happens when a network is damaged?*

Real networks — the internet, power grids, transportation systems, biological interaction networks — are subject to failure. Nodes can break down, edges can be severed, and components can become isolated from the rest of the system. The degree to which a network maintains its basic functionality despite such damage is called **network robustness** (Albert et al., 2000; Newman, 2010).

Two fundamentally different modes of failure must be distinguished from the outset:

1. **Random failures**: nodes or edges fail without targeting any particular structural property. A router breaks down due to hardware malfunction; a power line fails due to weather. These failures are essentially random with respect to the network's topology.
2. **Targeted attacks**: failures are directed at the most important nodes. A deliberate attack on infrastructure targets the highest-degree hubs, the most central nodes, or the most critical bridges.

These two modes produce strikingly different outcomes depending on the type of network. In particular, scale-free networks and Erdős–Rényi random graphs respond to damage in ways that are almost opposite to one another. Understanding this asymmetry is one of the central insights of network robustness theory (Albert et al., 2000; Callaway et al., 2000; Cohen et al., 2000).

8.2 Learning goals

By the end of this chapter, it should be possible to:

- define network robustness in terms of the giant component
- distinguish random failure from targeted attack
- formulate robustness as a percolation problem
- state and apply the Molloy–Reed criterion for the existence of a giant component
- derive the critical removal threshold for Erdős–Rényi networks
- explain why scale-free networks are robust to random failures but vulnerable to targeted attacks
- compute and plot robustness curves numerically in `R`

- discuss real-world implications of network robustness

8.3 Chapter roadmap

> Notation convention used in this chapter
>
> Throughout the chapter, f denotes the fraction of removed **nodes**, q denotes the fraction of removed **edges**, and $S(f)$ denotes the size of the giant component normalized by the **original** network size.

The chapter begins with robustness as a connectivity problem and introduces percolation as the natural mathematical framework. It then develops the Molloy–Reed criterion, applies it to Erdős–Rényi networks, and uses that comparison to explain why scale-free networks behave so differently under random failure and targeted attack. The later sections broaden the picture by considering bond percolation, cascading failures, centrality-based attacks, and alternative robustness measures.

8.4 Robustness and the giant component

8.4.1 What does it mean for a network to be robust?

The concept of robustness in a network context is tied to connectivity. The most widely used measure of functional integrity is the size of the **giant component** — the largest connected component — after a fraction f of nodes (or edges) have been removed.

Definition 8.1. The **robustness** of a network under a removal process is characterized by the relative size of the giant component S as a function of the fraction of removed nodes f:

$$S(f) = \frac{|G_{\text{giant}}(f)|}{N},$$

where N is the number of vertices in the original network and $|G_{\text{giant}}(f)|$ is the number of vertices in the largest connected component after removal.

This definition measures the surviving giant component relative to the **original** system size. That convention is standard in robustness studies because it tracks how much of the original network remains functionally connected. One may also normalize by the number of surviving vertices, but that answers a different question (Callaway et al., 2000; Newman, 2010).

When $S(f)$ becomes very small and no component of order N remains, the network has lost its global connectivity. The corresponding transition point is called the **critical threshold** f_c.

> Main idea
>
> Robustness is measured by how large a fraction f of nodes must be removed before the giant component disintegrates. A larger f_c means greater robustness.

8.4.2 Percolation theory

The mathematical framework for studying robustness is **percolation theory**. In the site percolation model (Callaway et al., 2000; Newman et al., 2001; Newman, 2010):

- each node is independently retained with probability $1 - f$, or removed with probability f,
- edges between removed nodes are also removed,
- the question is whether a giant component survives in the remaining subgraph.

This is precisely the model for random failure. Targeted attack requires a different formulation, studied later in this chapter.

The **critical percolation threshold** f_c is the value of f at which the giant component disappears **in the large-network limit**:

$$S(f) > 0 \text{ for } f < f_c, \qquad S(f) = 0 \text{ in the } N \to \infty \text{ sense for } f > f_c.$$

For finite simulated networks, the drop is not perfectly sharp and $S(f)$ usually does not become exactly zero at the theoretical threshold. Instead, one observes a rounded transition due to finite-size effects.

8.5 The Molloy–Reed criterion

8.5.1 Degree distribution moments

A key quantity is the ratio of the second moment to the first moment of the degree distribution. Let p_k denote the probability that a randomly chosen node has degree k.

Define:

$$\langle k \rangle = \sum_k k\, p_k, \qquad \langle k^2 \rangle = \sum_k k^2\, p_k.$$

8.5.2 Criterion for the giant component

Theorem 8.1. ***Molloy–Reed criterion (configuration-model form).*** *For a large, locally tree-like random network with degree distribution $\{p_k\}$, a giant component exists when (Molloy & Reed, 1995; Newman et al., 2001)*

$$\sum_k k(k-2)p_k > 0,$$

which is equivalent to

$$\kappa = \frac{\langle k^2 \rangle}{\langle k \rangle} > 2.$$

The quantity $\kappa = \langle k^2 \rangle / \langle k \rangle$ is closely related to the mean **excess degree**. It measures how many additional edges are expected when one reaches a node by following a random edge.

Intuitively: when following an edge to a neighbor, the expected number of *further* edges from that neighbor is $\langle k^2 \rangle / \langle k \rangle - 1$. When this quantity exceeds 1 — that is, $\kappa > 2$ — the local neighborhoods grow and a giant component forms.

> **Interpretation**
>
> The condition $\kappa > 2$ is equivalent to requiring that a random walk on the network starting from an edge endpoint has, on average, more than one onward step. This branching condition is the network analogue of the subcritical/supercritical transition in branching processes.

8.5.3 Why the second moment matters

A useful teaching point is that robustness is not controlled only by the average degree $\langle k \rangle$. It also depends strongly on how unevenly degree is distributed.

Consider two networks with the same mean degree, say $\langle k \rangle = 4$:

- one network has degrees tightly concentrated around 4,
- another has many low-degree vertices and a few very large hubs.

The second network has a much larger value of $\langle k^2 \rangle$, because squaring amplifies the contribution of hubs. Since

$$f_c = 1 - \frac{1}{\kappa_0 - 1} \qquad \text{with} \qquad \kappa_0 = \frac{\langle k^2 \rangle}{\langle k \rangle},$$

a larger second moment usually means a larger threshold against **random** failure. This is the mathematical reason heterogeneous networks can be surprisingly robust.

> **!** Pedagogical takeaway
>
> Two networks can have the same average degree but very different robustness. The crucial quantity is not only how many links vertices have on average, but how unevenly those links are distributed.

8.5.4 Stability condition under random removal

When nodes are removed independently with probability f, the effective degree distribution changes. A node of original degree k retains each neighbor independently with probability $1 - f$, so its residual degree follows a Binomial$(k, 1 - f)$ distribution.

After removal, the effective first and second moments become:

$$\langle k \rangle' = (1 - f)\langle k \rangle,$$

$$\langle k^2 \rangle' = (1 - f)^2 \langle k^2 \rangle + (1 - f) f \langle k \rangle.$$

Substituting into the Molloy–Reed criterion, the giant component survives if:

$$\kappa(f) = \frac{\langle k^2 \rangle'}{\langle k \rangle'} = (1 - f)\frac{\langle k^2 \rangle}{\langle k \rangle} + f > 2.$$

Setting $\kappa(f_c) = 2$ and solving for f_c:

$$\boxed{f_c = 1 - \frac{1}{\kappa_0 - 1}} \qquad \text{where} \quad \kappa_0 = \frac{\langle k^2 \rangle}{\langle k \rangle}.$$

This formula is exact for locally tree-like networks (networks with negligible number of short cycles in the large-N limit) and provides a good approximation for many real networks (Cohen et al., 2000; Newman et al., 2001; Newman, 2010).

With the general threshold formula in hand, the next step is to apply it to concrete graph families. The Erdős–Rényi model is the natural starting point because its degree distribution is analytically tractable and serves as a benchmark for more heterogeneous networks.

8.6 Robustness of Erdős–Rényi networks

8.6.1 Degree distribution and moments

In the Erdős–Rényi model $G(N, p)$ with large N and fixed $\langle k \rangle = p(N-1) \approx pN$, the degree distribution is approximately Poisson (Barabási & Pósfai, 2016; Newman, 2010):

$$p_k \approx e^{-\langle k \rangle} \frac{\langle k \rangle^k}{k!}.$$

For the Poisson distribution, the moments satisfy:

$$\langle k^2 \rangle = \langle k \rangle^2 + \langle k \rangle,$$

and therefore:

$$\kappa_0 = \frac{\langle k^2 \rangle}{\langle k \rangle} = \langle k \rangle + 1.$$

8.6.2 Critical threshold for ER networks

Substituting into the formula for f_c:

$$f_c^{\mathrm{ER}} = 1 - \frac{1}{\kappa_0 - 1} = 1 - \frac{1}{\langle k \rangle}.$$

This is a clean result. For an Erdős–Rényi network with average degree $\langle k \rangle$:

- when $\langle k \rangle = 2$, the threshold is $f_c = 1/2$: removing half the nodes destroys the giant component.
- when $\langle k \rangle = 5$, the threshold is $f_c = 4/5$: 80% of nodes must be removed.
- as $\langle k \rangle \to \infty$, $f_c \to 1$: a very dense ER network is highly robust.

The ER network thus has a finite, deterministic robustness threshold under random failure. It is neither extremely robust nor extremely fragile. This makes ER a useful baseline: once we turn to scale-free networks, the contrast becomes much sharper.

```
simulate_robustness_er <- function(n, avg_k, steps = 50, reps = 10) {
  p <- avg_k / (n - 1)
  f_vals <- seq(0, 0.98, length.out = steps)

  map_dfr(f_vals, function(f) {
    s_vals <- replicate(reps, {
      g  <- sample_gnp(n, p)
      n_remove <- floor(f * vcount(g))
      if (n_remove > 0) {
        remove_v <- sample(V(g), n_remove)
        g <- delete_vertices(g, remove_v)
      }
      if (vcount(g) == 0) return(0)
      max(components(g)$csize) / n
    })
    data.frame(f = f, S = mean(s_vals))
  })
}
```

```r
set.seed(101)
params <- list(
  list(avg_k = 2, col = course_cols$blue,  label = "k = 2"),
  list(avg_k = 4, col = course_cols$teal,  label = "k = 4"),
  list(avg_k = 8, col = course_cols$gold,  label = "k = 8")
)

rob_data <- map_dfr(params, function(par) {
  df <- simulate_robustness_er(n = 300, avg_k = par$avg_k, steps = 30, reps = 8)
  df$avg_k <- par$label
  df
})

fc_df <- data.frame(
  avg_k   = c("k = 2", "k = 4", "k = 8"),
  fc      = c(1 - 1/2, 1 - 1/4, 1 - 1/8),
  col_val = c(course_cols$blue, course_cols$teal, course_cols$gold)
)

ggplot(rob_data, aes(x = f, y = S, color = avg_k)) +
  geom_line(linewidth = 1.1) +
  geom_point(size = 1.5, alpha = 0.7) +
  geom_vline(data = fc_df, aes(xintercept = fc, color = avg_k),
             linetype = "dashed", linewidth = 0.8, alpha = 0.85) +
  scale_color_manual(values = c("k = 2" = course_cols$blue,
                                "k = 4" = course_cols$teal,
                                "k = 8" = course_cols$gold),
                     name = "Average degree") +
  labs(
    title    = "Random failure in Erdős--Rényi networks",
    subtitle = "Giant component size $S(f)$ versus removed fraction $f$",
    x        = "Fraction of nodes removed (f)",
    y        = "Giant component size S(f)"
  ) +
  scale_x_continuous(breaks = seq(0, 1, 0.2)) +
  scale_y_continuous(breaks = seq(0, 1, 0.2), limits = c(0, 1))
```

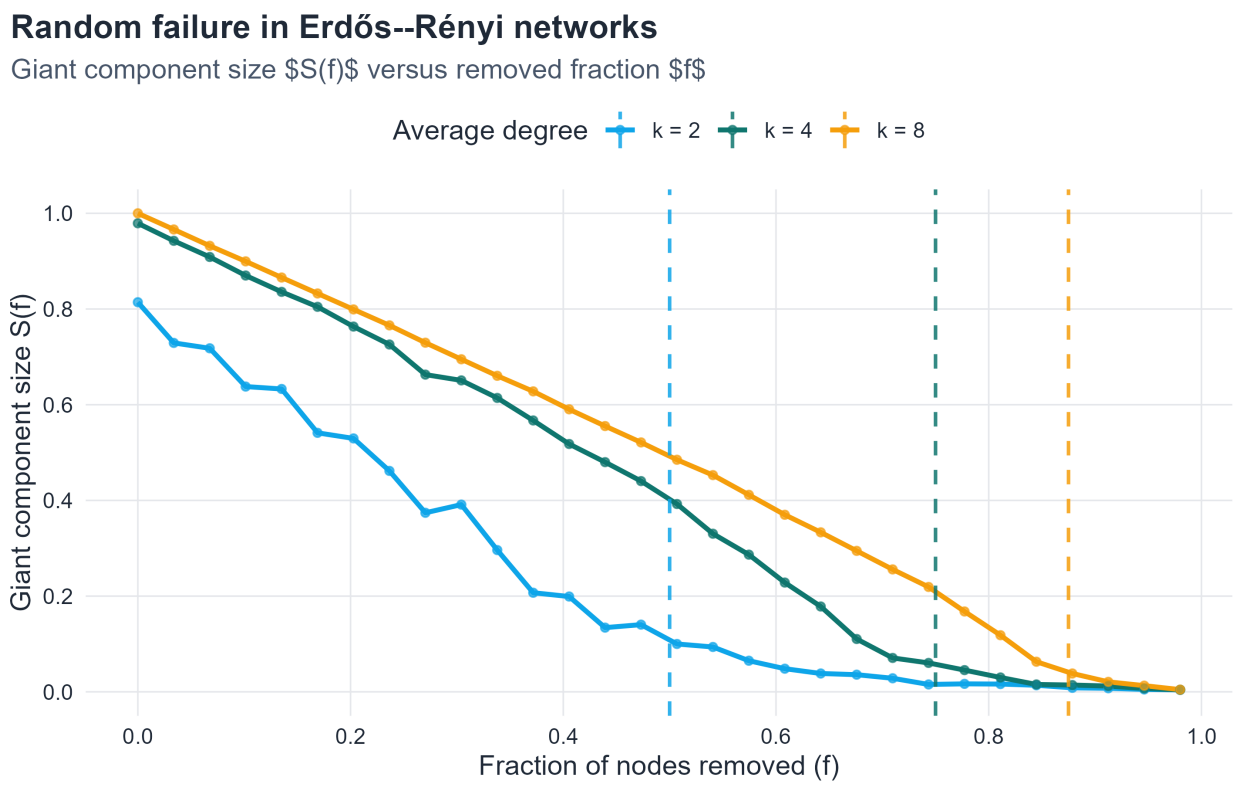


Figure 8.1: Robustness curves for Erdős–Rényi networks under random node removal. Each curve corresponds to a different average degree $\langle k \rangle$. Dashed vertical lines indicate the theoretical critical thresholds $f_c = 1 - 1/\langle k \rangle$. The giant component size is normalized by the original network size.

As shown in Figure 8.1, the decline of the giant component becomes slower as the average degree increases. This is exactly what the formula $f_c^{\mathrm{ER}} = 1 - 1/\langle k \rangle$ predicts: denser Erdős–Rényi networks can tolerate a larger fraction of random node removals before losing global connectivity.

The dashed vertical lines mark the theoretical thresholds, while the simulated curves remain smooth rather than perfectly sharp. That mismatch is expected. The formula describes the large-network transition, whereas the plotted curves come from finite simulations, where the breakdown is rounded by finite-size fluctuations.

The next question is whether this behavior is typical of all sparse networks. The answer is no. Once degree heterogeneity becomes extreme, the robustness picture changes qualitatively.

```
simulate_robustness_full <- function(g, strategy = c("random", "targeted"), steps = 40) {
  strategy <- match.arg(strategy)
  n0 <- vcount(g)
  f_vals <- seq(0, 0.98, length.out = steps)

  purrr::map_dfr(f_vals, function(f) {
    g_tmp <- g
    n_remove <- floor(f * n0)

    if (n_remove > 0) {
      removable <- min(n_remove, vcount(g_tmp))

      if (strategy == "random") {
```

```
        remove_v <- sample(V(g_tmp), removable)
      } else {
        deg_order <- order(degree(g_tmp), decreasing = TRUE)
        remove_v <- V(g_tmp)[deg_order[seq_len(removable)]]
      }

      g_tmp <- delete_vertices(g_tmp, remove_v)
    }

    S <- if (vcount(g_tmp) == 0) 0 else max(components(g_tmp)$csize) / n0
    data.frame(f = f, S = S)
  })
}
```

8.7 Robustness of scale-free networks

8.7.1 Power-law degree distribution and diverging moments

For a scale-free network with degree distribution $p_k \sim Ck^{-\gamma}$, the behavior of the moments $\langle k \rangle$ and $\langle k^2 \rangle$ depends critically on the exponent γ (Albert & Barabási, 2002; Barabási & Albert, 1999; Newman, 2010).

For $\gamma > 3$: both $\langle k \rangle$ and $\langle k^2 \rangle$ are finite.

For $2 < \gamma \leq 3$: $\langle k \rangle$ is finite but $\langle k^2 \rangle$ **diverges** as $N \to \infty$.

For $\gamma \leq 2$: both moments diverge.

The key case for many real networks is $2 < \gamma \leq 3$. In this regime:

$$\kappa_0 = \frac{\langle k^2 \rangle}{\langle k \rangle} \to \infty \quad \text{as } N \to \infty.$$

8.7.2 Infinite robustness to random failure

Substituting $\kappa_0 \to \infty$ into the critical threshold formula:

$$f_c = 1 - \frac{1}{\kappa_0 - 1} \to 1 \quad \text{as } N \to \infty.$$

This is a remarkable result. For a scale-free network with $2 < \gamma \leq 3$, the critical threshold under random failure approaches 1 in the large-N limit. This means that **no finite fraction of random node removals can destroy the giant component** (Cohen et al., 2000; Newman et al., 2001).

> **i** Error tolerance in scale-free networks
>
> Scale-free networks are almost perfectly robust to random failures. Because most nodes have very small degree, a randomly chosen node is almost certainly not a hub. The hubs — which hold the network together — are rarely hit by chance.

Albert, Jeong, and Barabási highlighted this phenomenon as **error tolerance**: the ability to withstand random failures, while contrasting it sharply with targeted attack vulnerability (Albert et al., 2000).

Finite-size caveat

The statement $f_c \to 1$ is asymptotic. In a simulated Barabási–Albert network with a few hundred or a few thousand nodes, the threshold is still below 1, and the robustness curve is rounded rather than singular. It is therefore better to say that scale-free networks are **highly** robust to random failures, not literally indestructible in finite size.

8.7.3 Simulation: random failure in a Barabási–Albert network

The asymptotic result $f_c \to 1$ is easier to understand after seeing a direct simulation. A Barabási–Albert (BA) network is a convenient classroom model of scale-free structure because it produces a heavy-tailed degree distribution with a small number of hubs and many low-degree vertices.

```
set.seed(123)

g_ba_rf <- sample_pa(n = 1000, m = 3, directed = FALSE)

df_ba_rf <- simulate_robustness_full(g_ba_rf, strategy = "random", steps = 50)

ggplot(df_ba_rf, aes(x = f, y = S)) +
  geom_line(color = course_cols$coral, linewidth = 1.2) +
  geom_point(color = course_cols$coral, size = 1.7, alpha = 0.85) +
  labs(
    title = "Random failure in a scale-free network",
    subtitle = "Barabási--Albert model with N = 1000 and m = 3",
    x = "Fraction of removed nodes, f",
    y = "Normalized giant component size, S(f)"
  ) +
  scale_x_continuous(limits = c(0, 1), breaks = seq(0, 1, 0.2)) +
  scale_y_continuous(limits = c(0, 1), breaks = seq(0, 1, 0.2))
```

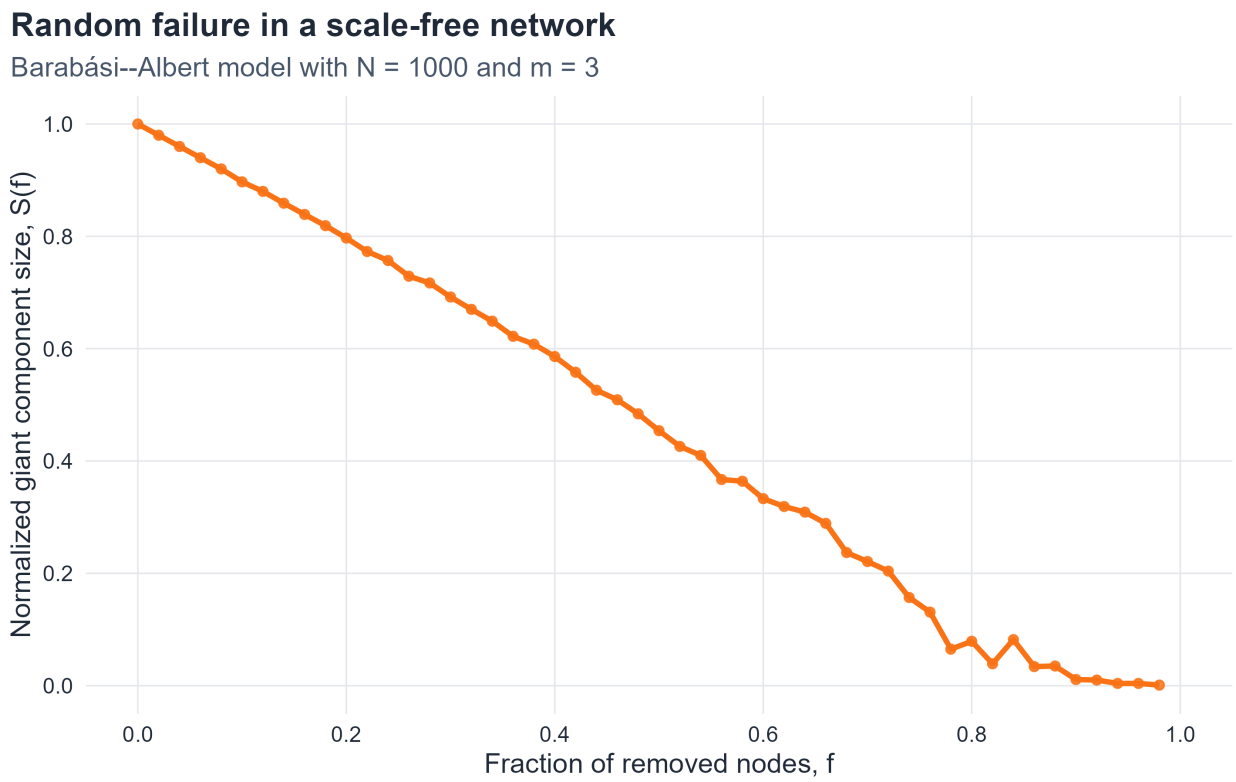


Figure 8.2: Robustness curve for a Barabási–Albert scale-free network under random node removal. Because random failures usually hit low-degree vertices rather than hubs, the giant component declines gradually.

As shown in Figure 8.2, the giant component decreases much more slowly than one might expect from a homogeneous random graph with similar average degree. The reason is structural: random deletion samples vertices almost uniformly, but the network's connectivity is distributed very non-uniformly. Most removals hit peripheral vertices, while the hubs that knit the network together survive for a long time.

This figure should not be overinterpreted as proving that a finite scale-free network is immune to random failure. The decline is still visible, and for sufficiently large f the giant component eventually collapses. The correct interpretation is more careful: compared with homogeneous networks, the onset of fragmentation is strongly delayed.

8.7.4 Simulation: different scale-free regimes under random failure

A deeper point is that not all scale-free networks behave in exactly the same way. The robustness depends on the power-law exponent γ. Smaller values of γ produce heavier tails, more extreme hubs, and therefore a larger second moment $\langle k^2 \rangle$.

```
sample_powerlaw_degrees <- function(n, gamma, k_min = 2, k_max = 80) {
  ks <- k_min:k_max
  probs <- ks^(-gamma)
  probs <- probs / sum(probs)
  deg <- sample(ks, size = n, replace = TRUE, prob = probs)
  if (sum(deg) %% 2 == 1) {
    i <- sample(seq_len(n), 1)
    deg[i] <- deg[i] + 1
  }
  deg
```

```
}

make_scale_free_cfg <- function(n, gamma, k_min = 2, k_max = 80) {
  deg <- sample_powerlaw_degrees(n, gamma, k_min, k_max)
  g <- igraph::sample_degseq(deg, method = "vl")
  g <- igraph::simplify(g, remove.multiple = TRUE, remove.loops = TRUE)
  comps <- igraph::components(g)
  giant_id <- which.max(comps$csize)
  igraph::induced_subgraph(g, which(comps$membership == giant_id))
}

set.seed(124)
gammas <- c(2.4, 3.0, 3.6)

df_sf_regimes <- map_dfr(gammas, function(gamma) {
  g <- make_scale_free_cfg(n = 1500, gamma = gamma, k_min = 2, k_max = 100)
  simulate_robustness_full(g, strategy = "random", steps = 50) %>%
    mutate(gamma = paste0("gamma = ", gamma))
})

ggplot(df_sf_regimes, aes(x = f, y = S, color = gamma)) +
  geom_line(linewidth = 1.15) +
  labs(
    title = "Random failure in different scale-free regimes",
    subtitle = "Heavier-tailed degree distributions remain connected for longer",
    x = "Fraction of removed nodes, f",
    y = "Normalized giant component size, S(f)",
    color = "Degree exponent"
  ) +
  scale_x_continuous(limits = c(0, 1), breaks = seq(0, 1, 0.2)) +
  scale_y_continuous(limits = c(0, 1), breaks = seq(0, 1, 0.2))
```

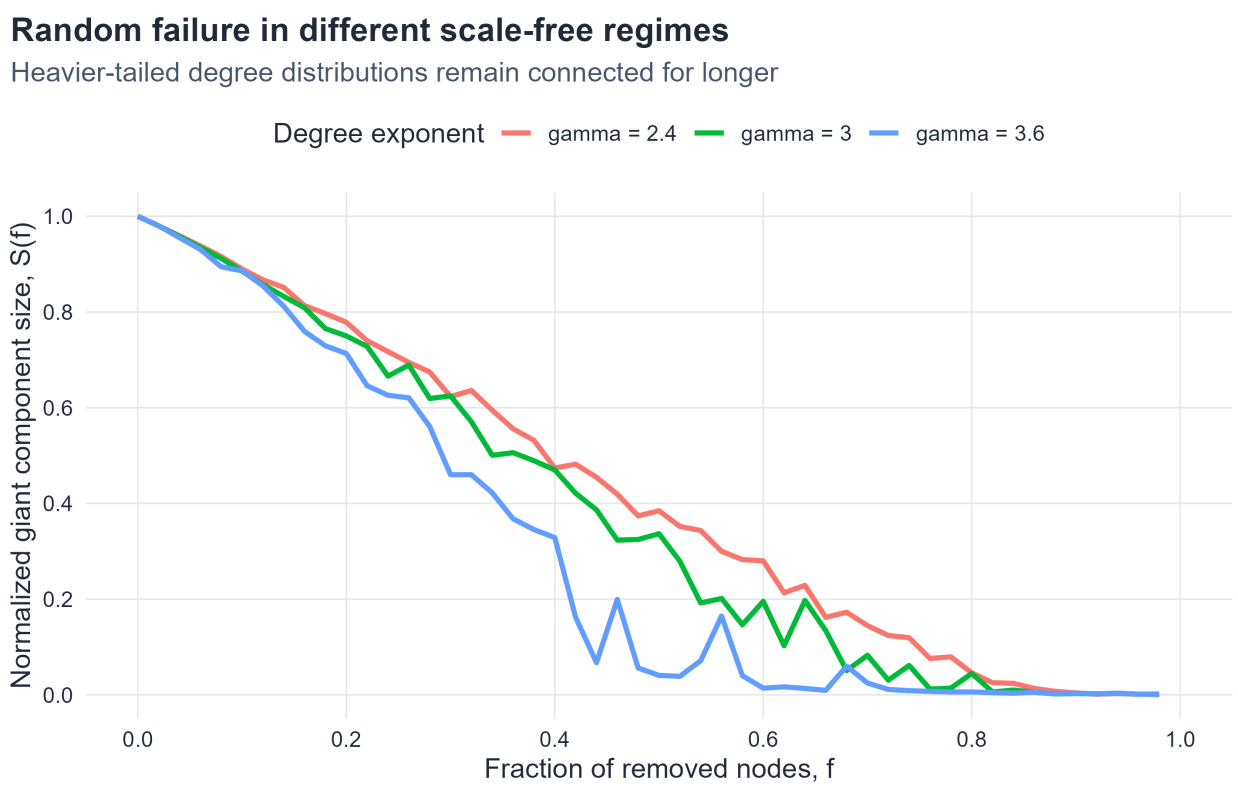


Figure 8.3: Random-failure robustness curves for scale-free networks with different degree exponents. Smaller values of γ produce heavier tails and greater robustness because connectivity is concentrated in hubs that are unlikely to be removed at random.

As shown in Figure 8.3, the curve for smaller γ stays higher for longer. This reflects the heavier tail of the degree distribution: when γ is small, a few extremely large hubs dominate connectivity, so random failure tends to remove many low-degree vertices before it reaches the structural core of the network.

The comparison is also a useful correction to an overly simple slogan such as "scale-free networks are robust." A more accurate statement is that robustness under random failure depends on the degree exponent and on network size. Scale-free structure is not a single regime, and Figure 8.3 makes that dependence visible.

8.7.5 Catastrophic vulnerability to targeted attack

The situation reverses completely when failures are targeted at the highest-degree nodes.

Under a targeted attack strategy:

- nodes are removed in decreasing order of degree,
- in a **static** attack, the ranking is computed once from the original network,
- in an **adaptive** attack, degrees are recalculated after each removal.

Because the hubs of a scale-free network carry a disproportionately large fraction of edges, removing even a small number of them causes $\langle k^2 \rangle$ to drop sharply. Once the high-degree tail is truncated, the network rapidly loses the heterogeneity that previously protected it against random failure.

For this reason, the critical fraction under targeted attack is typically **much smaller** than under random failure. In finite scale-free networks, it is common to observe severe fragmentation after removing only a few percent of the top hubs, although the exact threshold depends on network size, attack rule, and degree exponent.

Attack vulnerability in scale-free networks

Scale-free networks display a striking asymmetry: highly tolerant to random failures, yet extremely sensitive to attacks on their hubs. This is the *Achilles heel* of scale-free structure.

```
set.seed(2077)
n_net  <- 500
g_ba   <- sample_pa(n = n_net, power = 1, m = 2, directed = FALSE)
g_er   <- sample_gnp(n = n_net, p = mean(degree(g_ba)) / (n_net - 1))

df_ba_rand <- simulate_robustness_full(g_ba, "random",   40) %>% mutate(type = "Scale-free (BA)",      a
df_ba_targ <- simulate_robustness_full(g_ba, "targeted", 40) %>% mutate(type = "Scale-free (BA)",      a
df_er_rand <- simulate_robustness_full(g_er, "random",   40) %>% mutate(type = "Erdős-Rényi",          a
df_er_targ <- simulate_robustness_full(g_er, "targeted", 40) %>% mutate(type = "Erdős-Rényi",          a

rob_full <- bind_rows(df_ba_rand, df_ba_targ, df_er_rand, df_er_targ) %>%
  mutate(
    network  = type,
    scenario = attack,
    linetype = ifelse(attack == "Random failure", "solid", "dashed")
  )

ggplot(rob_full, aes(x = f, y = S,
                     color = network,
                     linetype = scenario)) +
  geom_line(linewidth = 1.1) +
  scale_color_manual(values = c("Scale-free (BA)" = course_cols$coral,
                                "Erdős-Rényi"     = course_cols$blue),
                     name = "Network type") +
  scale_linetype_manual(values = c("Random failure" = "solid",
                                   "Targeted attack" = "dashed"),
                        name = "Strategy") +
  labs(
    title    = "Random failure versus targeted attack",
    subtitle = "Scale-free networks are robust under random failure but fragile under attack",
    x        = "Fraction of nodes removed (f)",
    y        = "Giant component size S(f)"
  ) +
  scale_x_continuous(breaks = seq(0, 1, 0.2)) +
  scale_y_continuous(breaks = seq(0, 1, 0.2), limits = c(0, 1))
```

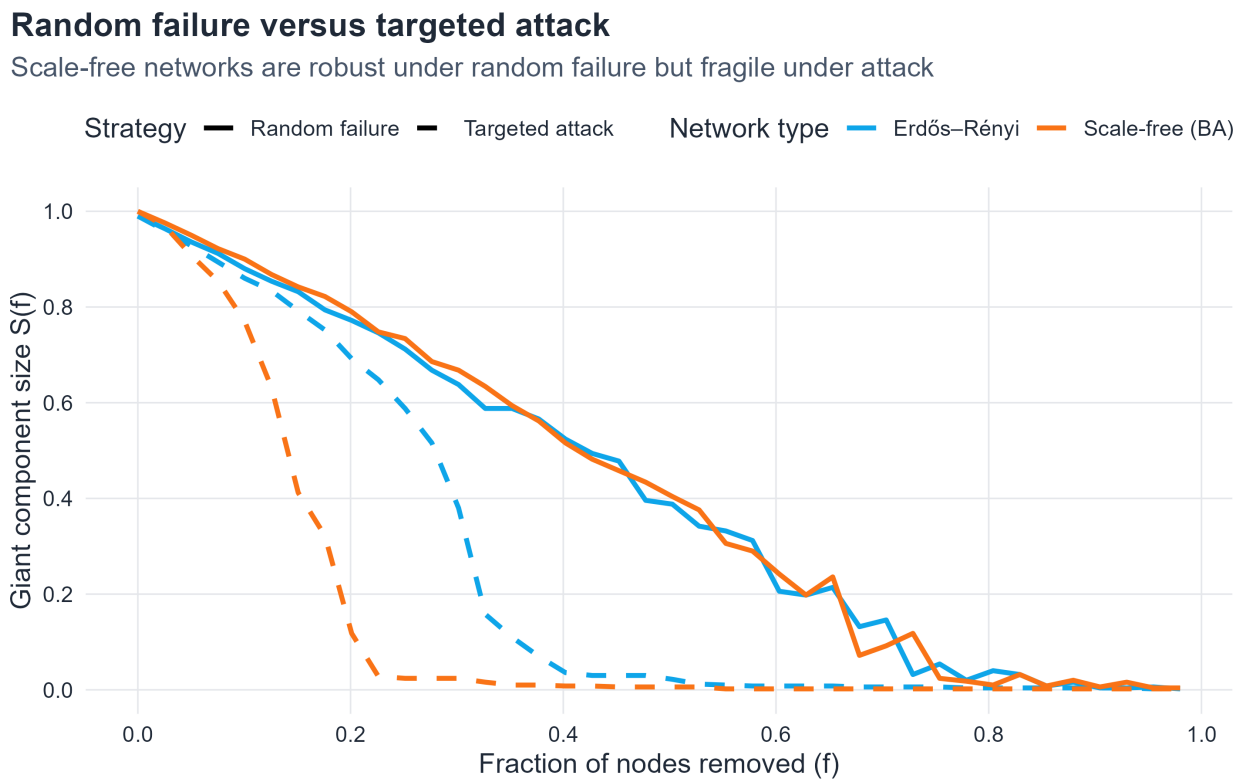


Figure 8.4: Robustness comparison of a scale-free Barabási–Albert network and an Erdős–Rényi network under random failure (solid lines) and static degree-based attack (dashed lines). Both networks have $N = 500$ and comparable average degree.

As shown in Figure 8.4, this comparison condenses the central message of the chapter into a single picture. Under random failure, the scale-free curve stays above the ER curve because random deletion rarely removes the hubs. Under targeted attack, the ordering reverses: the BA network fragments dramatically once the highest-degree vertices are removed, while the ER network degrades more steadily because it has no similarly dominant hubs.

It is also useful to compare the gap between the two curves inside each network family. In the ER case, the random-failure and targeted-attack curves are different, but not radically so. In the scale-free case, the separation is much larger. That large internal gap is the geometric expression of the famous robustness asymmetry of scale-free structure: protection against random errors comes from the same hubs that create extreme vulnerability under deliberate attack.

8.8 Static versus adaptive attack

The previous simulation uses a **static** degree ranking for the attack curve: for each chosen fraction f, the nodes with the largest degrees in the original graph are removed. This is useful for a first comparison, but in many robustness studies one also considers an **adaptive** attack, where degrees are recalculated after each deletion. Adaptive attacks are usually more destructive because the identity of the most important remaining node can change during the attack. This distinction is worth showing explicitly, since students often assume that "remove the top hubs" has only one meaning.

```
simulate_adaptive_attack <- function(g, steps = 40) {
  n0     <- vcount(g)
  f_vals <- seq(0, 0.95, length.out = steps)
```

```
  map_dfr(f_vals, function(f) {
    g_tmp <- g
    n_remove <- floor(f * n0)

    if (n_remove > 0) {
      for (i in seq_len(min(n_remove, vcount(g_tmp)))) {
        degs <- degree(g_tmp)
        v_star <- which.max(degs)
        g_tmp <- delete_vertices(g_tmp, v_star)
        if (vcount(g_tmp) == 0) break
      }
    }

    S <- if (vcount(g_tmp) == 0) 0 else max(components(g_tmp)$csize) / n0
    data.frame(f = f, S = S)
  })
}

set.seed(900)
g_ba_attack <- sample_pa(n = 300, m = 2, directed = FALSE)

df_static <- simulate_robustness_full(g_ba_attack, strategy = "targeted", steps = 35) %>%
  mutate(attack = "Static degree attack")

df_adaptive <- simulate_adaptive_attack(g_ba_attack, steps = 35) %>%
  mutate(attack = "Adaptive degree attack")

bind_rows(df_static, df_adaptive) %>%
  ggplot(aes(x = f, y = S, color = attack)) +
  geom_line(linewidth = 1.1) +
  geom_point(size = 1.5, alpha = 0.75) +
  scale_color_manual(values = c("Static degree attack" = course_cols$gold,
                                "Adaptive degree attack" = course_cols$coral)) +
  labs(
    title = "Static versus adaptive targeted attack",
    subtitle = "Adaptive attacks typically fragment the network sooner",
    x = "Fraction of nodes removed (f)",
    y = "Giant component size S(f)"
  ) +
  scale_x_continuous(breaks = seq(0, 1, 0.2)) +
  scale_y_continuous(limits = c(0, 1), breaks = seq(0, 1, 0.2))
```

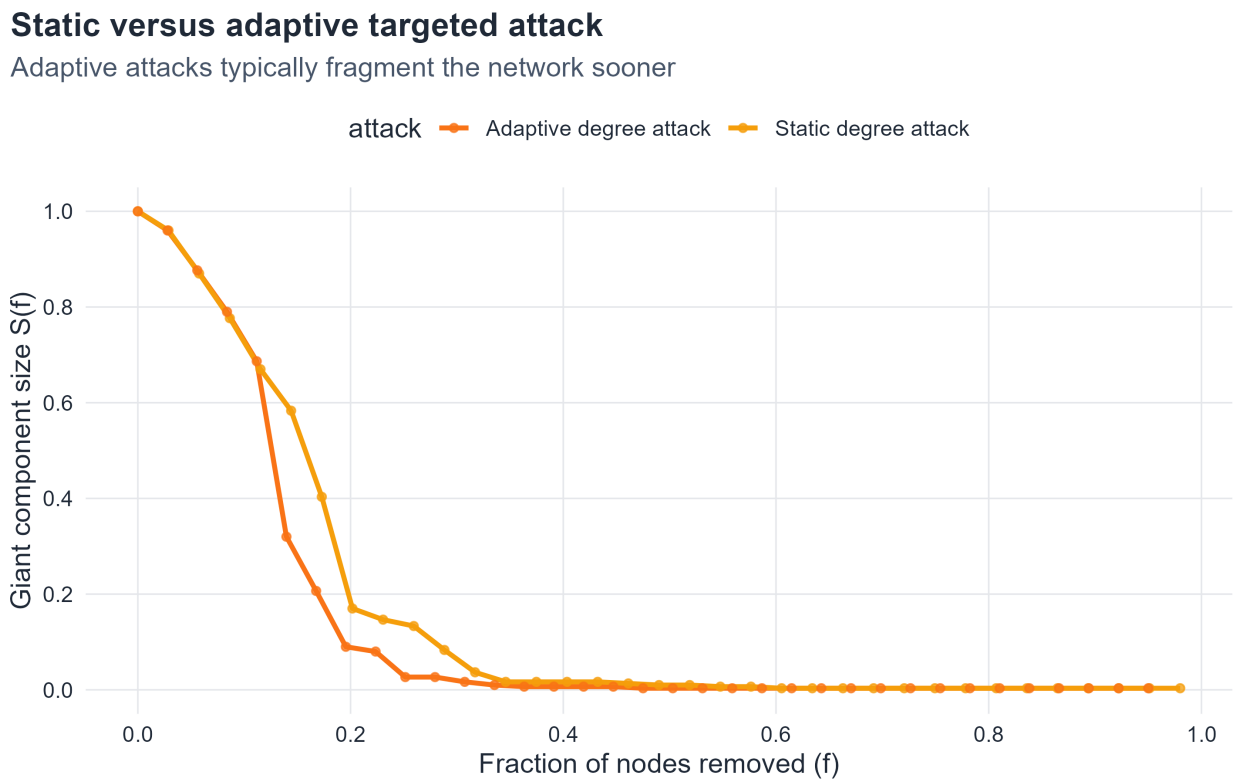


Figure 8.5: Static versus adaptive degree-based attack on a Barabási–Albert network. Recomputing degrees after each deletion usually fragments the network more rapidly.

As shown in Figure 8.5, the adaptive curve lies below the static one because the attack is updated after every deletion. Once an important hub is removed, another vertex may become the new structural bottleneck, and an adaptive strategy immediately identifies and removes it. A static strategy, by contrast, commits to the original ranking and may waste later deletions on vertices that are no longer the most damaging targets.

This distinction is conceptually important. "Remove the highest-degree nodes" is not a single, unambiguous rule. In practice, the effect of an attack depends on whether the attacker has current information about the damaged network. Adaptive attacks are therefore a better model for an intelligent adversary, while static attacks are often sufficient for illustrating the basic vulnerability of hub-dominated networks.

8.9 Edge percolation and bond percolation

The analysis above focused on **site percolation** (node removal). An analogous problem arises when edges rather than nodes are removed, known as **bond percolation**.

In bond percolation, each edge is independently retained with probability $1-q$ and removed with probability q. The critical threshold q_c is the fraction of edge removals at which the giant component disappears.

For an Erdős–Rényi network:

$$q_c^{\mathrm{ER}} = 1 - \frac{1}{\langle k \rangle}.$$

This is structurally identical to the site percolation formula for ER networks, which reflects the fact that the ER model's locally tree-like structure makes node and edge percolation equivalent at leading order.

For a scale-free network with $2 < \gamma \leq 3$, the bond percolation threshold also diverges: $q_c \to 1$ as $N \to \infty$, for the same reason as in site percolation.

8.10 The cascading failure problem

So far, robustness has been treated as a purely static problem: remove nodes or edges, measure the remaining giant component. In practice, failures often **cascade**.

When a node fails, it may increase the load on neighboring nodes. If those nodes then fail due to overload, further failures follow. This cascade can amplify a small initial failure into a large-scale collapse.

This phenomenon appears in:

- power grids (overload cascades leading to blackouts),
- financial networks (default cascades),
- biological networks (synthetic lethality in gene knockout experiments).

While a full treatment of cascading failure requires models beyond pure percolation theory, the basic robustness analysis provides intuition for when cascades are likely to be catastrophic: networks near their percolation threshold f_c are most susceptible, since only a small fraction of additional failures is needed to push the system over the edge.

8.11 Comparing ER and scale-free robustness

The fundamental difference between Erdős–Rényi and scale-free robustness can be summarized as follows.

In an ER network, degrees are homogeneous: no node is dramatically more connected than any other. Random removal and targeted attack behave similarly, and the critical threshold is finite for both.

In a scale-free network, degrees are highly heterogeneous: most nodes have small degree, and a few hubs have enormous degree. Random removal almost always hits low-degree nodes, leaving the hubs intact and the giant component connected. Targeted attack, conversely, destroys the hubs immediately, rapidly fragmenting the network.

```
set.seed(2077)
g_ba_deg <- sample_pa(n = 1000, power = 1, m = 2, directed = FALSE)
g_er_deg <- sample_gnp(n = 1000, p = mean(degree(g_ba_deg)) / 999)

df_deg <- bind_rows(
  data.frame(degree = degree(g_ba_deg), type = "Scale-free (BA)"),
  data.frame(degree = degree(g_er_deg), type = "Erdős-Rényi")
)

df_deg_summary <- df_deg %>%
  group_by(type, degree) %>%
  summarise(count = n(), .groups = "drop") %>%
  group_by(type) %>%
  mutate(prob = count / sum(count))

ggplot(df_deg_summary, aes(x = degree, y = prob, color = type)) +
  geom_line(linewidth = 1.0, alpha = 0.9) +
  geom_point(size = 1.5, alpha = 0.7) +
  scale_color_manual(values = c("Scale-free (BA)" = course_cols$coral,
                                "Erdős-Rényi"     = course_cols$blue),
                     name = "Network type") +
  scale_x_log10() +
```

```
  scale_y_log10() +
  labs(
    title    = "Degree distributions on a log--log scale",
    subtitle = "N = 1000 nodes; scale-free shows heavy tail, ER shows rapid decay",
    x        = "Degree k",
    y        = "P(k)"
  )
```

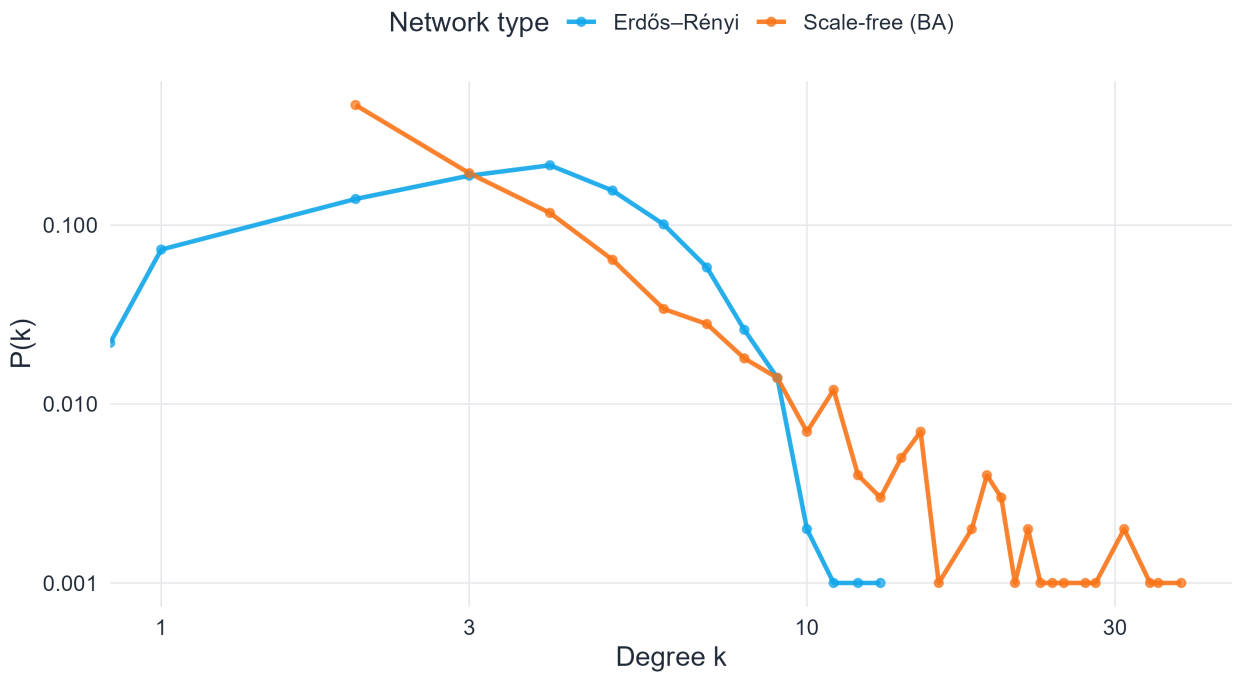


Figure 8.6: Degree distributions of a Barabási–Albert network and an Erdős–Rényi network with comparable average degree. On the log–log scale, the BA network shows a heavy tail, meaning that hubs are rare but exceptionally well connected.

As shown in Figure 8.6, the visual difference between the two degree distributions is already enough to anticipate the robustness contrast. The Erdős–Rényi network concentrates most vertices near a typical degree, whereas the Barabási–Albert network has a long tail extending toward much larger degrees. That heavy tail is precisely what increases robustness under random failure and vulnerability under targeted attack.

8.12 Betweenness centrality and attack vulnerability

Targeted attacks need not be restricted to removing the highest-degree nodes. A more sophisticated attack strategy targets nodes by **betweenness centrality**.

Definition 8.2. The **betweenness centrality** of a vertex v is

$$B(v) = \sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}},$$

where σ_{st} is the total number of shortest paths from s to t, and $\sigma_{st}(v)$ is the number of those paths that pass through v.

Nodes with high betweenness are critical bridges in the network: many shortest paths flow through them. Removing such nodes disrupts the communication structure more severely than removing high-degree nodes when the two sets differ.

In scale-free networks, high-degree hubs also tend to have high betweenness (since many paths route through them), so degree-based and betweenness-based attacks are often correlated. In other network types — such as networks with a core-periphery structure — they can diverge substantially.

```
set.seed(33)
g_hub <- sample_pa(n = 200, power = 1, m = 2, directed = FALSE)

hub_df <- data.frame(
  degree      = degree(g_hub),
  betweenness = betweenness(g_hub, normalized = TRUE)
)

ggplot(hub_df, aes(x = degree, y = betweenness)) +
  geom_point(color = course_cols$coral, alpha = 0.65, size = 2) +
  geom_smooth(method = "lm", formula = y ~ x, se = FALSE,
              color = course_cols$ink, linewidth = 0.8, linetype = "dashed") +
  labs(
    title    = "Degree vs. betweenness centrality (scale-free network)",
    subtitle = "Hubs accumulate both high degree and high betweenness",
    x        = "Degree k",
    y        = "Normalized betweenness B(v)"
  )
```

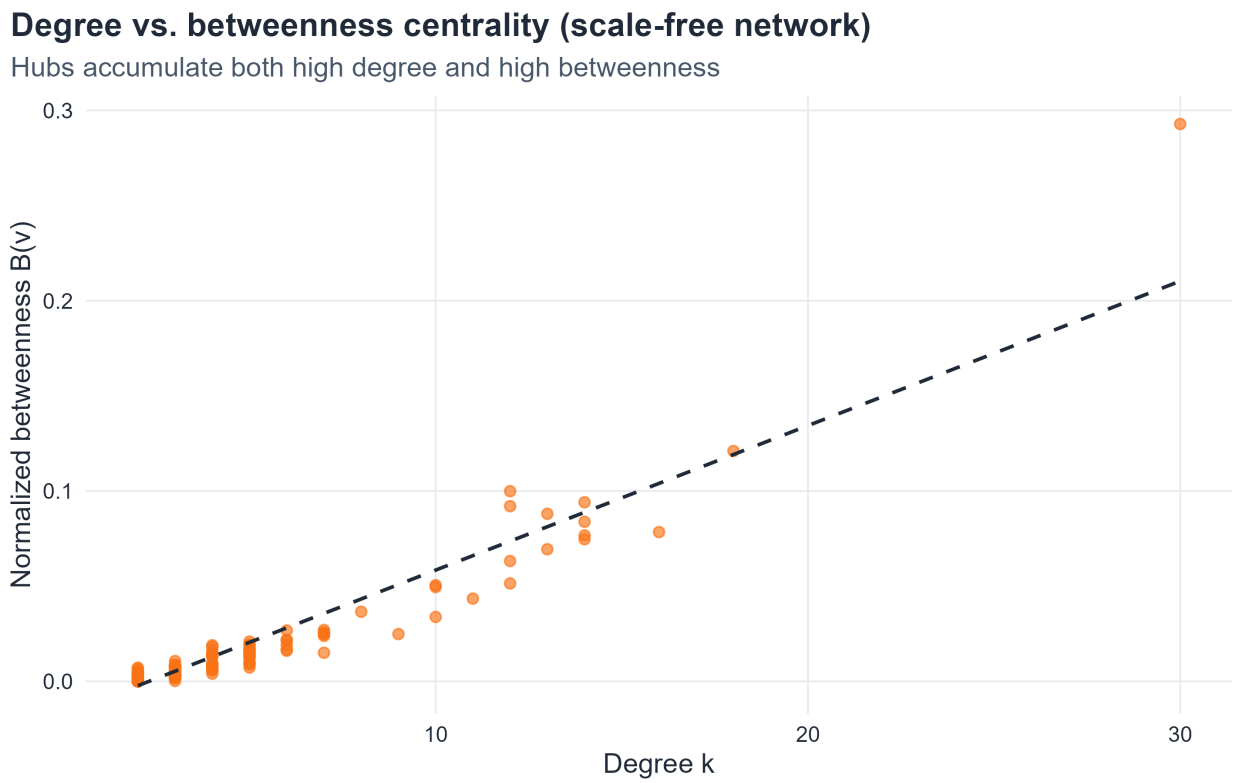


Figure 8.7: Betweenness centrality versus degree for vertices in a scale-free network. High-degree hubs often also have high betweenness, which makes them especially important for global connectivity.

As shown in Figure 8.7, vertices with large degree often also have large betweenness, although the relationship is not perfectly deterministic. This is why degree-based attack is often effective in scale-free networks: high-degree hubs are not only locally well connected, but also occupy strategically important positions in shortest-path structure.

8.13 Measuring robustness: the robustness index R

A single scalar summary of a network's robustness is useful for comparison. One such measure is the **robustness index** R, defined as the area under the robustness curve:

$$R \approx \int_0^1 S(f)\, df.$$

In numerical work, if $S(f)$ is evaluated on an evenly spaced grid of removal fractions, one often approximates this integral by the mean of the sampled values; a trapezoidal rule gives a slightly more accurate numerical estimate. In other words, R is best interpreted as a discrete approximation to the area under the curve, not as a fundamentally different quantity.

A network that maintains a large giant component for a wide range of f has a large R. A network that collapses quickly has a small R.

```
compute_R <- function(S_vec) mean(S_vec)

set.seed(888)
n_comp <- 300
```

```r
g_ba_r <- sample_pa(n = n_comp, power = 1, m = 2, directed = FALSE)
g_er_r <- sample_gnp(n = n_comp, p = mean(degree(g_ba_r)) / (n_comp - 1))

strategies <- c("random", "targeted")
networks   <- list("Scale-free (BA)" = g_ba_r, "Erdős-Rényi" = g_er_r)

R_results <- map_dfr(names(networks), function(nm) {
  g <- networks[[nm]]
  map_dfr(strategies, function(strat) {
    df <- simulate_robustness_full(g, strat, steps = 50)
    data.frame(
      network  = nm,
      strategy = strat,
      R        = compute_R(df$S)
    )
  })
})

R_results$strategy <- recode(R_results$strategy,
                             "random"   = "Random failure",
                             "targeted" = "Targeted attack")

ggplot(R_results, aes(x = network, y = R, fill = strategy)) +
  geom_col(position = position_dodge(0.7), width = 0.6, alpha = 0.85) +
  scale_fill_manual(values = c("Random failure"  = course_cols$blue,
                               "Targeted attack" = course_cols$coral),
                    name = "Strategy") +
  labs(
    title    = "Robustness index $R$ by network type and strategy",
    subtitle = "$R$ is the area under the robustness curve $S(f)$",
    x        = "Network type",
    y        = "Robustness index R"
  ) +
  scale_y_continuous(limits = c(0, 1), breaks = seq(0, 1, 0.2))
```

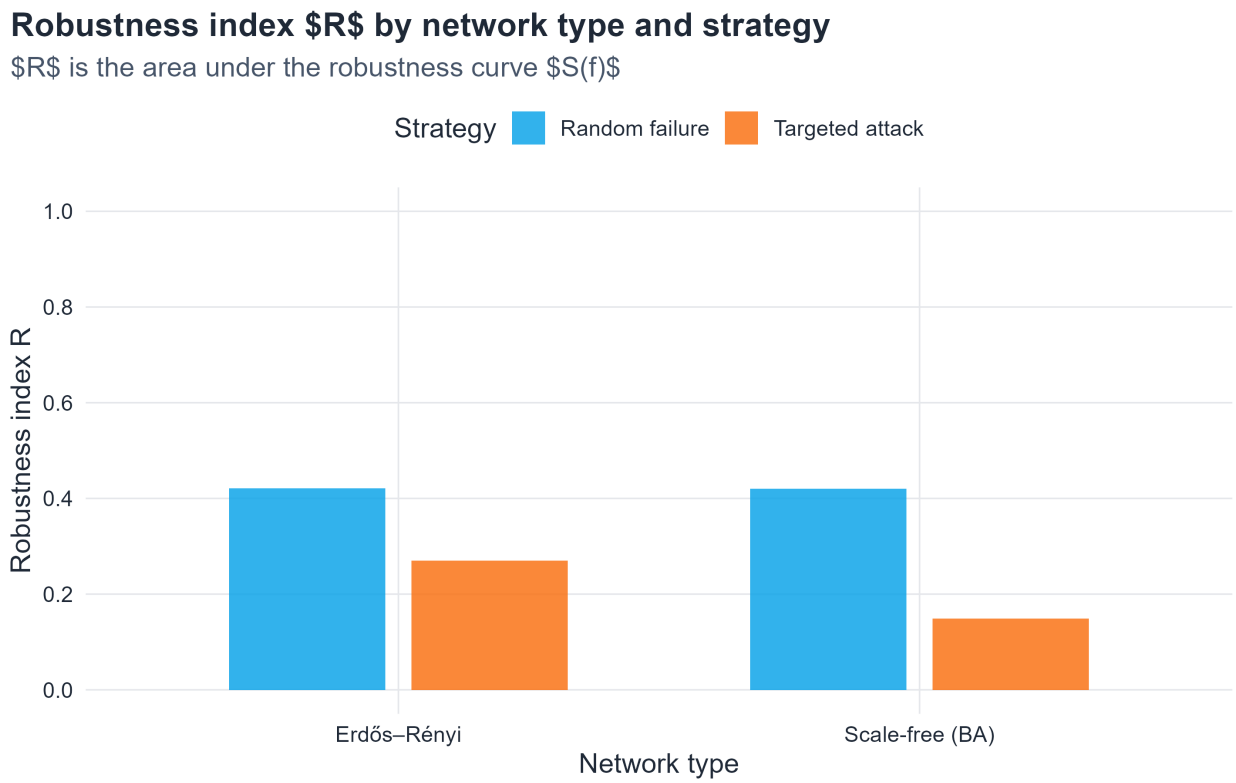


Figure 8.8: Robustness index R for BA and ER networks under random failure and targeted attack. Here R is the normalized area under the robustness curve; larger values indicate greater robustness.

As shown in Figure 8.8, the scalar summary R preserves the same qualitative ranking seen in the full robustness curves. Random failure produces a larger area under the curve for the scale-free network, while targeted attack reduces that area sharply. This makes R useful when many networks must be compared at once, although it should always be interpreted together with the underlying curve shape.

8.14 Beyond the giant component: other robustness observables

The giant component is the standard first observable, but in applications it is not always sufficient. Two damaged networks may have the same giant-component size while differing greatly in path efficiency or service quality.

Common complementary observables include:

- **average path length** inside the giant component,
- **network efficiency**, which measures how short communication paths remain,
- **number of connected components**,
- **size of the second-largest component**, which often peaks near the transition,
- **flow-based performance** in infrastructure networks.

For classroom purposes, this is an important conceptual reminder: robustness is not a single number until we decide what “functionality” means in the system under study.

8.15 Real-world implications

The theoretical results developed in this chapter have direct consequences for the design and protection of real systems (Albert et al., 2000; Cohen et al., 2000; Newman, 2010).

Internet and communication networks. The internet exhibits scale-free degree distributions. This means that most routers fail occasionally without disrupting global connectivity — error tolerance. However, a coordinated attack on the most connected routers could fragment the network.

Power grids. Power grids are not scale-free: their degree distributions are more narrow. As a result, they are more susceptible to random failures than scale-free networks, but they also lack the extreme hub vulnerability. Cascading failures — triggered when overloaded nodes fail — are a major concern.

Biological networks. Protein interaction networks and metabolic networks tend to be scale-free. The observation that essential genes (whose knockout is lethal) disproportionately encode high-degree hub proteins is a biological manifestation of the Achilles heel phenomenon.

Epidemic spreading. The same hubs that make scale-free networks robust also make them efficient channels for spreading processes. A disease or a piece of information propagates especially quickly when hubs are involved. This is studied in epidemic models on networks, where the epidemic threshold vanishes for scale-free networks with $\gamma \leq 3$ (Barabási & Pósfai, 2016; Newman, 2010).

8.16 Simulation: robustness of Watts–Strogatz networks

The Watts–Strogatz small-world model occupies an intermediate position between the homogeneous ER graph and the heterogeneous BA graph. Its degree distribution is narrow, making it similar to ER networks in terms of random failure.

```
set.seed(321)
p_rews <- c(0.01, 0.1, 0.5)
cols_ws <- c(course_cols$teal, course_cols$blue, course_cols$gold)

rob_ws <- map2_dfr(p_rews, cols_ws, function(p_r, col) {
  g <- sample_smallworld(dim = 1, size = 200, nei = 3, p = p_r)
  df <- simulate_robustness_full(g, "random", steps = 35)
  df$p_rew <- paste0("p_rew = ", p_r)
  df
})

ggplot(rob_ws, aes(x = f, y = S, color = p_rew)) +
  geom_line(linewidth = 1.1) +
  geom_point(size = 1.5, alpha = 0.7) +
  scale_color_manual(
    values = setNames(cols_ws, paste0("p_rew = ", p_rews)),
    name = "Rewiring prob."
  ) +
  labs(
    title    = "Random failure in Watts--Strogatz small-world networks",
    subtitle = "Higher rewiring probability increases disorder, sharpening the transition",
    x        = "Fraction of nodes removed (f)",
    y        = "Giant component size S(f)"
  ) +
  scale_x_continuous(breaks = seq(0, 1, 0.2)) +
  scale_y_continuous(limits = c(0, 1), breaks = seq(0, 1, 0.2))
```

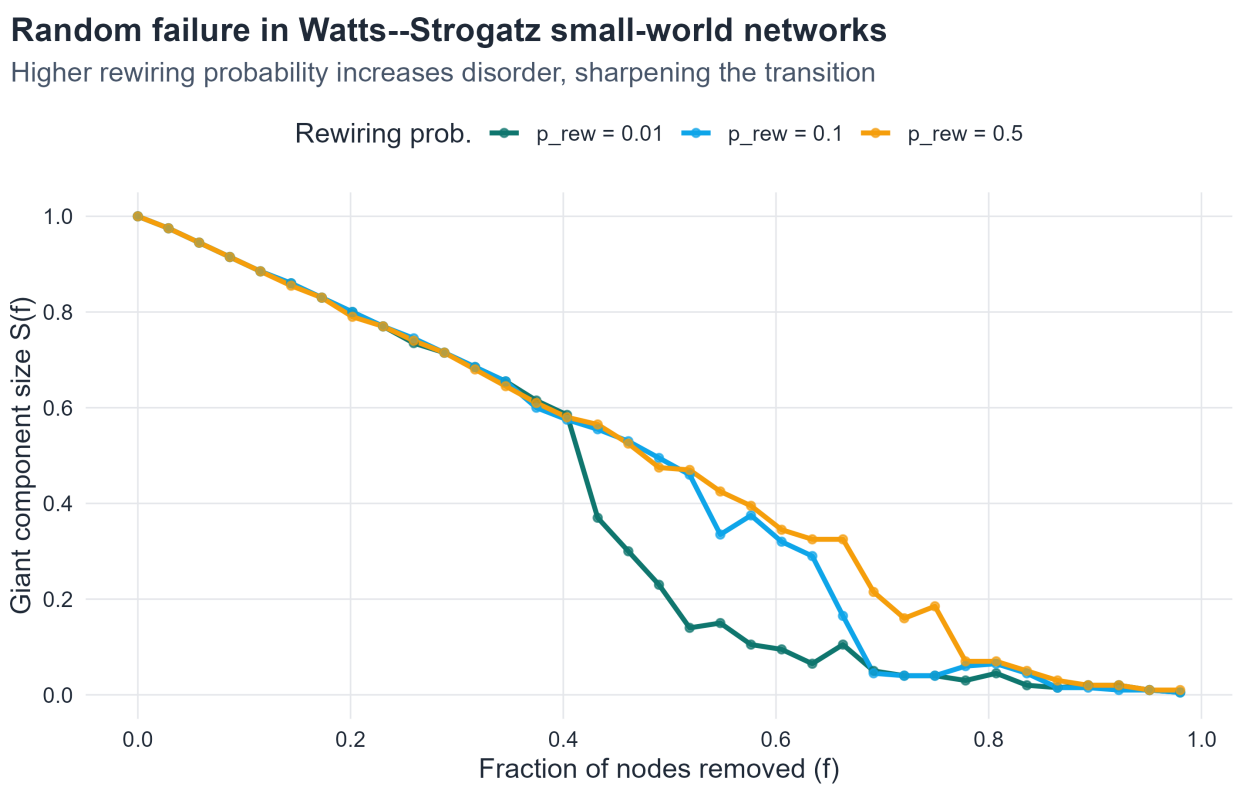


Figure 8.9: Robustness curves for Watts–Strogatz small-world networks with different rewiring probabilities p_{rew}. As rewiring increases, the network becomes more disordered and the transition becomes sharper.

As shown in Figure 8.9, changing the rewiring probability modifies the small-world network's robustness, but not as dramatically as changing from a homogeneous network to a scale-free one. The degree distribution remains relatively narrow, so random failure still produces a comparatively smooth degradation rather than the strong hub-protected behavior seen in scale-free networks.

Before moving to the exercises, it is helpful to pause and connect the chapter's main themes. Robustness depends on structure, but the relevant aspect of structure is not merely density. Degree heterogeneity, centrality concentration, and attack strategy all change how quickly connectivity is lost. The exercises below are designed to make that dependence concrete through both derivation and simulation.

8.17 Exercises

8.17.1 Part I. Computational and derivation exercises

8.17.1.1 Exercise 1. Molloy–Reed criterion

Let a network have degree distribution p_k for $k = 1, 2, 3, 4$ with $p_1 = 0.3$, $p_2 = 0.4$, $p_3 = 0.2$, $p_4 = 0.1$.

1. Compute $\langle k \rangle$ and $\langle k^2 \rangle$.
2. Compute $\kappa_0 = \langle k^2 \rangle / \langle k \rangle$.
3. Determine whether the network has a giant component.
4. Compute the critical threshold f_c under random node removal.

8.17.1.2 Exercise 2. Erdős–Rényi threshold derivation

Starting from the Poisson degree distribution,

1. Derive the formula $\kappa_0 = \langle k \rangle + 1$ for ER networks.
2. Derive the critical threshold $f_c^{\mathrm{ER}} = 1 - 1/\langle k \rangle$.
3. Verify this formula numerically for $\langle k \rangle = 3$ using a simulation.

8.17.1.3 Exercise 3. Effect of degree distribution on f_c

Consider two networks with the same average degree $\langle k \rangle = 4$ but different degree distributions:

- Network A: p_k uniform on $\{3, 4, 5\}$,
- Network B: p_k concentrated on $\{1, 7\}$ equally.

1. Compute $\langle k^2 \rangle$ for each network.
2. Compute κ_0 for each.
3. Compute f_c for each and interpret the difference.

8.17.1.4 Exercise 4. Bond percolation

For an Erdős–Rényi network with $\langle k \rangle = 5$:

1. Compute the bond percolation threshold q_c.
2. Compare it with the site percolation threshold f_c.
3. Explain why both thresholds are equal for ER networks.

8.17.1.5 Exercise 5. Betweenness centrality by hand

For the path graph P_5 with vertices $\{1, 2, 3, 4, 5\}$:

1. Identify all shortest paths between every pair of vertices.
2. Compute the betweenness centrality of each vertex.
3. Identify the vertex with highest betweenness and explain geometrically why it has this value.

8.17.1.6 Exercise 6. Robustness index

Let $S(f) = (1 - f/f_c)_+$ for a network with critical threshold f_c (where $(x)_+ = \max(x, 0)$).

1. Compute the robustness index $R = \int_0^1 S(f)\, df$.
2. Show that $R = f_c/2$.
3. What does this result say about networks with large f_c?

8.17.2 Part II. Conceptual and analytical exercises

8.17.2.1 Exercise 7. Why scale-free networks are robust to random failure

Provide a mathematical and conceptual explanation of why scale-free networks with $2 < \gamma \leq 3$ have $f_c \to 1$ under random node removal.

Your answer should address:

- the role of the second moment $\langle k^2 \rangle$ in the Molloy–Reed criterion,
- the probability that a randomly removed node is a hub,
- the implication for network connectivity.

8.17.2.2 Exercise 8. The Achilles heel of scale-free networks

Write a short analytical discussion of why scale-free networks are vulnerable to targeted attack.

Your answer should address:

- the definition of targeted attack in terms of degree ordering,
- what happens to $\langle k^2 \rangle$ after high-degree hubs are removed,
- why the critical threshold under targeted attack is small.

8.17.2.3 Exercise 9. Cascading failures

Explain qualitatively how a cascading failure differs from the simple percolation model.

Your answer should address:

- what triggers a cascade (overload, threshold dynamics),
- why networks near their percolation threshold are most vulnerable,
- one real-world example of a cascading failure.

8.17.2.4 Exercise 10. Comparing robustness of graph families

For each of the following network types — Erdős–Rényi, Watts–Strogatz, Barabási–Albert — describe qualitatively:

- the degree distribution,
- the expected robustness under random failure,
- the expected robustness under targeted attack,
- the location and sharpness of the percolation transition.

8.17.3 Part III. Simulation and coding exercises

8.17.3.1 Exercise 11. Robustness curve for ER networks

Write an R function that:

1. Generates a $G(N, p)$ network.
2. Simulates random node removal at steps $f = 0, 0.02, 0.04, \ldots, 0.98$.
3. Records the giant component size at each step.
4. Plots the robustness curve and overlays the theoretical critical threshold.

Run the function for $\langle k \rangle \in \{2, 3, 5, 10\}$ and compare.

8.17.3.2 Exercise 12. Targeted attack on BA networks

Write an R function that:

1. Generates a Barabási–Albert network using `sample_pa()`.

2. Removes nodes in decreasing order of degree (recalculate degrees after each removal).
3. Records and plots the giant component size.
4. Compares the targeted attack curve with the random failure curve on the same plot.

Verify that targeted attack destroys the giant component much sooner than random failure.

8.17.3.3 Exercise 13. Betweenness centrality attack

Modify the targeted attack simulation from Exercise 12 to remove nodes in decreasing order of **betweenness centrality** (recalculating after each removal).

1. Plot the betweenness-based attack curve alongside the degree-based attack curve.
2. Discuss which strategy is more destructive for the BA network.
3. Repeat for an ER network and compare the results.

8.17.3.4 Exercise 14. Robustness index comparison

For a grid of network types and attack strategies, compute the robustness index R for:

- ER networks with $\langle k \rangle \in \{3, 5, 8\}$,
- BA networks with $m \in \{2, 3, 4\}$,
- WS networks with rewiring probabilities $p \in \{0.05, 0.1, 0.5\}$,

under both random failure and targeted attack.

Present the results as a heatmap or grouped bar chart and discuss the patterns.

8.17.3.5 Exercise 15. Bond percolation simulation

Implement bond percolation (edge removal) in R:

1. Generate an ER network and a BA network.
2. Remove edges independently at random with probability q.
3. Measure the giant component size as a function of q.
4. Identify the empirical bond percolation threshold and compare with the theoretical value $q_c = 1 - 1/\langle k \rangle$ for the ER network.

8.17.3.6 Exercise 16. Finite-size effects

Repeat the robustness simulation for ER networks with the same average degree but different sizes, for example $N \in \{200, 500, 1000, 3000\}$.

1. Plot the robustness curves on the same figure.
2. Explain why the transition becomes sharper as N increases.
3. Compute the size of the second-largest component and show that it peaks near the empirical transition.

8.18 Suggested mini-project

A strong mini-project for this chapter is the following:

Conduct a full robustness analysis of two networks — one Erdős–Rényi and one scale-free — under random failure, targeted attack by degree, and targeted attack by betweenness centrality.

A complete submission should include:

- mathematical derivation of the theoretical critical threshold for each network,
- simulation of all three attack strategies with robustness curves,
- computation of the robustness index R for all combinations,
- discussion of the asymmetry between error tolerance and attack vulnerability in the scale-free case,
- a short concluding section on implications for the design of robust real-world networks.

8.19 Concluding remarks

Network robustness is one of the most practically important topics in network science. The theoretical results in this chapter connect abstract graph-theoretic quantities — degree moments, percolation thresholds, centrality — to concrete questions about resilience in technological, biological, and social systems.

The central lessons are:

- robustness is naturally measured through the survival of the giant component under node or edge removal,
- the Molloy–Reed criterion $\kappa_0 = \langle k^2 \rangle / \langle k \rangle > 2$ determines the existence of a giant component,
- the critical threshold $f_c = 1 - 1/(\kappa_0 - 1)$ quantifies how much damage a network can absorb,
- Erdős–Rényi networks have a finite, predictable critical threshold under both random failure and targeted attack,
- scale-free networks with $2 < \gamma \leq 3$ are asymptotically immune to random failure but catastrophically vulnerable to targeted attacks on their hubs,
- betweenness centrality provides an alternative measure of node criticality, sometimes more destructive than degree in targeted attack scenarios.

These insights motivate a fundamental design tension in engineering robust networks. A network that is robust to random failure by virtue of having hubs may be simultaneously fragile under deliberate attack. A network that is homogeneous — like a regular lattice or an Erdős–Rényi graph — avoids this asymmetry at the cost of lower robustness to random failures.

Understanding these trade-offs is essential for designing resilient infrastructure, biological systems, and communication networks.

8.20 References

Chapter 9

Community Structure in Graphs

Mesoscopic Organization, Graph Cuts, Spectral Methods, Modularity, and Statistical Inference

9.1 Introduction

After learning how random graphs generate baseline connectivity, how scale-free networks create hubs, and how small-world networks combine clustering with short paths, we now move to a different question: how do vertices assemble into *modules* or *communities*? In many real networks, the most informative structure is neither purely local nor purely global. Instead, it lives at an intermediate, **mesoscopic** scale. Social networks split into circles of friends, scientific collaboration networks split into research specialties, gene regulatory networks contain functional modules, and transportation networks organize into regional clusters. Community structure is therefore a canonical example of mesoscopic organization in complex systems (Barabási & Pósfai, 2016; Fortunato, 2010; Girvan & Newman, 2002).

In the language of network science, a community is a group of vertices that connect to one another more densely than to the rest of the graph. Yet this simple intuition immediately surfaces a fundamental difficulty: there is no unique, universally accepted mathematical definition of a community. Different definitions formalize different aspects of mesoscale cohesion. Some methods minimize sparse cuts between groups, some maximize a null-model deviation (modularity), and some treat block membership as a latent variable to be inferred from a probabilistic generative model (Fortunato, 2010; Newman, 2010). Each approach answers a slightly different question, and each inherits different assumptions, computational characteristics, and failure modes.

For master-level study, this plurality is not a problem to be avoided but a theoretical richness to be explored. The present chapter therefore develops community detection in stages: from informal intuition, through rigorous graph-cut quantities and spectral theory, through modularity and its optimization, to the stochastic block model as a full statistical inference framework. The chapter closes with a treatment of detectability limits that connects community structure to information theory, and with guidelines for honest interpretation of empirical results.

9.2 Chapter roadmap

This chapter develops in the following order.

1. Formal definitions: graphs, partitions, and notation
2. Why communities are mesoscale and not local or global properties

3. The distinction between clustering coefficient and community structure
4. Internal density, cut size, volume, and conductance
5. Cheeger's inequality and the spectral geometry of cuts
6. The graph Laplacian, the Fiedler vector, and spectral clustering
7. Bridge edges and the Girvan–Newman divisive algorithm
8. Modularity: derivation from null-model first principles and spectral formulation
9. Fast optimization: Louvain and the Leiden algorithm
10. The stochastic block model: likelihood, maximum-likelihood estimation, and EM outline
11. The degree-corrected stochastic block model
12. The Kesten–Stigum detectability threshold and its information-theoretic meaning
13. Comparing partitions: normalized mutual information, adjusted Rand index, and variation of information
14. Limitations: resolution limit, overlapping communities, and benchmark dependence
15. A practical workflow for empirical community analysis

9.3 Learning goals

By the end of this chapter, a student should be able to:

- define a graph partition formally and explain what makes a subset a good community candidate;
- state and apply the definitions of cut size, volume, conductance, and internal density;
- state Cheeger's inequality and explain what it connects;
- construct the (unnormalized and normalized) graph Laplacian and relate its eigenvalues to graph connectivity;
- describe spectral clustering and its connection to the normalized cut;
- explain the modularity null model, derive the community-level expression, and connect it to the modularity matrix eigenvalue problem;
- describe the Louvain heuristic and explain why the Leiden algorithm was proposed as an improvement;
- write down the likelihood of the stochastic block model, derive the maximum-likelihood estimator for Ω, and outline the EM algorithm;
- state the Kesten–Stigum threshold and explain why detectability failure is an information-theoretic impossibility, not merely a computational one;
- compute and interpret NMI, ARI, and variation of information between two partitions;
- discuss the resolution limit, overlapping communities, and benchmark-dependence as fundamental limitations.

9.4 Formal preliminaries: graphs, partitions, and notation

Throughout this chapter, let $G = (V, E)$ be a simple, undirected, connected graph with $n = |V|$ vertices and $m = |E|$ edges. The **adjacency matrix** is $A \in \{0,1\}^{n \times n}$ with $A_{ij} = 1$ if $\{i, j\} \in E$. The **degree** of vertex i is $k_i = \sum_j A_{ij}$, and $\mathbf{k} = (k_1, \dots, k_n)^T$ denotes the degree sequence. The **degree matrix** is $D = \mathrm{diag}(k_1, \dots, k_n)$. The total number of edge endpoints satisfies $\sum_i k_i = 2m$.

A **partition** of V into q communities is a collection $\mathcal{C} = \{C_1, C_2, \dots, C_q\}$ satisfying

$$C_r \cap C_s = \emptyset \ \text{ for } r \neq s, \qquad \bigcup_{r=1}^{q} C_r = V.$$

The **assignment function** $z : V \to \{1, \dots, q\}$ maps each vertex to its community label, so $z_i = r$ means vertex i belongs to C_r. For a subset $S \subseteq V$, write $\bar{S} = V \setminus S$.

For community C_r, define:

- $n_r = |C_r|$, the number of vertices;
- $m_r = \big|\{(i,j) \in E : i \in C_r,\, j \in C_r\}\big|$, the number of internal edges;
- $\ell_r = \big|\{(i,j) \in E : i \in C_r,\, j \notin C_r\}\big|$, the number of boundary edges (cut edges from C_r);
- $\mathrm{vol}(C_r) = \sum_{i \in C_r} k_i$, the volume.

9.5 Why communities matter

A useful analytical decomposition separates structural questions by scale:

- at the **local** level: degree, triangles, and clustering around individual vertices;
- at the **global** level: diameter, giant components, and overall connectivity;
- at the **mesoscopic** level: whether V decomposes into intermediate-scale modules.

Community structure belongs to the third category. It is therefore not visible from degree distribution alone, and it cannot be captured by a single global scalar such as average path length. Two networks can share the same degree distribution and the same global density while differing profoundly in their block organization (Barabási & Pósfai, 2016; Fortunato, 2010). This scale separation is precisely why community detection requires its own mathematical machinery.

In applications, a community can correspond to social role, biological function, thematic topic, geographic proximity, or hidden constraints in the formation process. Community analysis therefore transforms a large, difficult graph into an interpretable map of meso-level structure.

9.6 Communities are not the same as clustering

This distinction is pedagogically essential.

The **clustering coefficient** of vertex i is a local quantity:

$$c_i = \frac{\text{number of closed triangles centered at } i}{\binom{k_i}{2}},$$

measuring the probability that two neighbors of i are also neighbors of each other. Community structure, by contrast, is a mesoscopic quantity concerning whether groups of vertices cohere internally relative to their connections with the rest of the graph. A graph can have high average clustering without a clear block partition, and it can have a strong block partition without especially high local clustering.

Illustrative examples. A ring lattice has high local clustering because neighbors are spatially proximate and tend to connect, but its community structure is spatially smooth and not naturally partitioned into sharply separated modules. Conversely, a graph consisting of two moderately sparse cliques connected by a single bridge will have only moderate local clustering (interior vertices of each clique form triangles, but bridge-adjacent vertices do not), yet the two-community partition is visually and quantitatively obvious.

> **Key distinction**
>
> The clustering coefficient asks whether the neighborhood of a single vertex is internally dense. Community detection asks whether the graph decomposes into groups whose internal connectivity substantially exceeds their external connectivity. These are genuinely different questions.

9.7 A first synthetic example: planted communities

Before defining algorithms, it is valuable to construct a graph whose mesoscale structure is known in advance. The **planted partition model** (a special case of the stochastic block model treated formally later) assigns each vertex to a block and generates edges independently: edge probability p_{in} for vertex pairs within the same block, and $p_{\text{out}} \ll p_{\text{in}}$ for pairs in different blocks.

```
make_planted_partition <- function(sizes, p_in = 0.25, p_out = 0.03) {
  n <- sum(sizes)
  block <- rep(seq_along(sizes), sizes)
  A <- matrix(0, n, n)

  for (i in seq_len(n - 1)) {
    for (j in (i + 1):n) {
      p <- if (block[i] == block[j]) p_in else p_out
      A[i, j] <- rbinom(1, 1, p)
      A[j, i] <- A[i, j]
    }
  }

  g <- igraph::graph_from_adjacency_matrix(A, mode = "undirected", diag = FALSE)
  igraph::V(g)$block <- factor(block)
  g
}

p_in  <- 0.32
p_out <- 0.025

g_planted <- make_planted_partition(
  sizes = c(14, 14, 12),
  p_in = p_in,
  p_out = p_out
)

palette_blocks <- c(course_cols$blue, course_cols$gold, course_cols$violet)

set.seed(2081)
ggraph::ggraph(g_planted, layout = "fr") +
  ggraph::geom_edge_link(alpha = 0.22, colour = course_cols$slate) +
  ggraph::geom_node_point(aes(color = block), size = 3.1) +
  ggplot2::scale_color_manual(values = palette_blocks) +
  ggplot2::labs(
    title = "A graph with planted community structure",
    subtitle = bquote(p["in"] == .(p_in) * "," ~ p["out"] == .(p_out)),
    color = "Planted block"
  )
```

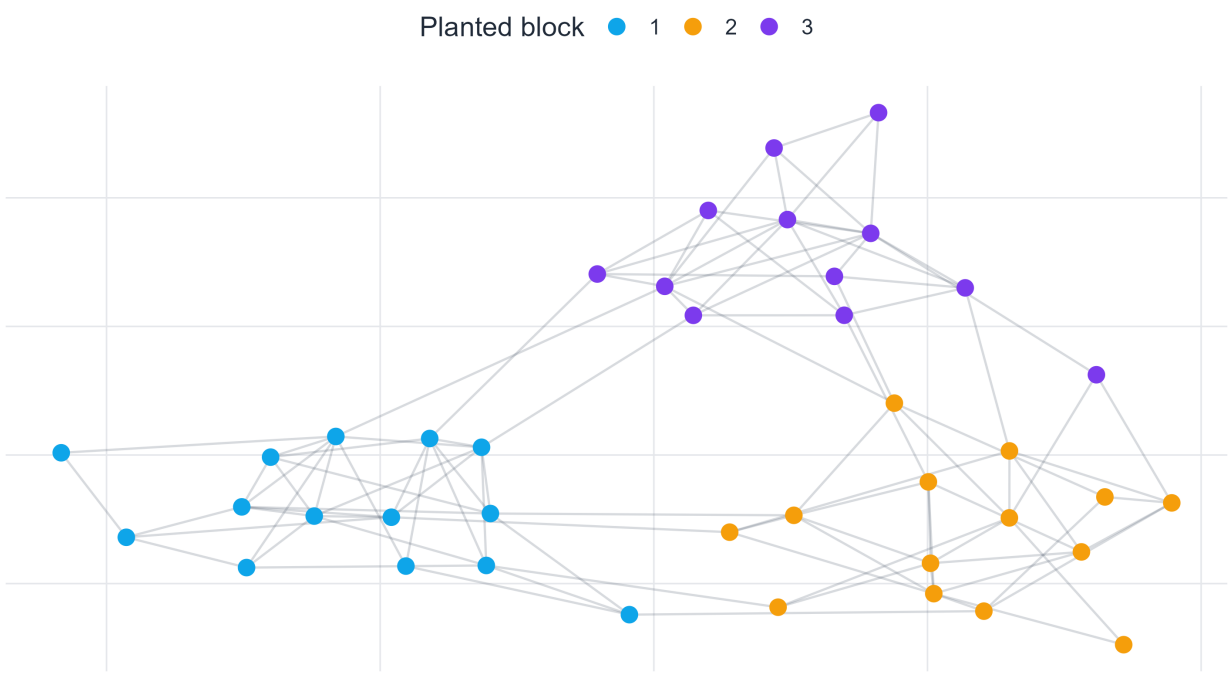


Figure 9.1: A synthetic graph with three planted communities. The within-block edge probability ($p_{\text{in}} = 0.32$) substantially exceeds the between-block probability ($p_{\text{out}} = 0.025$), producing visible mesoscale structure.

A second view, often more informative for matrix-analytic methods, is the **reordered adjacency matrix**.

```
A_planted <- as.matrix(igraph::as_adjacency_matrix(g_planted, sparse = FALSE))
ord_planted <- order(as.integer(igraph::V(g_planted)$block))
A_ord <- A_planted[ord_planted, ord_planted]

adj_df <- as.data.frame(as.table(A_ord)) |>
  dplyr::rename(row = Var1, col = Var2, edge = Freq) |>
  dplyr::mutate(row = as.integer(row), col = as.integer(col))

ggplot2::ggplot(adj_df, ggplot2::aes(x = col, y = row, fill = factor(edge))) +
  ggplot2::geom_tile() +
  ggplot2::scale_fill_manual(values = c("0" = "white", "1" = course_cols$ink), guide = "none") +
  ggplot2::scale_y_reverse() +
  ggplot2::coord_fixed() +
  ggplot2::labs(
    title = "Adjacency matrix reordered by planted labels",
    subtitle = "Communities appear as denser diagonal blocks",
    x = "Vertex index", y = "Vertex index"
  )
```

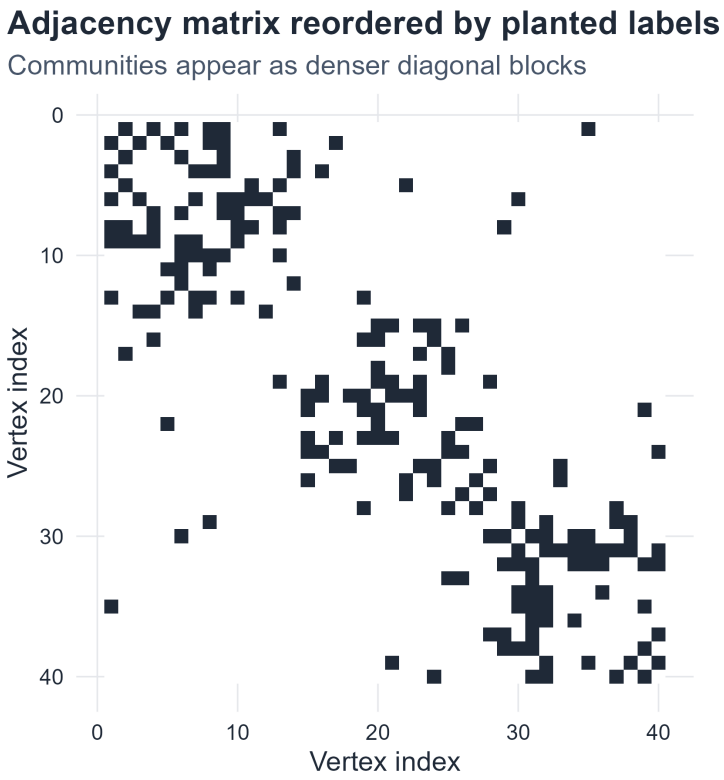


Figure 9.2: Adjacency matrix of the planted graph after reordering vertices by block label. Community structure manifests as denser diagonal blocks separated by sparse off-diagonal regions.

The matrix view in Figure 9.2 motivates the stochastic block model: the planted partition is designed so that the adjacency matrix has an approximate block structure, which the SBM formalizes as a statistical generative model.

9.8 Quantifying a candidate community

Visual intuition must be complemented by mathematics. For a subset $S \subseteq V$, we introduce four complementary quantities.

9.8.1 Internal density

$$\rho_{\text{in}}(S) = \frac{2m_{\text{in}}(S)}{|S|(|S|-1)},$$

the fraction of possible edges within S that are actually present. High internal density is necessary but not sufficient for a meaningful community: a very small dense clique surrounded by a large sparse graph may have $\rho_{\text{in}} \approx 1$ while being an isolated fragment rather than a coherent module.

9.8.2 Cut size

$$\text{cut}(S, \bar{S}) = m_{\text{out}}(S) = \big|\{(i,j) \in E : i \in S,\, j \notin S\}\big|.$$

A small cut is necessary for S to be a well-separated community. However, minimizing the cut alone is problematic: the global minimum of $\text{cut}(S, \bar{S})$ over all non-trivial bipartitions is often achieved by a very small S, such as a single vertex or a low-degree pair. This motivates normalization by size or degree.

9.8.3 Volume and conductance

Define the **volume** of S:

$$\text{vol}(S) = \sum_{i \in S} k_i.$$

Note that $\text{vol}(V) = 2m$. The **conductance** of S is then

$$\phi(S) = \frac{\text{cut}(S, \bar{S})}{\min\{\text{vol}(S),\, \text{vol}(\bar{S})\}}.$$

Conductance takes values in $[0, 1]$ and is small when S has a sparse boundary relative to its internal degree mass. It penalizes both small isolated sets and sets with excessively large boundaries, making it arguably the most natural cut-based quality measure for a community.

The **conductance of the graph** is defined as

$$\phi(G) = \min_{S \,:\, 0 < \text{vol}(S) \le m} \phi(S),$$

measuring how well-connected (or difficult to cut) the entire graph is.

9.8.4 Summary statistics for the planted graph

```
block_membership <- as.integer(igraph::V(g_planted)$block)
A_planted <- as.matrix(igraph::as_adjacency_matrix(g_planted, sparse = FALSE))

community_summary <- tibble::tibble(
  block = sort(unique(block_membership))
) |>
  dplyr::rowwise() |>
  dplyr::mutate(
    size         = sum(block_membership == block),
    int_edges    = sum(A_planted[block_membership == block, block_membership == block]) / 2,
    bnd_edges    = sum(A_planted[block_membership == block, block_membership != block]),
    int_density  = int_edges / choose(size, 2),
    volume       = sum(igraph::degree(g_planted)[block_membership == block]),
    conductance  = bnd_edges / min(volume, sum(igraph::degree(g_planted)) - volume)
  ) |>
  dplyr::ungroup()

community_summary
```

```
# A tibble: 3 x 7
  block  size int_edges bnd_edges int_density volume conductance
  <int> <int>     <dbl>     <dbl>       <dbl>  <dbl>       <dbl>
1     1    14        34         7       0.374     75      0.0933
2     2    14        28        10       0.308     66      0.152
3     3    12        27         9       0.409     63      0.143
```

9.8.5 Cheeger's inequality

The conductance $\phi(G)$ is NP-hard to compute exactly. A profound result from spectral graph theory provides both an efficient approximation and a deep connection between geometry and the eigenspectrum.

Theorem (Cheeger's inequality)

Let λ_2 be the second-smallest eigenvalue of the **normalized Laplacian** $\mathcal{L} = D^{-1/2}LD^{-1/2}$ (defined below). Then

$$\frac{\lambda_2}{2} \;\le\; \phi(G) \;\le\; \sqrt{2\lambda_2}.$$

The lower bound is due to Cheeger and Buser (Chung, 1997), and the upper bound is constructive: the sweep algorithm on the second eigenvector of $\mathcal{L}$ finds a cut achieving $\phi \le \sqrt{2\lambda_2}$. Cheeger's inequality is fundamental because it shows that a small spectral gap $\lambda_2 \approx 0$ is both *necessary* and *sufficient* (up to a square root) for the existence of a sparse cut. A graph with well-separated communities must have a small λ_2.

9.9 Graph Laplacian and spectral community detection

Cheeger's inequality tells us something deeper than a technical bound: a good community split is not only a combinatorial object, but also a spectral one. In other words, when a graph contains two weakly connected regions, that structural bottleneck leaves a trace in the eigenvalues and eigenvectors of the Laplacian. This is the real reason the Laplacian enters the story. We are not changing the problem; we are changing its language from discrete cuts to continuous geometry.

The educational payoff is important. A raw cut asks us to search over an exponential number of vertex subsets, whereas a spectral relaxation replaces that search by an eigenvector problem. The resulting vectors are not yet communities themselves, but they provide coordinates that reveal where the graph is easiest to separate. The rest of this section develops that logic in stages: first the Laplacian, then the normalized cut objective, then the spectral embedding that turns vertices into points in Euclidean space.

9.9.1 The unnormalized Laplacian

The **unnormalized graph Laplacian** is

$$L = D - A.$$

Its basic properties are:

1. L is symmetric positive semi-definite: $\mathbf{x}^T L\mathbf{x} = \sum_{\{i,j\}\in E}(x_i - x_j)^2 \ge 0$ for all $\mathbf{x} \in \mathbb{R}^n$.
2. $L\mathbf{1} = \mathbf{0}$, so $\mathbf{1}$ is an eigenvector with eigenvalue 0.
3. The multiplicity of eigenvalue 0 equals the number of connected components of G.
4. All eigenvalues satisfy $0 = \lambda_1 \le \lambda_2 \le \cdots \le \lambda_n$.

The second eigenvalue $\lambda_2(L)$ is the **algebraic connectivity** (or **Fiedler value**) of G. It measures how well-connected the graph is: $\lambda_2 = 0$ if and only if G is disconnected; a large λ_2 implies strong connectivity and therefore, by Cheeger, large conductance.

The corresponding eigenvector $\mathbf{v}_2$ is the **Fiedler vector**. A classical result is that thresholding $\mathbf{v}_2$ at its median value produces a bipartition of V with conductance bounded by $\sqrt{2\lambda_2}$. This is the geometric idea behind spectral bipartitioning.

9.9.2 The normalized Laplacian

The **normalized (or symmetric) Laplacian** is

$$\mathcal{L} = D^{-1/2} L D^{-1/2} = I - D^{-1/2} A D^{-1/2}.$$

Its eigenvalues lie in $[0, 2]$, and the normalized Laplacian appears naturally in random-walk analysis: the transition matrix of the simple random walk on G is $P = D^{-1}A = I - D^{-1}L$, so $\mathcal{L}$ and P share the same eigenvectors (up to the $D^{1/2}$ transformation).

9.9.3 The normalized cut and its spectral relaxation

For a bipartition $(S, \bar{S})$, the **normalized cut** is

$$\mathrm{NCut}(S, \bar{S}) = \frac{\mathrm{cut}(S, \bar{S})}{\mathrm{vol}(S)} + \frac{\mathrm{cut}(S, \bar{S})}{\mathrm{vol}(\bar{S})}.$$

Unlike the raw cut, the normalized cut is large when either part of the partition is small in degree-volume. Minimizing NCut over all bipartitions is NP-hard, but it has a natural spectral relaxation. Define the indicator $\mathbf{f} \in \mathbb{R}^n$ with $f_i = \sqrt{\mathrm{vol}(\bar{S})/\mathrm{vol}(S)}$ if $i \in S$ and $f_i = -\sqrt{\mathrm{vol}(S)/\mathrm{vol}(\bar{S})}$ if $i \in \bar{S}$. Then

$$\mathrm{NCut}(S, \bar{S}) = \frac{\tilde{\mathbf{f}}^T \mathcal{L} \tilde{\mathbf{f}}}{\tilde{\mathbf{f}}^T \tilde{\mathbf{f}}},$$

where $\tilde{\mathbf{f}} = D^{1/2}\mathbf{f}$, under the constraint $\tilde{\mathbf{f}}^T D^{1/2}\mathbf{1} = 0$. Relaxing $\tilde{\mathbf{f}}$ from a discrete vector to a continuous one and applying the Rayleigh quotient, the minimum is achieved by $\tilde{\mathbf{f}} = \mathbf{v}_2(\mathcal{L})$, the second eigenvector of $\mathcal{L}$. This is the **spectral relaxation of normalized cut** (Shi & Malik, 2000).

9.9.4 Why the spectral relaxation helps

The crucial approximation step is the relaxation from a discrete indicator vector to a continuous one. That move may seem abstract at first, so it helps to read it operationally. A discrete partition forces each vertex to belong entirely to one side of a cut. The relaxed eigenvector problem instead allows vertices to take continuous coordinates, and these coordinates measure how strongly each vertex is aligned with one side or the other of a low-conductance split. Vertices with similar coordinates are difficult to separate without paying a cut penalty; vertices with very different coordinates naturally belong to different sides.

For two communities, the sign and magnitude of the Fiedler vector already contain most of the geometric information. For more than two communities, we keep several leading eigenvectors and let them define an embedding of the vertices into a low-dimensional Euclidean space. Community detection is then performed in that embedded space rather than directly on the graph.

9.9.5 Spectral clustering algorithm

The full spectral clustering algorithm for q communities proceeds as follows.

> **i** Algorithm: Spectral Clustering
>
> **Input:** graph G with n vertices, target number of communities q.
> 1. Compute $\mathcal{L} = D^{-1/2}(D - A)D^{-1/2}$.
> 2. Find the q smallest eigenvectors $\mathbf{v}_1, \dots, \mathbf{v}_q$ of $\mathcal{L}$.
> 3. Form the matrix $U \in \mathbb{R}^{n \times q}$ with columns $\mathbf{v}_1, \dots, \mathbf{v}_q$.
> 4. Row-normalize U: let $\tilde{u}_{ir} = u_{ir} / \|\mathbf{u}_{i\cdot}\|$.

5. Treat each row $\tilde{\mathbf{u}}_{i\cdot}$ as a point in $\mathbb{R}^q$ and apply k-means clustering with $k = q$.
6. Assign vertex i to the community of its cluster.

Output: partition $\mathcal{C} = \{C_1, \ldots, C_q\}$.

The row normalization in step 4 projects vertices onto the unit sphere, which makes k-means more geometrically stable when block sizes and densities differ. The spectral embedding maps the discrete combinatorial structure of G onto a continuous Euclidean space where standard clustering applies (Luxburg, 2007; Ng et al., 2002).

```
A_planted <- as.matrix(igraph::as_adjacency_matrix(g_planted, sparse = FALSE))
D_planted <- diag(rowSums(A_planted))
Lsym_planted <- diag(nrow(A_planted)) -
  diag(1 / sqrt(pmax(diag(D_planted), 1e-12))) %*%
  A_planted %*%
  diag(1 / sqrt(pmax(diag(D_planted), 1e-12)))

eig_planted <- eigen(Lsym_planted, symmetric = TRUE)
embed_df <- tibble::tibble(
  x = eig_planted$vectors[, 2],
  y = eig_planted$vectors[, 3],
  block = igraph::V(g_planted)$block
)

ggplot2::ggplot(embed_df, ggplot2::aes(x = x, y = y, color = block)) +
  ggplot2::geom_point(size = 2.6, alpha = 0.9) +
  ggplot2::scale_color_manual(values = palette_blocks) +
  ggplot2::labs(
    title = "Spectral embedding separates the planted communities",
    subtitle = "Coordinates come from the second and third eigenvectors of the normalized Laplacian",
    x = "Second eigenvector coordinate",
    y = "Third eigenvector coordinate",
    color = "Planted block"
  )
```

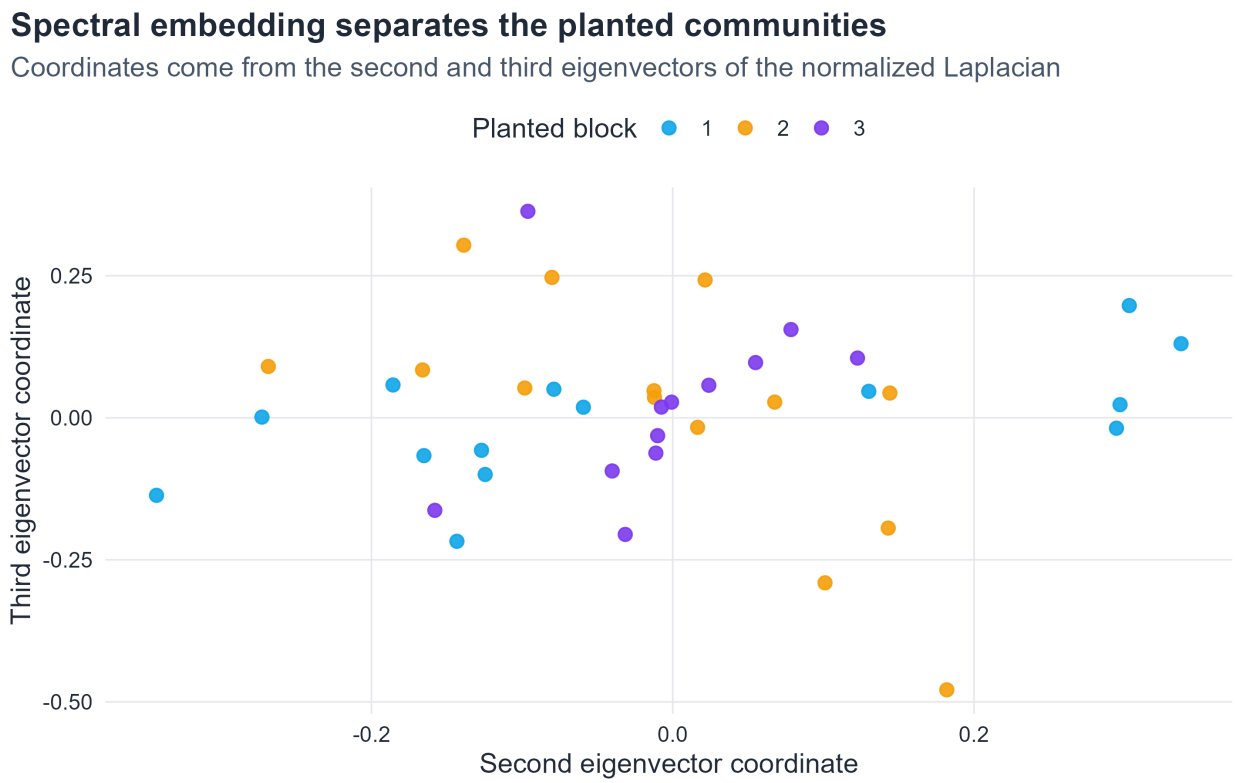


Figure 9.3: Two-dimensional spectral embedding of the planted graph. Vertices are colored by planted block. The Laplacian does not recover communities by directly cutting edges one by one; instead, it places vertices with similar structural roles close together in an embedding space, where standard clustering becomes feasible.

As shown in Figure 9.3, the Laplacian-based embedding converts a graph partitioning problem into a geometric clustering problem. This is the point that is easy to miss if one sees only the eigenvector formulas: the eigenvectors are useful because they supply coordinates in which structurally similar vertices become spatially close.

Spectral clustering has the following important properties:

- It is computationally efficient for sparse graphs: the dominant cost is the partial eigendecomposition, which can be done in $O(n \cdot q)$ time using iterative methods.
- It requires specifying q in advance, unlike modularity-maximization methods.
- The quality of the approximation degrades gracefully as $\lambda_{q+1} - \lambda_q$ (the eigengap) shrinks, which provides a natural diagnostic: a large eigengap suggests q is a good choice.

9.10 Bridge edges and the Girvan–Newman algorithm

The first historically influential modern algorithm for community detection is due to Girvan and Newman (Girvan & Newman, 2002). Its central intuition is that communities are held apart by a comparatively small number of bridge edges. Identifying and removing those bridges should reveal the modular structure.

9.10.1 Edge betweenness

For an edge $e = \{u, v\}$, the **edge betweenness centrality** is

$$B_e = \sum_{s \neq t} \frac{\sigma_{st}(e)}{\sigma_{st}},$$

where σ_{st} is the total number of shortest paths between s and t, and $\sigma_{st}(e)$ is the number of those shortest paths that use edge e. A bridge between two dense communities lies on many shortest paths between vertices in different communities and therefore acquires high betweenness.

This can be made precise: if G consists of two cliques of size $n/2$ connected by a single bridge e^*, then $B_{e^*} = (n/2)^2$ while interior edges have betweenness at most $O(n)$. In general, the betweenness of an intercommunity edge grows with the product of community sizes it separates.

```
edge_bet <- igraph::edge_betweenness(g_planted)
ig_plot  <- g_planted
igraph::E(ig_plot)$score       <- edge_bet
q90 <- stats::quantile(edge_bet, 0.90)
igraph::E(ig_plot)$bridge_flag <- ifelse(edge_bet >= q90, "Top 10%", "Other edges")

set.seed(2081)
ggraph::ggraph(ig_plot, layout = "fr") +
  ggraph::geom_edge_link(aes(edge_colour = bridge_flag, edge_width = score), alpha = 0.55) +
  ggraph::geom_node_point(aes(color = block), size = 3.0) +
  ggraph::scale_edge_color_manual(
    values = c("Other edges" = "grey75", "Top 10%" = course_cols$rose)
  ) +
  ggraph::scale_edge_width(range = c(0.25, 1.6), guide = "none") +
  ggplot2::scale_color_manual(values = palette_blocks) +
  ggplot2::labs(
    title    = "Edge betweenness identifies intercommunity bridges",
    subtitle = "Red edges: top 10% by betweenness centrality",
    color    = "Block", edge_colour = "Edge type"
  )
```

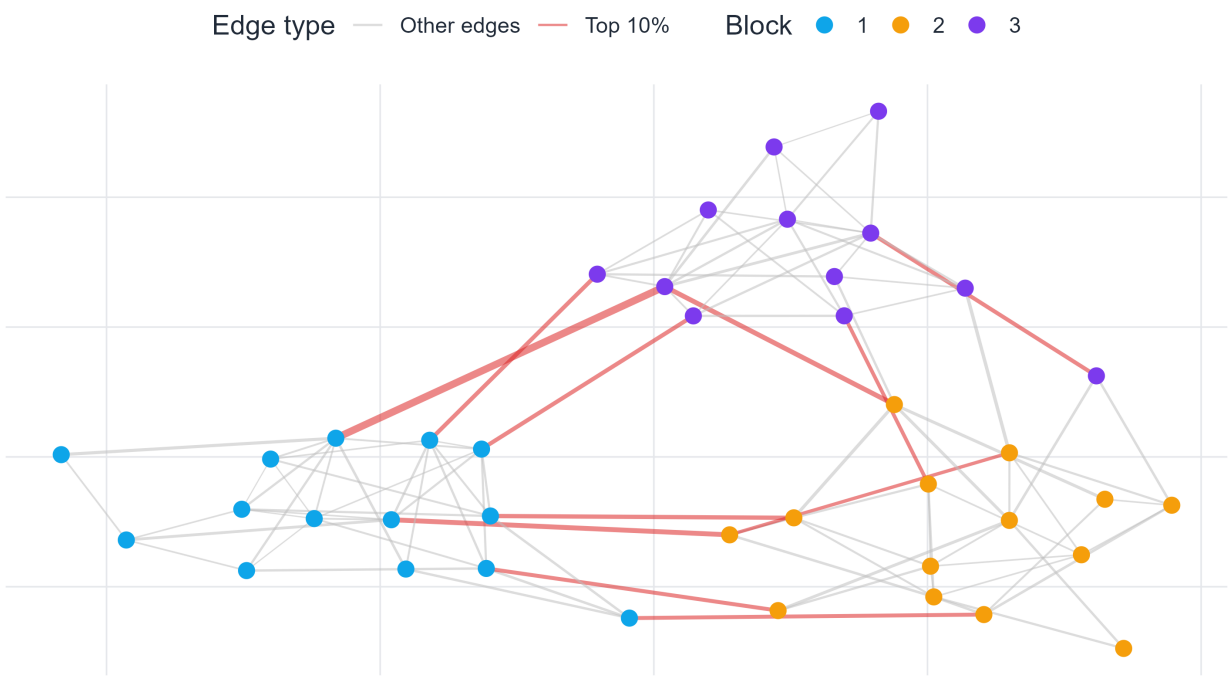


Figure 9.4: Edges in the top decile of edge betweenness highlighted in red. In a planted partition graph, high-betweenness edges concentrate between communities rather than deep inside them.

9.10.2 The divisive algorithm

The Girvan–Newman algorithm is **divisive**: it repeatedly removes the highest-betweenness edge and recomputes betweenness on the modified graph.

> **Algorithm: Girvan–Newman**
>
> **Input:** graph G.
>
> 1. Compute B_e for all edges using Brandes' algorithm in $O(nm)$ time.
> 2. Remove the edge $e^* = \arg\max_e B_e$.
> 3. Recompute B_e on the modified graph.
> 4. Repeat steps 2–3 until all edges have been removed or a stopping criterion is met.
> 5. The hierarchy of splits produces a **dendrogram**; cut at a level that maximizes modularity or satisfies a domain criterion.
>
> **Output:** hierarchical partition tree (dendrogram).

The algorithm is $O(nm^2)$ in the worst case (or $O(n^2 m)$ for sparse graphs), making it computationally prohibitive for large networks. Its chief value is pedagogical and conceptual: it demonstrates that community boundaries coincide with edges that carry large amounts of global routing, an insight that motivates both the modularity and the SBM approaches.

```
gn_fit <- igraph::cluster_edge_betweenness(g_planted)
ig_gn  <- g_planted
igraph::V(ig_gn)$community <- factor(igraph::membership(gn_fit))
```

```r
set.seed(2081)
ggraph::ggraph(ig_gn, layout = "fr") +
  ggraph::geom_edge_link(alpha = 0.22, colour = course_cols$slate) +
  ggraph::geom_node_point(aes(color = community), size = 3.1) +
  ggplot2::scale_color_brewer(palette = "Set2") +
  ggplot2::labs(
    title    = "Girvan--Newman partition",
    subtitle = paste0(
      "Detected communities: ", length(igraph::sizes(gn_fit)),
      "; Q = ", round(igraph::modularity(gn_fit), 3)
    ),
    color = "Detected block"
  )
```

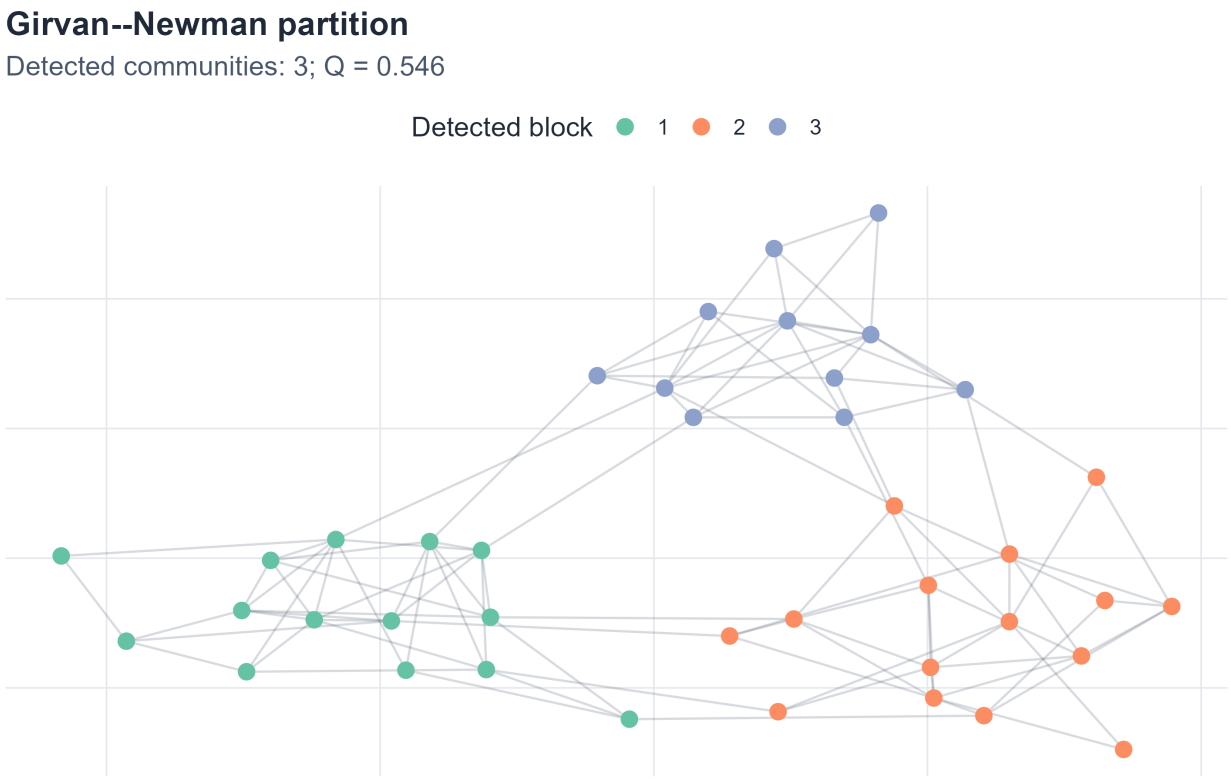


Figure 9.5: Partition returned by the Girvan–Newman algorithm on the planted graph. The algorithm recovers the planted structure because bridge edges between the three communities concentrate in the top betweenness decile.

9.11 Modularity: null-model viewpoint and spectral formulation

Instead of removing bridges, a complementary approach asks whether a proposed partition contains *more within-community edges than one would expect by chance given the degree sequence*. This is the **modularity** framework (Newman, 2006; Newman & Girvan, 2004).

9.11.1 Derivation from the configuration model null

The configuration model generates a random graph with prescribed expected degrees by connecting pairs of edge stubs uniformly at random. Under this null model, the probability of an edge between i and j is approximately

$$p_{ij}^{\text{null}} = \frac{k_i k_j}{2m},$$

which preserves the expected degree of every vertex while otherwise randomizing connectivity (Newman, 2010). The expected adjacency matrix of the null model has entries $k_i k_j/(2m)$, and the **modularity matrix** is the deviation of the observed adjacency from this expectation:

$$B_{ij} = A_{ij} - \frac{k_i k_j}{2m}.$$

Note that $B\mathbf{1} = \mathbf{0}$ because $\sum_j B_{ij} = k_i - k_i \sum_j k_j/(2m) = k_i - k_i = 0$.

9.11.2 The modularity formula

For a partition $\{c_i\}$, **modularity** is the normalized total excess of internal edges over the null expectation:

$$Q = \frac{1}{2m} \sum_{i,j} B_{ij}\, \delta(c_i, c_j) = \frac{1}{2m} \sum_{i,j} \left(A_{ij} - \frac{k_i k_j}{2m} \right) \delta(c_i, c_j),$$

where $\delta(c_i, c_j) = \mathbf{1}[c_i = c_j]$. Modularity $Q \in (-1, 1)$, with $Q = 0$ corresponding to a partition no better than the null model and Q close to 1 indicating strong community structure.

9.11.3 Community-level expression: derivation

Starting from the pairwise formula, we derive an equivalent expression in terms of community-level quantities. Define

$$e_{rs} = \frac{1}{2m} \sum_{i \in C_r} \sum_{j \in C_s} A_{ij}, \qquad a_r = \frac{\text{vol}(C_r)}{2m} = \frac{1}{2m} \sum_{i \in C_r} k_i.$$

Here e_{rs} is the fraction of all edge endpoints contributed by edges between C_r and C_s, and a_r is the fraction of total degree in community r.

For the within-community sum in Q, restricting $\delta(c_i, c_j) = 1$ to pairs (i, j) with $c_i = c_j = r$:

$$\sum_{i,j:\, c_i = c_j = r} A_{ij} = 2m\, e_{rr}, \qquad \sum_{i,j:\, c_i = c_j = r} \frac{k_i k_j}{2m} = \frac{1}{2m} \left(\sum_{i \in C_r} k_i \right)^2 = 2m\, a_r^2.$$

Therefore:

$$\boxed{Q = \sum_{r=1}^{q} \left(e_{rr} - a_r^2 \right).}$$

This form makes the null-model comparison transparent: for each community r, we compare the observed internal edge fraction e_{rr} to the fraction a_r^2 expected if edges were placed at random with the correct degree distribution. Communities that capture more internal edges than expected contribute positively to Q.

9.11.4 Spectral formulation and the modularity matrix eigenvalue problem

For a bipartition into two communities with assignment vector $s_i \in \{+1, -1\}$, write $\delta(c_i, c_j) = (s_i s_j + 1)/2$. Then

$$Q = \frac{1}{4m} \mathbf{s}^T B \, \mathbf{s},$$

maximized (relaxing s_i to $\mathbb{R}$) by the **leading eigenvector** of B. This is the Newman spectral bisection algorithm (Newman, 2006).

For $q > 2$ communities, we use indicator matrices $U \in \mathbb{R}^{n \times q}$ with $U_{ir} = \mathbf{1}[z_i = r]$. Then

$$Q = \frac{1}{2m} \operatorname{tr}(U^T B \, U).$$

Relaxing U to be orthogonal, this trace is maximized by the q leading eigenvectors of B. Community labels are then assigned by k-means in the eigenvector space, exactly analogous to spectral clustering. The modularity-spectral and Laplacian-spectral methods therefore share the same mathematical structure; the difference lies in whether B or $\mathcal{L}$ governs the embedding.

```
true_membership <- as.integer(igraph::V(g_planted)$block)
planted_Q <- igraph::modularity(g_planted, membership = true_membership)

random_Q <- replicate(400, {
  igraph::modularity(g_planted, membership = sample(true_membership))
})

modularity_df <- tibble::tibble(Q = random_Q)

ggplot2::ggplot(modularity_df, ggplot2::aes(x = Q)) +
  ggplot2::geom_histogram(
    bins = 28, fill = course_cols$blue, color = "white", alpha = 0.88
  ) +
  ggplot2::geom_vline(
    xintercept = planted_Q,
    color      = course_cols$rose,
    linewidth  = 1.2,
    linetype   = "dashed"
  ) +
  ggplot2::annotate(
    "text",
    x = planted_Q, y = Inf,
    label = paste0("Planted  Q = ", round(planted_Q, 3)),
    vjust = 1.5, hjust = 1.05,
    color = course_cols$rose, size = 3.8
  ) +
  ggplot2::labs(
    title    = "Planted modularity far exceeds the random baseline",
    subtitle = "400 random relabelings with identical community sizes",
    x = "Modularity Q", y = "Count"
  )
```

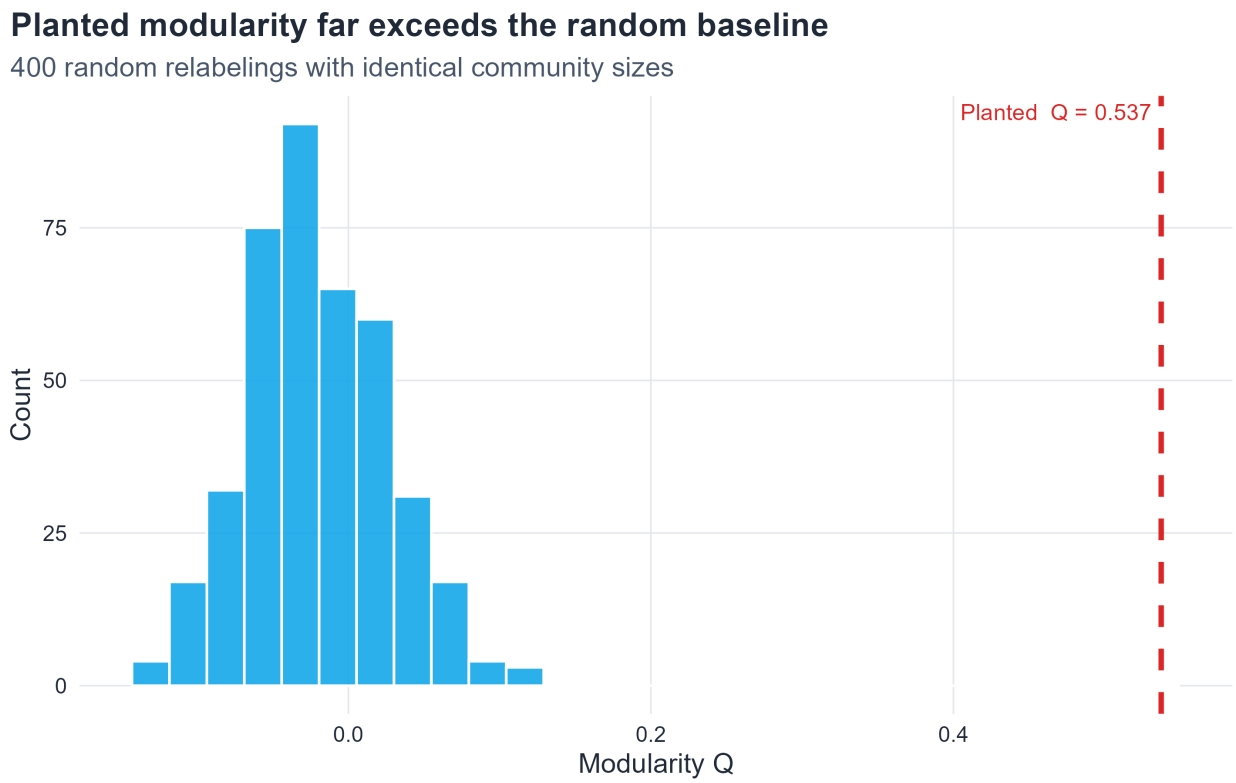


Figure 9.6: Modularity of the planted partition (red dashed line) compared to 400 random relabelings with the same community sizes. The planted partition far exceeds the random baseline, confirming its statistical significance as a mesoscale structure.

9.12 Fast community detection: Louvain and the Leiden algorithm

Exact modularity maximization is NP-hard (Brandes et al., 2008). For large networks, efficient heuristics are necessary.

9.12.1 The Louvain method

The **Louvain method** (Blondel et al., 2008) is a greedy multilevel algorithm organized in two alternating phases.

> **i** Algorithm: Louvain
>
> **Input:** graph $G = (V, E)$.
> **Phase 1 (local moves):** Initialize each vertex in its own community.
> For each vertex i (in arbitrary order, repeated until convergence): - Temporarily remove i from its current community. - For each neighboring community C, compute the modularity gain of placing i in C:
>
> $$\Delta Q = \left[\frac{\Sigma_{\text{in}} + 2k_{i,\text{in}}}{2m} - \left(\frac{\Sigma_{\text{tot}} + k_i}{2m}\right)^2\right] - \left[\frac{\Sigma_{\text{in}}}{2m} - \left(\frac{\Sigma_{\text{tot}}}{2m}\right)^2 - \left(\frac{k_i}{2m}\right)^2\right],$$

where Σ_{in} = internal degree of C, Σ_{tot} = total degree of C, $k_{i,\text{in}}$ = edges from i to C. - Place i in the community maximizing ΔQ if $\Delta Q > 0$; otherwise keep i isolated.
Phase 2 (aggregation): Collapse each detected community into a single supernode, accumulating edge weights. Apply Phase 1 to the aggregated (weighted) graph.
Repeat both phases until no further gain is possible.
Output: hierarchical partition with the highest modularity partition at each level.

The key efficiency gain of Louvain is that the modularity gain formula ΔQ can be computed in $O(\deg(i))$ time per vertex move, reducing the overall complexity to approximately $O(m \log n)$ per pass.

9.12.2 The Leiden algorithm: correcting Louvain's flaw

A significant theoretical problem with Louvain was identified by Traag et al. (Traag et al., 2019): **Louvain can produce internally disconnected communities**. In Phase 2, an aggregated supernode can represent a set of vertices that, when examined in the original graph, are not connected to each other within their community. Such a community is not a coherent module in any meaningful sense.

The **Leiden algorithm** corrects this by introducing a **refinement phase** between aggregation steps:

> **i** Algorithm: Leiden (summary)
>
> 1. Apply local moves as in Louvain Phase 1.
> 2. **Refinement:** For each community C returned in step 1, further sub-partition C into well-connected subcommunities using a constrained local-move procedure that enforces internal connectivity.
> 3. Aggregate based on the refined partition (not the Phase 1 partition).
> 4. Repeat until convergence.

Leiden guarantees that every community in the output is γ-separated: its internal edge density exceeds its boundary density by a factor γ, providing a formal quality certificate that Louvain does not. In practice, Leiden often produces partitions of equal or higher modularity than Louvain while being provably free of disconnected communities (Traag et al., 2019).

```
g_benchmark <- make_planted_partition(
  sizes = c(25, 25, 25, 25), p_in = 0.18, p_out = 0.015
)

fit_louvain  <- igraph::cluster_louvain(g_benchmark)
fit_walktrap <- igraph::cluster_walktrap(g_benchmark)
fit_gn_bench <- igraph::cluster_edge_betweenness(g_benchmark)

method_df <- tibble::tibble(
  method = c("Louvain", "Walktrap", "Girvan-Newman"),
  modularity = c(
    igraph::modularity(fit_louvain),
    igraph::modularity(fit_walktrap),
    igraph::modularity(fit_gn_bench)
  ),
  communities = c(
    length(igraph::sizes(fit_louvain)),
    length(igraph::sizes(fit_walktrap)),
    length(igraph::sizes(fit_gn_bench))
  )
)
```

```r
method_df |>
  dplyr::mutate(method = stats::reorder(method, modularity)) |>
  ggplot2::ggplot(ggplot2::aes(x = method, y = modularity, fill = method)) +
  ggplot2::geom_col(width = 0.72, show.legend = FALSE) +
  ggplot2::geom_text(
    ggplot2::aes(label = paste0("k = ", communities)),
    vjust = -0.5, size = 3.7, color = course_cols$ink
  ) +
  ggplot2::coord_flip() +
  ggplot2::scale_fill_brewer(palette = "Set2") +
  ggplot2::scale_y_continuous(limits = c(0, max(method_df$modularity) + 0.08)) +
  ggplot2::labs(
    title    = "Method comparison on a four-block benchmark graph",
    subtitle = "Labels show detected community count k",
    x = NULL, y = "Modularity Q"
  )
```

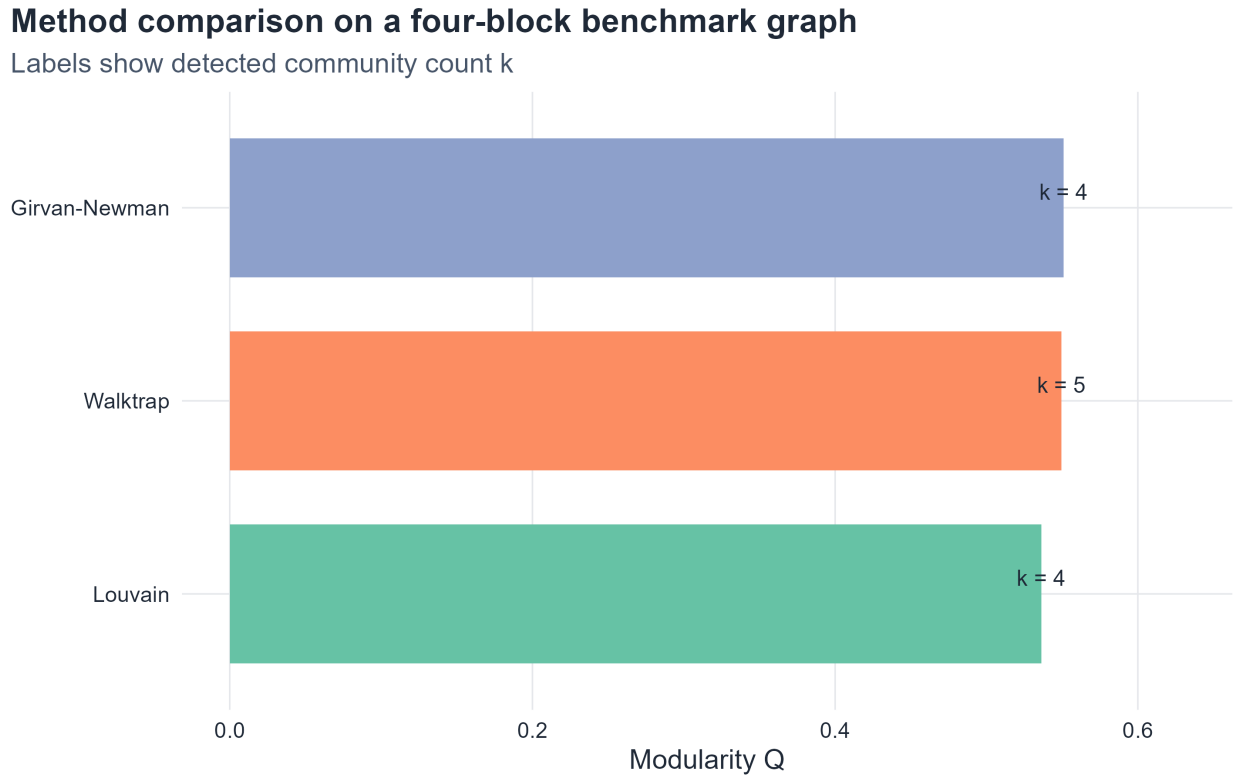


Figure 9.7: Modularity and number of detected communities for three methods on a four-community planted benchmark. Labels show detected community count k. Louvain and Walktrap are competitive on this scale; Girvan–Newman is most interpretable but slowest.

9.13 The stochastic block model: community detection as statistical inference

Up to this point, the chapter has treated community detection mainly as an optimization problem. We defined scores such as conductance and modularity, and then searched for partitions that optimize those scores. The stochastic block model changes the question. Instead of asking, "Which partition scores well?" it asks, "What latent block structure could plausibly have generated the graph we observe?"

This shift matters educationally and scientifically. Once we introduce a generative model, we can talk not only about finding a partition, but also about uncertainty, parameter estimation, model comparison, and even the limits of recoverability. The **stochastic block model** (SBM) therefore reframes community detection as **statistical inference**: we assume a probabilistic mechanism for edge formation and infer which latent community assignments are most consistent with the data (Holland et al., 1983; Nowicki & Snijders, 2001; Wang & Wong, 1987).

9.13.1 Model definition

Let each vertex $i \in V$ have a latent community label $z_i \in \{1, \dots, q\}$. The SBM assumes that edges are conditionally independent given the labels, with connection probability determined solely by the block membership of the two endpoints:

$$\Pr(A_{ij} = 1 \mid z_i = r,\, z_j = s) = \omega_{rs}, \quad i < j.$$

The matrix $\Omega = (\omega_{rs}) \in [0,1]^{q \times q}$ is symmetric. The SBM encompasses:

- **Assortative communities:** $\omega_{rr} > \omega_{rs}$ for $r \neq s$, describing cohesive groups.
- **Disassortative structure:** $\omega_{rr} < \omega_{rs}$, producing bipartite-like patterns.
- **Core–periphery structure:** one dense core block with lower-density peripheral blocks.

All three arise from the same likelihood, making the SBM a genuinely general model for latent network structure.

9.13.2 Likelihood function

Let $\pi = (\pi_1, \dots, \pi_q)$ with $\sum_r \pi_r = 1$ be the community size proportions. The **complete-data log-likelihood** (treating $\mathbf{z}$ as observed) is

$$\ell(\Omega, \pi \mid A, \mathbf{z}) = \sum_{r \le s} \left[m_{rs} \log \omega_{rs} + (M_{rs} - m_{rs}) \log(1 - \omega_{rs}) \right] + \sum_{r=1}^{q} n_r \log \pi_r,$$

where

$$m_{rs} = \sum_{\substack{i \in C_r,\, j \in C_s \\ i<j}} A_{ij}, \qquad M_{rs} = \begin{cases} \binom{n_r}{2} & r = s, \\ n_r n_s & r \neq s, \end{cases}$$

are the observed and possible edge counts between blocks r and s.

9.13.3 Maximum-likelihood estimators

Fixing $\mathbf{z}$, differentiating ℓ with respect to ω_{rs} and π_r, the MLEs are:

$$\widehat{\omega}_{rs} = \frac{m_{rs}}{M_{rs}}, \qquad \widehat{\pi}_r = \frac{n_r}{n}.$$

These are simply observed edge densities between (and within) blocks—a natural result. The difficult part is that $\mathbf{z}$ is unobserved.

9.13.4 EM algorithm for SBM inference

The **Expectation-Maximization (EM) algorithm** (Dempster et al., 1977) handles the latent labels by alternating between:

E-step: Compute the posterior probability that vertex i belongs to community r, given the current parameter estimates $(\hat{\Omega}, \hat{\pi})$ and the observed graph A:

$$\tau_{ir} = \Pr(z_i = r \mid A, \hat{\Omega}, \hat{\pi}) \propto \hat{\pi}_r \prod_{j \neq i} \hat{\omega}_{r s_j}^{A_{ij}} (1 - \hat{\omega}_{r s_j})^{1 - A_{ij}},$$

where the product runs over all other vertices j with their current soft assignments s_j. In practice, mean-field variational approximations or belief propagation are used for computational tractability.

M-step: Update the parameters using the soft assignments τ_{ir}:

$$\tilde{m}_{rs} = \sum_{i,j:\, i<j} \tau_{ir}\, \tau_{js}\, A_{ij}, \qquad \tilde{M}_{rs} = \sum_{i<j} \tau_{ir}\, \tau_{js}, \qquad \hat{\omega}_{rs} = \frac{\tilde{m}_{rs}}{\tilde{M}_{rs}}.$$

The EM algorithm converges to a local maximum of the marginal likelihood. Multiple initializations are recommended to mitigate local optima.

```
make_block_graph <- function(sizes, P) {
  n <- sum(sizes)
  block <- rep(seq_along(sizes), sizes)
  A <- matrix(0, n, n)
  for (i in seq_len(n - 1)) {
    for (j in (i + 1):n) {
      p <- P[block[i], block[j]]
      A[i, j] <- rbinom(1, 1, p)
      A[j, i] <- A[i, j]
    }
  }
  g <- igraph::graph_from_adjacency_matrix(A, mode = "undirected", diag = FALSE)
  igraph::V(g)$block <- factor(block)
  g
}

P_sbm <- matrix(
  c(0.24, 0.03, 0.02,
    0.03, 0.22, 0.025,
    0.02, 0.025, 0.20),
  nrow = 3, byrow = TRUE
)

g_sbm    <- make_block_graph(sizes = c(20, 18, 22), P = P_sbm)
A_sbm    <- as.matrix(igraph::as_adjacency_matrix(g_sbm, sparse = FALSE))
ord      <- order(as.integer(igraph::V(g_sbm)$block))
A_ord_sbm <- A_sbm[ord, ord]

adj_df_sbm <- as.data.frame(as.table(A_ord_sbm)) |>
  dplyr::rename(row = Var1, col = Var2, edge = Freq) |>
```

```
  dplyr::mutate(row = as.integer(row), col = as.integer(col))

ggplot2::ggplot(adj_df_sbm, ggplot2::aes(x = col, y = row, fill = factor(edge))) +
  ggplot2::geom_tile() +
  ggplot2::scale_fill_manual(
    values = c("0" = "white", "1" = course_cols$ink), guide = "none"
  ) +
  ggplot2::scale_y_reverse() +
  ggplot2::coord_fixed() +
  ggplot2::labs(
    title    = "Adjacency matrix of a three-block SBM",
    subtitle = expression(Omega[diag] %~~% 0.22 ~~ Omega[off] %~~% 0.027),
    x = "Vertex index", y = "Vertex index"
  )
```

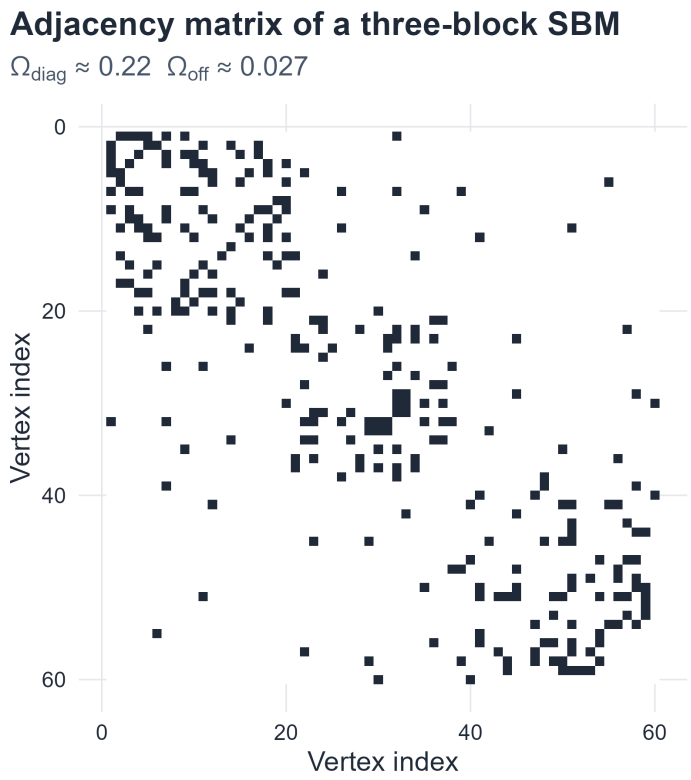


Figure 9.8: Adjacency matrix of a three-block SBM after reordering vertices by planted block. The block structure in Ω produces visible dense diagonal blocks in the reordered adjacency matrix.

```
omega_df <- expand.grid(row = 1:nrow(P_sbm), col = 1:ncol(P_sbm)) |>
  dplyr::mutate(prob = as.vector(P_sbm))

ggplot2::ggplot(omega_df, ggplot2::aes(x = factor(col), y = factor(row), fill = prob)) +
  ggplot2::geom_tile(color = "white") +
  ggplot2::geom_text(ggplot2::aes(label = sprintf("%.3f", prob)), color = "white", size = 4) +
  ggplot2::scale_fill_gradient(low = course_cols$blue, high = course_cols$violet) +
  ggplot2::labs(
    title = "The block matrix drives the generative mechanism",
```

```
    subtitle = "Higher diagonal probabilities encode assortative communities",
    x = "Block index",
    y = "Block index",
    fill = "Probability"
  )
```

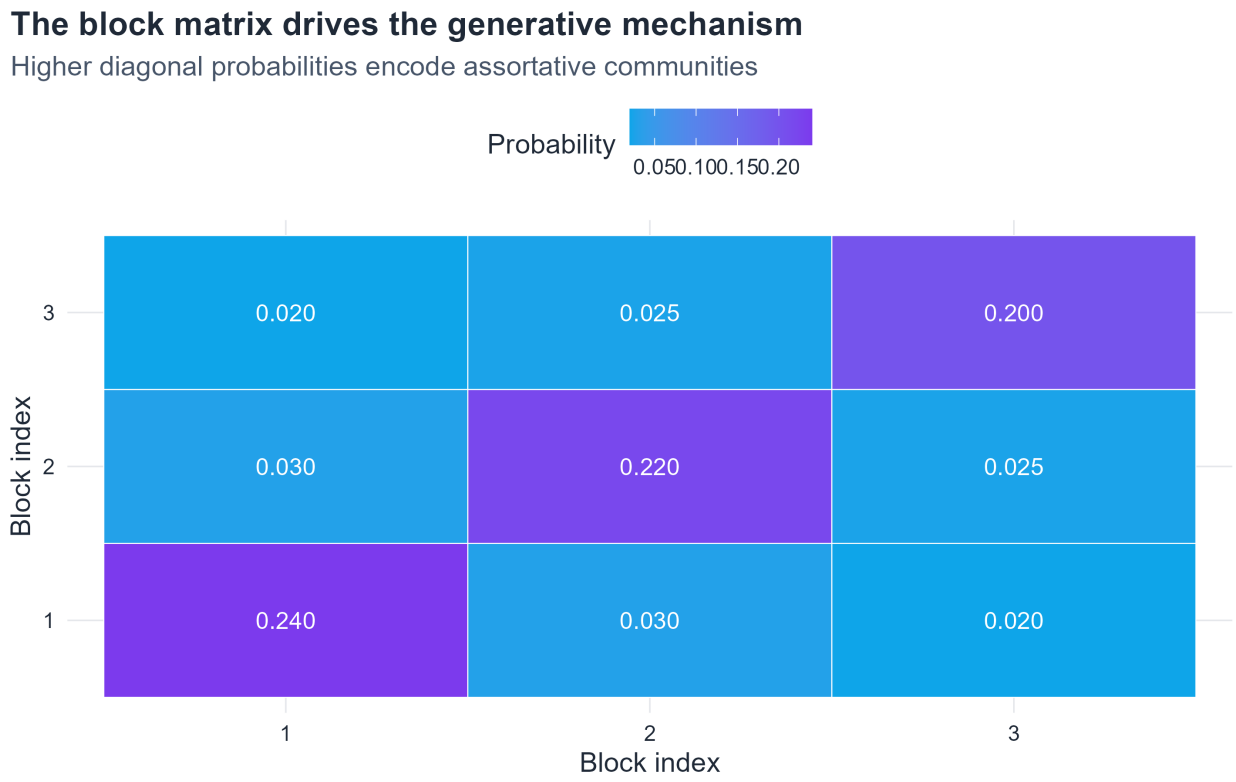


Figure 9.9: Block probability matrix Ω used to generate the synthetic SBM example. Diagonal entries are larger than off-diagonal entries, so the model encodes assortative communities before any graph is sampled.

Before looking at the sampled graph, it is worth pausing at Figure 9.9. The matrix Ω is the real object being specified by the model: it tells us which pairs of blocks should connect densely and which should connect sparsely. The adjacency matrix in Figure 9.8 is then just one random realization from that block-level recipe.

9.13.5 Recovering the planted structure

```
fit_sbm_louvain <- igraph::cluster_louvain(g_sbm)
truth     <- igraph::V(g_sbm)$block
estimated <- factor(igraph::membership(fit_sbm_louvain))
conf_df   <- as.data.frame(table(truth = truth, estimated = estimated)) |>
  dplyr::rename(count = Freq)

ggplot2::ggplot(conf_df, ggplot2::aes(x = estimated, y = truth, fill = count)) +
  ggplot2::geom_tile(color = "white") +
  ggplot2::geom_text(ggplot2::aes(label = count), color = "white", size = 4) +
  ggplot2::scale_fill_gradient(low = course_cols$blue, high = course_cols$violet) +
  ggplot2::labs(
    title    = "Detected versus planted communities",
```

```
    subtitle = "Label permutation is irrelevant; concentration matters",
    x = "Detected community", y = "Planted block", fill = "Vertices"
  )
```

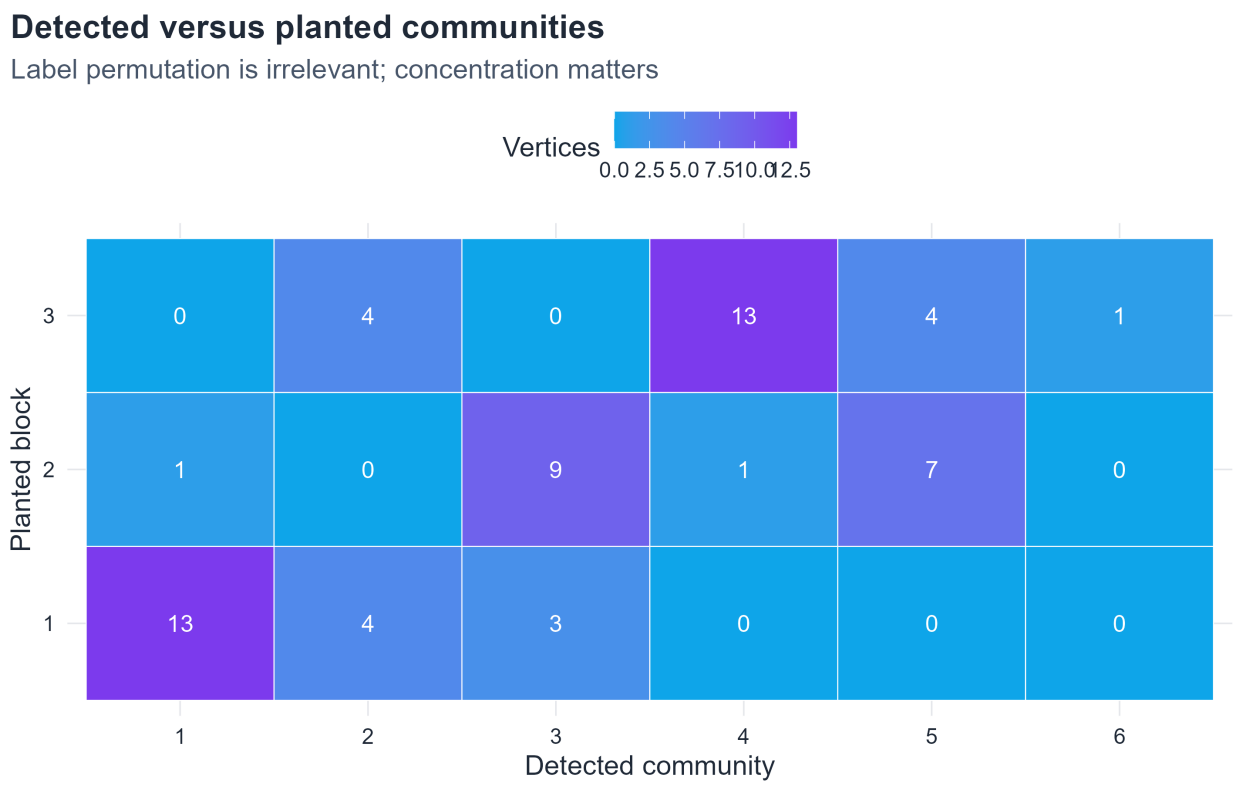


Figure 9.10: Confusion matrix between planted SBM blocks and Louvain-detected blocks. Strong concentration along a permutation of the diagonal indicates near-perfect recovery. Community labels are arbitrary up to permutation.

As shown in Figure 9.10, recovery should not be judged by literal agreement of community labels. Community labels are arbitrary up to permutation, so what matters is whether the mass of the confusion matrix concentrates near a permutation of the diagonal. This is the statistical analogue of a successful recovery plot.

9.13.6 Degree-corrected SBM

The standard SBM implicitly assumes that all vertices in a block have similar degrees, since $\Pr(A_{ij} = 1)$ depends only on block membership. In practice, degree heterogeneity is strong: recall from the scale-free chapter that real networks often have power-law degree distributions. A hub within a block will be connected to many vertices in other blocks purely because of its high degree, not because of genuine intercommunity affinity. The basic SBM may misidentify this degree-driven connectivity as evidence for multiple communities.

The **degree-corrected SBM** (DC-SBM) of Karrer and Newman (Karrer & Newman, 2011) introduces vertex-specific degree parameters $\theta_i > 0$:

$$\Pr(A_{ij} = 1 \mid z_i = r,\, z_j = s) = \theta_i \theta_j \omega_{rs},$$

where ω_{rs} now captures block-level affinity after controlling for individual degree propensities. Under this model, the expected degree of vertex i is

$$\mathbb{E}[k_i] = \theta_i \sum_{j \neq i} \theta_j \omega_{z_i z_j},$$

so θ_i is identifiable from the degree sequence. The DC-SBM can represent both heavy-tailed degree distributions and modular structure simultaneously, resolving the tension between hubs and communities that the standard SBM cannot handle. Fitting the DC-SBM by maximum likelihood leads to an extended EM algorithm or to a belief-propagation method that alternates between degree normalization and block assignment.

9.14 Detectability and the Kesten–Stigum threshold

Community detection is subject to a fundamental limit that is not merely computational but **information-theoretic**: even given unlimited computation, there exist regimes in which no algorithm can recover communities at a rate better than random guessing.

9.14.1 Setup: the symmetric planted partition model

Consider the symmetric two-block planted partition model: n vertices, two equal-size blocks, within-block edge probability $p_{\text{in}} = a/n$, and between-block probability $p_{\text{out}} = b/n$ (with $a > b$ for assortativity). This model is a special case of the sparse SBM, where average degree is $c = (a+b)/2$.

9.14.2 The Kesten–Stigum threshold

A remarkable phase transition was established by Decelle et al. (Decelle et al., 2011) (conjectured) and proved rigorously by Massoulié (Massoulié, 2014) and Mossel et al. (Mossel et al., 2018):

> **i** Theorem (Kesten–Stigum, community detection)
>
> Let $\mu = (a-b)/2$ be the signal-to-noise parameter and $c = (a+b)/2$ the average degree. Then:
> - **Above threshold:** If $\mu^2 > c$ (equivalently, $(a-b)^2 > 2(a+b)$), there exists a polynomial-time algorithm that detects communities with positive correlation with the planted partition.
> - **Below threshold:** If $\mu^2 < c$, no algorithm—regardless of computational cost—can detect communities with correlation better than chance.
> - **At the threshold:** $\mu^2 = c$ is a sharp phase transition.

The condition $(a-b)^2 > 2(a+b)$ can be rewritten as the signal-to-noise ratio

$$\text{SNR} = \frac{(a-b)^2}{2(a+b)} > 1.$$

Below this threshold, the graph looks statistically identical to an Erdős–Rényi random graph with the same average degree: the community signal is entirely absorbed into statistical fluctuations.

Why is this information-theoretic? The proof that recovery is impossible below the threshold proceeds by showing that the mutual information $I(\mathbf{z}; A)$ between the planted labels and the observed graph goes to zero as $n \to \infty$. Even an oracle with access to the entire graph cannot extract the label vector $\mathbf{z}$ from A if $\text{SNR} < 1$. This is not a statement about algorithms but about the data itself.

```
set.seed(2081)
mixing_ratios <- seq(0.05, 0.90, by = 0.05)
p_in_fixed    <- 0.30
```

```r
detect_df <- purrr::map_dfr(mixing_ratios, function(ratio) {
  p_out <- ratio * p_in_fixed
  g_tmp <- make_planted_partition(
    sizes = c(20, 20), p_in = p_in_fixed, p_out = p_out
  )
  fit_tmp <- igraph::cluster_louvain(g_tmp)
  tibble::tibble(
    mixing   = ratio,
    Q        = igraph::modularity(fit_tmp),
    n_comm   = length(igraph::sizes(fit_tmp))
  )
})

# Kesten-Stigum threshold for this model: (a-b)^2 = 2(a+b)
n_nodes <- 40
ks_ratio <- function(r, p_in, n) {
  a <- p_in * n
  b <- r * p_in * n
  (a - b)^2 / (2 * (a + b))
}

ks_snr <- purrr::map_dbl(mixing_ratios, ks_ratio, p_in = p_in_fixed, n = n_nodes)
ks_threshold_x <- mixing_ratios[which.min(abs(ks_snr - 1))]

ggplot2::ggplot(detect_df, ggplot2::aes(x = mixing, y = Q)) +
  ggplot2::geom_line(color = course_cols$blue, linewidth = 1.1) +
  ggplot2::geom_point(color = course_cols$blue, size = 2) +
  ggplot2::geom_vline(
    xintercept = ks_threshold_x,
    linetype   = "dashed",
    color      = course_cols$rose,
    linewidth  = 1.0
  ) +
  ggplot2::annotate(
    "text",
    x = ks_threshold_x + 0.02,
    y = 0.35,
    label = "Kesten-Stigum\nthreshold",
    color = course_cols$rose,
    hjust = 0,
    size = 3.5
  ) +
  ggplot2::labs(
    title    = "Detectability degrades near the Kesten-Stigum threshold",
    subtitle = expression(p["in"] == 0.30 ~~ "two equal blocks of 20 vertices"),
    x        = expression(p["out"] / p["in"] ~~ "(mixing ratio)"),
    y        = "Louvain modularity Q"
  )
```

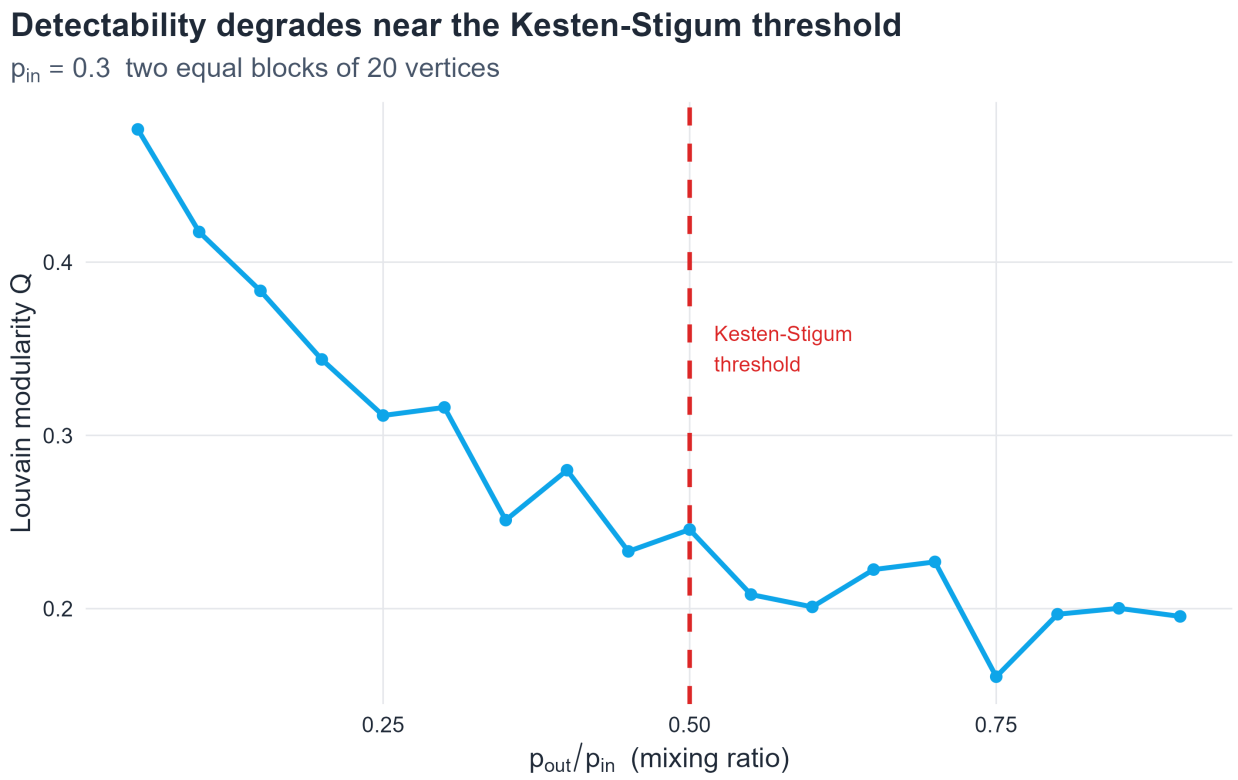


Figure 9.11: Modularity of the Louvain partition as $p_{\text{out}}/p_{\text{in}}$ increases toward 1. Recovery degrades sharply near the Kesten–Stigum threshold. At high mixing, the detected partition is no better than a random assignment.

9.15 Comparing partitions: NMI, ARI, and variation of information

When evaluating a community detection method against planted labels, or comparing two different methods, we need a principled way to measure partition similarity. Three measures are standard in the literature.

9.15.1 Normalized Mutual Information (NMI)

Given two partitions $\mathcal{C} = \{C_1, \dots, C_q\}$ and $\mathcal{C}' = \{C'_1, \dots, C'_{q'}\}$ of the same vertex set, let $n_{rs} = |C_r \cap C'_s|$, $n_r = |C_r|$, $n'_s = |C'_s|$.

The **mutual information** between the two partitions (treated as random variables on a uniformly drawn vertex) is

$$I(\mathcal{C}; \mathcal{C}') = \sum_{r=1}^{q} \sum_{s=1}^{q'} \frac{n_{rs}}{n} \log \frac{n\, n_{rs}}{n_r\, n'_s}.$$

The marginal entropies are $H(\mathcal{C}) = -\sum_r (n_r/n) \log(n_r/n)$ and $H(\mathcal{C}')$ defined similarly. The normalized mutual information is

$$\mathrm{NMI}(\mathcal{C}, \mathcal{C}') = \frac{2\, I(\mathcal{C}; \mathcal{C}')}{H(\mathcal{C}) + H(\mathcal{C}')}.$$

$\mathrm{NMI} \in [0, 1]$: value 1 indicates perfect agreement (up to permutation of labels), value 0 indicates statistical independence between the two partitions. NMI is invariant to label permutation, which is essential since community labels are meaningless beyond their identity.

9.15.2 Adjusted Rand Index (ARI)

The **Rand Index** counts the fraction of vertex pairs on which two partitions agree (both in the same community, or both in different communities):

$$\mathrm{RI}(\mathcal{C}, \mathcal{C}') = \frac{|\{(i,j) : (c_i = c_j) \Leftrightarrow (c'_i = c'_j)\}|}{\binom{n}{2}}.$$

The **Adjusted Rand Index** corrects for expected agreement under random partitions of the same size:

$$\mathrm{ARI} = \frac{\mathrm{RI} - \mathbb{E}[\mathrm{RI}]}{\max(\mathrm{RI}) - \mathbb{E}[\mathrm{RI}]}.$$

$\mathrm{ARI} \in [-1, 1]$: value 1 means perfect agreement, value 0 means agreement at chance level, and negative values indicate worse-than-chance agreement. ARI is particularly appropriate when community sizes are highly unbalanced, a regime where NMI can be misleading.

9.15.3 Variation of Information

The **Variation of Information** (VI) is a metric on the space of partitions (Meilă, 2003):

$$\mathrm{VI}(\mathcal{C}, \mathcal{C}') = H(\mathcal{C} \mid \mathcal{C}') + H(\mathcal{C}' \mid \mathcal{C}),$$

where $H(\mathcal{C} \mid \mathcal{C}') = H(\mathcal{C}) - I(\mathcal{C}; \mathcal{C}')$ is the conditional entropy. Unlike NMI, VI satisfies the triangle inequality, making it a true metric on the space of partitions. It is the theoretically cleanest measure but is not bounded by 1 in general (it grows with $\log n$).

```
set.seed(2081)
mixing_eval <- seq(0.05, 0.90, by = 0.05)

compute_nmi <- function(true_labels, detected_labels) {
  n      <- length(true_labels)
  tbl    <- table(true_labels, detected_labels)
  n_r    <- rowSums(tbl)
  n_s    <- colSums(tbl)
  H_C    <- -sum((n_r / n) * log(n_r / n + 1e-15))
  H_Cp   <- -sum((n_s / n) * log(n_s / n + 1e-15))
  MI     <- sum((tbl / n) * log(n * tbl / outer(n_r, n_s) + 1e-15))
  2 * MI / (H_C + H_Cp + 1e-15)
}

compute_ari <- function(true_labels, detected_labels) {
  tbl   <- table(true_labels, detected_labels)
  sum_sq_rows <- sum(choose(rowSums(tbl), 2))
  sum_sq_cols <- sum(choose(colSums(tbl), 2))
  sum_sq_all  <- sum(choose(tbl, 2))
```

```
  n_pairs     <- choose(length(true_labels), 2)
  expected    <- sum_sq_rows * sum_sq_cols / n_pairs
  maximum     <- (sum_sq_rows + sum_sq_cols) / 2
  (sum_sq_all - expected) / (maximum - expected + 1e-15)
}

eval_df <- purrr::map_dfr(mixing_eval, function(ratio) {
  p_out <- ratio * p_in_fixed
  g_tmp <- make_planted_partition(
    sizes = c(20, 20), p_in = p_in_fixed, p_out = p_out
  )
  true_lab <- as.integer(igraph::V(g_tmp)$block)
  fit_tmp  <- igraph::cluster_louvain(g_tmp)
  det_lab  <- igraph::membership(fit_tmp)
  tibble::tibble(
    mixing = ratio,
    NMI    = compute_nmi(true_lab, det_lab),
    ARI    = compute_ari(true_lab, det_lab)
  )
})

eval_long <- eval_df |>
  tidyr::pivot_longer(cols = c(NMI, ARI), names_to = "metric", values_to = "value")

ggplot2::ggplot(eval_long, ggplot2::aes(x = mixing, y = value, color = metric)) +
  ggplot2::geom_line(linewidth = 1.1) +
  ggplot2::geom_point(size = 2) +
  ggplot2::geom_vline(
    xintercept = ks_threshold_x,
    linetype = "dashed",
    color = course_cols$rose,
    linewidth = 0.9
  ) +
  ggplot2::scale_color_manual(
    values = c("NMI" = course_cols$blue, "ARI" = course_cols$gold)
  ) +
  ggplot2::labs(
    title    = "Recovery quality versus mixing strength",
    subtitle = "Both NMI and ARI collapse near the Kesten-Stigum threshold (red dashed)",
    x        = expression(p["out"] / p["in"]),
    y        = "Partition similarity",
    color    = "Metric"
  )
```

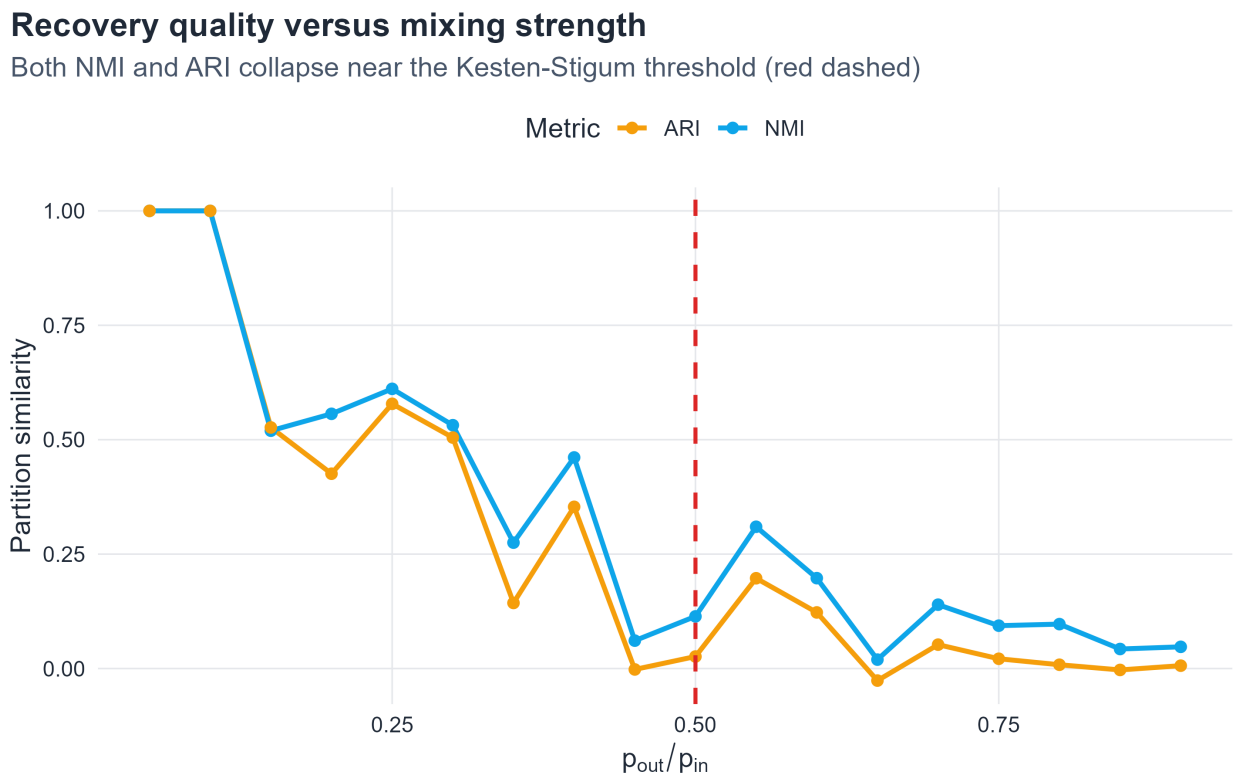


Figure 9.12: NMI (blue) and ARI (gold) between planted and Louvain-detected partitions as the mixing ratio increases. Both metrics approach zero near and beyond the Kesten-Stigum threshold, confirming that recovery becomes chance-level.

9.16 Limitations of community detection

9.16.1 The resolution limit of modularity

Modularity has a **resolution limit**: in a large graph, optimizing Q tends to merge small but well-defined communities into larger blocks (Fortunato & Barthélemy, 2007). The origin of this behavior can be understood from the null model: for a small community C_r with e_{rr} edges, the null-model term a_r^2 depends on the global size of the graph through $\mathrm{vol}(C_r)/(2m)$. When m is large, the modularity gain of splitting a small but genuine community becomes negligible relative to the gain from merging it with a neighbor.

Quantitatively, Fortunato and Barthélemy showed that communities with fewer than $\sqrt{m}$ internal edges may not be resolvable by modularity maximization, regardless of how isolated they actually are. This is a global property of the objective function, not a local artifact, and it cannot be fixed by using a different optimization algorithm. Resolution-limit-aware methods include the use of a tunable resolution parameter γ in the modified objective $Q_\gamma = \sum_r (e_{rr} - \gamma a_r^2)$, where $\gamma > 1$ favors finer partitions.

9.16.2 Overlapping and hierarchical communities

Real networks often violate the hard-partition assumption. A scientist may belong to multiple research fields; a gene may participate in multiple functional modules; a city may lie at the intersection of cultural regions. **Overlapping community** models such as the mixed-membership SBM (Airoldi et al., 2008) and the BigCLAM non-negative matrix factorization approach (Yang & Leskovec, 2013) extend the partition framework to soft assignments $u_{ir} \geq 0$ representing vertex i's strength of membership in community r.

Hierarchical community structure is equally common: communities often nest within super-communities, which nest within still larger groups. Flat partitions lose this hierarchical information. Methods that return a dendrogram (Girvan–Newman, Ward's linkage on graph distance) preserve it, at the cost of requiring the user to choose a resolution for the final partition.

9.16.3 Benchmark dependence

Method performance depends strongly on graph properties: size, density, degree heterogeneity, and mixing strength. The **LFR benchmark** (Lancichinetti et al., 2008) generates synthetic graphs with planted communities, power-law degree distributions, and power-law community size distributions, providing a more realistic test bed than the homogeneous planted partition model. A method that excels on the planted partition may fail on LFR benchmarks at high mixing, and vice versa. Any method comparison should therefore span a range of benchmark families.

> ⚠ Interpretive caution
>
> A detected community partition is a structured hypothesis about mesoscale organization, not a scientific fact. It should be interpreted alongside domain knowledge, validated against available metadata, and checked for stability under different methods and parameter choices.

9.17 Comparing the main approaches

Method	Core idea	Main strength	Main limitation
Girvan–Newman	Remove edges with high edge betweenness	Highly interpretable on small graphs	Too slow for large networks
Spectral clustering	Relax cut optimization into an eigenvector problem	Strong mathematical foundation; clear geometry	Requires choosing the number of communities
Louvain/Leiden	Greedy modularity optimization	Fast and scalable	Objective-function dependent; modularity has a resolution limit
SBM / DC-SBM	Infer latent blocks from a generative model	Probabilistic interpretation and uncertainty	More modeling choices; inference can be delicate

The point of this table is not to crown a universal winner. Rather, it helps separate four genuinely different philosophies of community detection: bridge removal, spectral relaxation, quality-function optimization, and probabilistic inference. A mature analysis often compares more than one of them.

9.18 A practical workflow for community analysis

For empirical data, principled method selection and careful interpretation matter as much as algorithmic choice.

1. **Pre-analysis.** Examine the degree distribution, connected components, and whether a raw adjacency matrix (reordered by degree) shows any block structure. Understand whether edge weights or vertex attributes are available.
2. **Run multiple methods.** At minimum, compare a modularity-based method (Louvain or Leiden) with a random-walk or spectral method. Note where they agree and where they diverge.

3. **Inspect the full partition, not just Q.** Examine community sizes, internal density, conductance per community, and the confusion matrix against any available ground truth.
4. **Assess stability.** Perturb the graph slightly (remove 5–10% of edges at random) and rerun. Communities that appear robustly across perturbations are more credible.
5. **Validate against metadata.** If node attributes (geographic region, function, topic) are available, compute NMI or ARI between the detected partition and the metadata. Agreement strengthens interpretation; disagreement motivates investigation.
6. **Report limitations explicitly.** State the method used, the value of any resolution parameter, the number of detected communities, and whether the result was sensitive to these choices.

9.19 Concluding remarks

Community structure is one of the clearest and most consequential examples of mesoscale organization in complex networks. It is neither a local property like clustering coefficient nor a global scalar like path length; it describes an intermediate level at which a network decomposes into cohesive modules that often carry scientific meaning.

This chapter has developed three complementary theoretical frameworks:

- The **graph-cut and spectral** viewpoint formalizes communities as low-conductance sets, connects them to the eigenspectrum of the Laplacian via Cheeger's inequality, and leads to spectral clustering as a principled polynomial-time approximation of normalized cut minimization.
- The **modularity and null-model** viewpoint defines community quality relative to a degree-preserving random baseline, leads to the modularity matrix eigenvalue problem, and motivates greedy multilevel heuristics such as Louvain and Leiden.
- The **stochastic block model** viewpoint treats block membership as latent structure, provides a full likelihood and an EM inference framework, and generalizes beyond assortative communities to any pattern of inter-block connectivity.

All three frameworks share a common theme: communities are meaningful only relative to an appropriate baseline. The cut viewpoint uses random walks; the modularity viewpoint uses the configuration model; the SBM viewpoint uses a parametric generative model. For master-level study, understanding *which baseline each method assumes* is as important as knowing *how to run* the algorithm.

The Kesten–Stigum threshold places fundamental limits on what any method can achieve: when the community signal is too weak relative to graph sparsity, recovery at better-than-chance accuracy is information-theoretically impossible. This result is one of the deepest connections between network science and statistical physics, and it sets a realistic ceiling on what community detection can deliver in practice.

9.20 Exercises

9.20.1 Part I. Computational and derivation exercises

Exercise 8.1. Internal density, cut size, and conductance.
Construct a graph with two candidate subsets S_1 and S_2. Compute $\rho_{\text{in}}(S)$, $\text{cut}(S, V \setminus S)$, and $\phi(S)$ for both subsets. Explain why these three quantities do not always rank the subsets in the same order.

Exercise 8.2. Edge betweenness and bridge identification.
For a small graph of your choice, compute the edge-betweenness score of every edge. Identify the edges that act as bridges between dense regions, and explain why their shortest-path usage is large.

Exercise 8.3. Interpreting the modularity formula.

Starting from

$$Q = \frac{1}{2m} \sum_{i,j} \left(A_{ij} - \frac{k_i k_j}{2m} \right) \delta(c_i, c_j),$$

derive the community-level form

$$Q = \sum_r (e_{rr} - a_r^2).$$

State clearly what e_{rr} and a_r mean.

9.20.2 Part II. Conceptual and analytical exercises

Exercise 8.4. Clustering versus community structure.
Give one example of a graph with high clustering coefficient but weak global community structure, and one example of a graph with modest clustering but a clear block partition. Explain why local triangles and mesoscopic communities are different notions.

Exercise 8.5. Modularity versus SBM.
Write a short comparison between modularity-based community detection and SBM-based inference. Your answer should discuss the following points: objective function versus generative model, degree effects, interpretability, and sensitivity to weak signal.

Exercise 8.6. Detectability threshold.
In the sparse two-block SBM, explain in words the meaning of the inequality

$$(a - b)^2 > 2(a + b).$$

Why does failure of this condition represent an information-theoretic limitation rather than merely a weakness of one algorithm?

Exercise 8.7. Resolution limit and interpretation.
Explain why a globally normalized objective such as modularity can merge small but meaningful communities inside a much larger graph. Relate your answer to the idea of scale in mesoscale structure (Fortunato & Barthélemy, 2007).

9.20.3 Part III. Simulation and coding exercises

Exercise 8.8. Failure of Girvan–Newman under weak separation.
Use the planted-partition generator in this chapter. Keep p_{in} fixed and gradually increase p_{out}. At what point does the Girvan–Newman partition cease to align well with the planted blocks?

Exercise 8.9. Comparing algorithms on the same benchmark.
Generate one benchmark graph with known planted communities and compare `cluster_edge_betweenness()`, `cluster_walktrap()`, and `cluster_louvain()`. Compare their modularity values, the number of detected groups, and their qualitative agreement with the planted partition.

Exercise 8.10. SBM recovery and confusion matrices.
Generate SBM graphs with fixed group sizes but progressively weaker block separation. For each graph, compute a confusion matrix between planted and detected communities. Describe how recoverability degrades as the diagonal and off-diagonal probabilities become closer.

Exercise 8.11. Degree correction experiment.
Create a benchmark in which one block contains several high-degree hubs. Compare the behavior of a modularity-based method to the qualitative expectations of a degree-corrected SBM. What kinds of mistakes arise when degree heterogeneity is confused with community structure?

9.20.4 Mini-project

Mini-project 8.1. A full community-detection workflow on a benchmark network.
Build a complete analysis pipeline for a synthetic network with planted communities. Your report should include: 1. visualization of the graph and its reordered adjacency matrix, 2. at least two community-detection methods, 3. one quality function or external comparison metric, 4. a discussion of whether the recovered partition is structurally meaningful, 5. a reflection on which limitations from this chapter are visible in your experiment.

Your final report should distinguish carefully between *detecting a partition*, *justifying a partition mathematically*, and *interpreting a partition scientifically*.

Airoldi, E. M., Blei, D. M., Fienberg, S. E., & Xing, E. P. (2008). Mixed membership stochastic blockmodels. *Journal of Machine Learning Research*, *9*, 1981–2014.

Albert, R., & Barabási, A.-L. (2002). Statistical mechanics of complex networks. *Reviews of Modern Physics*, *74*(1), 47–97. https://doi.org/10.1103/RevModPhys.74.47

Albert, R., Jeong, H., & Barabási, A.-L. (2000). Error and attack tolerance of complex networks. *Nature*, *406*(6794), 378–382. https://doi.org/10.1038/35019019

Barabási, A.-L., & Albert, R. (1999). Emergence of scaling in random networks. *Science*, *286*(5439), 509–512. https://doi.org/10.1126/science.286.5439.509

Barabási, A.-L., & Pósfai, M. (2016). *Network science*. Cambridge University Press.

Blondel, V. D., Guillaume, J.-L., Lambiotte, R., & Lefebvre, E. (2008). Fast unfolding of communities in large networks. *Journal of Statistical Mechanics: Theory and Experiment*, *2008*(10), P10008. https://doi.org/10.1088/1742-5468/2008/10/P10008

Bollobás, B. (2001). *Random graphs* (2nd ed.). Cambridge University Press. https://doi.org/10.1017/CBO9780511814068

Bollobás, B., Riordan, O., Spencer, J., & Tusnády, G. (2001). The degree sequence of a scale-free random graph process. *Random Structures & Algorithms*, *18*(3), 279–290. https://doi.org/10.1002/rsa.1009

Brandes, U., Delling, D., Gaertler, M., Görke, R., Hoefer, M., Nikoloski, Z., & Wagner, D. (2008). On modularity clustering. *IEEE Transactions on Knowledge and Data Engineering*, *20*(2), 172–188. https://doi.org/10.1109/TKDE.2007.190689

Callaway, D. S., Newman, M. E. J., Strogatz, S. H., & Watts, D. J. (2000). Network robustness and fragility: Percolation on random graphs. *Physical Review Letters*, *85*(25), 5468–5471. https://doi.org/10.1103/PhysRevLett.85.5468

Cayley, A. (1889). A theorem on trees. *Quarterly Journal of Pure and Applied Mathematics*, *23*, 376–378.

Chung, F. R. K. (1997). *Spectral graph theory* (Vol. 92). American Mathematical Society.

Clauset, A., Shalizi, C. R., & Newman, M. E. J. (2009). Power-law distributions in empirical data. *SIAM Review*, *51*(4), 661–703. https://doi.org/10.1137/070710111

Cohen, R., Erez, K., ben-Avraham, D., & Havlin, S. (2000). Resilience of the internet to random breakdowns. *Physical Review Letters*, *85*(21), 4626–4628. https://doi.org/10.1103/PhysRevLett.85.4626

Decelle, A., Krzakala, F., Moore, C., & Zdeborová, L. (2011). Asymptotic analysis of the stochastic block model for modular networks and its algorithmic applications. *Physical Review E*, *84*(6), 066106. https://doi.org/10.1103/PhysRevE.84.066106

Dempster, A. P., Laird, N. M., & Rubin, D. B. (1977). Maximum likelihood from incomplete data via the EM algorithm. *Journal of the Royal Statistical Society. Series B (Methodological)*, *39*(1), 1–38.

Diestel, R. (2017). *Graph theory* (5th ed.). Springer. https://doi.org/10.1007/978-3-662-53622-3

Dorogovtsev, S. N., Mendes, J. F. F., & Samukhin, A. N. (2003). Metric structure of random networks. *Nuclear Physics B*, *653*(3), 307–338. https://doi.org/10.1016/S0550-3213(03)00095-2

Erdős, P., & Gallai, T. (1960). Graphs with prescribed degrees of vertices. *Matematikai Lapok*, *11*, 264–274.

Erdős, P., & Rényi, A. (1959). On random graphs i. *Publicationes Mathematicae*, *6*, 290–297.

Erdős, P., & Rényi, A. (1960). On the evolution of random graphs. *Publications of the Mathematical Institute of the Hungarian Academy of Sciences*, *5*, 17–61.

Euler, L. (1736). Solutio problematis ad geometriam situs pertinentis. *Commentarii Academiae Scientiarum Imperialis Petropolitanae*, *8*, 128–140.

Fiedler, M. (1973). Algebraic connectivity of graphs. *Czechoslovak Mathematical Journal*, *23*(2), 298–305.

Fortunato, S. (2010). Community detection in graphs. *Physics Reports*, *486*(3–5), 75–174. https://doi.org/10.1016/j.physrep.2009.11.002

Fortunato, S., & Barthélemy, M. (2007). Resolution limit in community detection. *Proceedings of the National Academy of Sciences*, *104*(1), 36–41. https://doi.org/10.1073/pnas.0605965104

Gilbert, E. N. (1959). Random graphs. *The Annals of Mathematical Statistics*, *30*(4), 1141–1144. https://doi.org/10.1214/aoms/1177706098

Girvan, M., & Newman, M. E. J. (2002). Community structure in social and biological networks. *Proceedings of the National Academy of Sciences*, *99*(12), 7821–7826. https://doi.org/10.1073/pnas.122653799

Holland, P. W., Laskey, K. B., & Leinhardt, S. (1983). Stochastic blockmodels: First steps. *Social Networks*, *5*(2), 109–137. https://doi.org/10.1016/0378-8733(83)90021-7

Humphries, M. D., & Gurney, K. (2008). Network 'small-world-ness': A quantitative method for determining canonical network equivalence. *PLoS ONE*, *3*(4), e0002051. https://doi.org/10.1371/journal.pone.0002051

Karrer, B., & Newman, M. E. J. (2011). Stochastic blockmodels and community structure in networks. *Physical Review E*, *83*(1), 016107. https://doi.org/10.1103/PhysRevE.83.016107

Kőnig, D. (1916). Über graphen und ihre anwendung auf determinantentheorie und mengenlehre. *Mathematische Annalen*, *77*(4), 453–465. https://doi.org/10.1007/BF01456961

Lancichinetti, A., Fortunato, S., & Radicchi, F. (2008). Benchmark graphs for testing community detection algorithms. *Physical Review E*, *78*(4), 046110. https://doi.org/10.1103/PhysRevE.78.046110

Luxburg, U. von. (2007). A tutorial on spectral clustering. *Statistics and Computing*, *17*(4), 395–416. https://doi.org/10.1007/s11222-007-9033-z

Massoulié, L. (2014). Community detection thresholds and the weak ramanujan property. *Proceedings of the 46th Annual ACM Symposium on Theory of Computing*, 694–703. https://doi.org/10.1145/2591796.2591857

Meilă, M. (2003). Comparing clusterings by the variation of information. In B. Schölkopf & M. K. Warmuth (Eds.), *Learning theory and kernel machines* (Vol. 2777, pp. 173–187). Springer. https://doi.org/10.1007/978-3-540-45167-9_14

Milgram, S. (1967). The small world problem. *Psychology Today*, *2*(1), 60–67.

Molloy, M., & Reed, B. (1995). A critical point for random graphs with a given degree sequence. *Random Structures & Algorithms*, *6*(2-3), 161–180. https://doi.org/10.1002/rsa.3240060204

Mossel, E., Neeman, J., & Sly, A. (2018). A proof of the block model threshold conjecture. *Combinatorica*, *38*(3), 665–708. https://doi.org/10.1007/s00493-016-3291-z

Newman, M. E. J. (2006). Modularity and community structure in networks. *Proceedings of the National Academy of Sciences*, *103*(23), 8577–8582. https://doi.org/10.1073/pnas.0601602103

Newman, M. E. J. (2010). *Networks: An introduction*. Oxford University Press.

Newman, M. E. J., & Girvan, M. (2004). Finding and evaluating community structure in networks. *Physical Review E*, *69*(2), 026113. https://doi.org/10.1103/PhysRevE.69.026113

Newman, M. E. J., Strogatz, S. H., & Watts, D. J. (2001). Random graphs with arbitrary degree distributions and their applications. *Physical Review E*, *64*(2), 026118. https://doi.org/10.1103/PhysRevE.64.026118

Ng, A. Y., Jordan, M. I., & Weiss, Y. (2002). On spectral clustering: Analysis and an algorithm. *Advances in Neural Information Processing Systems 14*, 849–856.

Nowicki, K., & Snijders, T. A. B. (2001). Estimation and prediction for stochastic blockstructures. *Journal of the American Statistical Association*, *96*(455), 1077–1087. https://doi.org/10.1198/016214501753208735

Shi, J., & Malik, J. (2000). Normalized cuts and image segmentation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *22*(8), 888–905. https://doi.org/10.1109/34.868688

Strogatz, S. H. (2001). Exploring complex networks. *Nature*, *410*(6825), 268–276. https://doi.org/10.1038/35065725

Telesford, Q. K., Joyce, K. E., Hayasaka, S., Burdette, J. H., & Laurienti, P. J. (2011). The ubiquity of small-world networks. *Brain Connectivity*, *1*(5), 367–375. https://doi.org/10.1089/brain.2011.0038

Traag, V. A., Waltman, L., & Eck, N. J. van. (2019). From louvain to leiden: Guaranteeing well-connected communities. *Scientific Reports*, *9*, 5233. https://doi.org/10.1038/s41598-019-41695-z

Wang, Y. J., & Wong, G. Y. (1987). Stochastic blockmodels for directed graphs. *Journal of the American Statistical Association*, *82*(397), 8–19. https://doi.org/10.1080/01621459.1987.10478385

Watts, D. J., & Strogatz, S. H. (1998). Collective dynamics of 'small-world' networks. *Nature*, *393*(6684),

440–442. https://doi.org/10.1038/30918
West, D. B. (2001). *Introduction to graph theory* (2nd ed.). Prentice Hall.
Yang, J., & Leskovec, J. (2013). Overlapping community detection at scale: A nonnegative matrix factorization approach. *Proceedings of the Sixth ACM International Conference on Web Search and Data Mining*, 587–596. https://doi.org/10.1145/2433396.2433471